

Hardware and Software
Engineered to Work Together



Oracle Database 12c: New Features for Administrators

Student Guide – Volume II

D77758GC20

Edition 2.0 | November 2014 | D87749

Learn more from Oracle University at oracle.com/education/

Authors

Dominique Jeunot
Jean-François Verrier

Technical Contributors and Reviewers

Andy Rivenes
James Spiller
Donna Keesling
Maria Billings
Lachlan Williams
Peter Fusek
Mark Fuller
Gregg Christman
Dimpri Sarmah
Kevin Jernigan
Branislav Valny
Frank Fu
Joel Goodman
Gerlinde Frenzen
Harald Van Breederode
Hermann Baer
Jim Stenoish
Mark Drake
Beda Hammerschmidt
Prabhaker Gongloor
Patrick Wheeler
Maria Colgan
Jesse Kamp
Paul Needham
Pat Huey
Roy F Swonger
Ron Soltani
Sue Lee
Sharath Bhujani

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Disclaimer

This document contains proprietary information and is protected by copyright and other intellectual property laws. You may copy and print this document solely for your own use in an Oracle training course. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice. If you find any problems in the document, please report them in writing to: Oracle University, 500 Oracle Parkway, Redwood Shores, California 94065 USA. This document is not warranted to be error-free.

Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT RIGHTS

The U.S. Government's rights to use, modify, reproduce, release, perform, display, or disclose these training materials are restricted by the terms of the applicable Oracle license agreement and/or the applicable U.S. Government contract.

Trademark Notice

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Editors

Anwasha Ray
Malavika Jinka
Smita Kommini

Graphic Designers

Seema Bopaiah
Maheshwari Krishnamurthy

Publishers

Jobi Varghese
Pavithran Adka
Joseph Fernandez

Contents

1 Introduction

- Overview 1-2
- Oracle Database Innovation 1-3
- Enterprise Cloud Computing 1-4
- Oracle Database 12c New and Enhanced Features 1-5

2 Enterprise Manager Cloud Control and Other Tools

- Oracle Database 12c New and Enhanced Features 2-2
- Objectives 2-3
- Key Challenges for Administrators 2-4
- Enterprise Manager Cloud Control 2-5
- Cloud Control Components 2-7
- Components and Communication Flow 2-8
- Oracle Management Repository 2-9
- Controlling the Enterprise Manager Cloud Control Framework 2-10
- Starting the Enterprise Manager Cloud Control Framework 2-11
- Stopping the Enterprise Manager Cloud Control Framework 2-12
- Different Target Types 2-13
- Target Discovery 2-14
- Enterprise Manager Cloud Control 2-15
- User Interface 2-16
- Security: Overview 2-17
- Managing Securely with Credentials 2-18
- Distinguishing Credentials 2-19
- Quiz 2-21
- Enterprise Manager Database Express Architecture 2-22
- Configuring EM Database Express 2-23
- Home Page 2-24
- Menus 2-25
- Quiz 2-26
- Database Configuration Assistant 2-27
- Oracle SQL Developer: Connections 2-28
- Oracle SQL Developer: DBA Actions 2-29
- Quiz 2-30
- Summary 2-31
- Practice 2-32

3 Module – Multitenant Container Database and Pluggable Databases

Basics of Multitenant Container Database and Pluggable Databases 3-2

Oracle Database 12c New and Enhanced Features 3-3

Objectives 3-4

Challenges 3-5

Oracle Database in 11g Release 2 3-6

New Multitenant Architecture: Benefits 3-7

Other Benefits of Multitenant Architecture 3-9

Configurations 3-11

Multitenant Container Database 3-12

Pristine Installation 3-13

Adding User Data 3-14

Separating SYSTEM and User Data 3-15

SYSTEM Objects in the USER Container 3-16

Naming the Containers 3-17

Provisioning a Pluggable Database 3-18

Interacting Within Multitenant Container Database 3-19

Multitenant Container Database Architecture 3-20

Containers 3-21

Questions: Root Versus PDBs 3-22

Questions: PDBs Versus Root 3-23

Terminology 3-24

Common and Local Users 3-25

Common and Local Privileges and Roles 3-26

Shared and Non-Shared Objects 3-27

Data Dictionary Views 3-28

Impacts 3-29

Quiz 3-31

Summary 3-34

Practice 3-35

4 Creating Container Databases and Pluggable Databases

Oracle Database 12c New and Enhanced Features 4-2

Objectives 4-3

Goals 4-4

Tools 4-5

Steps to Create a Container Database 4-6

Creating a Container Database: Using SQL*Plus 4-7

Creating a Container Database: Using DBCA 4-9

New Clause: SEED FILE_NAME_CONVERT 4-10

New Clause: ENABLE PLUGGABLE DATABASE 4-11

After CDB Creation: What Is New in CDBs 4-12

Data Dictionary Views: DBA_xxx	4-13
Data Dictionary Views: CDB_xxx	4-14
Data Dictionary Views: Examples	4-15
Data Dictionary Views: V\$xxx Views	4-16
After CDB Creation: To-Do List	4-17
Automatic Diagnostic Repository	4-18
Automatic Diagnostic Repository: alert.log File	4-19
Quiz	4-20
Practice	4-22
Provisioning New Pluggable Databases	4-23
Tools	4-24
Method 1: Create New PDB from PDB\$SEED	4-25
Steps: With Location Clauses	4-26
Steps: Without Location Clauses	4-28
Synchronization	4-29
Method 2: Plug a Non-CDB into a CDB	4-30
Plug a Non-CDB in to CDB Using DBMS_PDB	4-32
Method 3: Clone Local PDBs	4-33
Method 3: Clone a Non-CDB or Remote PDB	4-34
Method 4: Plug Unplugged PDB in to CDB	4-35
Method 4: Flow	4-36
Plug Sample Schemas PDB: Using DBCA	4-38
Dropping a PDB	4-39
Migrating pre-12.1 Databases to 12.1 CDB	4-40
Quiz	4-41
Summary	4-43
Practice	4-44

5 Managing Multitenant Container Databases and Pluggable Databases

Oracle Database 12c New and Enhanced Features	5-2
Objectives	5-3
Connection	5-4
Connection with SQL*Developer	5-7
Switching Connections	5-8
Starting Up a CDB Instance	5-9
Mounting a CDB	5-10
Opening a CDB	5-11
Opening a PDB	5-12
Closing a PDB	5-13
Shutting Down a CDB Instance	5-14
Automatic PDB Opening	5-15
Changing PDB Open Mode	5-16

Changing PDB Mode: With SQL Developer 5-17
 Modifying PDB Settings 5-18
 Instance Parameter Change Impact 5-19
 Instance Parameter Change Impact: Example 5-20
 Quiz 5-21
 Summary 5-23
 Practice 5-24

6 Managing Tablespaces and Users in CDB and PDBs

Oracle Database 12c New and Enhanced Features 6-2
 Objectives 6-3
 Tablespaces in PDBs 6-4
 Creating Permanent Tablespaces in a CDB 6-5
 Assigning Default Tablespaces 6-6
 Creating Local Temporary Tablespaces 6-7
 Assigning Default Temporary Tablespaces 6-8
 Users, Roles, and Privileges 6-9
 Local Users, Roles, and Privileges 6-10
 Creating a Local User 6-11
 Common Users 6-12
 Creating Users 6-13
 Common and Local Schemas/Users 6-14
 Common and Local Privileges 6-15
 Granting and Revoking Privileges 6-16
 Creating Common and Local Roles 6-17
 Granting Common or Local Privileges/Roles to Roles 6-18
 Granting Common and Local Roles to Users 6-19
 Creating Shared and Non-Shared Objects 6-20
 Restriction on Definer's Rights 6-21
 Quiz 6-22
 Summary 6-24
 Practice 6-25

7 Backup, Recovery, and Flashback CDBs and PDBs

Oracle Database 12c New and Enhanced Features 7-2
 Objectives 7-3
 New Syntax and Clauses in RMAN 7-5
 CDB Backup: Whole CDB Backup 7-6
 CDB Backup: Partial CDB Backup 7-7
 PDB Backup: Whole PDB Backup 7-8
 PDB Backup: Partial PDB Backup 7-9
 PDB Backup: User-Managed Hot PDB Backup 7-10

Practice 7-11
 Recovery 7-12
 Instance Failure 7-13
 NOARCHIVELOG Mode 7-14
 Media Failure: CDB Temporary File Recovery 7-15
 Media Failure: PDB Temporary File Recovery 7-16
 Media Failure: Control File Loss 7-17
 Media Failure: Redo Log File Loss 7-18
 Media Failure: Root SYSTEM or UNDO Data File 7-19
 Media Failure: Root SYSAUX Data File 7-20
 Media Failure: PDB Data File 7-21
 Media Failure: PITR 7-22
 Flashback CDB 7-24
 Special Situations 7-26
 Quiz 7-27
 Summary 7-29
 Practice 7-30

8 Module – Automatic Data Optimization and Storage Enhancements

Heat Map, Automatic Data Optimization, and Online Data File and Partition Move 8-2
 Oracle Database 12c New and Enhanced Features 8-3
 Objectives 8-4
 ILM Challenges and Solutions 8-5
 ILM Components 8-6
 ILM Challenges 8-7
 Solutions 8-8
 Components 8-10
 What Is Automatic Data Optimization? 8-12
 Data Classification Levels 8-13
 Heat Map and ADO 8-14
 Enabling Heat Map Segment-Level Statistics 8-15
 DBA_HEAT_MAP_SEGMENT View 8-16
 Block-Level Statistics 8-17
 Extent-Level Statistics 8-18
 Defining Automatic Detection Conditions 8-19
 Defining Automatic Actions 8-20
 Compression Scopes and Types 8-21
 Creating Compression Policies Tablespace and Group 8-22
 Creating Compression Policies Segment and Row 8-23
 Creating Storage Tiering Policy 8-25
 Storage Tiering: Priority 8-26
 Storage Tiering: READ ONLY 8-27

- Policy Relying on Function 8-28
- Multiple SEGMENT Policies on a Segment 8-29
- Only One Single ROW Policy on a Segment 8-31
- Policy Inheritance 8-32
- Displaying Policies DBA_ILMPOLICIES/DBA_ILMDATAMOVEMENTPOLICIES 8-33
- Displaying Policies DBA_ILMDATAMOVEMENTPOLICIES 8-34
- Preparing Evaluation and Execution 8-35
- Customizing Evaluation and Execution 8-36
- Monitoring Evaluation and Execution 8-37
- ADO DDL 8-39
- Turning ADO Off and On 8-40
- Stop Activity Tracking and Clean Up Heat Map Statistics 8-41
- Specific Situations of Activity Tracking 8-42
- Quiz 8-44
- Online Move Data File 8-46
- Compression 8-47
- REUSE and KEEP 8-48
- States 8-49
- Compatibilities 8-50
- Flashback Database 8-51
- Online Move Partition 8-52
- Online Move Partition: Benefits 8-53
- Online Move Partition: Compress 8-54
- Quiz 8-55
- Summary 8-56
- Practice 8-57

9 In-Database Archiving and Temporal

- Oracle Database 12c New and Enhanced Features 9-2
- Objectives 9-3
- Archiving Challenges 9-4
- Archiving Solutions 9-5
- In-Database Archiving: HCC 9-6
- Archiving Challenges and Solutions 9-8
- In-Database Archiving 9-10
- ORA_ARCHIVE_STATE column 9-11
- Session Visibility Control 9-12
- Disable Row-Archival 9-13
- Quiz 9-14
- PERIOD FOR Clause Concept 9-16
- Filtering on Valid-Time Columns: Example 1 9-17
- Filtering on Valid-Time Columns: Example 2 9-18

DBMS_FLASHBACK_ARCHIVE 9-19
 Quiz 9-20
 Temporal History Enhancements: FDA Optimization 9-21
 Temporal History Enhancements: User Context Metadata 9-22
 Summary 9-23
 Practice 9-24

10 Module – Security

Auditing 10-2
 Oracle Database 12c New and Enhanced Features 10-3
 Objectives 10-4
 Types of Auditing 10-5
 Audit Trail Implementation 10-7
 Oracle Database 12c Auditing 10-9
 Security and Performance: Audit Architecture 10-10
 Consolidation 10-11
 Data Pump Audit Policy 10-12
 Unified Audit Implementation 10-13
 Quiz 10-15
 Security 10-17
 Simplicity: Audit Policy 10-18
 Step 1: Creating the Audit Policy 10-19
 Step 2: Enabling/Disabling the Audit Policy 10-21
 Viewing the Audit Policy 10-22
 Using Predefined Audit Policies 10-23
 Including Application Context Data 10-24
 Dropping the Audit Policy 10-25
 Audit Cleanup 10-26
 Quiz 10-27
 Summary 10-28
 Practice 10-29

11 Privileges

Oracle Database 12c New and Enhanced Features 11-2
 Objectives 11-3
 Major Challenges 11-4
 New Administrative Privileges 11-5
 OS Authentication and OS Groups 11-6
 Password Authentication for SYSBACKUP 11-8
 Oracle Database Vault Data Protection and Administration Privileged Users 11-10
 Quiz 11-11
 New System Privilege: PURGE DBA_RECYCLEBIN 11-13

Privilege Analysis	11-14
Privilege Analysis Flow	11-15
Used Privileges Results	11-17
Compare Used and Unused Privileges	11-18
Listing Captures	11-19
Dropping an Analysis	11-20
Quiz	11-21
Privilege Checking During PL/SQL Calls	11-22
New Privilege Checking During PL/SQL Calls	11-23
INHERIT (ANY) PRIVILEGES Privileges	11-24
Privilege Checking with New BEQUEATH Views	11-25
Quiz	11-26
Summary	11-28
Practice	11-29

12 Oracle Data Redaction

Oracle Database 12c New and Enhanced Features	12-2
Objectives	12-3
Oracle Data Redaction: Overview	12-4
Oracle Data Redaction and Operational Activities	12-6
Available Redaction Methods	12-7
Oracle Data Redaction: Examples	12-8
What Is a Redaction Policy?	12-9
Managing Redaction Policies	12-11
Applying a Redaction Policy to a Table or View	12-12
Full Redaction: Examples	12-13
Partial Redaction: Examples	12-14
Regular Expression	12-15
Modifying the Redaction Policy	12-16
Exempting Users from Redaction Policies	12-17
Using Oracle Data Redaction with Other Oracle Database Security Solutions	12-18
Oracle Database Security Features	12-19
Best Practices: Preventing Unauthorized Policy Modifications and Exemptions	12-21
Best Practices: Considerations	12-22
Summary	12-23
Practice 12: Overview	12-24

13 Module – High Availability

Recovery Manager: New Features	13-2
Oracle Database 12c New and Enhanced Features	13-3
Objectives	13-4
Separation of DBA Duties	13-5

Using SQL in RMAN	13-6
Backing Up and Restoring Very Large Files	13-7
RMAN Duplication Enhancements	13-8
Duplicating an Active Database	13-9
What Is New?	13-10
NOOPEN Option	13-11
Duplicating Multitenant Container Databases	13-12
Recovering Databases with Third-Party Snapshots	13-13
Quiz	13-14
Transporting Data Across Platforms	13-15
Data Transport	13-16
Transporting Database: Process Steps - 1	13-17
Transporting Database: Process Steps - 2	13-18
Transporting Tablespace: Process Steps - 1	13-19
Transporting Tablespace: Process Steps - 2	13-20
Quiz	13-21
Table Recovery	13-22
Recovering Tables from Backups	13-23
Table Recovery: Graphical Overview	13-24
Specifying the Recovery Point-in-Time	13-25
Process Steps of Table Recovery - 1	13-26
Customization	13-27
Quiz	13-28
Summary	13-29
Practice 13: Overview	13-30

14 Module – Manageability

Real-Time Database Operation Monitoring	14-2
Oracle Database 12c New and Enhanced Features	14-3
Objectives	14-4
Real-Time Database Operation Monitoring: Overview	14-5
Use Cases	14-6
Current Tools	14-7
Defining a DB Operation	14-8
Scope of a Composite DB Operation	14-9
Database Operation Concepts	14-10
Identifying a Database Operation	14-11
Enabling Monitoring of Database Operations	14-12
Identifying, Starting, and Completing a Database Operation	14-13
Monitoring the Progress of a Database Operation	14-14
Monitoring Load Database Operations	14-15
Monitoring Load Database Operation Details	14-16

Reporting Database Operations Using Views	14-17
Reporting Database Operations by Using Functions	14-19
Database Operation Tuning	14-21
Quiz	14-22
Summary	14-24
Practice 14: Overview	14-25

15 Emergency Monitoring, Real-Time ADDM, Compare Period ADDM, and ASH Analytics

Oracle Database 12c New and Enhanced Features	15-2
Objectives	15-3
Emergency Monitoring: Challenges	15-4
Emergency Monitoring: Goals	15-5
Real-Time ADDM: Challenges	15-7
Real-Time ADDM: Goals	15-8
Flow	15-10
Using the DBMS_ADDM Package	15-11
Quiz	15-12
AWR Compare Periods Report	15-13
Method: Preserved Snapshot Sets	15-14
What Is Missing?	15-15
Compare Period ADDM: Analysis	15-16
Workload Commonality	15-17
Comparison Modes	15-18
Report: Configuration	15-19
Report: Finding	15-21
Using the DBMS_ADDM Package	15-22
Quiz	15-24
ASH: Overview	15-25
Top Activity Page	15-26
ASH Analytics Page: Activity	15-27
Summary	15-28
Practice 15: Overview	15-29

16 ADR and Network Enhancements

Oracle Database 12c New and Enhanced Features	16-2
Objectives	16-3
Automatic Diagnostic Repository	16-4
ADR File Types	16-5
ADR Files: Location	16-6
ADR Files: DDL and DEBUG Log Files	16-7
New ADRCI Command	16-8
Network Performance: Compression	16-9

Setting Up Compression 16-10
 Session Data Unit (SDU) Size 16-11
 Setting SDU Size 16-12
 Quiz 16-13
 Summary 16-14
 Practice 16: Overview 16-15

17 Module – Performance

In-Memory Column Store 17-2
 Oracle Database 12c New and Enhanced Features 17-3
 Objectives 17-4
 Goals of In-Memory Column Store 17-5
 Benefits 17-7
 Overview 17-8
 Row Store Versus Column Store: 2D Vision 17-10
 In-Memory Column Unit 17-11
 In-Memory Column Store Cache Versus Buffer Cache 17-12
 Dual Format In Memory 17-13
 No More Indexes Issues 17-14
 Process 17-15
 Deploying IM Column Store 17-16
 Deploying IM Column Store: Objects Setting 17-18
 Deploying IM Column Store: Columns Setting 17-19
 Objects Candidates for IM Column Store 17-20
 Columns Candidates for IM Column Store 17-22
 Defining IM Column Store Priority 17-23
 Segments Populated into the IM Column Store 17-24
 Defining IM Column Store Compression 17-25
 IM Column Store Compression Advisor 17-27
 Computing Compression Ratio 17-29
 Default In-Memory Setting 17-30
 Impact of Changing In-Memory Attributes 17-31
 Moving or Splitting In-Memory Segments 17-32
 INMEMORY Inheritance 17-33
 After Objects Settings 17-34
 Retrieving CREATE DDL Statement of In-Memory Objects 17-35
 Quiz 17-36
 Query Benefits 17-38
 Testing and Comparing Query Performance 17-39
 Queries on In-Memory Tables: Simple Predicate 17-40
 MINMAX Pruning Statistics 17-41
 IM Column Store Statistics 17-42

Execution Plan: TABLE ACCESS IN MEMORY FULL 17-43
 Queries on In-Memory Tables: Join 17-44
 Execution Plan: JOIN FILTER CREATE / USE 17-46
 Queries on In-Memory and Non-In-Memory Tables 17-47
 Queries on In-Memory and Non-In-Memory Columns 17-48
 DMLs and In-Memory Column Store 17-49
 Recommendations 17-50
 Views 17-51
 Interaction with Other Products 17-52
 Optimizer 17-53
 IM Column Store and RAC 17-55
 IM Column Store and Data Pump 17-57
 Data Pump TRANSFORM Names 17-58
 Summary 17-59
 Practice 17: Overview 17-60

18 In-Memory Caching

Oracle Database 12c New and Enhanced Features 18-2
 Objectives 18-3
 Full Database In-Memory Caching 18-4
 Setting Up Force Full Database Caching 18-6
 Monitoring Full Database In-Memory Caching 18-8
 In-Memory Parallel Query Before Automatic Big Table Caching 18-9
 Automatic Big Table Caching 18-11
 Configuring Automatic Big Table Caching 18-12
 Using Automatic Big Table Caching 18-14
 Monitoring Automatic Big Table Caching 18-15
 Summary 18-17
 Practice 18: Overview 18-18

19 SQL Tuning Enhancements

Oracle Database 12c New and Enhanced Features 19-2
 Objectives 19-3
 Road Map 19-4
 SQL Plan Baseline: Architecture 19-5
 SQL Plan Management: Overview 19-7
 Adaptive SQL Plan Management 19-8
 Automatically Evolving SQL Plan Baseline 19-9
 SQL Management Base Enhancements 19-10
 Quiz 19-11
 Lesson Road Map 19-12
 Adaptive Execution Plans 19-13

Dynamic Plans	19-14
Dynamic Plan: Adaptive Process	19-15
Dynamic Plans: Example	19-16
Reoptimization: Statistics Feedback	19-17
Statistics Feedback: Monitoring Query Executions	19-18
Statistics Feedback: Reparsing Statements	19-19
Automatic Reoptimization	19-20
Quiz	19-22
Lesson Road Map	19-23
SQL Plan Directives	19-24
Creating SQL Plan Directives	19-25
Using SQL Plan Directives	19-26
SQL Plan Directives: Example	19-27
Online Statistics Gathering for Bulk-Load	19-28
Concurrent Statistics Enhancements in Oracle Database 12c	19-29
Statistics for Global Temporary Tables	19-30
Histogram Enhancements	19-32
Top Frequency Histograms	19-33
Hybrid Histograms	19-34
Hybrid Histograms: Example	19-35
Extended Statistics Enhancements	19-36
Capturing Column Group Usage	19-37
Capturing Column Group Usage: Running the Workload	19-38
Creating Column Groups Detected During Workload Monitoring	19-40
Automatic Dynamic Sampling	19-41
Quiz	19-42
Summary	19-43
Practice	19-44

20 Resource Manager and Other Performance Enhancements

Oracle Database 12c New and Enhanced Features	20-2
Objectives	20-3
Resource Manager and Pluggable Databases	20-4
Managing Resources Between PDBs	20-5
CDB Resource Plan Basics: Share	20-6
CDB Resource Plan Basics: Limits	20-8
CDB Resource Plan: Full Example	20-10
Creating a CDB Resource Plan	20-11
Setting Default Directives	20-12
Viewing CDB Resource Plan Directives	20-13
Maintaining a CDB Resource Plan	20-14
Managing Resources Within a PDB	20-15

Managing PDB Resource Plans	20-16
Putting It Together	20-17
Considerations	20-18
Runaway Queries and Resource Manager	20-19
Controlling IM Column Store Repopulation Resource Consumption	20-21
Default UNIX/Linux Architecture	20-22
Multi-Process Multi-Threaded UNIX/Linux Architecture	20-23
Multi-Process Multi-Threaded Architecture: Benefits and Setup	20-24
Multi-Process Multi-Threaded Architecture: Considerations	20-25
Multi-Process Multi-Threaded Architecture: Monitoring	20-26
Database Smart Flash Cache Enhancements	20-27
Enabling and Disabling Flash Devices	20-28
In-Memory PQ Algorithm: Benefits	20-29
Smart Flash Cache: New Statistics	20-30
Temporary Undo: Overview	20-31
Temporary Undo: Benefits and Setup	20-32
Temporary Undo Monitoring	20-33
Limiting the Size of the Program Global Area	20-34
Summary	20-35
Practice 20: Overview	20-36

21 Tables, Indexes, and Online Operations Enhancements

Oracle Database 12c New and Enhanced Features	21-2
Objectives	21-3
Why Multiple Indexes on the Same Set of Columns?	21-4
Creating Multiple Indexes on the Same Set of Columns	21-5
Quiz	21-7
Invisible and Hidden Columns in SQL*Plus	21-8
SET COLINVISIBLE and DESCRIBE Commands	21-9
Quiz	21-10
Online Redefinition: Tables with VPD	21-11
Online Redefinition: dml_lock_timeout	21-12
Advanced Row Compression: New Feature Name and Syntax	21-13
LOB Compression: New Name	21-14
Using the Compression Advisor	21-15
Enhanced Online DDL Capabilities	21-16
DROP INDEX / CONSTRAINT	21-17
Index UNUSABLE	21-18
SET UNUSED Column	21-19
Summary	21-20
Practice 21: Overview	21-21

22 Module – Miscellaneous

- Oracle Data Pump, SQL*Loader, and External Tables 22-2
- Oracle Database 12c New and Enhanced Features 22-3
- Objectives 22-4
- Full Transportable Export/Import: Overview 22-5
- Full Transportable Export/Import: Usage 22-6
- Full Transportable Export/Import: Example 22-8
- Transporting a Database Over the Network: Example 22-9
- Disabling Logging for Oracle Data Pump Import 22-10
- Exporting Views as Tables 22-11
- Specifying the Encryption Password 22-13
- Compressing Tables During Import 22-14
- Creating SecureFile LOBs During Import 22-15
- Quiz 22-16
- SQL*Loader Support for Direct-Path Loading of Identity Columns 22-17
- SQL*Loader and External Table Enhancements 22-18
- SQL*Loader Express Mode 22-19
- Summary 22-21
- Practice 22: Overview 22-22

23 Partitioning Enhancements

- Oracle Database 12c New and Enhanced Features 23-2
- Objectives 23-3
- Enhancements to Reference Partitioning 23-4
- Interval Reference Partitioning 23-5
- TRUNCATE TABLE CASCADE 23-6
- Multipartition Maintenance Operations 23-7
- Adding Multiple Partitions 23-8
- Creating a Range-Partitioned Table 23-9
- Adding Multiple Partitions 23-10
- Truncating Multiple Partitions 23-11
- Dropping Multiple Partitions 23-12
- Splitting into Multiple Partitions 23-13
- Splitting into Multiple Partitions Rules 23-14
- Splitting into Multiple Partitions: Examples 23-15
- Merging Multiple Range Partitions 23-16
- Merging List and System Partitions 23-17
- Quiz 23-18
- Partitioned Indexes: Review 23-19
- Partial Indexes for Partitioned Tables 23-20
- Partial Index Creation on a Table 23-21
- Specifying the INDEXING Clause at the Partition and Subpartition Levels 23-22

Creating a Partial Local or Global Index	23-23
Explain Plan: LOCAL INDEX ROWID	23-24
Explain Plan: GLOBAL INDEX ROWID	23-25
Affected Data Dictionary Views: Overview	23-26
Asynchronous Global Index Maintenance	23-27
DBMS_PART Package	23-28
Global Index Maintenance Optimization During Partition Maintenance Operations	23-29
Quiz	23-30
Summary	23-31
Practice 23: Overview	23-32

24 SQL Enhancements and Migration Assistant for Unicode

Oracle Database 12c New and Enhanced Features	24-2
Objectives	24-3
Increased Length Limits of Data Types	24-4
Configuring Database for Extended Data Type	24-5
Using VARCHAR2, NVARCHAR2, and RAW Data Types	24-6
Database Migration Assistant for Unicode	24-7
SecureFiles	24-8
SQL Row-Limiting Clause	24-9
SQL Row-Limiting Clause: Examples	24-10
Quiz	24-11
Summary	24-13
Practice 25: Overview	24-14

A New Processes, Views, Parameters, Packages, and Privileges

Instance and Database	A-2
Multitenant Architecture: General Architecture Poster	A-3
CDB and PDB	A-4
Heat Map and ADO	A-6
In-Database Archiving and Temporal Validity	A-8
Security: Auditing	A-9
Security: Privilege Analysis	A-10
Security: Privilege Analysis and New Privileges	A-11
Security: Oracle Data Redaction	A-12
HA: Flashback Data Archive	A-13
Manageability: Database Operations	A-14
Manageability: ADDM	A-16
Performance: In-Memory Column Store	A-17
Performance: Full Database In-Memory Caching	A-19
Performance: Automatic Big Table Caching	A-20
Performance: SQL Tuning	A-21

Performance: Resource Manager A-22
 Performance: Multi-Process Multi-Threaded A-23
 Performance: Database Smart Flash Cache A-24
 Performance: Temporary UNDO A-25
 Performance: Online Operations A-26
 Miscellaneous: Partitioning A-27
 Miscellaneous: SQL A-28
 Appendix C: Data Comparison A-29

B Pluggable Databases: Other Creation Methods

Plugging a Non-CDB into CDB Using Data Pump B-2
 Plugging a Non-CDB into CDB Using Replication B-3
 Cloning PDBs Across CDBs B-4
 Plugging Unplugged PDB: Using SQL Developer B-5

C Schema and Data Change Management

Objectives C-2
 Database Lifecycle Management Pack: New Features C-3
 Change Management Pack Features C-4
 Change Management Pack Components C-5
 Dictionary Baselines C-6
 Dictionary Comparisons C-8
 Dictionary Synchronization C-10
 Comparing Change Propagation and 11g SQL Scripts C-11
 Database Lifecycle Management Pack Schema Change Plans C-12
 Change Requests C-14
 Schema Synchronization C-16
 Database Lifecycle Management Pack Data Comparisons C-18
 DBMS_COMPARISON C-19
 Flow C-21
 Guidelines C-22
 Creating a Data Comparison C-24
 Comparison Job and Results C-25
 Results: Reference Only Rows C-26
 Results: Candidate-Only Rows C-27
 Results: Non-Identical Rows C-28
 Quiz C-29
 Summary C-31
 Practice C-32

D New and Enhanced Features in Other Courses

Further Information D-2
 Suggested Oracle University ILT Courses D-3

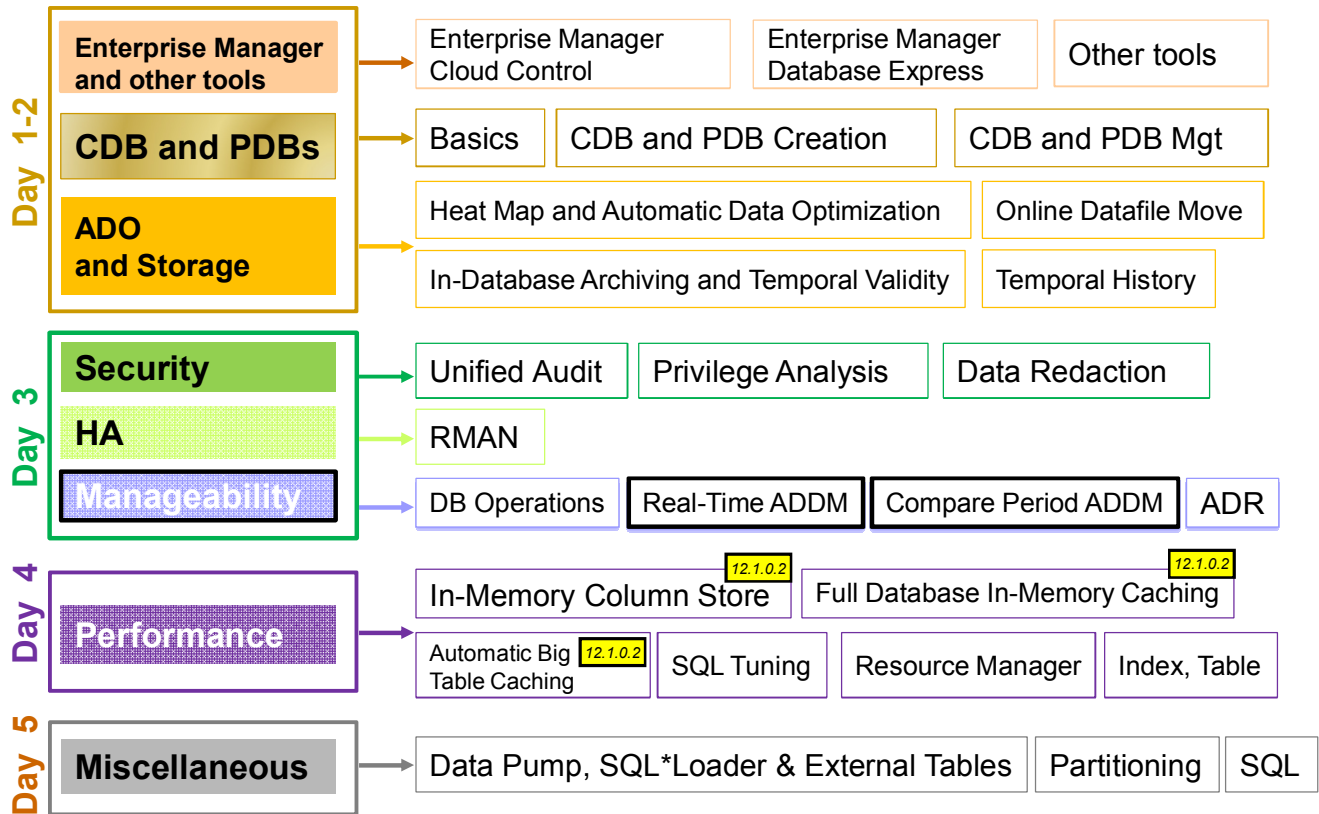
15

Emergency Monitoring, Real-Time ADDM, Compare Period ADDM, and ASH Analytics

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are new features in Automatic Database Diagnostic Monitor (ADDM): **Emergency Monitoring, Real-Time ADDM, Compare Period ADDM, and Active Session History (ASH) Analytics.**

Objectives

After completing this lesson, you should be able to:

- Explain Emergency Monitoring
- Describe Real-Time ADDM
- Use Real-Time ADDM
- Describe Compare Period ADDM
- Use Compare Period ADDM
- Generate a Compare Period ADDM report
- Identify the enhanced functionality used to view ASH data

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For a complete understanding of the Real-Time ADDM, Emergency Monitoring, Compare Period ADDM, and ASH Analytics features and usage, refer to the following sources of information:

- *“Oracle Enterprise Manager Cloud Control 12c: Database Management”* self-paced online course
- *Oracle Enterprise Manager Cloud Control 12c Demo Series* demonstrations:
 - *Use Real-Time ADDM*
 - *Compare Period ADDM*
 - *Use Active Session History (ASH) Analytics*

Emergency Monitoring: Challenges

- Sick systems
- Slow database
 - All users' queries are very slow.
 - Performance screens show slow data refresh rates.
 - There is a significant reduction in throughput.
- Database is hung due to internal contention for resources
 - Database is totally unresponsive; no logon is allowed.
 - Users' queries are hanging.
 - Performance screens do not refresh.



Solution: Shut the database instance down?

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The DBA detects that the system is sick based on different symptoms:

- Users complain that their queries do not respond.
- The refresh rate of the EM Performance Page is slower and slower.
- The throughput is abnormally degrading.

You expect to be able to perform a regular performance analysis with ADDM, but there might be serious limitations:

- On slowly performing systems, the snapshots may not be available for that time period.
- Depending on the severity of the performance issue, it may not be advisable or even possible to take AWR snapshots.
- You may just not be able to connect to the database because it is hanging.

Is a database instance shutdown the only solution? There might be another solution that is less drastic than bouncing the database instance.

To perform this analysis, you need to be able to connect and have a quick, lightweight analysis in order to check who is blocking the database and why it is hanging. This analysis should not require I/O nor global resources such as enqueuees or latches.

Emergency Monitoring: Goals



Running Emergency Monitoring should be the final resort before bouncing the database.

1. Switching to Emergency Monitoring allows you to:
 - Connect to the instance in diagnostic mode
 - View the Emergency Performance page with data collected refreshed in real time
 - View ASH data and Hang Analysis table of top blocking and blocked sessions and deadlocks, refreshed in real time
2. Viewing Hang Analysis data helps you in:
 - Killing the root blocker responsible for hangs or deadlocks
 - Shutting down and starting up the instance

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Before shutting down the instance, you can launch the Emergency Monitoring. It allows you to have a quick look and perform a performance analysis even when you cannot log on to the instance with a normal connection.

Emergency Monitoring allows DBAs with `SYSDBA` credentials to connect in diagnostic mode and have a quick, lightweight analysis to check who is blocking the database, why it is hanging. This type of connection does not require I/O nor global resources.

In Enterprise Manager 11g, “Memory Access Mode” is enabled and disabled explicitly with click of a button. This is done to start/stop the collector process, which reads performance data from SGA.

In the new approach there is no collector process. You do not have to switch between modes. When you navigate to Emergency Monitoring, the agent connects directly to the SGA and starts collecting SGA data bypassing SQL retrieval layer to get performance statistics. It will stop collecting after you return back to the regular performance monitoring.

It shows data refreshed in real time, and ASH data and Hang Analysis table of top blocking and blocked sessions, deadlocks, refreshed in real time.

It uses historical data obtained from ASH buffers to populate the performance pages.

ASH buffers generally contain history of wait data over the past 60 minutes, but this is not guaranteed. ASH buffers may not have enough history in the following cases:

- If there is too much active session activity, ASH buffer may get filled up in less than one hour and flushed to the disk.
- If there is high resource contention, the process responsible for writing data into ASH buffers may get stuck and therefore result in loss of data.

It will, on some occasions, reduce the amount of history provided to the user from the usual 30 minutes to something lesser.

When Emergency Monitoring shows the session that blocks other users, you can kill that session. Otherwise, you can either shut down the database instance or attempt a deeper analysis.

Real-Time ADDM: Challenges

- Sick systems
- Slow database
 - All users' queries are still very slow.
 - Performance screens still show slow data refresh rates.
 - There is still a significant reduction in throughput.
- Database is hung due to other contention for resources
 - Database might still be unresponsive; logon is allowed or is not allowed.
 - Users' queries are still waiting.
 - Performance screens do not refresh faster.
 - *You did not find any blocking session to kill.*
 - *Emergency Monitoring does not provide the root cause.*

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered within a solid red rectangular bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You did not find any blocking session and the database is still slow. Nevertheless you do not want to bounce the database instance before trying another analysis.

Real-Time ADDM offers you the possibility to perform a complementary analysis, a root-cause analysis that Emergency Monitoring does not provide.

Real-Time ADDM: Goals



1. Switch to Real-Time ADDM before bouncing the instance.
 - Starts collecting performance data from all database instances
 - Analyzes recent data for systems paralyzed because of severe contention on local or global resources
 - Provides holistic analysis for systems experiencing unusually high (though not severe) database activity
 - Detects findings for the recent activity (past 10 minutes)
 - Offers actionable recommendations
2. Use the recommendations to solve.
3. Return back to regular Performance Monitoring.

Note: Can be invoked for a RAC environment

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Real-Time ADDM works in a very similar fashion as ADDM to analyze performance.

Regular ADDM runs using AWR snapshots and provides findings and recommendations to let you know what needs to be changed to improve the performance.

Real-Time ADDM does not access the AWR snapshots but ASH recent activity from SGA data.

You connect in either modes with `SYSDBA` privilege, depending on the state of the instance:

- Diagnostic mode: If you cannot get a connection
- Normal mode: If you can still connect

Then the analysis starts, collecting recent performance data from all database instances from SGA, analyzing data for systems that are paralyzed because of severe contention on local or global resources.

It provides holistic analysis for systems experiencing unusually high (though not severe) database activity. After the analysis is complete, you can ask for the findings.

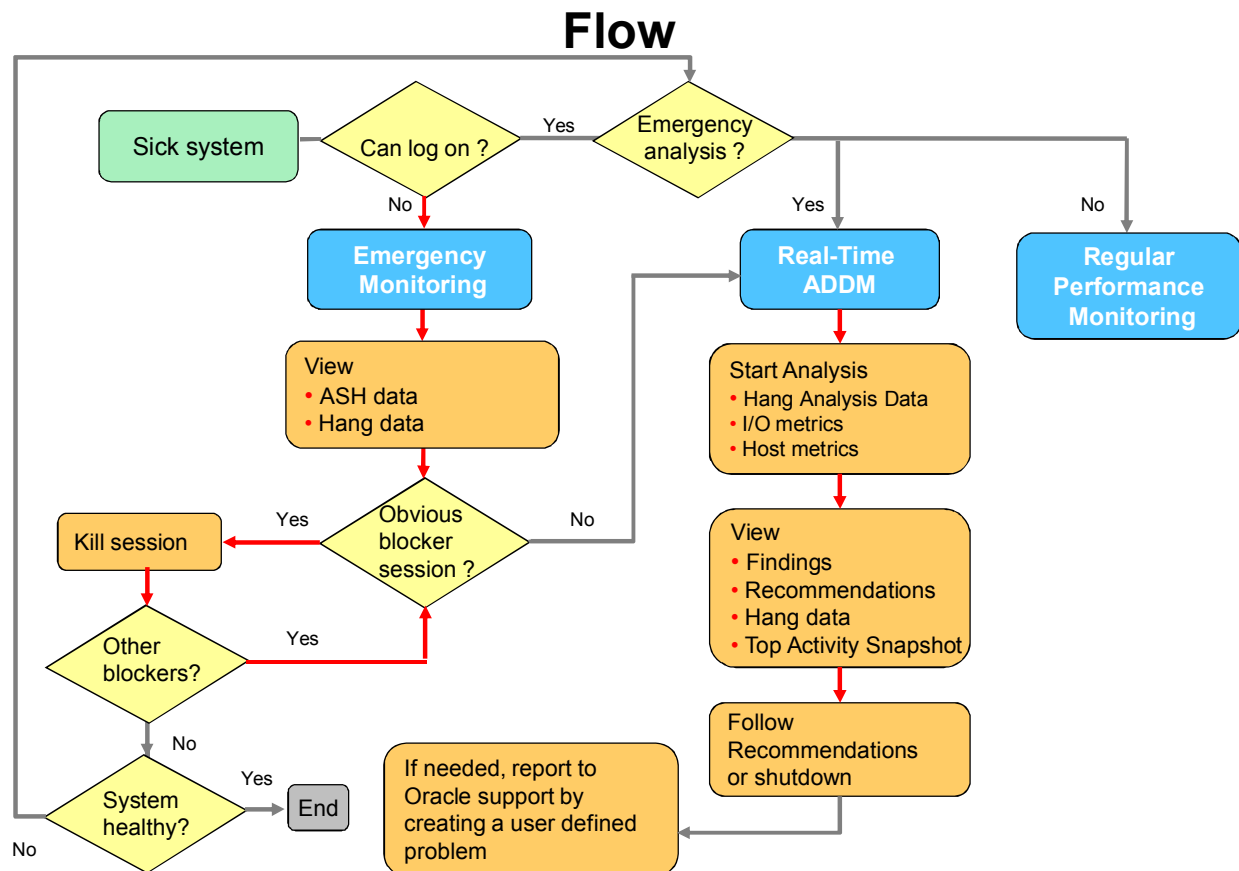
Real-Time ADDM finds that audit tracing cannot be performed due to lack of disk space and prevents connections, that a session needs to be killed because a logon trigger attempts to lock a table already locked, hence preventing any further connections. Real-Time ADDM identifies long-running queries that consume all resources.

When a session blocks other users, the Hang Data tab presents the Hang Analysis page with the Final Blockers and the Blocked Sessions. You will see the details of the blocker session and thus be able to get the process ID to be killed.

The Top Activity Snapshot presents the top activity of the instance for the past 10 minutes. Because Real-Time ADDM focuses on the last recent period, it is useful only for clarifying current performance issues.

Always review the recommendations to get help on different solutions. Then return back to regular performance monitoring.

Real-Time ADDM can be invoked for a RAC environment in the same way as single instances.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can combine both Emergency Monitoring and Real-Time ADDM.

When you have to find a quick solution while your system is getting very slow or is hanging, use Emergency Monitoring to get a quick action to perform. Either find a blocking session to kill, or shut down the database instance and start it up.

If you do not want to shut down the database instance right away, perform Real-Time ADDM deeper analysis. It will collect data from SGA, perform an analysis to provide you with a report of the root causes of the situation with recommendations and possible actions. Follow the recommendations if there are any. If not, shut down your database instance.

In either cases, you can create a user-defined problem and use Support Workbench to report your issue to My Oracle Support.

Using the DBMS_ADDM Package

```
SQL> SELECT dbms_addm.real_time_addm_report()  
FROM dual;
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can use new functions of the DBMS_ADDM package. The REAL_TIME_ADDM_REPORT returns a clob containing a Real-Time ADDM report for the past five minutes.

Quiz

The difference between Real-Time ADDM and regular ADDM resides in the source used for analysis. Which statements are true?

- a. Real-Time ADDM uses the AWR snapshots of the last 10 minutes.
- b. Real-Time ADDM uses the ASH recent activity from SGA data.
- c. Regular ADDM uses the AWR snapshots that are not yet purged.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

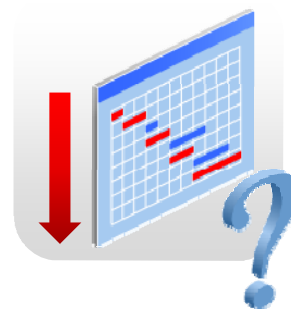
Answer: b, c

Real-Time ADDM works in a very similar fashion as regular ADDM to analyze performance; Real-Time ADDM does not access the AWR snapshots but accesses ASH recent activity from SGA data.

AWR Compare Periods Report

Production performance inconsistency:

- Today, it is very bad.
 - Yesterday, it was excellent.
-
- What has changed? (unknown reason)
 - Why is there performance degradation?



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

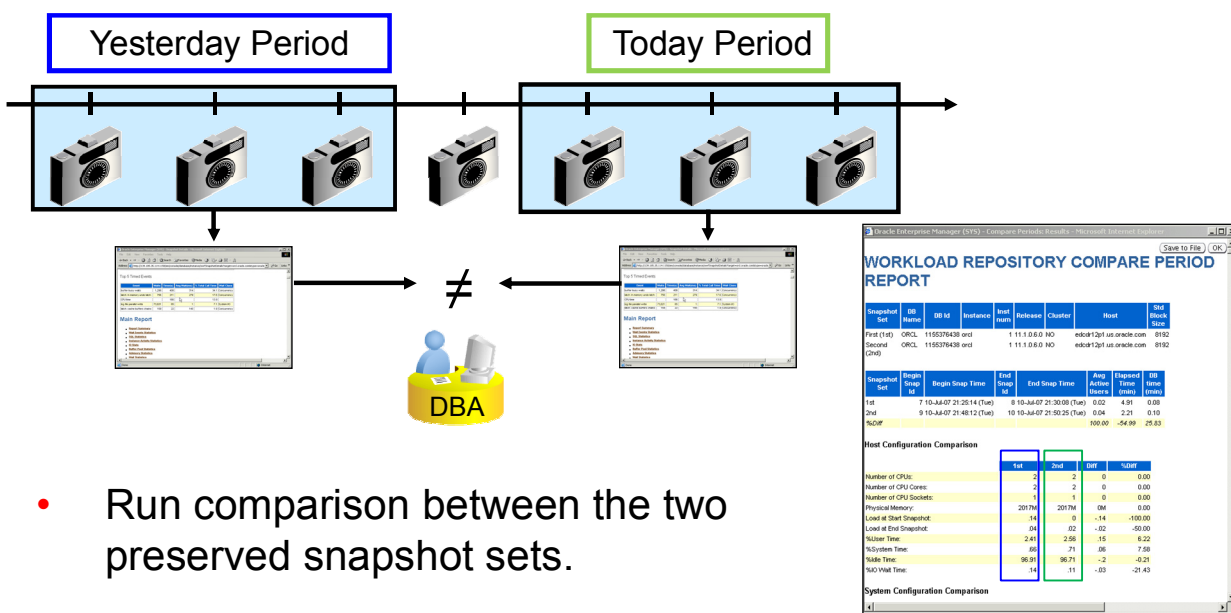
The challenge for DBAs about performance, when there is a degradation or improvement, is to identify what has changed and why, and which changes may have impacted the performance between two periods of time.

It can be normal if a different workload was executed, or if the storage devices changed, or if an ETL was executing while end users were executing their usual work, or simply the weekly ETL execution ran for three more hours than last week.

DBAs need to compare comparable periods when analyzing performance changes.

Method: Preserved Snapshot Sets

- Create a “yesterday” preserved snapshot set and a “today” preserved snapshot set.



- Run comparison between the two preserved snapshot sets.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The previous method provides the possibility to create preserved snapshot sets for the two periods and run a comparison report. You created a “yesterday” preserved snapshot set for the base period with your own choice of snapshots and a “today” preserved snapshot set for the compare period again with your own choice of snapshots. When you created preserved snapshot sets, you were sure that the snapshots would not be purged from AWR after the retention period. The action of “Compare Periods” compared the respective AWR data sets of two periods. The report displayed an HTML report comparing the two periods, showing differences in areas such as wait events, OS statistics, services, SQL statistics, instance activity, I/O statistics, segment statistics, and so on.

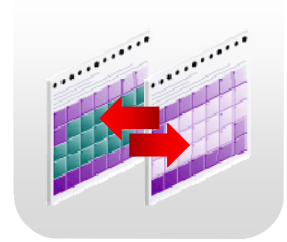
The DBA in a situation where the performance had degraded or improved, and knew the changes that had been completed could perform tests to detect:

- Database upgrade
- Schema changes
- Parameter changes, use of new optimizer enhancements
- System or I/O statistic collection, use of a new type of storage such as ASM instead of file systems
- Memory increase or decrease to verify the impact
- CPU added or new nodes in the RAC environment

What Is Missing?

Only AWR statistics reported in both cases:

- Comparison between two time periods
- Comparison between DB Replay capture and replay or two replays



Missing intelligent reporting with analysis:

- Changes that happened
- Mapping root causes to performance degradation effects



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The AWR Compare Periods report is interesting but you still have to perform an analysis on these metrics to find the effects that were mapped to the root causes of the performance degradation or improvement.

Compare Period ADDM: Analysis

Causes Identify system changes	Effects Identify performance difference	Map effects to causes with a rule set
• Configuration changes – DB version • Workload changes – Change in SQLs – Database time consumed by these SQLs	• Computes problem impacts with ADDM methodology – In base period – In compare period • Measures the difference	• Rules triggered by performance changes – SGA_TARGET decrease can cause I/O increase

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Compare Period ADDM, as opposed to the former method, performs a cause-to-effect analysis.

1. It first identifies the system changes that may have caused the performance change. For example, it detects a configuration change in DB version, or a workload change with SQL changes. These causes can cause performance difference.
2. Then it identifies the effects of these particular changes. For that purpose, it runs an ADDM analysis for the base period and one for the compare period and then measures the differences between both periods.
3. Finally, it maps the effects to the causes with rule sets. For example, an SGA_TARGET decrease can cause an I/O increase.

Compare Period ADDM provides more than what you had before with former methods, the identification of changes, and an intelligent cause-to-effect analysis.

Is the change due to changes in the workload? Did some new application start running? Were there some changes at the OS level? Was an instance parameter changed?

The report displays changes in the resource demand at the hardware and software levels.

These and other questions are more easily answered by using this advisor than with the previous AWR Compare Periods reports.

Workload Commonality

Comparison “current” Period

to

Base “earlier” Period

- The test time period → • Baseline
- The “current” production period → • DB Replay capture period
- The 1st RAT replay → • 1st RAT replay before modifications
- The 2nd RAT replay →

Workload Commonality

- Run same application?
- Gauge the workloads’ similarity, taking into account SQL statements and load.
- Is ideally 100% similar between DB Replay capture and replay

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To generate a Compare Period ADDM report, you have to perform the following steps:

1. Define the Base Period as being the normal one.
2. Define the Reference Period against which you compare the problematic period. The problematic period can cover situations like a period in a production database, a period when tests are performed in a test database, a period when using new optimizer enhancements, or another type of storage, or adding more memory, or more CPU or new nodes in a RAC environment.

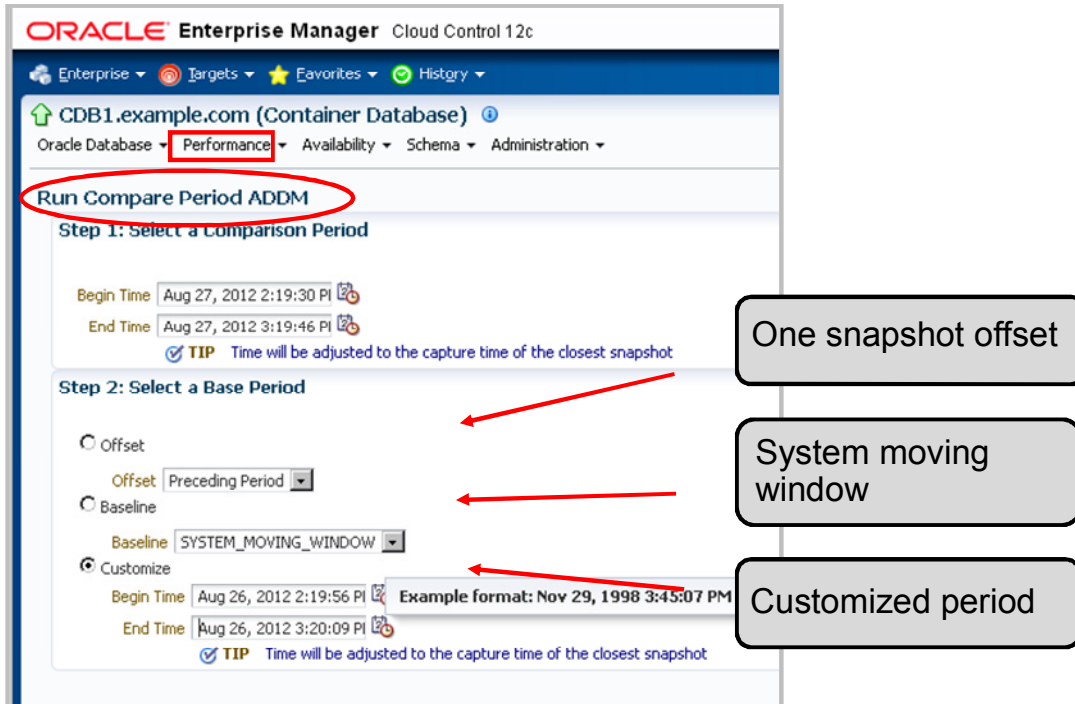
In some cases, you use DB Replay, capturing and replaying where the Base Period would be the capture period, and the Compare Period the replay one.

A new factor comes into play during the comparison, the workload commonality between two periods. Do you compare similar things? Are the SQL statements similar in both periods? Is it the same application running in both periods? This is called “SQL Commonality.”

This is a good way to gauge the workloads’ similarity, taking into account SQL statements and their respective load. In a live system, it is not 100% certain that DBAs compare exact or even similar application periods. If the result of the report shows an 80% or 90% commonality level, the DBA can rely on the findings provided by Compare Period ADDM.

Testing with Real Application Testing, you ideally get a 100% commonality between a DB Replay capture and a replay or between two replays because the workload is packaged by DB Replay.

Comparison Modes

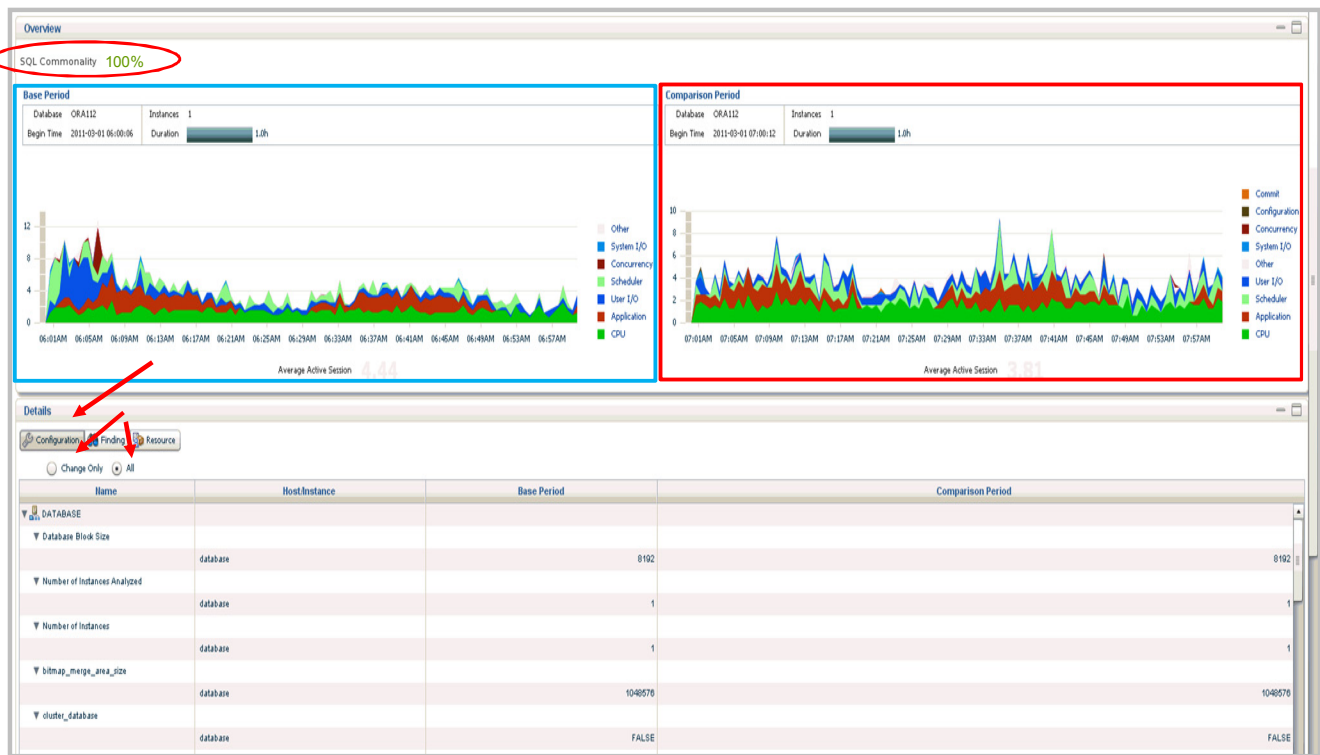


ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

1. First, select the Comparison Period that you want to inspect, that is, where you noticed a performance degradation.
2. Compare it to the Base Period where the performance was acceptable and that reflects a similar workload.
So you have three options to define the Base Period.
 - The first option allows you to select an offset of one snapshot. In this example, the comparison period starts at 2 and ends at 3. Therefore, the base period will start at 1 and end at 2. In this same option, you can select an offset of one day, so it will use the previous day, same time as the comparison period. You can even choose to compare the performance in a particular day with the performance in the previous week at the same time.
 - The second option allows you to select a baseline, either the out-of-box system moving window baseline or any other baseline you created to reflect your normal application.
 - The third option allows you to choose any other period of time.
3. In any case, it will automatically adjust the base period to a range of AWR snapshots that are as close as possible to the selected period of time.
4. Run the report. It should come back with the details of the differences between both periods.

Report: Configuration



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The reporting shows various types of information, and not only a list of statistics as with AWR Compare Periods report. It displays graphics that are easier and faster to interpret.

The overview at the top of screen shows the two periods.

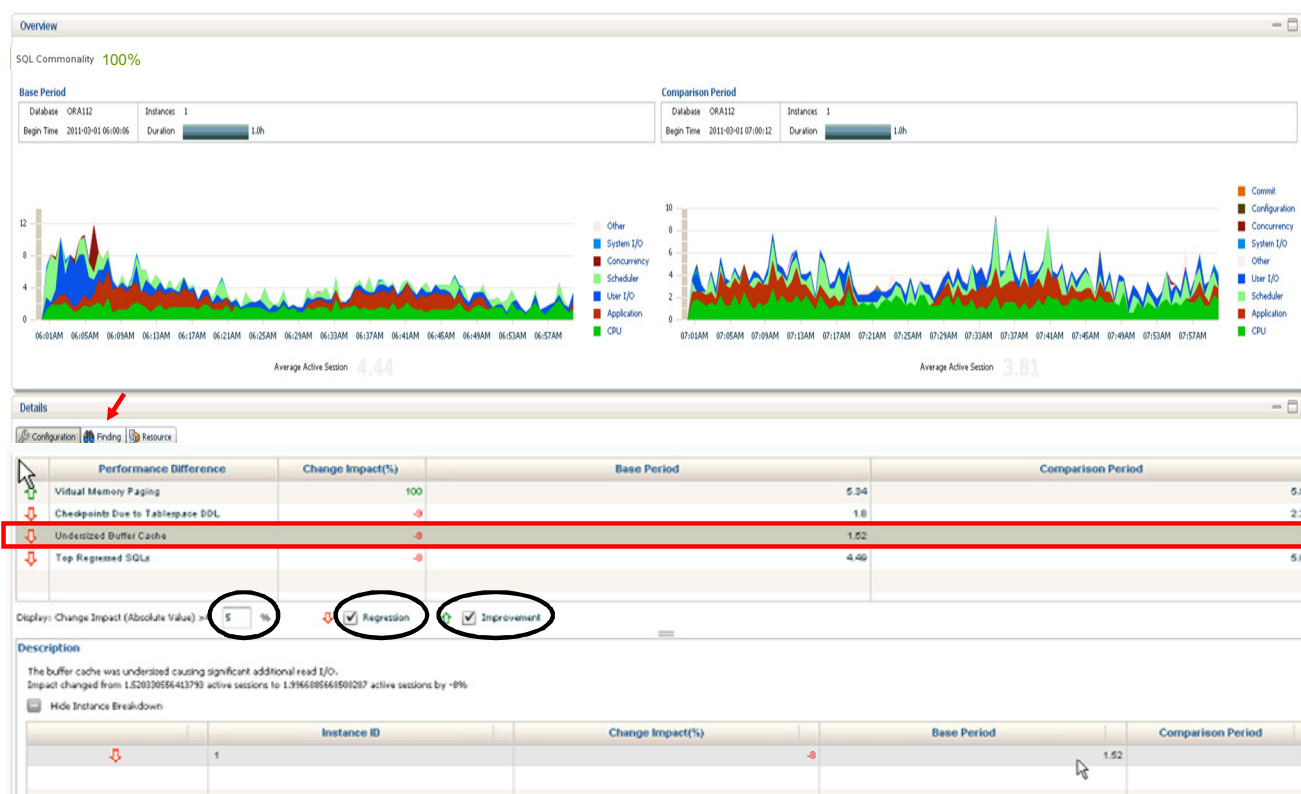
Between the two periods being compared, the report shows that they are 100% identical in terms of SQL commonality. If the report shows a commonality under 80%, you should not rely on this comparison because you compared two periods that are not comparable. So if you want a good, reliable comparison, check that the SQL commonality is at least 80% or 90%. Even if the commonality is less than 80%, it is still worthwhile to take a look at the findings. It will give an idea of what new SQLs were run in the system and how it affected the performance.

Now with this intelligent reporting, you get the changes that occurred between the two periods. In the details below the overview, you can choose to view only the configuration changes that occurred between the two periods, and not necessarily all the configuration information that is identical between the two periods.

If there was an instance parameter change, it will appear with the base period value and the comparison period value.

This information informs you about the existing changes but it does not explain why the performance degraded (or improved) and if this change had an impact on the performance degradation (or improvement). It is possible that this change did not impact the performance at all. It might even have helped in performance. So some other change might be the responsible for degradation of performance.

Report: Finding



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You need to get the findings that explain what caused the performance degradation or improvement.

Here again, you can choose to view only the areas where it degraded or improved, or all of them, and also set the percentage of impact for which you need to know the findings.

In the example in the slide, one of the reasons the performance degraded is that the buffer cache size was undersized from 2 GB to 1.52 GB, which was not sufficient. For each of the performance differences analyzed, you can get a detailed description of each finding and the SQL statements that were impacted positively or negative.

The Resource tab displays the CPU consumption, Memory and I/O usage. Regarding the CPU consumption, it can display CPU consumed by Oracle and Oracle Run Queue consumed. The I/O page shows the relative I/Os in both periods and tells you that neither period was I/O Bound. The Memory page may show swapping in both periods.

Using the DBMS_ADDM Package

Compare two periods within the same instance:

Comparison “current” Period
Snap_ID **123** to **124**

to

Base “earlier” Period
Snap_ID **121** to **122**

```
SQL> SELECT dbms_addm.compare_instances (
        base_dbid           => 1319927350,
        base_instance_id    => 1,
        base_begin_snap_id  => 121,
        base_end_snap_id    => 122,
        comp_dbid           => 1319927350,
        comp_instance_id    => 1,
        comp_begin_snap_id  => 123,
        comp_end_snap_id    => 124,
        report_type         => 'XML')
FROM dual;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can use new functions of the DBMS_ADDM package. The DBMS_ADDM.COMPARE_INSTANCES returns a clob containing a compare period ADDM report that compares the performance of a single instance over two different time periods.

The parameters used are the following:

- BASE_DBID: DB ID of the base period
- BASE_INSTANCE_ID: Instance ID of the base period
- BASE_BEGIN_SNAP_ID: Snapshot ID of the beginning of the base period
- BASE_END_SNAP_ID: Snapshot ID of the end of the base period
- COMP_DBID: DB ID of the comparison period
- COMP_INSTANCE_ID: Instance ID of the comparison period
- COMP_BEGIN_SNAP_ID: Snapshot ID of the beginning of the comparison period
- COMP_END_SNAP_ID: Snapshot ID of the end of the comparison period
- REPORT_TYPE: Output type for the report—XML or HTML (Default is HTML)

In the slide, you see an ADDM comparison between two periods of time, each delimited in time by a begin and end snapshot.

Using the DBMS_ADDM Package

Functions to compare periods:

- COMPARE_INSTANCES
- COMPARE_DATABASES
- COMPARE_CAPTURE_REPLAY_REPORT
- COMPARE_REPLAY_REPLAY_REPORT

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Additional PL/SQL functions:

- COMPARE_INSTANCES: Generates a Compare Period ADDM report for an instance-level performance comparison.
- COMPARE_DATABASES: Generates a Compare Period ADDM report for a database-wide performance comparison.
- COMPARE_CAPTURE_REPLAY_REPORT: Generates a Compare Period ADDM report to compare performance for a Workload Capture to a Workload Replay.
- COMPARE_REPLAY_REPLAY_REPORT: Generates a Compare Period ADDM report to compare performance for one Workload Replay to another Workload Replay.

All the preceding functions return a clob.

Note: For a complete description of the available procedures, see the *Oracle Database 12c PL/SQL References and Types* documentation.

Quiz

The difference between a Compare Period ADDM report and an AWR Compare Periods report resides in the output format.

- a. True
- b. False

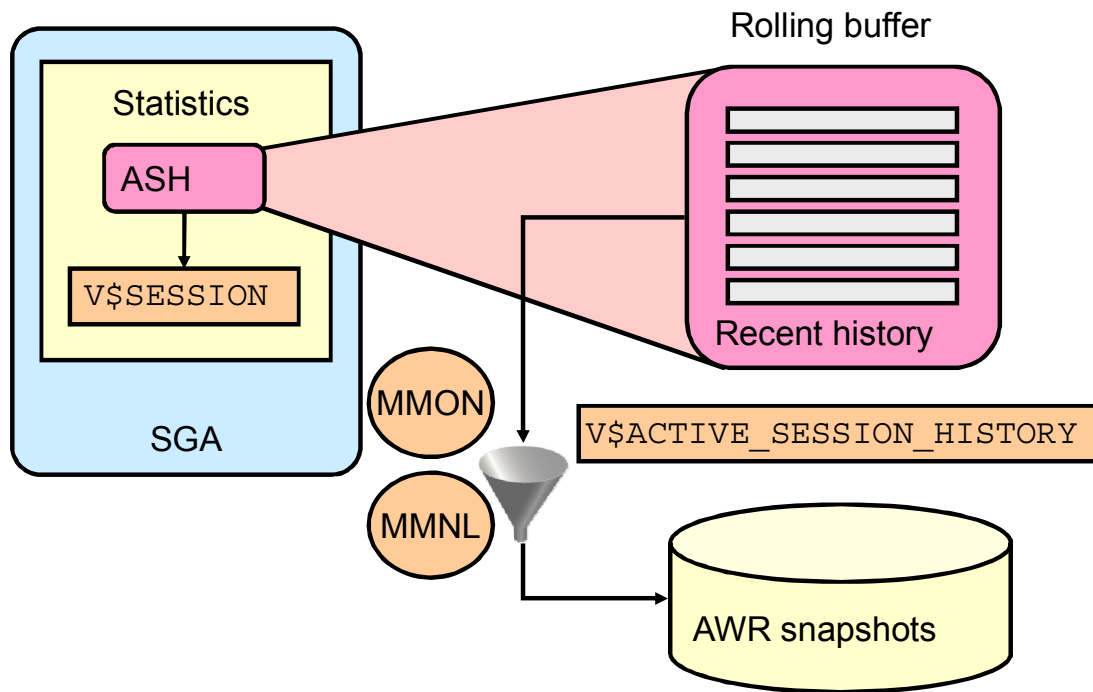
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

The Compare Period ADDM, compared to the AWR Compare Periods report, performs a cause-to-effect analysis.

ASH: Overview



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

One of the problems with AWR data is that, by its very nature, analysis of what is currently happening in a system requires detailed information about the activity in the last 5 to 10 minutes. Because the AWR takes snapshots of the system every 60 minutes, the last snapshot could be almost an hour old. Therefore, the AWR does not contain enough information to perform current analysis.

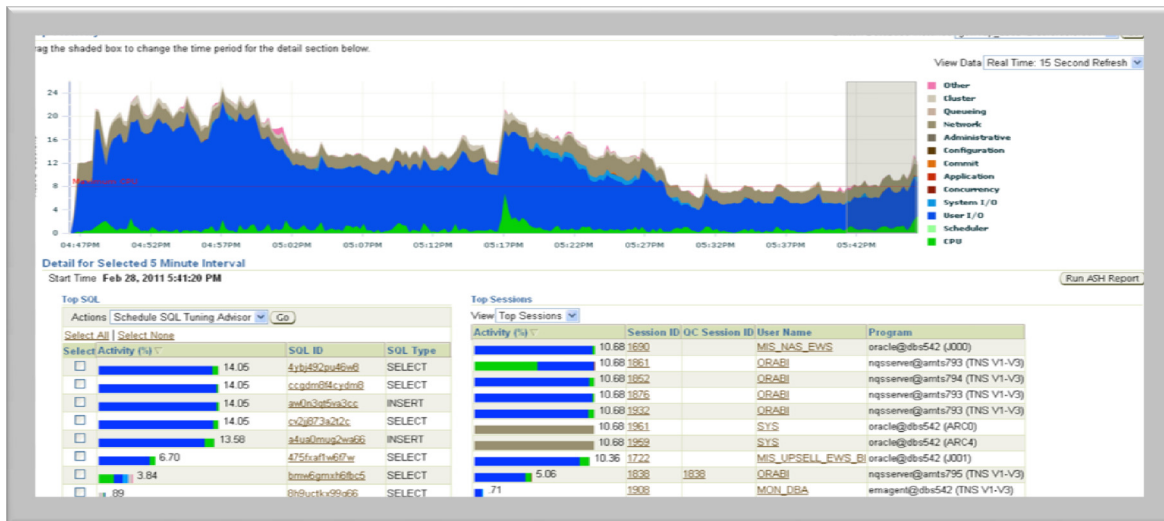
The Active Session History contains the history of recent session activity. Because recording session activity is expensive, ASH samples `V$SESSION` every second and records the events that the sessions are waiting for. Inactive sessions are not sampled. The sampling facility is very efficient because it directly accesses the internal database structures.

ASH is designed as a rolling buffer in memory; earlier information is overwritten when needed. The ASH statistics are available through the `V$ACTIVE_SESSION_HISTORY` view. This view contains one row for each active session per sample.

Flushing all ASH data to disk is unacceptable because of its volume. The approach is to filter the data while flushing it to disk. This is done automatically by MMON every 60 minutes and by Manageability Monitor Light (MMNL) whenever the buffer is full.

The ASH memory comes from the SGA and is fixed for the lifetime of the instance. It represents 2 MB of memory per CPU. However, the size of ASH cannot exceed 5% of the shared pool size.

Top Activity Page



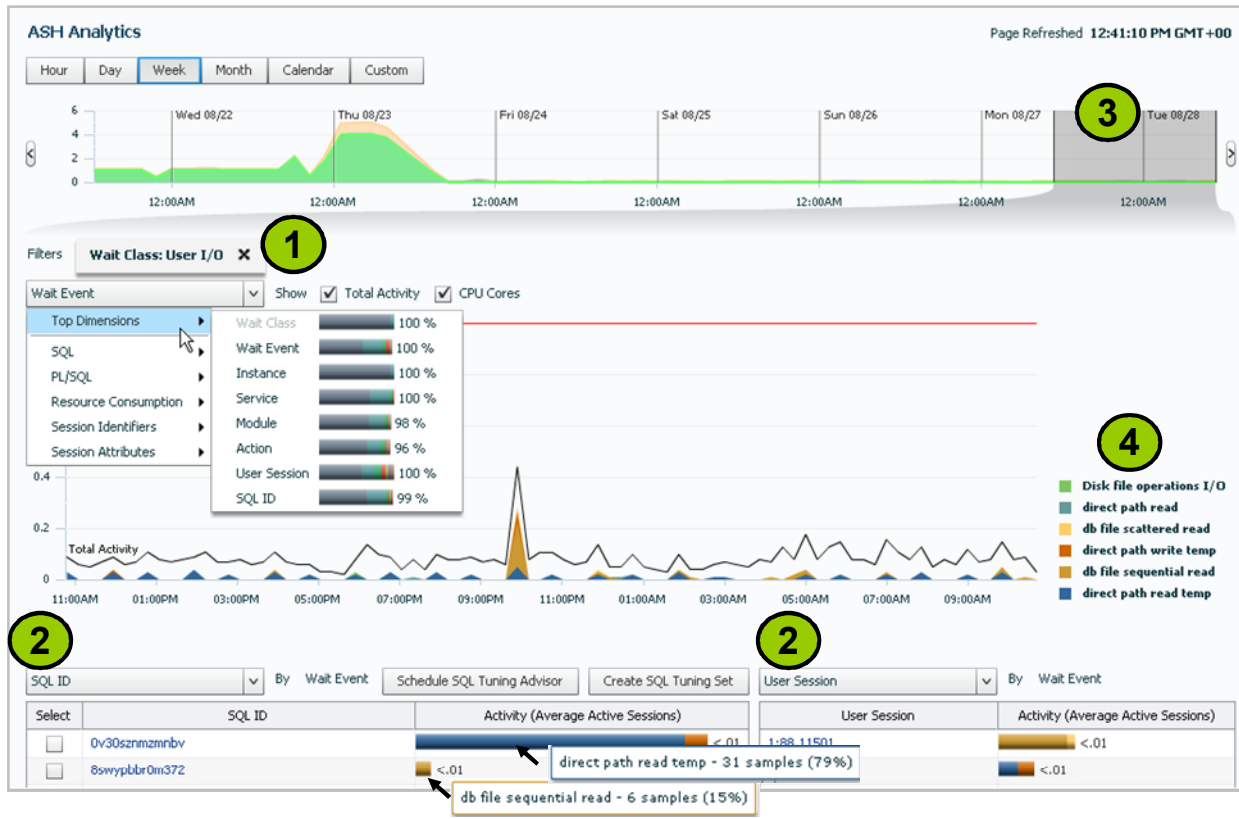
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Enterprise Manager 11g, the Top Activity page displays a lot of the ASH information. An example of this page is shown in the slide. There are a number of limitations in the way this information is displayed. These include:

1. You cannot switch dimensions on the area chart shown in the top part of the screen.
2. The left-hand table below that is fixed to displaying Top SQL, while the right-hand table has only a few dimensions that it can be displayed by, such as Top Sessions or Top Modules.
3. The information does not harness the full value of ASH data as some key dimensions that are actually captured with the ASH data are not displayed at all.
4. The slider used to select the time period for the detailed section is a fixed width (5 minutes real time, 30 minutes historical).
5. The visualization is restricted to a stacked area chart by wait classes so you always see the wait classes stacked over one another.
6. The data cannot be displayed as an active report, that is, it cannot be sent offline to other users for review.
7. There are limited drilldown capabilities that simply send you to other pages.
8. ASH reports are currently in text format and are not interactive.

ASH Analytics Page: Activity



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The new ASH Analytics page overcomes all these limitations to allow the user to perform the following tasks:

1. Filter dimensions on the Filters shown in the middle left part of the page.
2. Select the dimensions for the left- and right-hand tables in the bottom-left and right parts of the page.
3. Vary the slider width to select the time period for the detailed section.
4. Drill into a load map view to display the different waits indicating the importance of each by the size of the box.

Summary

In this lesson, you should have learned how to:

- Describe Real-Time ADDM
- Use Real-Time ADDM
- Explain Emergency Monitoring
- Describe Compare Period ADDM
- Use Compare Period ADDM
- Generate a Compare Periods ADDM reports
- Identify the enhanced functionality used to view ASH data

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 15: Overview

- 15-1: Using Emergency Monitoring
- 15-2: Cleaning Up
- 15-3: Using Compare Period ADDM (*Optional demo*)

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

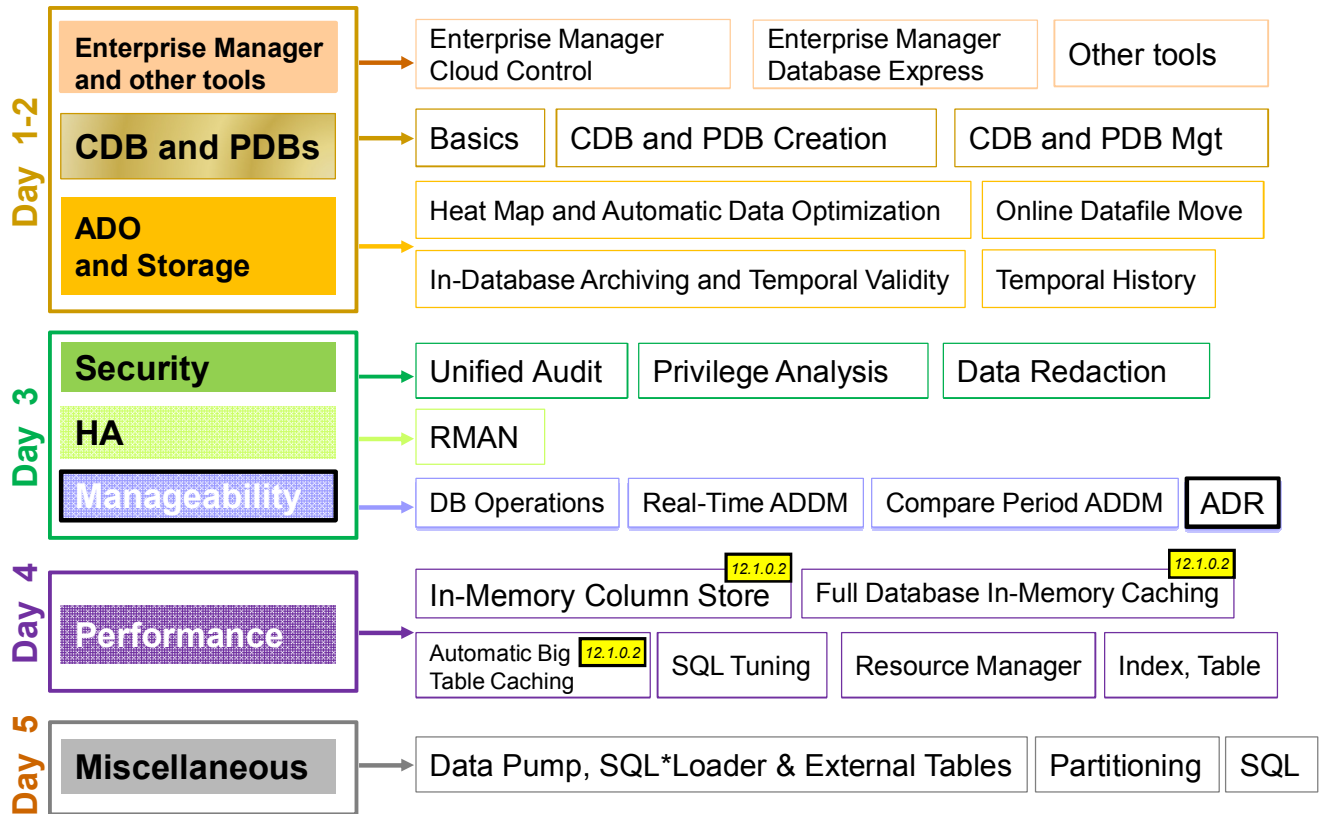
ADR and Network Enhancements

16

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are several **Automatic Diagnostic Repository (ADR)** and **Network** enhancements.

Objectives

After completing this lesson, you should be able to:

- Describe the new ADR DDL and debug log files
- List and view log files using ADRCI utility commands
- Describe benefits of network data compression
- Show change in `DEFAULT_SDU_SIZE`

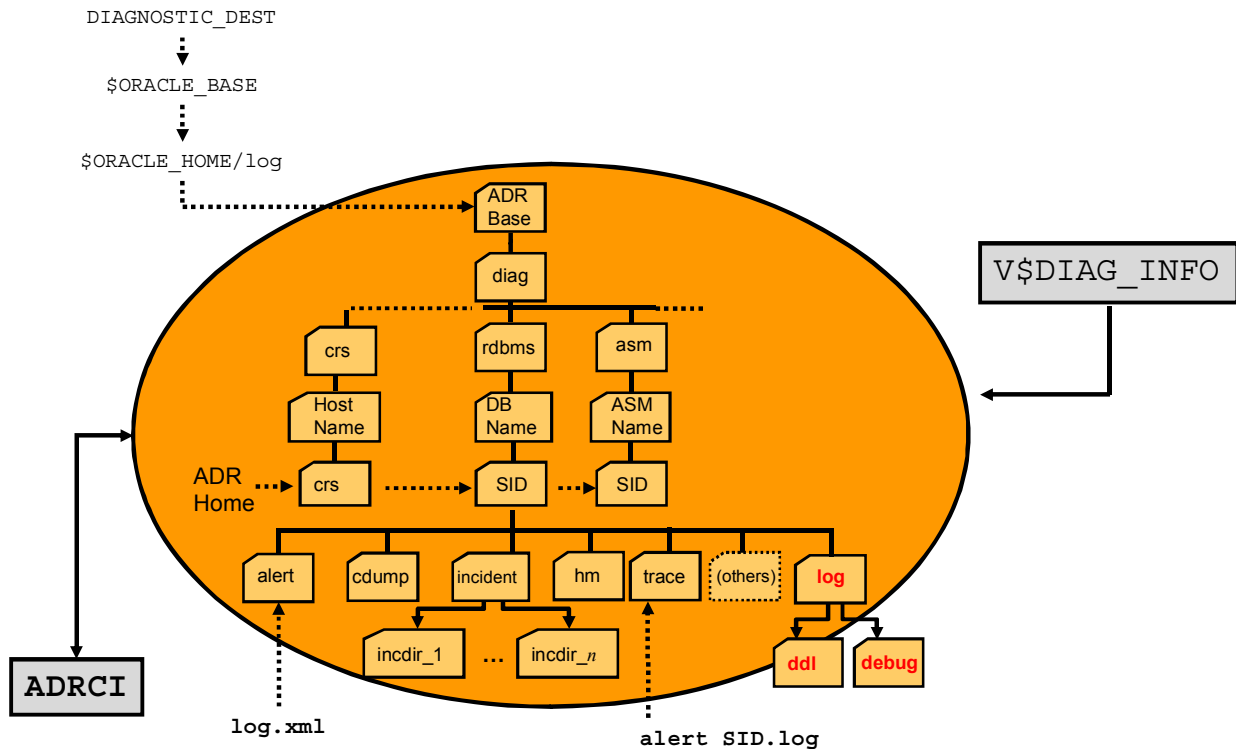
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For a complete understanding of the Automatic Diagnostic Repository enhancements and usage, refer to the following guide in the Oracle documentation:

- *Oracle Database Administration Guide 12c Release 1 (12.1) – “Managing Diagnostic Data” Chapter*
- *Oracle Enterprise Manager Licensing Information 12c Release 1 (12.1) – “Enterprise Database Management” Chapter – “Legacy: Lifecycle Management Pack for Oracle Database” Section*
- *Oracle Database Utilities 12c Release 1 (12.1) – “ADRCI: ADR Command Interpreter” Chapter*
- *Oracle Database Net Services Administrator's Guide 12c Release 1 (12.1) – “Optimizing Performance” Chapter*
- *Oracle Database Net Services Reference 12c Release 1 (12.1) – “Parameters for the sqlnet.ora File” Chapter*

Automatic Diagnostic Repository



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Automatic Diagnostic Repository ADR is a file-based repository for database diagnostic data such as traces, incident dumps and packages, the alert log, Health Monitor reports, core dumps, and more. It has a unified directory structure across multiple instances and multiple products stored outside of any database. It is, therefore, available for problem diagnosis when the database is down. Beginning with Oracle Database 11g Release 1, the database server, Automatic Storage Management (ASM), and other Oracle products or components store all diagnostic data in ADR. Each instance of each product stores diagnostic data underneath its own ADR home directory. For example, in a Real Application Clusters environment with shared storage and ASM, each database instance and each ASM instance has a home directory within ADR. ADR's unified directory structure uses consistent diagnostic data formats across products and instances. A unified set of tools enables customers and Oracle Support to correlate and analyze diagnostic data across multiple instances.

Two alert log files are generated. You can view the alert log in text format (with the XML tags stripped) with Enterprise Manager and with the ADR Command Interpreter (ADRCI) utility.

The graphic in the slide displays the directory structure of an ADR home.

A new `log` directory contains two subdirectories, `ddl` and `debug`.

ADR File Types

- Log: High-level information of activity
- Trace:
 - Background trace files
 - SQL trace files
- Dump:
 - Specific type of trace file
 - Detailed point-in-time information of an incident
- Core:
 - Memory dump
 - All-binary, port-specific format

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Log files are shared files that contain high-level information. The RDBMS alert log is a very good example of this kind of log.

Trace, dump, and core files contain diagnostic data that are used to investigate problems, all of them involving writing messages to files.

- **Trace** files contain fairly detailed information covering a long period of time, such as state transitions, or structures being worked on. Each server and background process can write to an associated trace file. Trace files are updated periodically over the life of the process and can contain information about the process environment, status, activities, and errors. In addition, when a process detects a critical error, it writes information about the error to its trace file. The SQL trace facility also creates trace files, which provide performance information on individual SQL statements.
- **Dump** files contain very detailed point-in-time information about some state or some structure. A dump is a specific type of trace file, a one-time output of diagnostic data in response to an event (such as an incident), whereas a trace tends to be continuous output of diagnostic data. When an incident occurs, the database writes one or more dumps to the incident directory created for the incident. Incident dumps also contain the incident number in the file name.
- **Core** files contain a memory dump, in an all-binary, port-specific format. Core file names include the string “core” and the operating system process ID. Core files are useful only to Oracle Support engineers. Core files are not found on all platforms.

ADR Files: Location

Diagnostic Data	ADR Location
Foreground process traces	<ADR_HOME>/trace
Background process traces	<ADR_HOME>/trace
Alert log data	<ADR_HOME>/alert&trace
Core dumps	<ADR_HOME>/cdump
Incident dumps	<ADR_HOME>/incident/incdir_n
→ CRS logs	<ADR_HOME>/<hostname>/crs
→ DDL logs	<ADR_HOME>/log/ddl
→ Debug logs	<ADR_HOME>/log/debug

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

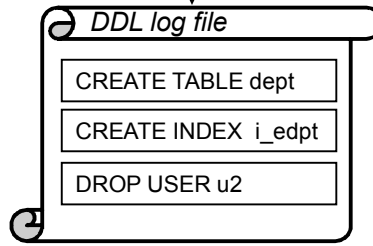
- **CRS logs:** Cluster Ready Services Daemon (CRSD) log files are found in <ADR_HOME>/<host_name>/crs.
- **DDL log:** A log.xml file that contains only DDL statements and details moved from the alert log file. It is created in the <ADR_HOME>/log/ddl directory.
- **Debug log:** A file that contains entries describing unusual events. It is created in the <ADR_HOME>/log/debug directory.

ADR Files: DDL and DEBUG Log Files

New specific ADR log files:

- DDL log

```
SQL> ALTER SYSTEM SET enable_ddl_logging=TRUE;
```



- No DDL log file
- No DDL recording in alert log

```
SQL> ALTER SYSTEM SET enable_ddl_logging=FALSE;
```

- Debug log

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

DDL log

Oracle Database 11g added some support for DDL logging of RDBMS DDL statements into the alert log. Setting the instance parameter `ENABLE_DDL_LOGGING` to `TRUE` activates DDL logging. Oracle Database 12c turns off DDL logging by default, by setting `ENABLE_DDL_LOGGING` to `FALSE`. If turned on, the RDBMS DDL logging is written to a new ADR file type that has the same format and basic behavior as the alert log, but it only contains DDL statements and dates. The `init.ora` parameter `ENABLE_DDL_LOGGING` is licensed as part of the Lifecycle Management Pack for Oracle Database when set to `TRUE`.

Debug Log

An Oracle Database component can detect conditions, states, or events that are unusual, but which do not inhibit correct operation of the detecting component. The component can issue a warning about these conditions, states, or events. These warnings are not serious enough to warrant an incident or a write to the alert log. But they do warrant a record in a log file because they might be needed to diagnose a future problem. Developers may find it useful to create a record of such events, but currently lack appropriate mechanism. The debug log is a file that records these warnings. The debug log has the same format and basic behavior as the alert log, but it contains information only about possible problems that might need to be corrected, reducing the amount of information in the alert log and trace files.

The debug log is included in IPS incident packages. The debug log's contents are intended for Oracle Support. Database administrators should not use the debug log directly.

New ADRCI Command

Show DDL log file content:

```
adrci> SHOW LOG;
```

- The <ADR_HOME>/log/ddl/log.xml file content is displayed with an editor:
 - vi for UNIX
 - Notepad for Windows

```
2012-08-23 15:01:00.200000 +00:00
create table t(a varchar(40), b number, c varchar(240), d varchar(240))
2012-08-23 15:03:49.121000 +00:00
create table scott.tabjfv(c number) tablespace users
2012-08-23 15:11:58.017000 +00:00
drop user test cascade
~
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To view the content of the DDL log file which is in XML format, you can use the ADRCI SHOW LOG command.

This command automatically opens the ddl log file by using the vi editor on UNIX or Notepad on Windows. Leave the vi editor by using :q command.

Network Performance: **Compression**

- Reduces data volume
- Reduces bandwidth use
- Is transparent to application
- Compression schemas

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Network performance is generally limited by two factors: the bandwidth and the data volume. In simple language, the bandwidth is the size of the pipe. The larger the pipe, more data bits can be pushed through it. The data volume is the number of bits that need to be transmitted. So to improve the network performance, either the bandwidth must be increased or the data volume decreased. In many cases, increasing the bandwidth is not cost effective, or impossible. The only option in such cases is to reduce the data volume.

Compression of the data in the network layer reduces the number of bits transmitted. This reduces the bandwidth used, allowing more data to be transmitted. By performing the compression in the network layer, the compression is transparent to the application.

Several compression schemes are possible. In the advanced compression option (ACO), GZIP and LZO compression schemes can be used to compress data in the network layer as they have the ability to handle data unit (SDU size) separately.

Compression exchanges computation for data volume. So compression will help in situations where the network bandwidth is the bottleneck. If the operation is already CPU bound, network compression will only make the problem worse.

Setting Up Compression

Set the following parameters for compression operations in the `sqlnet.ora` file:

- `SQLNET.COMPRESSION`
- `SQLNET.COMPRESSION_LEVELS`
- `SQLNET.COMPRESSION_THRESHOLD`

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Three new parameters in the `sqlnet.ora` file allow the DBA to set the parameters for compression operations. Network compression is available as part of Advanced Compression Option (whereas network encryption is no longer a cost option).

`SQLNET.COMPRESSION`: Can be set to `ON` or `OFF` to enable or disable compression. The default is `OFF`.

`SQLNET.COMPRESSION_LEVELS`: The compression levels are used at the time of negotiation to verify which levels are used at both ends, and to select one level. For Database Resident Connection Pooling (DRCP), only the compression level `LOW` is supported. The compression level may be set to `HIGH` to use high CPU usage and high compression ratio, or `LOW` to use low CPU usage and low compression ratio. The default is `LOW`.

`SQLNET.COMPRESSION_THRESHOLD`: This parameter specifies minimum data size required to perform compression. Compression will not be done if the size of data to be sent is less than this value. The default is 1024 (bytes).

Session Data Unit (SDU) Size

Modify SDU size when:

- The data coming from the server is fragmented
- You are on a wide area network (WAN) that has long delays
- The packet size is consistently the same
- Large amounts of data are returned

Do not modify SDU size when:

- The application can be tuned to reduce network use
- You have a high speed network where the effect of the data transmission is negligible
- Your requests return small amounts of data from the server

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The session data unit (SDU) size is the size of the data packet sent over the network. Setting the SDU size to a larger value can improve network performance. This value can range from 512 bytes to 2 MB bytes. The default SDU for the client and a dedicated server is 8192 bytes. The default SDU for a shared server is 65535 bytes. The actual SDU size used is negotiated between the client and the server at connect time and is the smaller of the two values.

After trace files statistics are collected with `trcasst -s` utility, consider changing the SDU size when the predominant message size is not equal to 8192. A larger message forces the network layer to split the message into multiple packets, called fragments. The SDU size should be 70 bytes larger than the predominant message size. If the predominant message size plus 70 bytes exceeds the maximum SDU, then the SDU should be set such that the message size is divided into the smallest number of equal parts where each part is 70 bytes less than the SDU size. To change the SDU size, change the `DEFAULT_SDU_SIZE` parameter in the `sqlnet.ora` file.

For example, if the majority of the messages sent and received by the application are smaller than 8 KB, taking into account the 70 bytes for overhead, then setting the SDU to 8 KB will probably produce good results. If sufficient memory is available, using the larger values for the SDU reduces the number of system calls and overhead for Oracle Net Services.

Before you change the SDU size, tune the application to use less network bandwidth. Do not change the SDU size if you are using a high speed network where the latency of network transmission is very small, or if only small amounts of data are transmitted across the network.

Setting SDU Size

DEFAULT_SDU_SIZE in the `sqlnet.ora` file:

- Default size is 8 KB.
- Maximum size for Oracle Database 12c is 2 MB compared to 64 KB in previous Oracle versions.
- Example: `DEFAULT_SDU_SIZE=4096`

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The `DEFAULT_SDU_SIZE` parameter in the `sqlnet.ora` file specifies the overall SDU size for a server or a client. When the configured values of client and database server do not match for a session, the lower of the two values is used, thus the setting should be the same on both. You can override the default setting of the SDU by specifying the SDU parameter in the connect descriptor for a client.

The optimal SDU size depends on the network characteristics, data generation rate, and message size. One case where a larger SDU could help is when the sender data generation rate is much lower than the network transmission rate.

The parameter `DEFAULT_SDU_SIZE` is used to set this value. In Oracle Database 11g, the default value is 8 KB.

Note: On a network that is near capacity, larger packets can produce in more collisions, thus reducing performance.

Quiz

DDL commands can be logged in the alert log and in a new DDL log.xml file.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

DDL commands may or may not be logged. If `ENABLE_DDL_LOGGING` is set to `TRUE`, DDL commands are logged in ADR repository in the `log/ddl` directory in the `log.xml` file. If `ENABLE_DDL_LOGGING` is set to `FALSE`, DDL commands are not logged at all.

Summary

In this lesson, you should have learned how to:

- Describe the new ADR DDL and debug log files
- List and view log files using `ADRCI` utility commands
- Describe the benefits of network data compression
- Show change in `DEFAULT_SDU_SIZE`

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 16: Overview

16-1: Showing ADR DDL log file and content

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Module

Performance

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

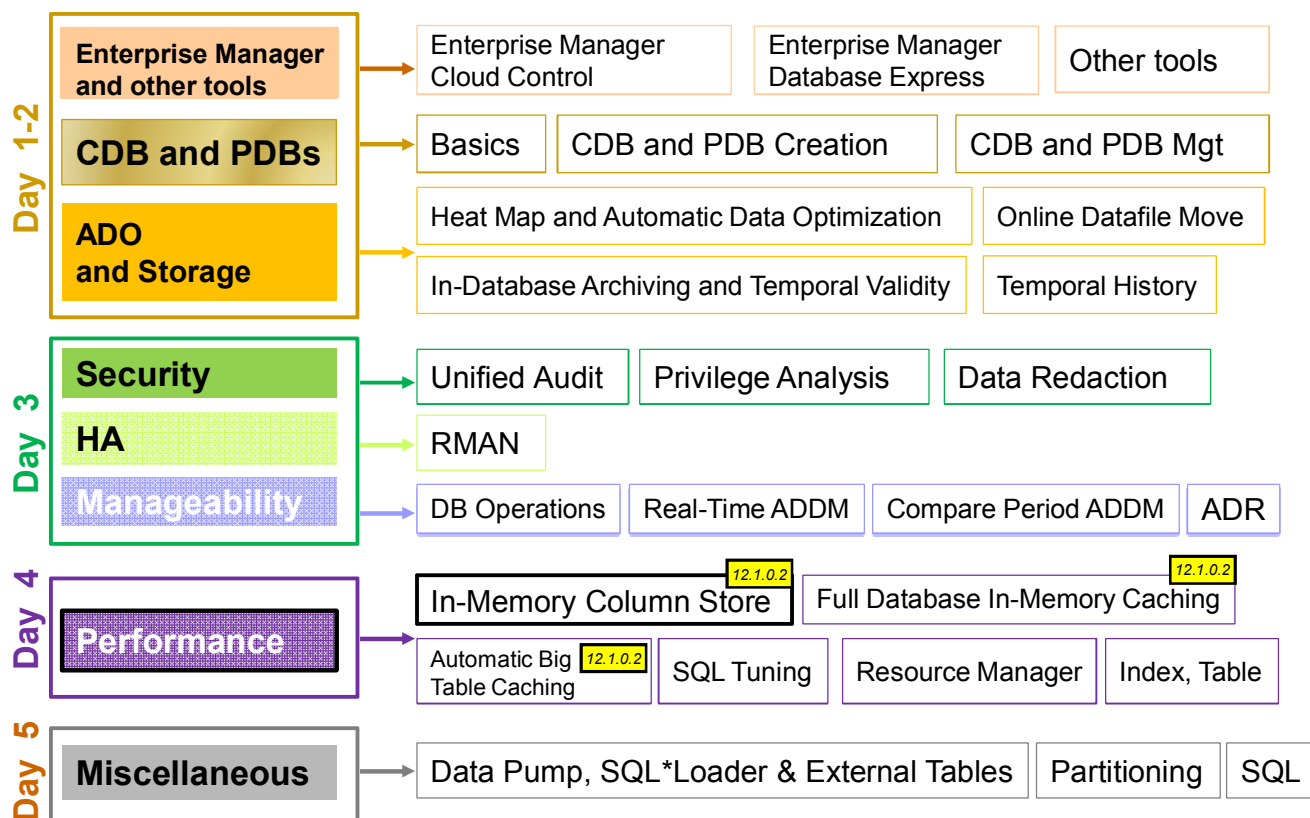
17

In-Memory Column Store

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The In-Memory Column Store feature brings the solution for accelerating database-driven business decision-making to real-time speeds. The In-Memory Column Store feature enables objects to be stored in memory in a new format known as the columnar format. This format enables scans, joins, and aggregates to perform much faster than the traditional on-disk formats, providing fast reporting performance for both OLTP and DW environments. This is particularly useful for analytic applications that operate on few columns returning many rows rather than for OLTP operating on few rows returning many columns. The DBA has to define which segments to be populated into the in-memory column store, such as hot tables, partitions, and more precisely, the frequently accessed columns. There are other new In-Memory features available with Oracle Database 12c R1 PS1. The In-Memory Column Store is included with the Oracle Database In-Memory option.

Objectives

After completing this lesson, you should be able to:

- Explain the goals, benefits, and architecture of the in-memory column store
- Deploy the in-memory column store in a database
- Show how queries and DMLs execution benefit from the in-memory column store
- Use new views and statistics to display in-memory column store usage
- Describe the interaction with other products and features

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Goals of In-Memory Column Store

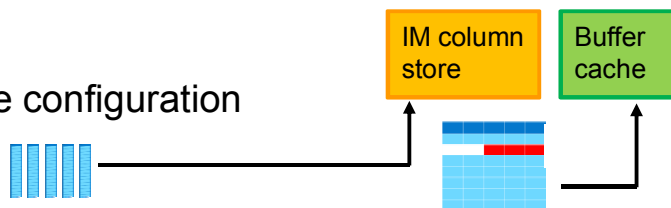
- **Instant query response:**
 - Faster queries on **very large** tables on **any** columns (**100x**)
 - Using scans, joins, and aggregates
 - Without indexes
 - Best suited for analytics: few columns/many rows
- **Faster DML:** Removal of most analytics indexes (**3 to 4x**)

- **Full application transparency**



- **Easy setup:**

- In-memory column store configuration
- Segment attributes



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The In-Memory Column Store enables objects (tables, partitions, and other types) to be stored in memory in a new format known as the columnar format. This format enables scans, joins, and aggregates to perform much faster than the traditional on-disk format, providing fast reporting and DML performance for both OLTP and DW environments. This is particularly useful for analytic applications that operate on few columns returning many rows rather than for OLTP operating on few rows returning many columns. The DBA has to define which segments to be populated into the in-memory column store (IM column store), such as hot tables, partitions and more precisely more frequently accessed columns.

The in-memory columnar format does not replace the on-disk or buffer cache format. It is a consistent copy of the table or of some columns of a table converted to the new columnar format that is independent of the disk format and only available in memory. Because of this independence, applications are able to transparently use this option without any changes. For the data to be converted into the new columnar format, a new pool is requested in the SGA. The pool is the IM column store.

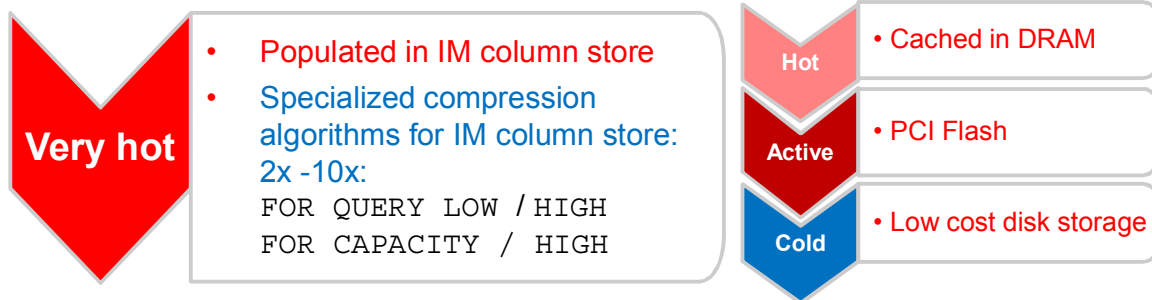
If sufficient space is allocated for the IM column store, a query accessing objects that are candidates to be populated into the IM column store performs much faster. The improved performance allows ad-hoc analytic queries to be executed directly on the real-time transaction data without impacting the existing workload.

There are three main advantages:

- Queries run a lot faster: All data can be populated in memory in a compressed columnar format. No index is required and used. Queries run at least 100 times faster than when fetching data from buffer cache thanks to the columnar compressed format.
- DMLs are faster: Analytics indexes can be eliminated being replaced by scans of the IM column store representation of the table.
- Arbitrary ad-hoc queries run with good performance, since the table behaves as if all columns are indexed.

Benefits

- Active data in In-Memory column store



- Fully supported on RAC
- Fully supported with multitenant architecture
- No SQL tuning anymore
- No restriction on SQL
- No change for backup and recovery operations
- No migration of data

ORACLE

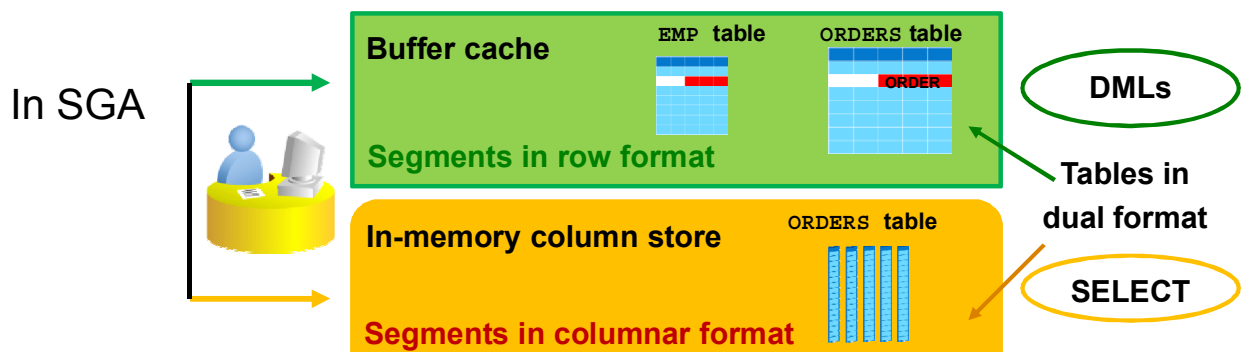
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The In-Memory Column Store feature brings many benefits apart from the major one that is increased queries performance.

- The amount of data that can fit in memory is greatly increased. The data as it is populated into the IM column store is automatically compressed using a specialized compression algorithm delivering 2 times to 10 times compression rates depending on the data types and the distribution of the data. There are three different types of compression and queries execute directly against the compressed data. Data can be stored in a multiple tiered environment, with the hottest data in-memory, active data on flash and older or colder data on disk.
- All OLTP operations are supported on RAC.
- The multitenant architecture can make use of the In-Memory Column Store capabilities.
- The response time for some queries is so fast that no more SQL tuning is required in these cases. Nevertheless, if only some columns are in the column store and a query accesses a not in-memory column, tuning might be required.
- No restriction exists on SQL statements.
- Backup and recovery procedures still operate the same way as without the IM column store.
- No migration of data is needed.

Overview

- A new pool in SGA: In-Memory column store
 - Segments populated into the IM column store are converted into a columnar format
 - In-Memory segments are transactionally consistent with the buffer cache
- Only one segment on disk and in row format



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The in-memory columnar format does not replace the on-disk or buffer cache format. This means that when a segment such as a table or a partition is populated into the IM column store, the on-disk format segment is automatically converted into a columnar format and optionally compressed. The columnar format is a pure in-memory format. There is no columnar format storage on disk. It never causes additional writes to disk and therefore does not require any logging or undo space.

All data is stored on disk in the traditional row format.

Moreover, the columnar format of a segment is a transaction-consistent copy of the segment either on disk or in the buffer cache. Transaction consistency between the two pools is maintained.

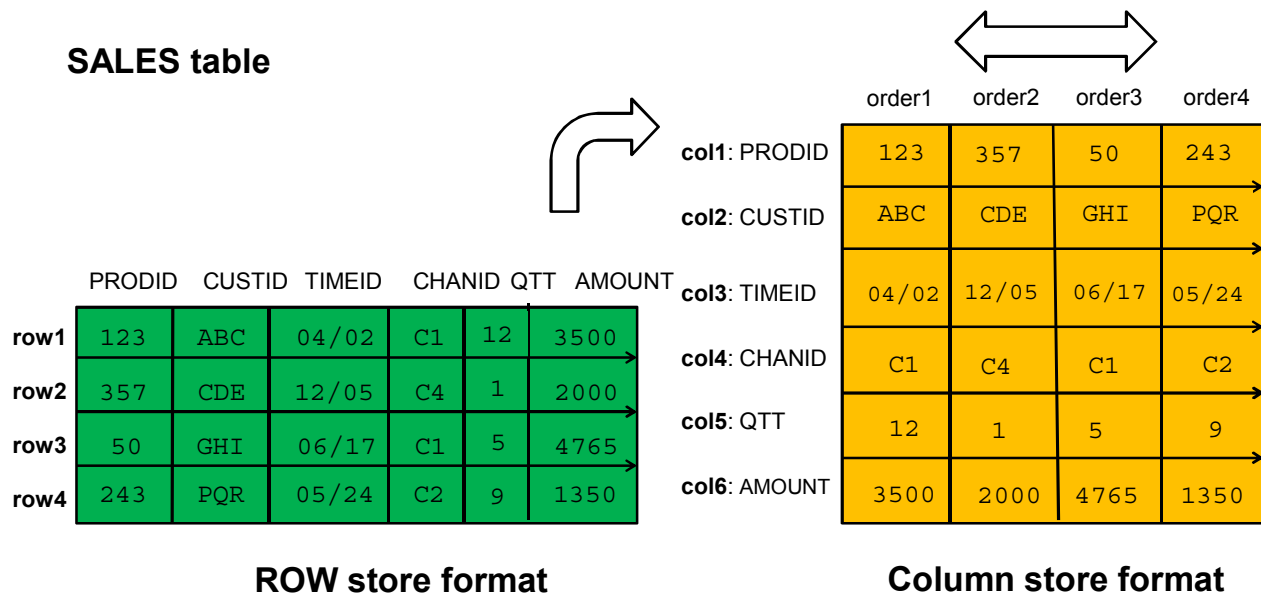
If sufficient space is allocated to the IM column store in SGA, a query accessing objects that are populated into the IM column store performs much faster. The improved performance allows more ad-hoc analytic queries to be executed directly on the real-time transaction data without impacting the existing workload. A lack of IM column store space does not prevent statements to execute against tables that could have been populated into IM column store.

The DBA must decide, according to the types of queries and DMLs executed against the segments, which segments should be defined as non in-memory segments and those as in-memory segments . The DBA can also define more precisely which columns are good candidates for IM column store:

- In **row format exclusively**: The segments being frequently accessed by OLTP style queries, operating on few rows returning many columns are good candidates for the buffer cache. These segments should not necessarily be defined as in-memory segments and will be sent to the buffer cache only.
- In **dual format simultaneously**: The segments being frequently accessed by analytical style queries, operating on many rows returning few columns are good candidates for IM column store. If a segment is defined as an in-memory segment but has some columns defined as non in-memory columns, the queries that select any non in-memory columns are sent to the buffer cache and those selecting in-memory columns only are sent to the IM column store. Any fetch-by-rowid is performed on the segment through the buffer cache.

Any DML performed on these objects is executed via the buffer cache.

Store Versus Column Store: 2D Vision



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The traditional RDBMS style of storing tables in data files follows the above example on the left in the slide. The `SALES` table contains rows and each row represents one order which is described by columns: Product ID, customer ID, a date, a channel ID, quantity sold for that product, and the corresponding amount of money.

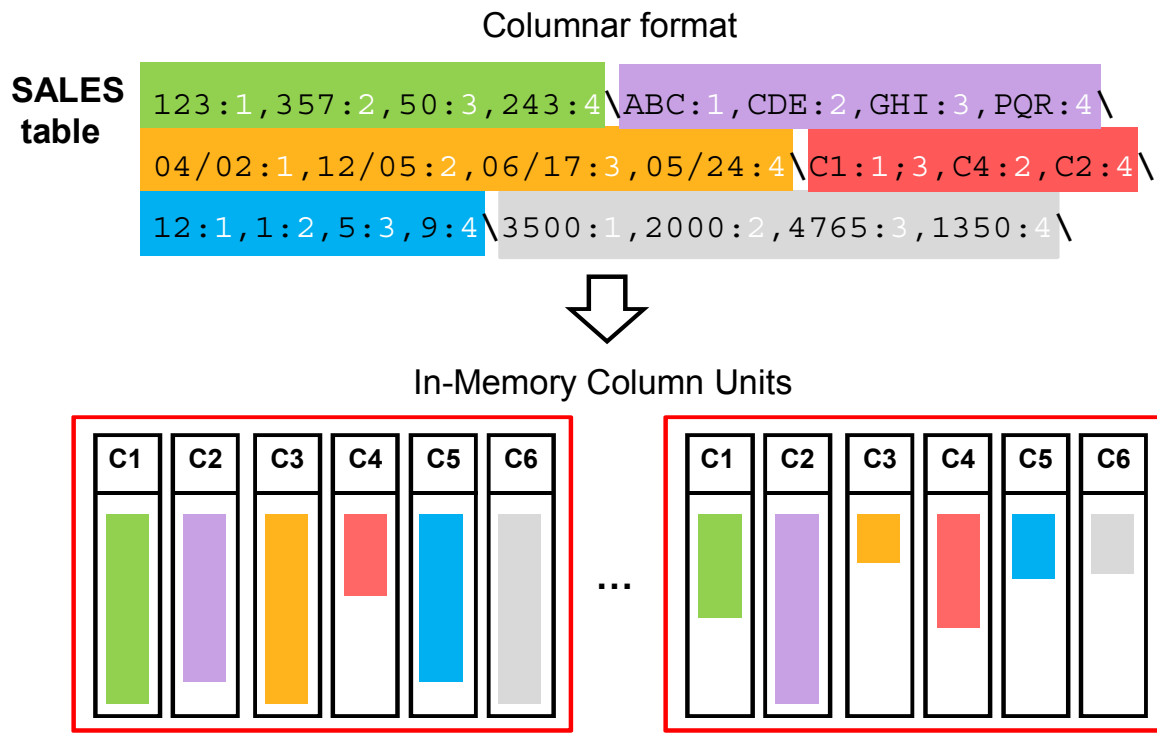
This representation is well suited to manipulate the individual orders. However it is not really convenient for searches for just one attribute on all orders when no indexes exist on columns. For example, you want calculate the profit made on all orders done in June through channel C1 from the table containing millions of orders. The query must read each and every row, and look for the right date and channel. Note that each time the query reads a row, it reads all columns although only two are requested.

How can we do better than this row representation?

Imagine all values of the same column are stored together, and this for each column of the previous row storage representation. You end up with a representation similar to the one on the right of the above slide.

The same query has to scan only two columns with this columnar representation. This represents far fewer IOs than in the row storage case. Reading only the columns you need and ignoring the rest are referred to as “columnar projection.”

In-Memory Column Unit



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In-Memory Column Unit

Conceptually when a table is populated into the IM column store, the segment rows are divided into large chunks (In-Memory Column Units or IMCUs), and within those chunks each column is stored separately in a contiguous region of memory. The chunks will be large enough to achieve the maximum possible scan speeds.

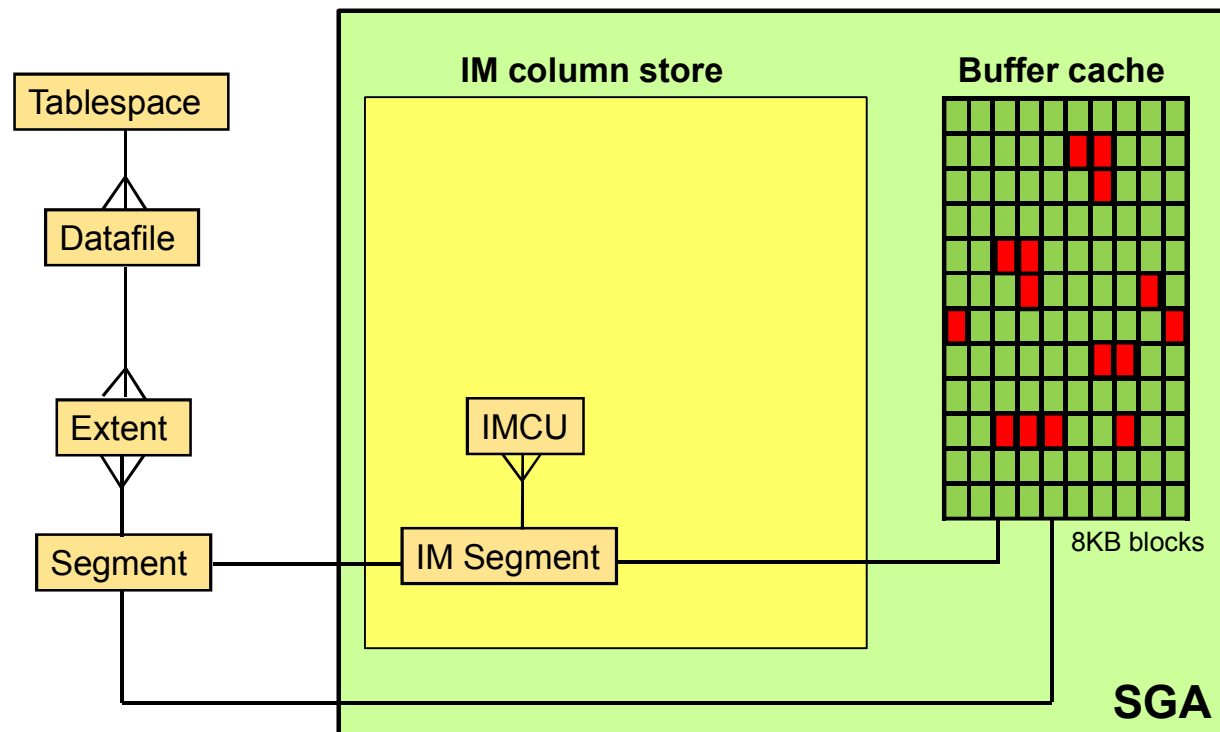
The fundamental in memory data structure is called an IMCU.

Unlike conventional database blocks which are typically 8k bytes in size, IMCUs will be orders of magnitude larger. The exact number of rows per IMCU is determined at runtime and based on table size and structure and memory constraints.

In-Memory Compression Algorithm

The data converted to the columnar format in the IM column store uses new specialized compression algorithms. There are three compression levels.

In-Memory Column Store Cache Versus Buffer Cache



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In-Memory Column Store

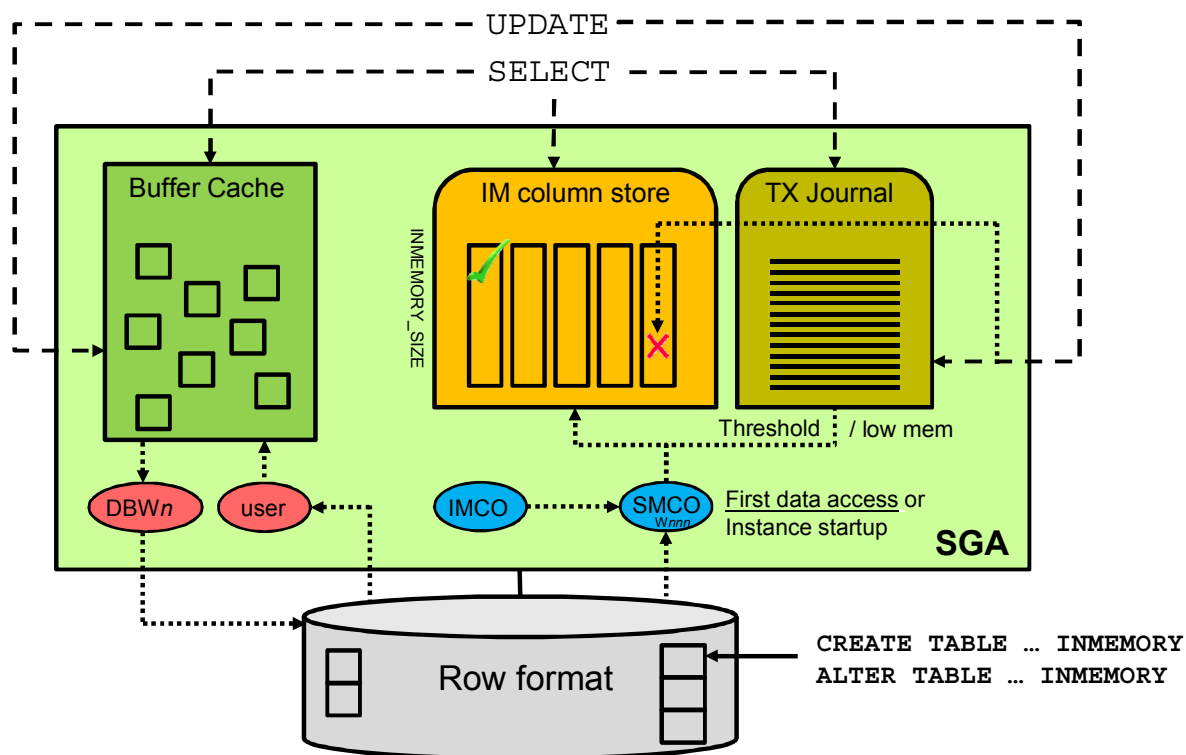
The IM column store stores converted data of nonpartitioned tables, table partitions or table subpartitions and resides in a single in-memory area in an instance.

In-Memory Data Segments

There is one-to-one correspondence between an in-memory segment and an on-disk segment. Every existing on-disk data format for permanent heap tables and all object types (partitions, materialized views, materialized join views, materialized view logs, and inline LOBs) is supported by IM column store.

An in-memory column store segment is a collection of used IMCUs.

Dual Format In Memory



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After allocating space to the IM column store in SGA, the DBA turns on the `INMEMORY` attribute at the object creation or when altering it to convert an existing object to an in-memory candidate. An in-memory table gets IMCUs allocated in the IM column store at first table data access or at database startup. An in-memory copy of the table is made by doing a conversion from the on-disk format to the new in-memory columnar format. This conversion is done each time the instance restarts as the IM column store copy resides only in memory. When this conversion is done, the in-memory version of the table gradually becomes available for queries. If a table is partially converted, queries are able to use the partial in-memory version and go to disk for the rest, rather than waiting for the entire table to be converted. There is a new background process, `IMCO`, that creates and refreshes IMCUs to populate and re-populate the IM column store. `IMCO` is the background coordinator process which schedules the objects to be populated/repopulated in the IM column store. `SMCO` and `Wnnn` are background processes which actually populate the objects in memory. When rows in the table are updated, the corresponding entries in the IMCUs are marked stale. The row version recorded in the journal is constructed from the buffer cache, and is unaffected by what subsets of columns are in the IMCU. IMCU synchronization is performed by `IMCO/SMCO/Wnnn` background processes with the updated rows populated in the transaction journal, based on events:

- An internal threshold including a number of invalidations to the rows per IMCU
- Transaction journal running low on memory
- RAC invalidations

Indexes

- Which columns to index?

For OLTP: 1 to 3

For analytics: 10 or 20

Without IM Column Store

- Poor performance due to missing indexes
- Heavy maintenance during DMLs
- Storage space required

With IM
Column Store

- No more analytic indexes for in-memory tables
- Both predefined and ad-hoc analytic queries run fast.
- OLTP & batch run up to 300% faster.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Up until Oracle Database 12c Release 12.1.0.1, the only way to run analytic queries with an acceptable response on an OLTP environment was to create specific indexes for these queries on commonly accessed columns. They work well in-memory and are extremely efficient on-disk since they minimize disk IO needed to find the requested data. But all these additional indexes need to be maintained as the data changes, which increase the elapsed time for each of these changes. Moreover, the indexes require additional storage space.

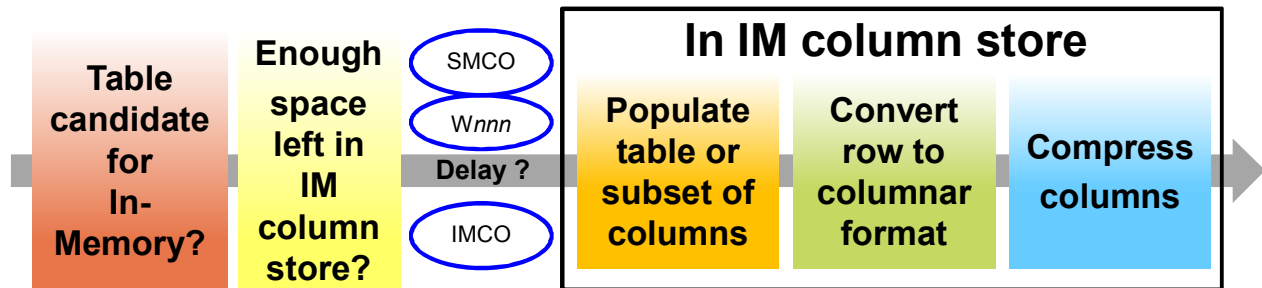
Now with the In-Memory Column Store feature of Oracle Database 12c Release 12.1.0.2, all but the indexes needed for referential integrity (for example, primary key indexes) can be replaced with scans of the IM column store representation of the table when the table fits in IM column store. 75% of the indexes on an in-memory table can be removed, which speed up DML statements. Ad-hoc queries that were not able to be run efficiently before because of the lack of appropriate indexes will run at in-memory speeds.

Removing the analytic indexes greatly simplifies tuning and eliminates ongoing administration.

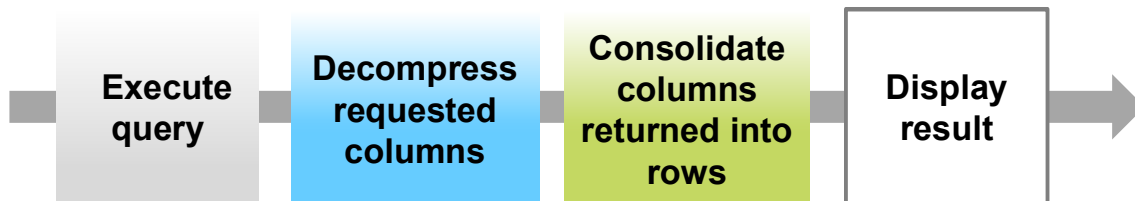
Process

The IM column store is initialized.

1. Query an in-memory table:



2. Results are displayed much faster than with row format:



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

What are the different steps when a query is executed on an in-memory table?

The first prerequisite is that the DBA pre-allocated space for the IM column store in SGA.

1. If the table being queried is defined as a candidate to be populated into the IM column store, background processes convert the table rows row format from disk into a columnar format and populate it into the IM column store. By default, population of an in-memory table into IMCUs may be delayed until it will be useful.
 - If there is not enough space in the IM column store to populate the entire object, then the object will be partially populated. When a query starts accessing the entire object, as much of the data from the column store is retrieved and the rest is retrieved either from the buffer cache, flash cache or disk. The DBA can set priorities on in-memory objects so that populating tables into the IM column store processes with different delays.
 - A possible compression may occur if memory compression is defined on the table or on a subset of the table columns. According to the attributes defined on the table or columns, it is possible that only a subset of columns fall under the population and compression steps.
2. After it is in the right format in IM column store, the optimizer plays its role to generate a plan. The query is executed on the columnar format data in memory. After the execution is completed, the columns data are decompressed, consolidated into rows to show the result to the user. This type of query is called an in-memory query.

Deploying IM Column Store

1. Verify the database compatibility value.

```
COMPATIBLE = 12.1.0.0.0
```

2. Configure the IM column store size.

```
INMEMORY_SIZE = 100G
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

No special installation is required to set up the feature. It is shipped in Oracle Database 12c Release 1 PS1.

1. The database compatibility must be set to 12.1.0.0 or higher.
2. Configure the IM column store size by setting the `INMEMORY_SIZE` instance parameter. Use the K or M or G letters to define the amount unit.
This pool, like the log buffer, the KEEP, RECYCLE and other block sizes pools is a static pool and is not affected by ASMM. The memory allocated to this pool is fixed and deducted from the total available memory for `SGA_TARGET` when ASMM is enabled. There is no LRU algorithm to manage the in-memory objects. In-memory objects may be partially populated into the IM column store, if there is not enough space to accommodate the entire object. When this object is queried, as much of the data from the column store is retrieved and the rest is retrieved either from the buffer cache, flash cache, or disk. The DBA can set priorities on objects to define which in-memory objects should be populated in priority in the IM column store.
Set the `INMEMORY_SIZE` parameter to a value at least the sum of the estimated size of all in-memory tables. You can use the `DBMS_COMPRESSION` package to determine how well the tables can be compressed in-memory.
3. If the instance was already started, you must restart the database to initialize the IM column store in the SGA with the new value.

This parameter can also be set per-PDB to limit the maximum size used by each PDB. Note that the sum of the per-PDB value does not necessarily have to be to equal the CDB value, and it may even be greater.

Deploying IM Column Store: Objects Setting

3. Enable/disable objects to be populated into IM column store.
 - Enable/disable a whole segment:
 - IMCUs are initialized and populated at query access only.

```
SQL> CREATE TABLE large_tab (c1 ...) INMEMORY; → Dual format
```

```
SQL> ALTER TABLE t1 INMEMORY ; → Dual format
```

```
SQL> ALTER TABLE sales NO INMEMORY; → Row format only
```

```
SQL> CREATE TABLE countries_part ... PARTITION BY LIST ..
      ( PARTITION p1 .. INMEMORY, PARTITION p2 .. NO INMEMORY);
```

- IMCUs can be initialized at database open.

```
SQL> CREATE TABLE test (...) INMEMORY PRIORITY CRITICAL;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

4. Turn on the `INMEMORY` attribute at the object creation or when you alter it, to convert the object into a columnar representation in IM column store. All columns of the in-memory table are populated into memory unless some columns are disabled by using the `NO INMEMORY` clause. It is recommended to specify all columns at once rather than having an `ALTER TABLE` for each column, as it is more efficient. Two `INMEMORY` subattributes define the following behaviors:
 - The loading priority of the object data in the IM column store: The `INMEMORY` clause can be decorated with the `PRIORITY` subclause. An in-memory table is populated into memory at the first data access by default. This default behavior is the “on demand” behavior. Using different priority levels, a table data can be populated into the IM column store soon after the database starts up. The different `PRIORITY` values are covered later in the lesson.
 - The degree of compression of the columns of the object in the IM column store
- The segments that are compatible with the `INMEMORY` attribute are the tables, partitions, subpartitions, inline LOBs, materialized views, materialized join views, and materialized view logs. `INMEMORY` partitions setting is shown in the fourth example of the slide.
- Clustered tables and IOTs are not supported with the `INMEMORY` clause.

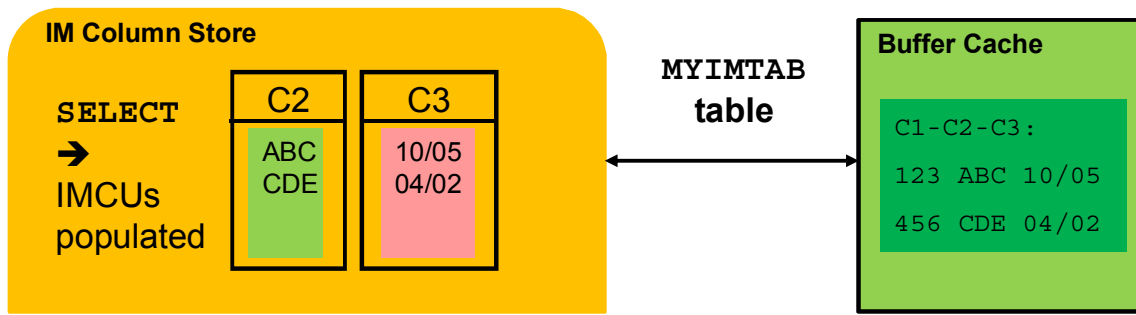
Deploying IM Column Store: Columns Setting

- Enable/disable a subset of columns:

```
SQL> CREATE TABLE myimtab (c1 NUMBER, c2 CHAR(2), c3 DATE)
      INMEMORY NO INMEMORY (c1);
```

```
SQL> ALTER TABLE myimtab NO INMEMORY (c2);
```

```
SQL> CREATE TABLE t (c1 NUMBER, c2 CHAR(2)) NO INMEMORY INMEMORY (c2);
ORA-64361: column INMEMORY clause may only be specified for an inmemory
table
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If a table is defined as `INMEMORY`, populating a subset of columns only into the IM column store is possible by using the `NO INMEMORY` on the non in-memory columns as shown in the first and second examples in the slide. You can group non in-memory columns in the `NO INMEMORY` clause as in the following example:

```
SQL> ALTER TABLE tab INMEMORY MEMCOMPRESS FOR CAPACITY LOW (c2, c3)
      NO INMEMORY (c1, c4);
```

If a table is defined as `NO INMEMORY`, populating a subset of columns into the IM column store is not possible as shown in the third example in the slide.

Columns that are not associated with a compression clause (`INMEMORY` or `NO INMEMORY`), and columns that have the `INMEMORY` clause specified without a compression level (`HIGH` | `MEDIUM` | `LOW`), inherit the compression clause of the parent table or partition. A column's compression level, if specified, overrides the compression level of the parent table or partition.

Changing the compression clause of columns with the `ALTER TABLE` statement results in an update of the dictionary, but does not force a repopulation on any existing data into the IM column store. The new compression levels are only reflected in newly repopulated data.

Setting one or the other attribute at both levels, object and column levels, can be defined while creating or altering the object.

Objects Candidates for IM Column Store

The DBA_TABLES view displays tables candidates for IM column store:

```
SQL> SELECT table_name tab, inmemory_compression Comp,
       inmemory_priority Priority,
       inmemory_distribute RAC, inmemory_duplicate DUP
FROM   dba_tables;
```

TABLE	COMP	PRIORITY	RAC	DUP
TEST1	FOR QUERY HIGH	NONE	AUTO	DUPLICATE ALL
TEST2	NO MEMCOMPRESS	CRITICAL	AUTO	DUPLICATE
EMP	FOR DML	LOW	BY PARTITION	NO DUPLICATE

- **NO MEMCOMPRESS:**
Not compressed
- **FOR QUERY / FOR CAPACITY:**
Compressed

- **NONE:**
Populated on demand
- **Other values:**
Populated according to priority level

- **AUTO:** Object distributed by Oracle among the IM column stores of the RAC nodes
- **BY PARTITION:** The object is distributed by partitions among the RAC nodes

- **DUPLICATE ALL**
- **DUPLICATE**
- **NO DUPLICATE**

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To indicate that a table or partition or subpartition is not candidate for the IM column store, the INMEMORY_COMPRESSION and INMEMORY_PRIORITY new columns in DBA_TABLES, DBA_TAB_PARTITIONS, DBA_TAB_SUBPARTITIONS are set to null. To indicate the current IM column store attributes of a table, partition or subpartition, three columns have been added to the DBA_TABLES, DBA_TAB_PARTITIONS, DBA_TAB_SUBPARTITIONS views:

- **INMEMORY_COMPRESSION:** Each object that is brought into the IM column store is compressed. Oracle offers multiple compression techniques which provide different levels of compression and performance. By default, data is compressed using the FOR QUERY option. This provides the best balance between compression and performance. The compression option used is shown in the INMEMORY_COMPRESSION column of the DBA_TABLES views. The compression attribute is covered later.
- **INMEMORY_PRIORITY:** The population of an in-memory table into the IM column store depends on the priority set with the PRIORITY subclause. The default is PRIORITY NONE. The priority attribute is covered later.

- **INMEMORY_DISTRIBUTE:** In a RAC environment, each database node has its own IM column store. If an object is too big to fit in the IM column store on a single node, it is possible for pieces of an object to be distributed among each of the RAC nodes. The default is "AUTO" which means that Oracle decides the best way to distribute tables across RAC instances. You can specify `BY ROWID RANGE` to distribute by rowid range, `BY PARTITION` or `BY SUBPARTITION` to distribute partitions or subpartitions to different nodes.
- **INMEMORY_DUPLICATE:** To duplicate a table across all RAC instances, so that the entire table, rows and columns, are available on every instance, specify the `DUPLICATE ALL` option in the `DUPLICATE` clause. When the table is actually distributed among the nodes, there are two options for the `DUPLICATE` clause.
 - If you want one copy, use `NO DUPLICATE`.
 - If you want two copies across the cluster, use `DUPLICATE`.

Columns Candidates for IM Column Store

The V\$IM_COLUMN_LEVEL view also displays columns in in-memory tables that are NOT candidates for IM column store:

```
SQL> SELECT obj_num, segment_column_id, inmemory_compression
FROM    v$im_column_level;
```

obj_num	segment_column_id	inmemory_compression
98202	1	DEFAULT
98202	2	NO MEMCOMPRESS
98202	3	DEFAULT
98202	4	NO INMEMORY

In-memory columns overriding compression clause of the parent table

In-memory columns of in-memory tables inheriting compression clause of the parent table

Non in-memory columns in in-memory tables

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

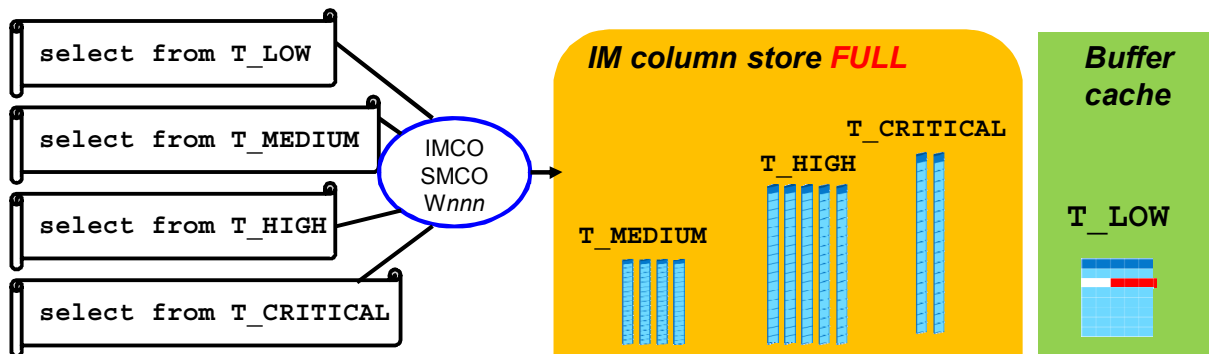
When queries do not perform through the IM column store even though the tables are defined as in-memory segments, find whether non in-memory columns exist in these in-memory tables using the view V\$IM_COLUMN_LEVEL view. Querying this view displays the list of non in-memory columns and in-memory columns that exist in in-memory tables.

Note that you can only specify the compression or NO INMEMORY for selective columns. Columns cannot have different priority or distribution since they are all populated together.

Defining IM Column Store Priority

Use **PRIORITY** to define the population priority level.

```
SQL> CREATE TABLE t_low (code NUMBER) INMEMORY PRIORITY LOW;
```



```
SQL> CREATE TABLE countries_part ... PARTITION BY LIST ..
      ( PARTITION p1 .. INMEMORY PRIORITY HIGH,
        PARTITION p2 .. INMEMORY MEMCOMPRESS FOR CAPACITY LOW,
        PARTITION p3 .. , .. );
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

By default, the population of an in-memory table or partition or subpartition or even column depends on the **PRIORITY** value set for the table or partition or subpartition or column.

- The **NONE** value means an “on demand” population when the data is queried only. The default is **PRIORITY NONE** when no **PRIORITY** is defined.
- There are four priority levels: **LOW**, **MEDIUM**, **HIGH**, and **CRITICAL**. If one of these levels is specified, in-memory objects are populated in priority order at instance startup on the next **IMCO** cycle (default 2 minute cycles).
 1. **IMCO** initiates background population and repopulation of in-memory enabled objects, queuing population tasks.
 2. The population tasks are queued to **SMCO** background process for execution and thus may have to wait for available worker processes (**Wnnn**). **SMCO** dynamically spawns slave processes (**Wnnn**) to implement these tasks.

The **IMCO** background process queues population tasks for objects with priority other than **NONE** only. The queue is drained in priority order, from **CRITICAL** to **LOW**.

If the population runs out of IM column store space, all tables with higher priority take precedence over those with lower priority. The statement that does not populate data into IM column store is committed in order to not break any application semantics and uses the buffer cache.

Segments Populated into the IM Column Store

- View segments populated into the IM column store:

```
SQL> SELECT owner, segment_name name FROM v$sqlim_segments
WHERE segment_name = 'SALES';
```

Priority NONE →
No rows populated yet

```
SQL> SELECT /*+ full(s) noparallel (s)*/ count(*) FROM sales s;
```

- Check if segments are **fully populated** in IM column store:
- Count blocks on **disk** versus **blocks in IM column store**:

```
SQL> SELECT segment_name, bytes, inmemory_size,
           bytes_not_populated
FROM v$sqlim_segments;
```

SEGMENT_NAME	BYTES	INMEMORY_SIZE	BYTES_NOT_POPULATED
SALES	228231	36299	0
T1	11475615744	4664262656	5004262678



BYTES in IM column store

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If an in-memory object is created with the **NONE** priority, it is populated into the MC only when it is accessed by a query. The IMCUs are not created and populated in the IM column store until a query accesses at least one column.

If in-memory objects are created with a non **NONE** priority, the IMCUs are created and populated in the IM column store on the next **IMCO** cycle after the instance starts up and in the priority order. Populating a table in the background has no effect on any concurrently running DML operations on the table. Nothing will wait on the background population.

You can compare the number of bytes stored under the columnar format by using the **INMEMORY_SIZE** column of **V\$IM_SEGMENTS** view to the actual bytes of the segments as it is stored on disk under the row format by using the **BYTES** column of **V\$IM_SEGMENTS**.

Large segments are not necessarily populated into the IM column store within one shot. The **SMCO** and the **Wnnn** background processes populate the segment progressively. To check that the segment gets fully populated into the IM column store, check the **BYTES_NOT_POPULATED** column of the **V\$IM_SEGMENTS** view. If the value is **<> 0**, then the segment is not fully populated.

Table eviction only happens when a table is disabled for the IM column store using an **ALTER TABLE** command. Since **ALTER** is a DDL statement, it automatically invalidates dependent cursors. There is no automatic memory management for the IM column store, potentially leading to evictions of infrequently used tables.

Defining IM Column Store Compression

- Use **MEMCOMPRESS** to define the compression level.

```
SQL> CREATE MATERIALIZED VIEW mv1 INMEMORY NO MEMCOMPRESS;
```

```
SQL> CREATE TABLE large_tab (c1 ...) INMEMORY
MEMCOMPRESS FOR QUERY;
```

- Compare stored bytes on disk with columnar format compressed bytes:



- Change **MEMCOMPRESS** to a higher compression level:

```
SQL> ALTER TABLE t1 INMEMORY MEMCOMPRESS FOR CAPACITY HIGH;
```

SEGMENT_NAME	BYTES	INMEMORY_SIZE	 No automatic repopulation
T1	11475615744	4664262656	

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

MEMCOMPRESS attribute distinguishes in-memory compression from on-disk compression. There are different compression levels in IM column store using different compression algorithms:

- NO MEMCOMPRESS** does no compression at all.
- MEMCOMPRESS FOR QUERY [LOW|HIGH]** uses a compression algorithm fit for better query performance than for lower space used in in-memory column store. **FOR QUERY LOW** is the default when only **MEMCOMPRESS** is defined.
- MEMCOMPRESS FOR CAPACITY LOW** corresponds to a compression algorithm fit for lower space used in in-memory column store. The **FOR CAPACITY LOW** level is for getting a balance between high compression and better query performance. If **FOR CAPACITY** is specified without **LOW** or **HIGH**, it defaults to **FOR CAPACITY LOW**.
- MEMCOMPRESS FOR CAPACITY HIGH** corresponds to a compression algorithm that compresses data at a higher ratio than **LOW**, and therefore providing higher storage savings in in-memory column store than **LOW**.
- MEMCOMPRESS FOR DML**

Use `V$IM_SEGMENTS` to know the actual size of the segments on disk compared – `BYTES` column – to the IM column store size of the same segments – `INMEMORY_SIZE` column. When the compression level is updated, the information is updated in the data dictionary with the new segment compression level. The IMCUs data repopulation with the new compression level takes place only if the segment is moved or manually repopulated. When none of these actions are used, the new compression levels are only reflected in newly populated data.

IM Column Store Compression Advisor

A compression advisor analyzes how much space in IM column store is required to enable INMEMORY MEMCOMPRESS with different levels on a table.

```
BEGIN
  DBMS_COMPRESSION.GET_COMPRESSION_RATIO (
    'TS_DATA', 'SSB', 'LINEORDER', NULL,
    DBMS_COMPRESSION.COMP_INMEMORY_QUERY_LOW,
    blkcnt_cmp, blkcnt_uncmp, row_cmp, row_uncmp, cmp_ratio,
    comptype_str,10000,1);
  DBMS_OUTPUT.PUT_LINE('Block count uncompressed = ' || blkcnt_uncmp);
  DBMS_OUTPUT.PUT_LINE('Row count per block uncompressed=' || row_uncmp);
  DBMS_OUTPUT.PUT_LINE('Compression type = ' || comptype_str);
  DBMS_OUTPUT.PUT_LINE('Comp ratio= ' || cmp_ratio );
  DBMS_OUTPUT.PUT_LINE(' ');
  DBMS_OUTPUT.PUT_LINE('The IM column store space needed is about 1 byte
in IM column store for ' || cmp_ratio || ' bytes on disk.');
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Use the `DBMS_COMPRESSION.GET_COMPRESSION_RATIO` procedure to analyze how much space in the IM column store is needed to enable INMEMORY MEMCOMPRESS on a table or partition or subpartition or LOB or other compatible type of object. Dividing the expected on-disk size by the reported ratio gives a proper estimate of the IM column store space required for the segment. Depending on whether `DISTRIBUTE` or `DUPLICATE` is specified, the size is either the total size or the size per RAC node.

Use new constants for the `COMPTYPE` parameter:

- `COMP_INMEMORY_NOCOMPRESS`: to test IM column store basic compression.
- `COMP_INMEMORY_QUERY_LOW` / `HIGH`: to test IM column store compression for query.
- `COMP_INMEMORY_CAPACITY_LOW` / `COMP_INMEMORY_CAPACITY_HIGH`: to test columnar format for capacity low/high compression.
- `COMP_INMEMORY_DML`

The result of the procedure execution could be the following:

Block count uncompressed = 146

Row count per block uncompressed = 68

Compression type = "In-memory Memcompress Query Low"

Comp ratio= 18

The IM column store space needed is about 1 byte in IM column store for 18 bytes on disk.

Computing Compression Ratio

Use V\$IM_SEGMENTS view to display compression ratio:

```
SQL> SELECT segment_name, bytes Disk, inmemory_size,
           inmemory_compression COMPRESSION,
           bytes / inmemory_size comp_ratio
FROM v$im_segments;
```

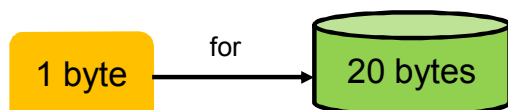
OBJECT_NAME	DISK	INMEMORY_SIZE	COMPRESSION	COMP_RATIO
SALES	212284915	200104115	FOR QUERY LOW	1.06
EMPLOYEES	3456956	17984	FOR CAPACITY HIGH	192.22



The highest, the best: 20



The lowest, the worse:
1.06



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The columnar format in the IM column store compresses the segments or not during population. It depends on the compression level assigned to the object.

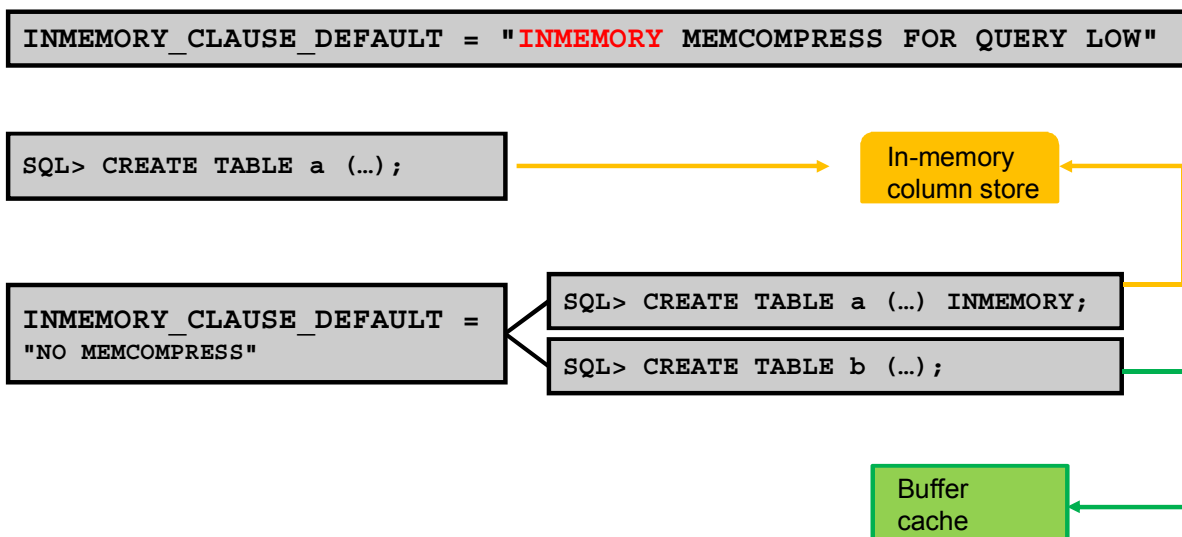
The query in the slide shows the actual number of bytes of the objects as they are stored on disk under the row format using the BYTES column of V\$IM_SEGMENTS view and compares these values to the number of bytes stored under the columnar format in the IM column store using the INMEMORY_SIZE column of V\$IM_SEGMENTS view. The query shows you how to get the compression ratio for each of the objects populated in the IM column store.

In the result of the query in the slide, the SALES table gets a low compression ratio with the FOR QUERY compression level whereas the EMPLOYEES table gets an extremely high compression ratio with the FOR CAPACITY HIGH compression level.

However, accessing the EMPLOYEES table requires a lot more CPU than accessing the SALES table as data compressed FOR CAPACITY HIGH must be decompressed to a certain degree before WHERE clause predicates can be applied to it. This is not the case for the SALES table and MEMCOMPRESS FOR QUERY, which allows WHERE clause predicates can be applied to the compressed data directly.

Default In-Memory Setting

Assign new tables default in-memory attributes:



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Instead of setting the `INMEMORY` attribute each time the user creates an object, the DBA can specify a default mode for in-memory tables at instance level by setting the new `INMEMORY Clause Default` parameter. By default, the parameter is set to an empty string. This means that only explicitly specified tables are made to be in memory. Setting the parameter to a value makes all new tables in memory or force certain in-memory options by default if not explicitly specified in the syntax. The parameter is dynamic at instance and session levels. The possible values are:

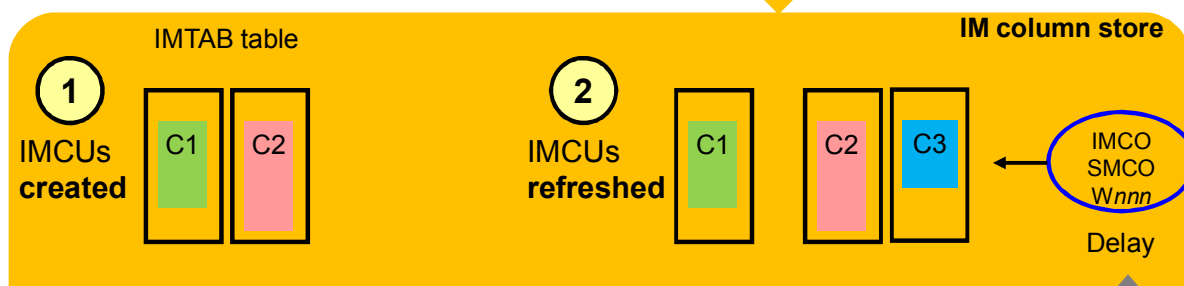
- `INMEMORY [NO MEMCOMPRESS|MEMCOMPRESS [FOR [QUERY [LOW|HIGH] | [CAPACITY[LOW|HIGH]]]]] [PRIORITY [LOW|MEDIUM|HIGH|CRITICAL|NONE]] [DISTRIBUTE [AUTO|BY ROWID|BY PARTITION|BY SUBPARTITION]] [DUPLICATE ALL|DUPLICATE|NO DUPLICATE]`: All tables are made in-memory with the defined compression, priority level, distribution and duplication unless specifically specified as `NO INMEMORY`.
- `NO MEMCOMPRESS|MEMCOMPRESS [FOR [QUERY [LOW|HIGH] | [CAPACITY[LOW|HIGH]]]]] [PRIORITY [...]] [DISTRIBUTE [...]] [DUPLICATE ALL|DUPLICATE|NO DUPLICATE]`: All tables are made in-memory with the defined compression, priority level distribution and duplication if specifically specified as `INMEMORY`. If the parameter value is set to `MEMCOMPRESS FOR CAPACITY` and a table is created as `INMEMORY MEMCOMPRESS DUPLICATE`, it is the same as if the table was declared as `INMEMORY MEMCOMPRESS FOR CAPACITY DUPLICATE`.

Impact of Changing In-Memory Attributes

1. An in-memory table is created:

```
SQL> CREATE TABLE imtab (c1 number, c2 VARCHAR2(30), c3 DATE)
      INMEMORY NO INMEMORY(c3);
```

```
SQL> INSERT INTO imtab VALUES (...)
SQL> SELECT c1, c2 FROM imtab;
```



2. Change or set the INMEMORY attribute of a column:

```
SQL> ALTER TABLE imtab INMEMORY(c3);
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Whenever a table is created with the `INMEMORY` attribute and a defined priority, at query time or after instance startup, an IMCU is created for all or some columns defined to be in-memory. Population of an in-memory table may be delayed depending on the priority assigned. If you want an in-memory table to be populated with a higher priority than another in-memory table, use the `PRIORITY` subclause. You can also change the compression level of the segment or columns.

Changing the `INMEMORY` attribute of a column does not force a refresh of existing IMCUs. It does not necessarily repopulate the existing IMCUs even after the column query. The new compression level is only reflected in newly (re)populated data.

Moving or Splitting In-Memory Segments

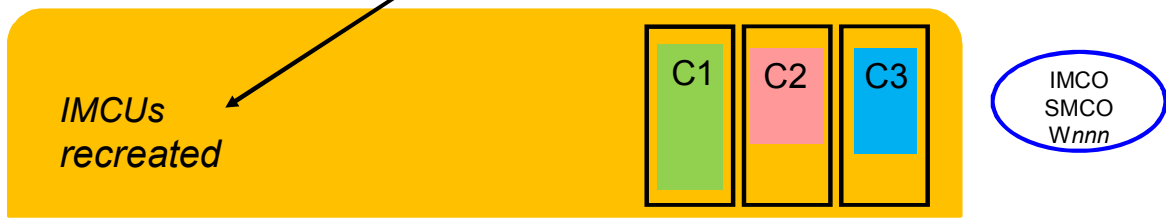
1. An in-memory segment is created:

```
SQL> CREATE TABLE imt (c1 number, c2 VARCHAR2(30), c3 DATE) INMEMORY;
SQL> INSERT INTO imt ...      SQL> SELECT * FROM imt;
```



2. Move the in-memory table or split the in-memory partition:

```
SQL> ALTER TABLE imt MOVE;
SQL> SELECT * FROM imt;
```

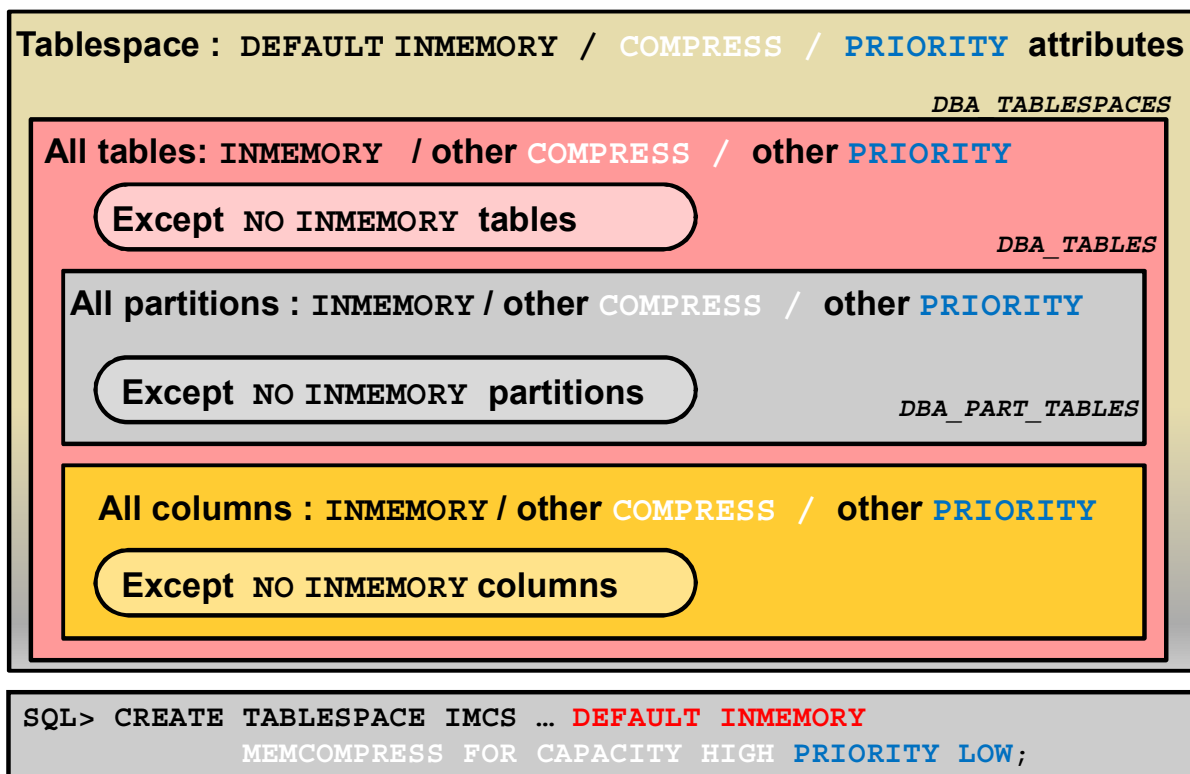


ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- A DDL operation such as ALTER TABLE MOVE or SPLIT / MERGE PARTITION requires recreating the IM column store representation for the table or the partition which means the recreation of the IMCUs. This requires that the table is being accessed by a query or detected by the IMCO background process (this depends on the priority set on the object). While the IM column store representation recreation is ongoing, other operations are unimpaired.
- DDL operations on an in-memory table such as adding/dropping in-memory columns force the IMCUs repopulation.

INMEMORY Inheritance



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The INMEMORY attribute set at the tablespace level as the default behavior is inherited by all INMEMORY compatible segments in the tablespace (tables, partitions, inline LOBs, MVs, MV logs). Any future table created in the tablespace is automatically an in-memory table. The default values are recorded in DBA_TABLESPACES view in three columns:

- DEF_INMEMORY_COMPRESSION
- DEF_INMEMORY_PRIORITY
- DEF_INMEMORY_DISTRIBUTE
- DEF_INMEMORY_DUPLICATE

A PRIORITY change (different from NONE) at the tablespace level applies only to newly created tables and not to existing tables. All new tables are immediately queued for populating.

The INMEMORY attributes set for a partitioned table as the default ones are inherited by all partitions and are visible in the DBA_PART_TABLES view in three columns:

- DEF_INMEMORY_COMPRESSION
- DEF_INMEMORY_PRIORITY
- DEF_INMEMORY_DISTRIBUTE
- DEF_INMEMORY_DUPLICATE

After Objects Settings

Drop analytics indexes.

```
SQL> DROP INDEX i_sales_timeid PURGE;
```

```
SQL> DROP INDEX i_sales_chanid PURGE;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The in-memory tables do not require the analytics indexes any more provided all corresponding columns are set as in-memory. They are replaced with scans of the in-memory columnar representation of the table. The indexes can be marked invisible as long as these do not guarantee uniqueness or referential integrity, such as primary keys, unique indexes, and foreign keys.

The guidance could be as follows:

- Mark analytic style indexes invisible
- Confirm workload no longer benefits for indexes
- Drop indexes

Retrieving CREATE DDL Statement of In-Memory Objects

```
SQL> SELECT DBMS_METADATA.GET_DDL ('TABLE', 'CUSTOMER', 'SSB')
        FROM dual;

DBMS_METADATA.GET_DDL('TABLE', 'CUSTOMER', 'SSB')
-----
CREATE TABLE "SSB"."CUSTOMER"
  ( "C_CUSTKEY" NUMBER, "C_NAME" VARCHAR2(25),
    "C_ADDRESS" VARCHAR2(25), "C_CITY" CHAR(10),
    "C_NATION" CHAR(15), "C_REGION" CHAR(12),
    "C_PHONE" CHAR(15), "C_MKTSEGMENT" CHAR(10)
  ) SEGMENT CREATION IMMEDIATE
PCTFREE 10 PCTUSED 40 INITRANS 1 MAXTRANS 255
NOCOMPRESS LOGGING ...
BUFFER_POOL DEFAULT FLASH_CACHE DEFAULT CELL_FLASH_CACHE
DEFAULT) TABLESPACE "TS_DATA"
INMEMORY PRIORITY MEDIUM MEMCOMPRESS FOR QUERY LOW
      AUTO DISTRIBUTE

CACHE
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The existing `DBMS_METADATA.GET_DDL` procedure is updated to be able to generate the new clauses.

Quiz

What is the purpose of the in-memory column store?

- a. Store any type of data
- b. Keep a duplicate of data in memory in a format different than the one in the buffer cache
- c. Keep a duplicate of data in memory when the buffer cache ages out blocks
- d. Compress data in memory just as it does on disk

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

Quiz

What happens when the in-memory column store runs low on free space?

- a. A query on an in-memory object results in an error.
- b. A warning or alert is issued by ADDM.
- c. A query on an in-memory object that could not be populated executes against the buffer cache instead of the in-memory column store.
- d. Portions of an in-memory object can be populated into space left in the in-memory column store.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: c, d

b is not true. ADDM is not aware of the In-Memory Column Store.

Query Benefits

Ideal for :

- Scanning large numbers of rows and applying filters
- Querying a subset of columns in a table
- Joining small tables (dimensions) to larger tables (facts)
 - Leverage repeated dimension values in fact table
- Aggregating data

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In-memory column store capability is ideal for queries that perform the operations like:

- Scanning large numbers of rows and applying filters like: =, <, >, in-lists
- Querying a subset of columns in a table, for example, select 5 of 100 columns in a table
- Joining small tables (dimensions) to larger tables (facts)
 - Leverage repeated dimension values in fact table
- Aggregating data

Testing and Comparing Query Performance

- Test a query on an in-memory table executed in IM column store:

```
SQL> SET timing ON
SQL> SELECT max(lo_ordtotalprice) most_expensive_order
FROM lineorder;
```



Elapsed: 00:00:01.80

Transparent for the application

- Re-execute the same query against the buffer cache:

```
SQL> ALTER SESSION SET INMEMORY_QUERY="DISABLE";
SQL> SELECT max(lo_ordtotalprice) most_expensive_order
FROM lineorder;
```



Elapsed: 00:12:21.90

Use the `INMEMORY_QUERY` session parameter to execute the query against the buffer cache.

ORACLE

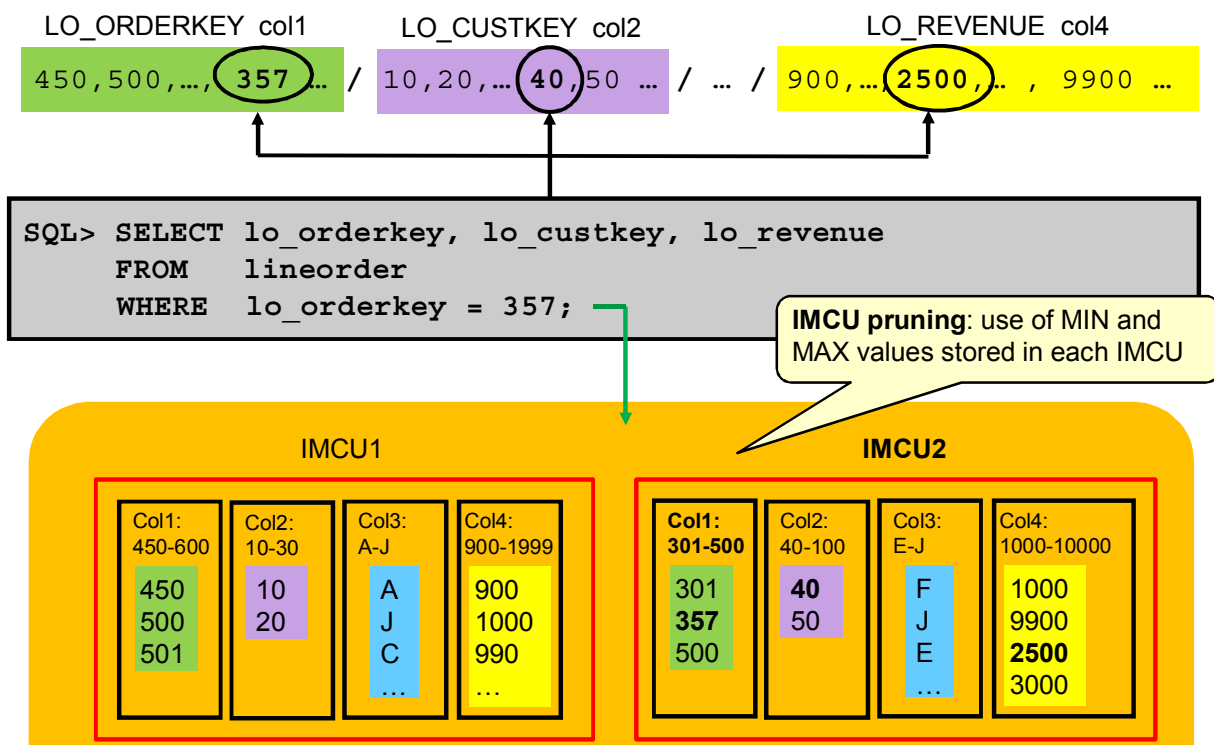
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The optimizer must be aware of when tables are in the columnar format and costs queries appropriately. Potentially the plans queries against in-memory objects can be very different than those for buffer cache queries.

The query in the slide executed extremely quickly in both cases (first and second examples) because this is purely an in-memory scan. However, the performance of the query against the in-memory column store is significantly faster than on the traditional buffer cache because the IM column store has to scan the single `LO_ORDTOTALPRICE` column only while the row store format of the buffer cache has to scan all of the columns in each of the rows until it reaches the `LO_ORDTOTALPRICE` column. The query also benefits from the excellent compression ratio and the fact that the columnar format requires no additional manipulation for join operations.

During the testing phase of workloads with and without in-memory, the user can enable or disable in-memory queries at the session or even at the system level by using the `INMEMORY_QUERY` parameter. The default is "ENABLE". The user sets `INMEMORY_QUERY` to "DISABLE" when the user wants to test the performance of the same query against the buffer cache.

Queries on In-Memory Tables: Simple Predicate



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The performance of queries with simple predicates against the IM column store is much better than the same query against the buffer cache because the IM column store has access to the MINIMUM and MAXIMUM values of each of the columns, for each IMCU.

The column in the WHERE predicate is compared to the MINIMUM and MAXIMUM range of values stored in each IMCU of the column, and if the value does not fall in the specified range, the IMCU is completely skipped. The IMCU pruning applies to queries using predicate filters on a single column such as equality, inequality (<>), comparison operators (<, <=, >, >=, NULL, NOT NULL, LIKE, SUBSTRING).

In the example of the slide, IMCU1 of the LO_ORDERKEY column is pruned out based on MINIMUM and MAXIMUM values and only IMCU2 of the same column is scanned.

```
SELECT COLUMN_NUMBER, MINIMUM_VALUE, MAXIMUM_VALUE
FROM   V$IM_COL_CU i, dba_objects o
WHERE  i.objd = o.data_object_id
AND    o.object_name = 'LINEORDER'
GROUP BY COLUMN_NUMBER, MINIMUM_VALUE, MAXIMUM_VALUE
ORDER BY column_number;
```

Note: Just like Exadata has storage indexes, the in-memory column store records the MINIMUM and MAXIMUM values for each of the IMCUs or extents.

MINMAX Pruning Statistics

Does MINMAX pruning occur?

MINMAX pruning occurs for WHERE clause with equality and non-equality.

```
SQL> SELECT display_name, value
       FROM v$mystat m, v$statname n
       WHERE m.statistic# = n.statistic#
       AND   display_name IN (
               'IM scan segments minmax eligible',
               'IM scan CUs pruned',
               'IM scan CUs optimized read',
               'IM scan CUs predicates optimized');

```

DISPLAY_NAME	VALUE
IM scan segments minmax eligible	250
IM scan CUs pruned	249
IM scan CUs optimized read	0
IM scan CUs predicates optimized	249

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Verify that minmax pruning occurred by looking at four of the IM column store session statistics.

- IM scan segments minmax eligible: Count of CUs that are eligible for minmax pruning.
- IM scan CUs optimized read: Count of CUS with all rows passing minmax comparison. This means that IMCUs had all rows matching the value of the WHERE clause.
- IM scan CUs predicates optimized: Count of predicates for which either all rows pass minmax comparison or no rows pass minmax comparison.
- IM scan CUs pruned: Count of CUs with no rows passing minmax comparison. This means that IMCUs were never scanned.

In the example in the slide, the "IM scan CUs pruned" and "IM scan CUs predicates optimized" statistics show 249. This means that 249 IMCUs did not contain any rows matching the value of LO_ORDERKEY of 357. Therefore only one CU had to be read to satisfy the query predicate.

IM Column Store Statistics

- Use V\$MYSTAT and V\$STATNAME: %IM% statistics.

```
SQL> SELECT display_name, value
FROM   v$mystat m, v$statname n
WHERE  m.statistic# = n.statistic#
AND    display_name IN (
        'session logical reads - IM',
        'IM scan rows', 'IM scan rows valid',
        'IM scan blocks cache',
        'IM scan CUs columns accessed' );
```

DISPLAY_NAME	VALUE
session logical reads - IM	3063604
IM scan rows	211307905
IM scan rows valid	879059
IM scan blocks cache	0
IM scan CUs columns accessed	3

Note: If the query is not executed in the IM column store, all statistics show 0.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are two ways to find out if a query performed against the IM column store and not against the buffer cache. View In-Memory session statistics or the execution plan.

The session level statistics report how the SQL was executed. Query the V\$MYSTAT and V\$STATNAME performance views. If all IM statistics show a value of 0, the query was executed against the buffer cache. Most of the statistics related to the IM column store begin with IM:

- session logical reads -IM: Count of the number of blocks scanned in an IMCU
- IM scan blocks cache: Count of blocks fetched from disk because they were on the IMCU fetch list
- IM scan rows: Count of rows scanned from an IMCU before applying valid bloom filtering
- IM scan rows valid: Count of rows scanned from an IMCU after applying valid bloom filtering
- IM scan CUs columns accessed: Count of columns accessed by the scan

In the example in the slide, the IM column store scan blocks cache statistic shows that the LINEORDER table was populated in IM column store and was not accessed from the buffer cache during the query. The LO_ORDERKEY, LO_CUSTKEY, LO_REVENUE columns only were queried in the example as the IM scan CU columns accessed states.

The V\$SEGMENT_STATISTICS view also relates IM statistics for each segment.

Execution Plan: TABLE ACCESS IN MEMORY FULL

Executed in IM column store or in buffer cache?

```
SQL> SELECT * FROM table(dbms_xplan.display_cursor());
```

Id	Operation	Time
...		
* 4	TABLE ACCESS INMEMORY FULL	LINEORDER
Predicate Information (identified by operation id):		
4	- inmemory("LO_ORDERKEY">=357) filter("LO_ORDERKEY">=357)	

* 4	TABLE ACCESS FULL	
4	- access (:Z>=:Z AND :Z<=:Z) filter("LO_ORDERKEY">=357)	

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In the cases of simple predicates on a single column, the optimizer chooses table scans unless there is a single column index available. The optimizer is likely to select the index. The optimizer will only favor the IM column store if there is no index or if there is only a multiple column index available.

The execution plan for the query executed in the slide 39 shows that the operation performed a TABLE ACCESS INMEMORY FULL. The addition of the keyword 'INMEMORY' into the TABLE ACCESS FULL operation is exactly the same as the addition of the word 'STORAGE' on an Exadata system.

The keyword 'INMEMORY' indicates that this operation is eligible to have some or all of the data returned from that step in the plan coming from the IM column store.

If a table is partially converted to a columnar format due to ongoing population operation or if there is a lack of space in the IM column store, queries are able to use that partial in-memory version and go to disk for the rest, rather than waiting for the entire table to be converted.

Queries on In-Memory Tables: Join

Query several in-memory tables:

```
SQL> SELECT SUM(lo_extendedprice * lo_discount) revenue
FROM   lineorder l, date_dim d
WHERE  l.lo_orderdate = d.d_datekey
AND    l.lo_discount BETWEEN 2 AND 3
AND    l.lo_quantity < 24
AND    d.d_date='December 24, 1996';
```



Elapsed: 00:00:00.28

```
SQL> SELECT SUM(lo_extendedprice * lo_discount) revenue
FROM   lineorder l, date_dim d
WHERE  l.lo_orderdate = d.d_datekey
AND    l.lo_discount BETWEEN 2 AND 3
AND    l.lo_quantity < 24
AND    d.d_date='December 24, 1996';
```



Elapsed: 00:02:28.85

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The IM column store has no problem executing a query with a join on several tables because it is able to take advantage of bloom filter joins. The performance of the first example is better than the one of the second example because it benefits from the column store representation of the columns. The first example executes the query in an inmemory enabled session whereas the second example executes the query in an inmemory disabled session.

A bloom filter join transforms a join to a filter that can be applied as part of the scan of the fact table. The join conversion is possible with the use of bloom filters. Bloom filters were originally introduced in Oracle Database 10g to enhance hash join performance by reducing the number of rows sent via a table queue between the producer slave set scanning a table and the consumer slave set performing the probe of a hash join. Bloom filters are not specific to Oracle Database In-Memory. However, they are very efficiently applied to columnar data. The basic idea is to avoid sending rows that are going to be discarded after the join because they do not match the join condition.

Bloom filters consist of two steps. In the first step, the bloom filter is created during the scan of the build side of the hash join. In the second step, the bloom filter is used during the scan of the probe side of the same hash join.

When two tables are joined via a hash join, the first table (typically the smaller table) is scanned and the rows that satisfy the `WHERE` clause predicates (for that table) are used to create a hash table. During the hash table creation, a bloom filter is also created based on the join column. The bloom filter is then sent as an additional predicate to the second table scan. After the `WHERE` clause predicates have been applied to the second table scan, the resulting rows will have their join column hashed and it will be compared to values in the bloom filter. If a match is found in the bloom filter that row will be sent to the hash join. If no match is found then the row will be disregarded.

Execution Plan: JOIN FILTER CREATE / USE

Id	Operation	Name
0	SELECT STATEMENT	
1	SORT AGGREGATE	
*	HASH JOIN	
3	JOIN FILTER CREATE	
*	TABLE ACCESS INMEMORY FULL	:BF0000
4	TABLE ACCESS INMEMORY FULL	DATE_DIM
5	JOIN FILTER USE	
*	TABLE ACCESS INMEMORY FULL	:BF0000
6	TABLE ACCESS INMEMORY FULL	LINEORDER


```

2 - access ("L"."LO_ORDERDATE"="D"."D_DATEKEY")
4 - inmemory ("D"."D_DATE"='December 24, 1996')
   filter ("D"."D_DATE"='December 24, 1996')
6 - inmemory (( "L"."LO_DISCOUNT"<=3 AND L"."LO_QUANTITY"<24
   AND "L"."LO_DISCOUNT">=2
   AND SYS_OP_BLOOM_FILTER (:BF0000, "L"."LO_ORDERDATE")))
  
```

If tables queried against buffer cache: **access**

If tables queried against buffer cache: **TABLE ACCESS FULL**

ORACLE

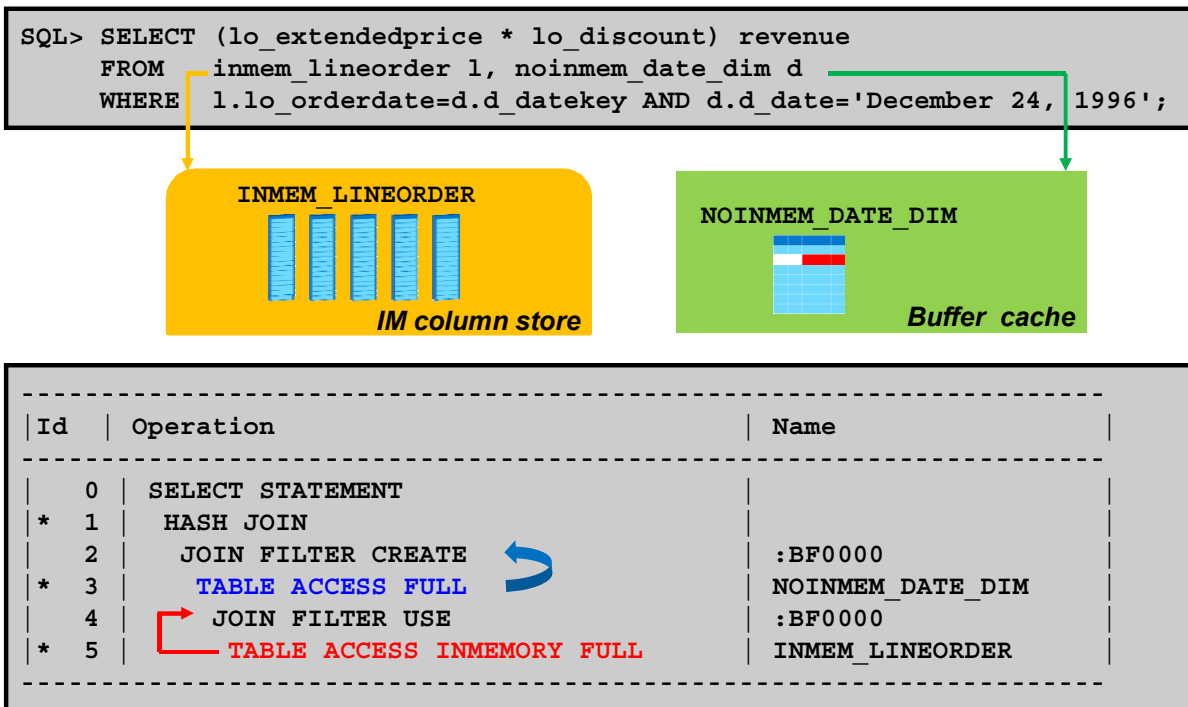
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Generate the execution plan of the query executed in the in-memory enabled session to identify bloom filters. A bloom filter appears in two places, at creation time and when it is used.

In the example of the slide, below are the execution steps:

- When the LINEORDER and DATE_DIM tables are joined via a hash join, the DATE_DIM table is scanned first (step 4). The rows that satisfy the WHERE clause predicate for that table (D_DATE='December 24, 1996') are used to create a hash table (step 2).
- During the hash table creation, a bloom filter (:BF0000) is also created based on the D_DATEKEY join column (step 3).
- The bloom filter is then sent as an additional predicate to the LINEORDER table scan (SYS_OP_BLOOM_FILTER (:BF0000, "L"."LO_ORDERDATE")).
 - The table is scanned to find the rows that satisfy both the WHERE clause predicates for that table (LO_DISCOUNT<=3 AND LO_QUANTITY<24 AND LO_DISCOUNT>=2) (step 6).
 - The resulting rows have their join column LO_ORDERDATE hashed and it is compared to D_DATEKEY values in the bloom filter (:BF0000) (step 5).
- If a match is found in the bloom filter that row is sent to the hash join. If no match is found then the row is disregarded.

Queries on In-Memory and Non-In-Memory Tables



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In some cases, only some segments in a query are enabled for IM column store. In this case, the optimizer would still choose an optimal plan that makes sense for both those portions of the query that can go to memory and those that must go to disk. A similar situation arises when not all of the table fits in memory.

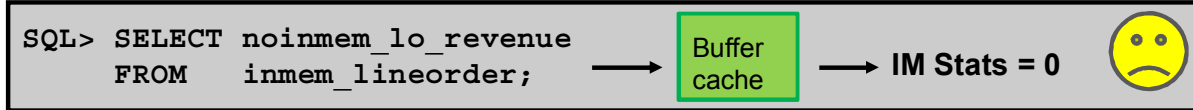
In the example of the slide, the optimizer chooses to work from both the buffer cache and the IM column store in different portions of the plan.

The execution steps are as follows:

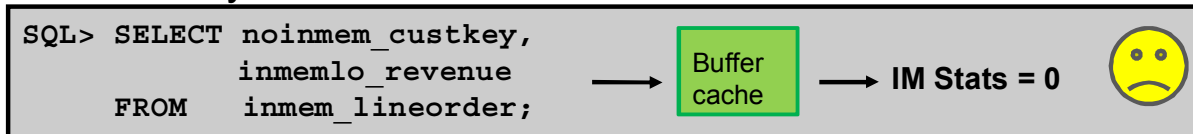
- When the INMEM_LINEORDER and NOINMEM_DATE_DIM tables are joined via a hash join, the NOINMEM_DATE_DIM table is scanned first from the buffer cache (step 3).
- The rows that satisfy the WHERE clause predicate for that table (D_DATE= 'December 24, 1996') are used to create a hash table (step 1).
- During the hash table creation, a bloom filter (:BF0000) is also created based on the D_DATEKEY join column (step 2).
- The bloom filter is then sent as an additional predicate to the INMEM_LINEORDER table scan (SYS_OP_BLOOM_FILTER (:BF0000, "L". "LO_ORDERDATE"). The table is scanned in IM column store (step 5).
- The resulting rows have their join column LO_ORDERDATE hashed and it is compared to D_DATEKEY values in the bloom filter (:BF0000) (step 4). If a match is found in the bloom filter that row is sent to the hash join. If no match is found then the row is disregarded.

Queries on In-Memory and Non-In-Memory Columns

- Query on non in-memory columns of an in-memory table:



- Query on non in-memory and in-memory columns of an in-memory table:



DISPLAY_NAME	VALUE
-----	-----
IM scan rows	0
IM scan rows valid	0
IM scan blocks cache	0
IM scan CUs columns accessed	0

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In some cases only some columns in a query are disabled for IM column store whereas the table is enabled for IM column store.

If a segment is defined as `INMEMORY` but has some columns defined as `NO INMEMORY`, the queries that select any non-in-memory columns are sent to the buffer cache as this is the case in the first and second examples in the slide and those selecting in-memory columns only are sent to the IM column store.

The IM column store statistics are all set to 0 in both examples.

DMLs and In-Memory Column Store

- In-Memory column store manages DML transactions.

```
SQL> SELECT display_name, value FROM v$mystat m, v$statname n
      WHERE m.statistic# = n.statistic#
      AND   display_name like 'IM transactions%';
```

DISPLAY_NAME	VALUE
IM transactions	1
IM transactions rows journaled	10224
...	

- In-Memory column store manages bulk loads.
 - Into the columnar representation of a table
 - Into the row format representation of the same table
 - If analytic indexes dropped 
 - If analytic indexes kept 
- Fully compatible with OLTP

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The IM column store manages any DML operation. The data in the IM column store remains transactionally consistent with disk format data. DML transactional operations are performed in memory as well as on disk and to provide read consistency. IM statistics provide information about the number of transactions performed against data in the IM column store and rows journaled into the In-Memory column store journal.

The IM column store is truly integrated into the database environment because it is able to handle both bulk data loads and online transaction processing. A direct-path load can populate the IM column store version along with loading the on-disk version. In this case, a background population performs under the covers. The bulk load operation does not return or even commit until the background population completes. If the population fails due to memory undersizing, the bulk load is committed so as to avoid breaking application semantics. The on-disk version is fully populated through the buffer cache whereas the IM column store representation of the populated table is partially populated. In the case of bulk load operations against data in the IM column store, there are no IM transactions statistic.

Scans are fast enough so that fewer indexes are necessary on OLTP tables. Primary key indexes still need to be maintained, but indexes that are less essential can be eliminated. This leads to faster DML operations because fewer indexes need to be updated.

Recommendations

The performance of queries improves as follows:

- IM column store and //
- IM column store only
- No IM column store but //
- No IM column store nor //



Possibly no tuning

Enabling IM column store means:

- Find how much memory is available for IM column store.
- Compute $\sum$ (Estimated size of objects) to load in IM column store.
- Do NOT set `DEFAULT INMEMORY PRIORITY xxx` at tablespace level.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When IM column store is enabled, the queries performance benefit when IM column store is coupled with parallelization. If auto DOP is enabled, depending on the number of CPUs, the optimizer is able to find the best parallel query execution plan. If auto DOP is disabled for any reason, the IM column store still provides a very high performance. The DBA does not have to tune the queries any more. When the IM column store is disabled, the volume of data to be queried affect the execution performance. Parallelization still helps to improve the queries performance. If both IM column store and parallelization are disabled, the SQL tuning is required.

When the DBA has to decide whether to enable IM column store or not, he or she has to answer the following questions:

- How much available memory he or she has to give for the IM column store?
- Which objects would benefit from the columnar format?
- How much space these objects once compressed will take in the IM column store?
- Which indexes can be removed after the IM column store would be enabled for objects?

Note: It is highly recommended not to set `DEFAULT INMEMORY PRIORITY` at the tablespace level if a `PRIORITY` other than `NONE` is specified. This IM column store setting is will apply to all future tables in the tablespace. All such tables are immediately queued for populating.

Views

- `V$IM_SEGMENTS` / `V$IM_USER_SEGMENTS` / `V$IM_SEGMENTS_DETAIL`: In-memory status of all / user segments
- `V$IM_COLUMN_LEVEL`: In-memory status of all columns of in-memory segments
- `V$IM_TBS_EXT_MAP`: In-memory extents of on-disk DBA ranges given TSN and data object ID
- `V$IM_SEG_EXT_MAP`: Mapped by TSN, on-disk DBA and data object number, this view list all in-memory extents.
- `V$IM_COL_CU`: Each row has information about in-memory column units for a particular column (including min and max values)

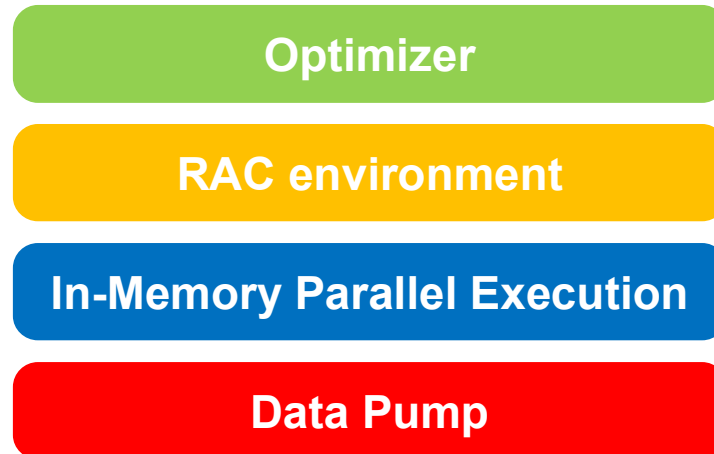
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Through the previous examples, you discovered a new `V$IM_SEGMENTS` view. Other new IM column store views have been added to monitor IM segments. Other views exist to provide additional information about the details of IM column store segments such as the extents, the IMCUs, and details about the global IM column store area.

- `V$IM_SEGMENTS`: In-memory status for all segments enabled for IM column store
- `V$IM_USER_SEGMENTS`: In-memory status for the user segments enabled for IM column store
- `V$IM_TBS_EXT_MAP`: Points to the in-memory extents for on-disk DBA ranges given TSN (TableSpace Number) and data object ID.
- `V$IM_SEG_EXT_MAP`: Mapped by TSN, on-disk DBA and data object number, this view will list all in-memory extents
- `V$IM_COL_CU`: Each row has information about in-memory column units for a particular column (including min and max values)
- `V$IM_SMU_HEAD`: Each row has information for a particular in-memory transactional snapshot metadata unit (SMU).
- `V$IM_SMU_CHUNK`: Each row has information for a particular in-memory transactional snapshot metadata unit (SMU) chunk.

Interaction with Other Products

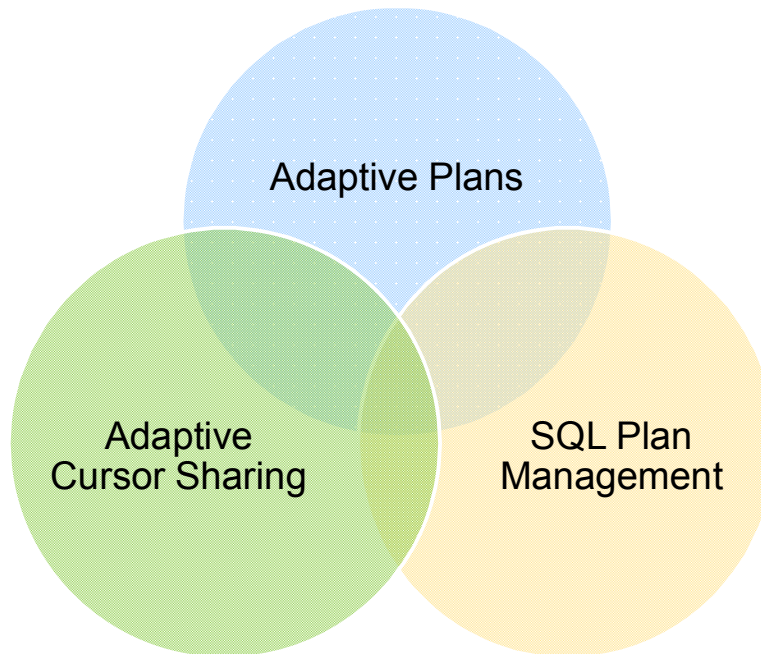


ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The In-Memory Column Store feature may interact with other products or features such as Real Application Clusters, In-Memory PX, Data Pump.

Optimizer

**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Adaptive Plans

Adaptive plans are a new feature in 12.1.0.1 allowing the optimizer to choose alternates for certain operations in the plan. Imagine the situation where one of the tables of the query is partially populated into IM column store or a populated table has some in-memory rows in row format in the transaction journal or one table is evicted from IM column store. These are only a few examples where the in-memory quotient is continuously changing. One possible solution to still improve the performance of such queries is to use adaptive plans instead of Adaptive Cursor Sharing. The adaptive plan has two access paths for each IM column store table, full table scan and another access path (typically index). At execution time, the current in-memory quotient is retrieved, compared to a threshold in-memory quotient stored in the adaptive plan mode to choose the appropriate access path. The plan is continuously adaptive.

SQL Plan Management

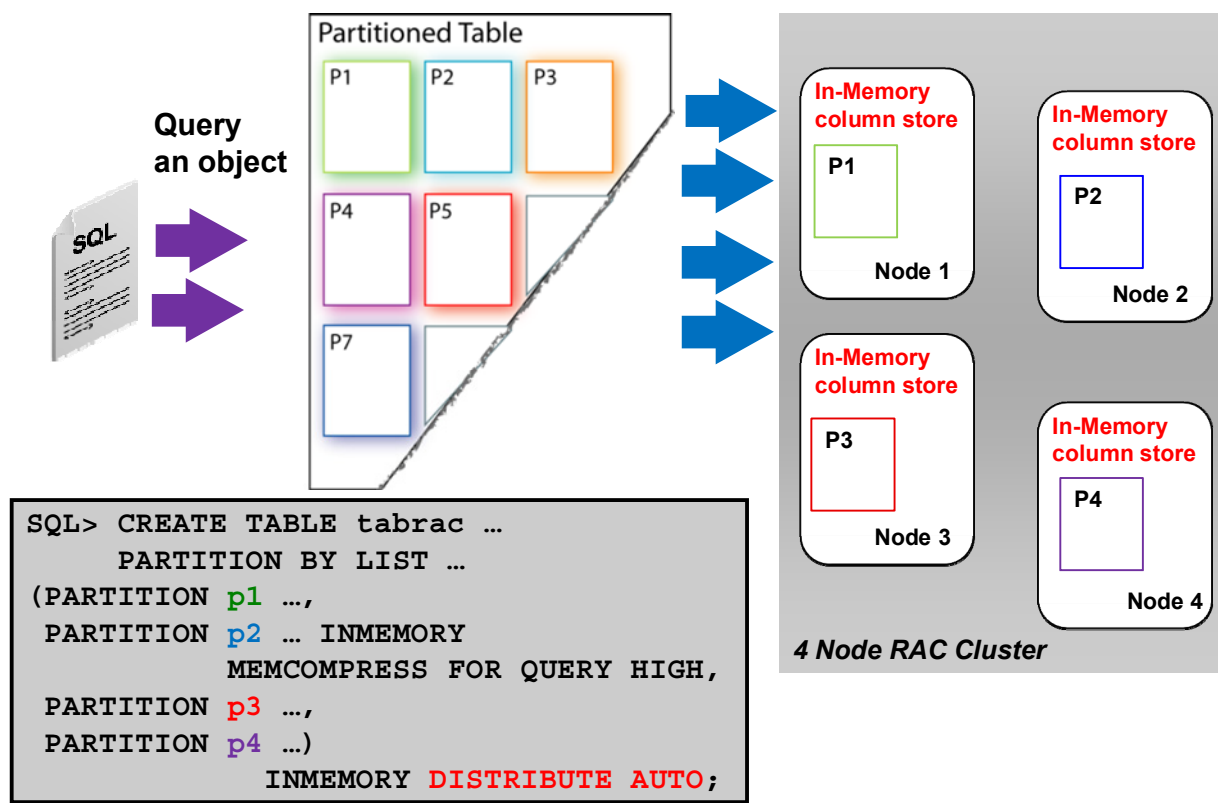
SQL plan baselines are compatible with IM column store queries. Tables may be disabled for IM column store and revert to on-disk access only. The currently accepted plans, which may be using `FULL TABLE IN MEMORY ACCESS` scans, may perform poorly for on-disk accesses. This is similar to scenarios where schema changes make existing plans sub-optimal. The reverse scenario can also happen. A SQL plan baseline may be created when the table is on-disk and later is enabled for IM column store. This is similar to cases where new access structures are created for tables. In both of these cases, during hard parse, the optimizer most likely finds new plans that reflect the current in-memory quotient of the tables. The automatic SPM evolve task will then verify and accept these plans if they are better than currently accepted plans.

Adaptive Cursor Sharing

ACS is fully compatible with IM column store. Consider an in-memory table configured for an on-demand load. The first query that uses a full table scan for the table triggers a background process to start loading the table into IM column store. During hard parse of the first query, regular cost functions for on-disk tables generate a plan accordingly. The cursor is shared while the background load proceeds. If the same query is executed multiple times while the initial load is still active, the same plan is shared but later executions are faster because more rows are in IM column store.

Once the initial load process completes, all dependent on that table are invalidated.

IM Column Store and RAC



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In a RAC database enabled for IM column store, each instance has its own IM column store area. The default behavior is that the Oracle Database automatically decides how to split tables among instances in the IM column stores. In a RAC configuration, you can specify for a given object (table or partition or subpartition) whether that object should be fully populated into each instance's in-memory column store, or whether that object should be distributed across each instance's in-memory column store. The latter approach makes sense when you have a table that is too big to fit into a single instance's in-memory column store, but it can fit in the aggregate space available across all the instances—and after you distribute a large table across RAC instances, in-memory parallel query will work extremely well in utilizing all of the CPU and memory resources available in the cluster for executing superfast queries on huge data sets.

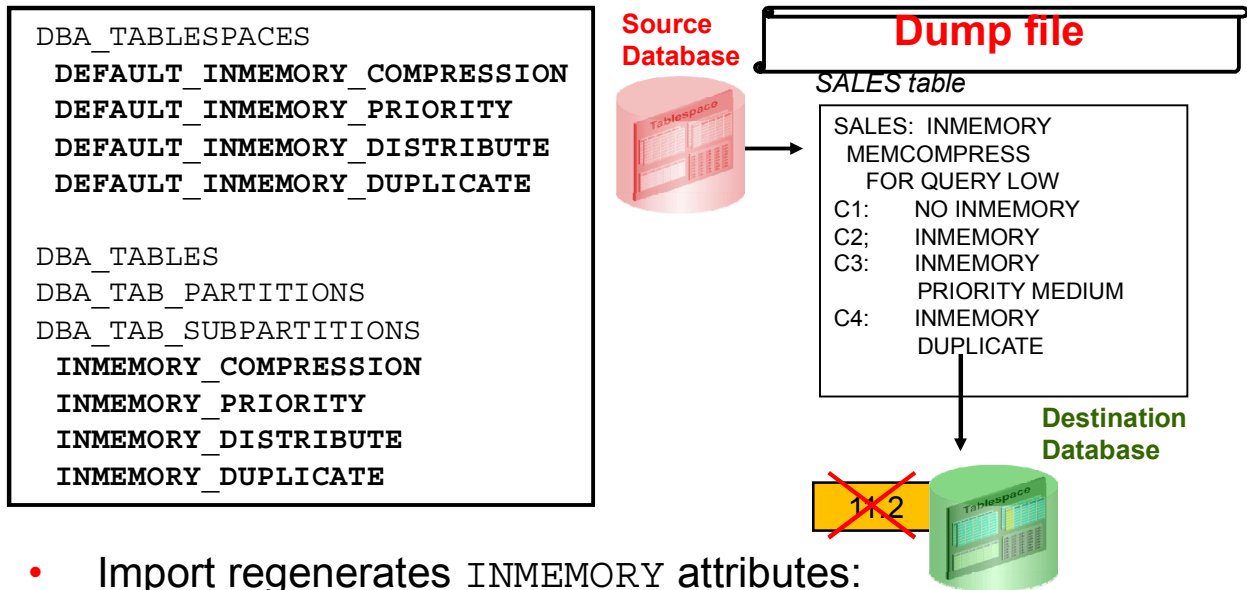
- The `DISTRIBUTE AUTO` attribute distributes portions of large tables across RAC instances. You can also distribute `BY ROWID RANGE`, `BY PARTITION` or `BY SUBPARTITION` to distribute partitions or subpartitions to different nodes. When the table is actually distributed among the nodes, there are two options for the `DUPLICATE` clause.
 - If you want one copy, use `NO DUPLICATE`.
 - If you want two copies across the cluster, use `DUPLICATE`.

- If a smaller table or a subset of partitions has to be duplicated across all RAC instances, so that the entire table, rows and columns, or a subset of partitions, rows and columns are available on every instance, the default `DUPLICATE ALL` attribute is used.

The cache fusion has been extended to ensure transaction consistency maintained across the nodes. In case of RAC row invalidations during DML transactions, the row invalidation occurs both in the buffer cache and in the IM column store to ensure transaction consistency between the buffer cache and the IM column store .

IM Column Store and Data Pump

- Export transports INMEMORY attributes:



- Import regenerates INMEMORY attributes:

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Data Pump export preserves the INMEMORY clause for all objects and selective columns that support it. When those objects are recreated at import time, the Data Pump import generates the INMEMORY clause that matches the setting for those objects at export time.

Note: The INMEMORY clause is not generated when importing into databases that are at version versions earlier than 12.1.

Data Pump TRANSFORM Names

- Data Pump import transports INMEMORY attributes by default.
- Importing INMEMORY objects **keeps/does not keep** the INMEMORY attribute:

```
$ impdp ... TRANSFORM=INMEMORY:Y|N
```

- Importing INMEMORY objects **transforms** the INMEMORY attributes:

```
$ impdp ... TRANSFORM=INMEMORY_CLAUSE:\
    "INMEMORY MEMCOMPRESS FOR CAPACITY HIGH
    PRIORITY MEDIUM\"
```

```
$ impdp ... TRANSFORM=INMEMORY_CLAUSE:\"NO INMEMORY\"
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Oracle Data Pump adds two new transform names to the TRANSFORM parameter. The TRANSFORM parameter is a way to tell Data Pump to modify the DDL generated during import.

- The first TRANSFORM name is INMEMORY and it has a value of N or Y. A value of N tells Oracle Data Pump to drop the INMEMORY clause from all objects that have the attribute assigned. A value of Y tells Oracle Data Pump to keep the INMEMORY clause, which is the default behavior. If there is no INMEMORY clause for an object that is stored in a tablespace, then the object inherits the INMEMORY clause for the tablespace. If a user is migrating a database and wants the new database to use INMEMORY features, the user could pre-create the tablespaces with the appropriate INMEMORY clause and then use TRANSFORM=INMEMORY:N on the IMPDP command. Without this parameter, the user would have to alter every object to add the appropriate INMEMORY clause.
- The second TRANSFORM name is INMEMORY_CLAUSE. The value is a string with a valid in-memory clause. When the user uses this TRANSFORM, Oracle Data Pump uses the contents of the string as the INMEMORY_CLAUSE for all objects being imported that have an in-memory clause in their DDL. This TRANSFORM is useful when you want to override the in-memory clause for the object in the dump file as shown in the second example in the slide. If the DBA wants to disable INMEMORY compression and make sure the table does not inherit the compression of the parent tablespace, the DBA would use the third example in the slide.

Note: Put options in a parameter file to avoid OS command line processing.

Summary

In this lesson, you should have learned how to:

- Explain the goals, benefits and architecture of the in-memory column store
- Deploy the in-memory column store in a database
- Show how queries and DMLs execution benefit from the in-memory column store
- Use new views and statistics to display in-memory column store usage
- Describe the interaction with other products and features

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 17: Overview

- 17-1: Configuring In-Memory Column Store
- 17-2: Configuring in-memory objects
- 17-3: Querying on in-memory objects
- 17-4: Exporting and importing in-memory objects
(*optional*)
- 17-5: Using In-Memory Column Store (*demons*)

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

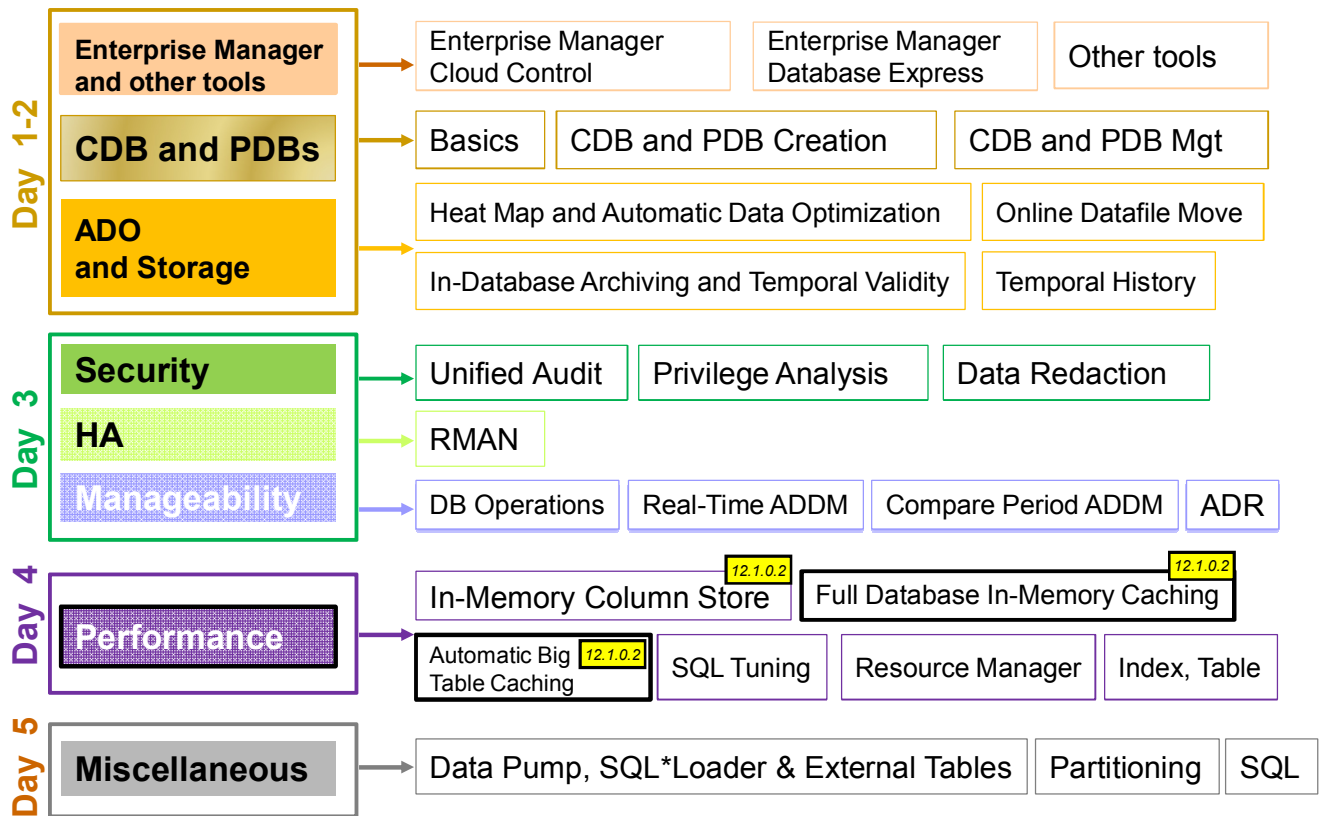
18

In-Memory Caching

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are other new In-Memory Caching features available with Oracle Database 12c R1 PS1 such as *Full Database In-Memory Caching* and *Automatic Big Table Caching*.

Objectives

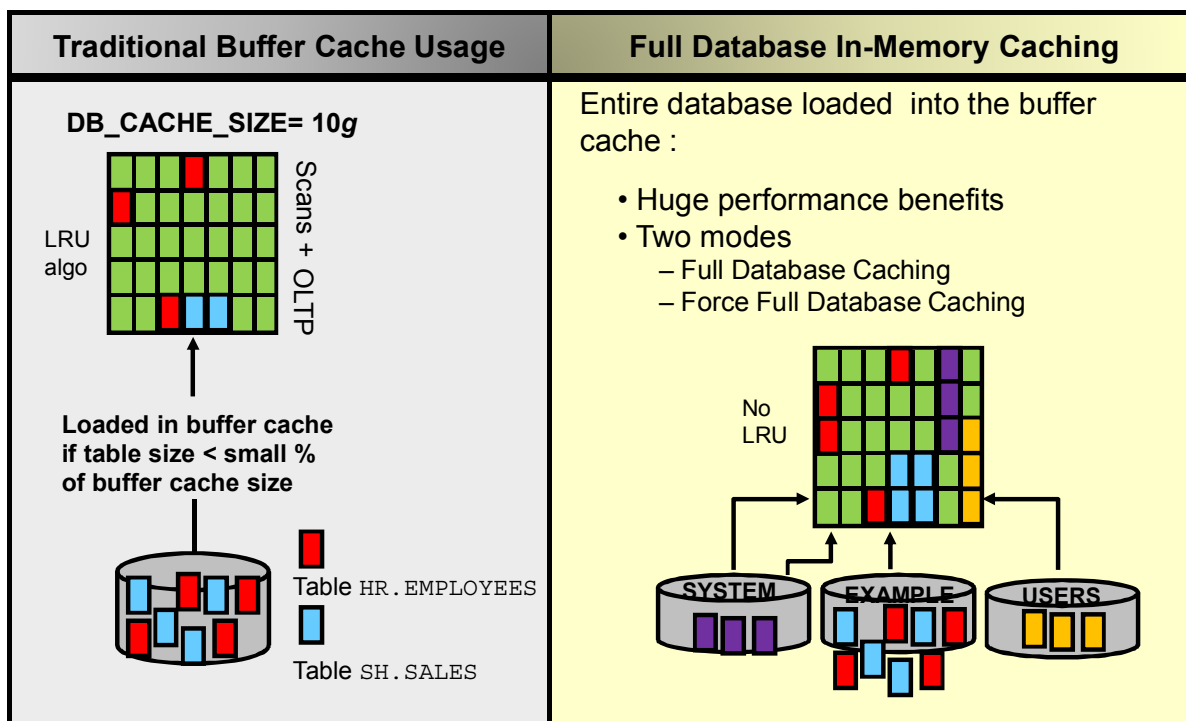
After completing this lesson, you should be able to:

- Describe the Full Database In-Memory Caching purpose
- Use the Full Database In-Memory Caching
- Describe the Automatic Big Table Caching purpose
- Explain the two buffer replacement algorithms
- Use the Automatic Big Table Caching

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Full Database In-Memory Caching 12.1.0.2



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

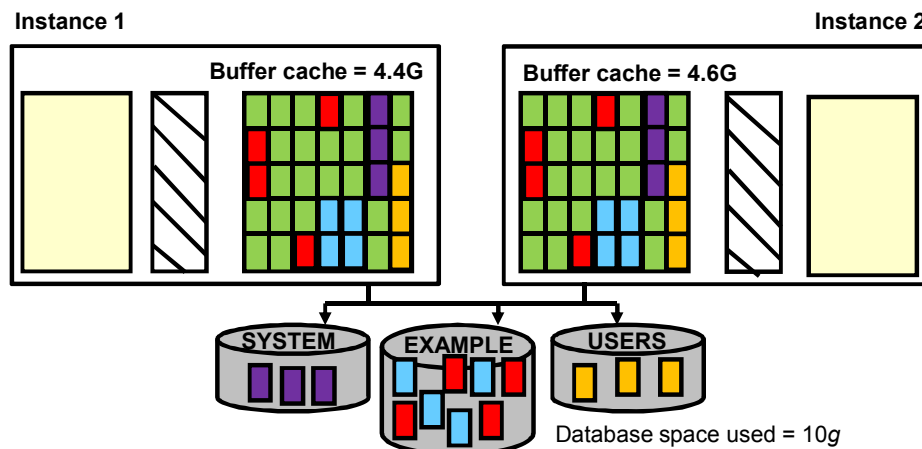
The current algorithm for table scans loads a table into the buffer cache only when the table size is less than a small percent of the buffer cache size. For very large tables, the database uses a direct path read, which loads blocks directly into the PGA and bypasses the SGA, to avoid flooding the buffer cache. The DBA has to explicitly declare small lookup tables, accessed frequently, as `CACHE` to load data into memory and avoid bypassing the SGA. This clause indicates that the blocks retrieved for these tables are placed at the most recently used end of the least recently used (LRU) list in the buffer cache when a full table scan is performed.

The Full Database In-memory Caching feature enables an entire database to be cached in memory when the database size (sum of all data files, `SYSTEM` tablespace, LOB `CACHE` files minus `SYSAUX`, `TEMP`) is smaller than the buffer cache size. Caching and running a database from memory leads to huge performance benefits. Two modes can be used:

- **Full Database Caching:** Implicit default and automatic mode in which an internal calculation determines if the database can be fully cached for an instance. `NOCACHE` LOBs are not cached in Full Database Caching but in Force Full Database Caching mode even `NOCACHE` LOBs are cached.

- **Force Full Database Caching:** Neither Full Database Caching nor Force Full Database Caching force/prefetch data into memory. Workload has to access the data first for them to be cached. It considers the entire database as eligible to be completely cached in the buffer cache. This mode requires the DBA to execute the `ALTER DATABASE FORCE FULL DATABASE CACHING` command. This mode takes precedence over Full Database Caching mode. To revert to traditional caching, use the `ALTER DATABASE NO FORCE FULL DATABASE CACHING` command.

Setting Up Force Full Database Caching



- Logical size of the database space used < each instance buffer cache
- Logical database size < 80% ($\sum$ buffer cache size of all instances)

```
SQL> STARTUP MOUNT
SQL> ALTER DATABASE FORCE FULL DATABASE CACHING;
SQL> SELECT force_full_db_caching FROM v$database;
```



Backup controlfile

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Different Configurations

- In a RAC environment, the DBA can use the Force Full Database Caching if any of the following conditions is met:
 - Logical database size (actually used space) is less than each instance private buffer cache.
 - For a well partitioned workload (by instance access), the logical database size is less than 80% of the combined buffer cache size of all instances.

The Force Full Database Caching mode impacts all instances of the RAC database or none.
- Primary and standby databases are independent in the Force Full Database Caching mode. There is no redo generated by the `ALTER DATABASE FORCE FULL DATABASE CACHING` command and the primary database does not trigger the standby into or out of this mode.
- In a multitenant configuration, the Force Full Database Caching mode applies to the entire multitenant container database (CDB), including all of its pluggable databases (PDBs).

Recommendations

The control files are not automatically backed up after an `ALTER DATABASE FORCE FULL DATABASE CACHING` command is completed. Therefore, it is highly recommended to back up the control files in case you would have to restore an old version of the control files.

To be certain about the status of the Force Full Database Caching mode after any control file restoration, check the `FORCE_FULL_DB_CACHING` column of `V$DATABASE` view.

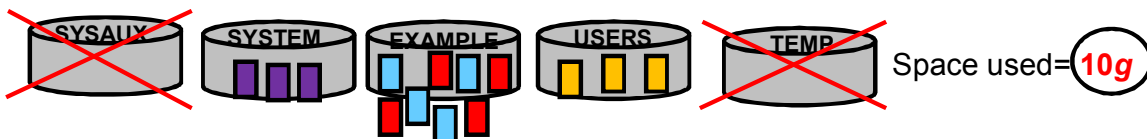
Monitoring Full Database In-Memory Caching

1. Database is elected for `FORCE_FULL_DB_CACHING`.
2. Is the database fully loaded into the buffer cache?
 - a. What is the current buffer cache size?

```
SQL> SELECT sum(cnum_set * blk_size) BC FROM X$KCBWDS;
```

12G

- b. How much is the stored data size?



Σ (All data files – SYSAUX data files – TEMP tempfiles)

- c. Set the `SGA_SIZE` or `DB_CACHE_SIZE` so that stored data size < 80% buffer cache:

```
SQL> ALTER SYSTEM SET SGA_TARGET = 13G SCOPE=SPFILE;
```

- d. Restart the instance.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

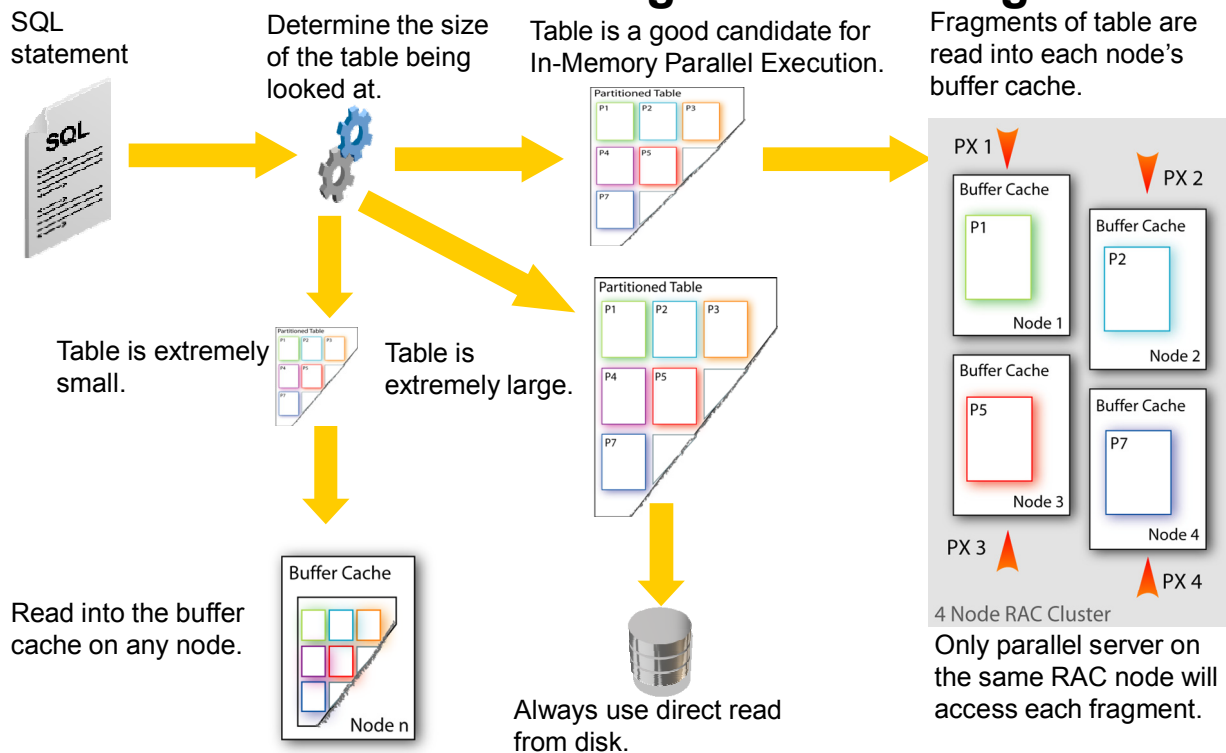
Monitoring the Full Database Caching means that you regularly check that the data stored in segments in data files can all be loaded into the buffer cache.

Compute how much buffer cache size is needed to load the database data, including all data from data files except the SYSAUX tablespace and temporary data stored in tempfiles.

1. Display the current cache size by using the `X$KCBWDS` view like in the first example in the slide.
2. Display the total number of bytes used by all segments in all data files except in the SYSAUX data files. The temporary segments stored in temporary tempfiles are excluded.


```
SQL> SELECT sum(bytes) Tot_size FROM dba_segments
        WHERE tablespace_name <> 'SYSAUX';
```
3. Verify that the 80% of the buffer cache size can load all application data.
4. If this is not the case, set the `SGA_TARGET` or `DB_CACHE_SIZE` to the appropriate value and restart the instance.

In-Memory Parallel Query Before Automatic Big Table Caching



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle 11g Release 2, queries can benefit from auto DOP when `PARALLEL_DEGREE_POLICY` is set to `AUTO`. Oracle automatically decides if a statement:

- Should execute in parallel or not and what degree of parallelism (DOP) it should use
- Can execute immediately or if it should be queued until more system resources are available
- Can take advantage of the aggregated cluster memory or not (In-Memory Parallel Query or In-Memory PX)

Oracle begins by determining if the working set (group of database blocks) necessary for a query fits into the aggregated buffer cache of the system.

- If the working set does not fit, the objects are accessed via direct path IO just as they were before. The object size should not exceed 80% of the buffer cache.
- If the working set fits into the aggregated buffer cache, the blocks are distributed among the nodes and the blocks are associated with that node.

In-Memory PX uses data of database segments (such as tables, indexes) cached on the buffer cache. It offers high-speed SQL execution environment eliminating the issue of storage performance.

If the target data is not cached yet, response time may be slower than direct path read because there can be some performance degradation due to the overhead to load data into the buffer cache from disks. After the target data is distributed and cached, In-Memory PX can scan all the data without accessing the storage system and yield to very good performance.

If PX deals with more data than the capacity of buffer size, it automatically detects it during generating SQL execution plan and changes to use direct path read instead.

When using partitioned tables, there is a greater tendency for the data to be cached in the buffer cache because partition pruning reduces the actual data size to be used. There is no need to scan the irrelevant partitions increasing the chances that the data fits within 80% of the buffer cache.

Only PX servers on the same RAC node will access each fragment of the object, all fragments being dispersed in all buffer caches. There is no copy of the missing partition from one instance to another.

Automatic Big Table Caching 12.1.0.2

- Improves in-memory PX performance for large objects with sizes that do not fit completely in the buffer cache
 - Eliminates disk reads for large objects
 - Facilitates efficient caching for large objects in data warehousing environment
 - Stores scanned objects in the big table cache instead of the buffer cache
 - Nonpartitioned tables
 - Partitions and subpartitions of partitioned tables
- Uses a different cache replacement algorithm

	Replacement Base	Replacement Level
9i, 10g, 11g, 12.1.0.1	Access-based	Block-level
12.1.0.2	Temperature-based	Object-level

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Automatic Big Table Caching enhances the in-memory parallel query capabilities of the Oracle Database in both single instance and Oracle RAC environments. An optional section of the buffer cache, called the big table cache, is used to store data for parallel table scans.

If a large table is approximately the size of the combined size of the Big Table Cache of all instances, the table is partitioned and cached, or mostly cached, on all instances. With in-memory PX, this could eliminate most disk reads for queries on the table, or the database could intelligently read from disk only for those portions of the table that do not fit in the Big Table Cache.

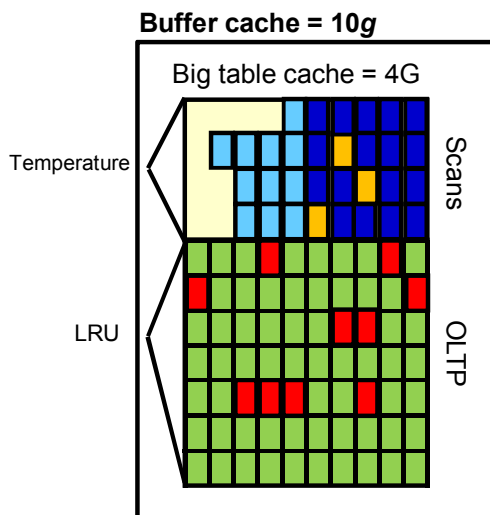
Parallel query scans that use automatic DOP can use a different cache replacement algorithm. When a table does not fit in memory, the database decides which buffers to cache based on access patterns. This provides efficient caching for large tables, even if they do not fully fit in the buffer cache. If the Big Table Cache cannot cache all the tables to be scanned, only the most frequently accessed table are cached, and the rest are read through direct read automatically.

The big table cache is integrated with the buffer cache and uses a temperature-based, object-level replacement algorithm to manage the big table cache contents, different from the access-based, block level LRU algorithm used by the buffer cache.

Configuring Automatic Big Table Caching

Configure a portion of the database buffer cache:

- `PARALLEL_DEGREE_POLICY=AUTO` or `ADAPTIVE`
- `DB_BIG_TABLE_CACHE_PERCENT_TARGET=40`



- Selects the "hot" objects:

- Table HR.EMPLOYEES : Hot object $\nearrow 30^\circ$
- Table SH.SALES : Hot object $\nearrow 80^\circ$
- Table SH.BIG : Hot object $\nearrow 40^\circ$

When space is required in the big table cache, the HR.EMPLOYEES table is replaced by a new object with a higher temperature.

- Avoids thrashing: Caches partial objects

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The performance of queries on big tables improves by:

- **Selectively caching the "hot" objects:** Each time an object is accessed, Oracle Database increments the object temperature. An object in the big table cache can be replaced only by another object whose temperature is higher than its own. In the example provided in the slide, when an object in the buffer cache or in the big table cache gets a higher temperature than the HR.EMPLOYEES table and there is no more space to load both objects, the HR.EMPLOYEES table is replaced by the hotter object.
- **Avoiding thrashing:** Partial objects are cached when objects cannot be fully cached. For example, if only 95% of a popular table fits in memory, then the database may choose to leave 5% of the blocks on disk rather than cyclically reading blocks into memory and evicting the least recently used ones.

To use Automatic Big Table Caching, you must enable the big table cache:

- Set `DB_BIG_TABLE_CACHE_PERCENT_TARGET` initialization parameter to a nonzero value. The value represents a percentage of the buffer cache. The default value is 0 (disabled) and the upper limit is 90(%) reserving at least 10% buffer cache for usage besides table scans.
- Set `PARALLEL_DEGREE_POLICY` is set to `AUTO` or `ADAPTIVE`.

- Ensure that Force Full Database Caching is disabled. Force Full Database Caching disables In-memory PX affinity, hence Automatic Big Table Caching. This instructs the database that the database is small enough to fit into every instance's memory and there is no need to partition and rather have every instance to have a copy of all database blocks.

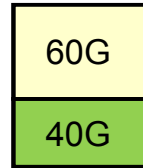
Using Automatic Big Table Caching

DB_CACHE_SIZE=100G

DB_BIG_TABLE_CACHE_PERCENT_TARGET=60

→ Large table scans use 60G.

→ OLTP uses 40G.



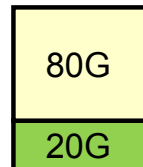
Workload changes → Increase big table cache

```
SQL> ALTER SYSTEM SET db_big_table_cache_percent_target=80;
```

DB_BIG_TABLE_CACHE_PERCENT_TARGET=80

→ Large table scans use 80G.

→ OLTP uses 20G.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When setting the DB_BIG_TABLE_CACHE_PERCENT_TARGET parameter, you should consider the workload mix: how much of the workload is for OLTP; insert, update, and random access; and how much of the workload involves table scans. Because data warehouse workloads often perform large table scans, you may consider giving Big Table cache section a higher percentage of buffer cache space for data warehouses.

This parameter can be dynamically changed if the workload changes. The change could take some time depending on the current workload to reach the target, because buffer cache memory might be actively used at the time.

Monitoring Automatic Big Table Caching

Following are the two views that provide more information about Automatic Big table Caching:

- `V$BT_SCAN_CACHE`
- `V$BT_SCAN_OBJ_TEMPS`

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Additional object-level statistics for a data warehouse load and scan buffers are added to represent the number of parallel query (PQ) scans on the object on the particular instance.

The `V$BT_SCAN_CACHE` view provides information about the Automatic Big Table Cache:

- **BT_CACHE_ALLOC:** Current ratio of the Big Table cache section to the buffer cache
- **BT_CACHE_TARGET:** Target ratio of the Big Table cache section to the buffer cache
- **OBJECT_COUNT:** Number of objects tracked by the Big Table cache section
- **MEMORY_BUF_ALLOC:** Number of memory buffers allocated by the Big Table cache section to objects
- **MIN_CACHED_TEMP:** Minimum temperature of the object currently cached by the Big Table cache section
- **CON_ID:** Identifier of the container ID if the database is a multitenant container database. If the database is a noncontainer database, the `CON_ID` is always 0.

The V\$BT_SCAN_OBJ_TEMPS view provides more information:

- **TS#:** Tablespace where the object resides
- **DATAOBJ#:** Data object number (objd)
- **SIZE_IN_BLKs:** Size of the object being scanned on this instance, in blocks
- **TEMPERATURE:** Temperature of this object
- **POLICY:** Caching policy of this object.
 - **MEM_ONLY:** This object will be fully cached in memory.
 - **MEM_PART:** This object will be partially cached in memory and some portion will remain on disk and will not be cached.
 - **DISK:** This object will not be cached in memory or flash for the scan at all.
- **CACHED_IN_MEM:** The number of blocks that are cached/allocated in memory for this object
- **CON_ID:** Identifier of the container ID if the database is a multitenant container database. If the database is a noncontainer database, the CON_ID is always 0.

Summary

In this lesson, you should have learned how to:

- Describe the Full Database In-Memory Caching feature
- Use the Full Database In-Memory Caching
- Describe the Automatic Big Table Caching feature
- Explain the two buffer replacement algorithms
- Use the Automatic Big Table Caching

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 18: Overview

- 18-1: Using Automatic Big Table Caching feature
- 18-2: Setting and Monitoring Force Full Database Caching

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The reason the practices do not follow the sequence of the two features introduced is that it allows skipping the recreation of a big table. The previous practice on In-Memory Column Store creates a very large table that Automatic Big Table Caching can reuse.

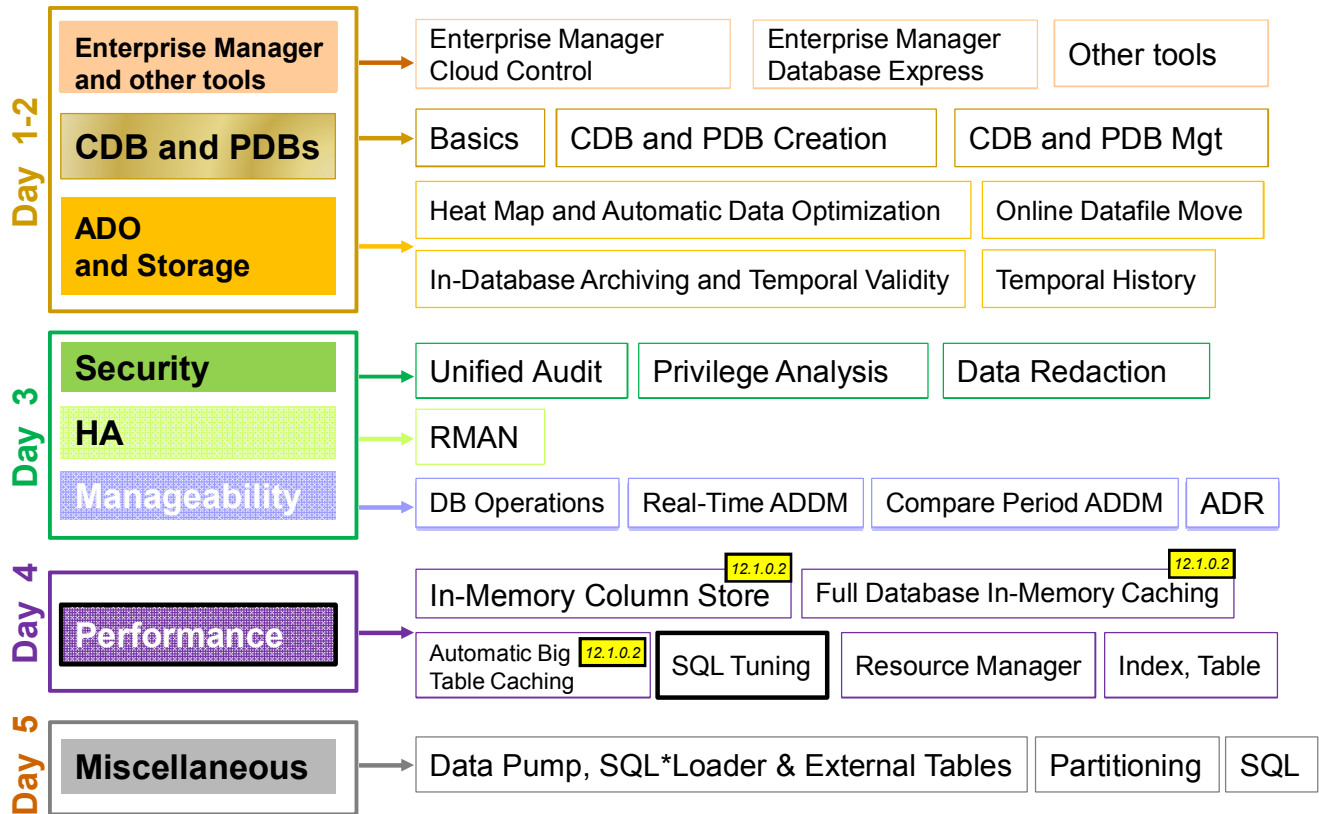
19

SQL Tuning Enhancements

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are several optimizer and statistics enhancements, and a new SQL plan directives feature.

Objectives

After completing this lesson, you should be able to:

- Describe Adaptive SQL Plan Management
- Describe enhancements to the SQL management base
- Use a dynamic plan to improve query performance
- Describe reoptimization for adaptive execution plans
- Use SQL plan directives to generate a better plan
- Describe statistics gathering performance improvements
- Use new histograms
- Describe enhancements to extended statistics and detect useful column groups for a specific workload
- Describe enhancements to dynamic sampling



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Refer to other sources of information:

- *Oracle Database 12c New Features Demo Series* demonstrations under Oracle Learning Library

Road Map

- Adaptive SQL Plan Management
- Adaptive Execution Plans
- Optimizer Statistics Management

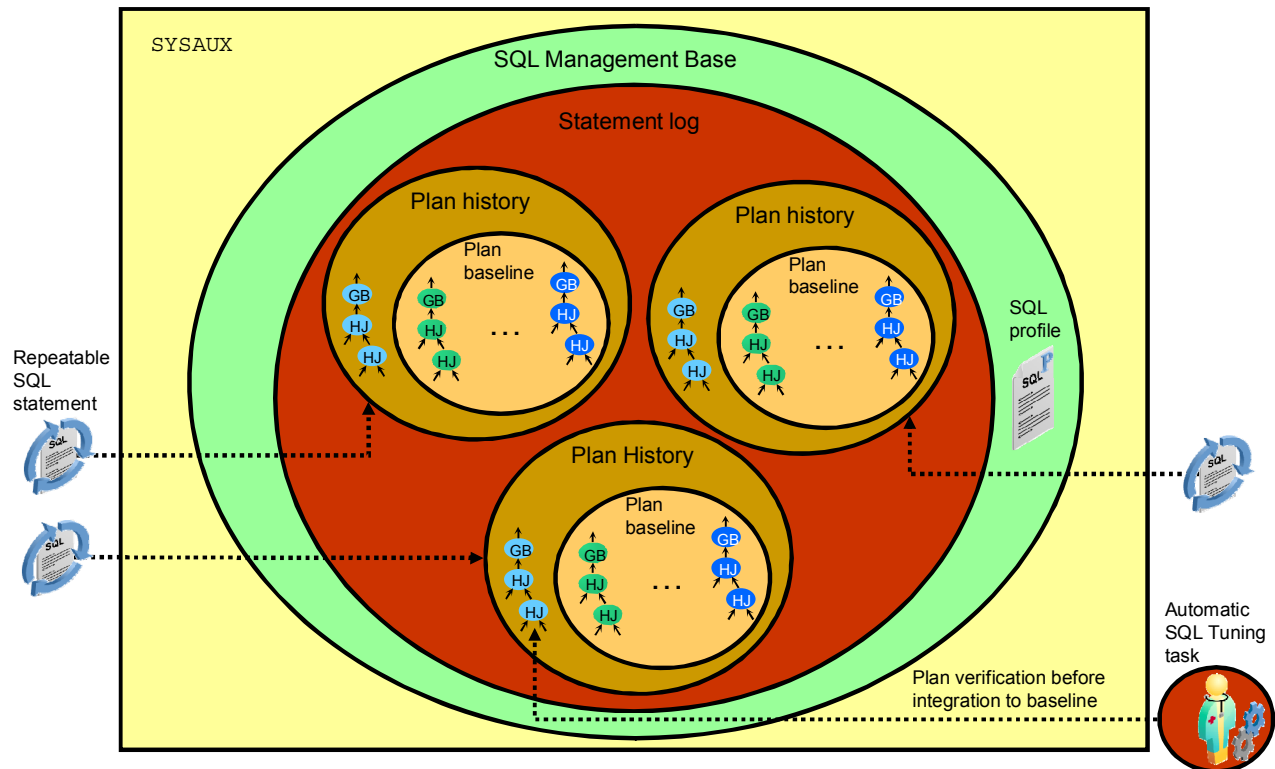
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

This lesson consists of three major parts:

- Adaptive SQL Plan Management
- Adaptive Execution Plans
- Optimizer Statistics Management

SQL Plan Baseline: Architecture



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The SQL Plan Management (SPM) feature introduces necessary infrastructure and services in support of plan maintenance and performance verification of new plans.

For SQL statements that are executed more than once, the optimizer maintains a history of plans for individual SQL statements. The optimizer recognizes a repeatable SQL statement by maintaining a statement log. A SQL statement is recognized as repeatable when it is parsed or executed again after it has been logged. After a SQL statement is recognized as repeatable, various plans generated by the optimizer are maintained as a plan history containing relevant information (such as SQL text, outline, bind variables, and compilation environment) that is used by the optimizer to reproduce an execution plan.

As an alternative or complement to the automatic recognition of repeatable SQL statements by setting the parameter `optimizer_capture_sql_plan_baselines` to `TRUE`, manual seeding of plans for a set of SQL statements is also supported.

A plan history contains different plans generated by the optimizer for a SQL statement over time. However, only some of the plans in the plan history may be accepted for use. For example, a new plan generated by the optimizer is not normally used until it has been verified not to cause a performance regression. Plan verification is done “out of the box” as part of Automatic SQL Tuning running as an automated task in a maintenance window.

An Automatic SQL Tuning task targets only high-load SQL statements. For them, it automatically implements actions such as making a successfully verified plan an accepted plan. A set of acceptable plans constitutes a SQL plan baseline. The very first plan generated for a SQL statement is obviously acceptable for use; therefore, it forms the original plan baseline. Any new plans subsequently found by the optimizer are part of the plan history but not part of the plan baseline initially.

The statement log, plan history, and plan baselines are stored in the SQL Management Base (SMB), which also contains SQL profiles. The SMB is part of the database dictionary and is stored in the `SYSAUX` tablespace. The SMB has automatic space management (for example, periodic purging of unused plans). You can configure the SMB to change the plan retention policy and set space size limits.

Note: If the database instance is up but the `SYSAUX` tablespace is `OFFLINE`, the optimizer is unable to access SQL management objects. This can affect the performance of some of the SQL workload.

SQL Plan Management: Overview

SQL Plan Management ensures that runtime performance never degrades due to the change of an execution plan.

Optimizer automatically manages "execution plans."

- Only known plans are used.
- Plans can be manually or automatically captured.

Plan changes are verified.

- Only better-performing plans are used.
- Manual verification is performed either through EM or by running `DBMS_SPM.EVOLVE_SQL_PLAN_BASELINES` in Oracle Database 11g.
- Automatic verification is performed by nightly Automatic SQL Tuning job running `DBMS_SPM.EVOLVE_SQL_PLAN_BASELINES` in Oracle Database 11g.

SPM consists of the following components:

- SQL plan baseline capture
- SQL plan baseline selection
- SQL plan baseline evolution

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

SQL Plan Management (SPM) ensures that runtime performance never degrades due to the change of an execution plan. It has two main objectives:

- It prevents performance regressions resulting from sudden changes to the execution plan of a SQL statement by providing components for capturing, selecting, and evolving SQL plan information.
- It offers performance improvements by adapting to database system changes.

With SPM, the optimizer automatically manages execution plans and ensures that only known or verified plans are used. When a new plan is found for a SQL statement, the plan is not used until the database verifies that its performance is comparable to or better than the current plans. Verification is a manual process performed either through Oracle Enterprise Manager (EM) or by running `DBMS_SPM.EVOLVE_SQL_PLAN_BASELINES` in Oracle Database 11g. Verification is an automatic process performed by the Automatic SQL Tuning (AST) job running `DBMS_SPM.EVOLVE_SQL_PLAN_BASELINES` in Oracle Database 11g.

The components of SPM are SQL plan baseline capture, SQL plan baseline selection and SQL plan baseline evolution. There are two ways to seed or populate a SQL management base (SMB) with execution plans:

- Automatic capture of execution plans (available starting with Oracle Database 11g)
- Bulk load execution plans or pre-existing SQL plan baselines

Adaptive SQL Plan Management

- The new evolve auto task runs in the nightly maintenance window.
- It ranks all nonaccepted plans and runs the evolve process for them. Newly found plans are ranked the highest.
- Poor performing plans are not retried for 30 days. After that, they are retired only if the statement is active.
- The new task is `SYS_AUTO_SPM_EVOLVE_TASK`.
- Information about the task is in `DBA_ADVISOR_TASKS`.
- Use `DBMS_SPM.REPORT_AUTO_EVOLVE_TASK` to view the results of the auto job.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

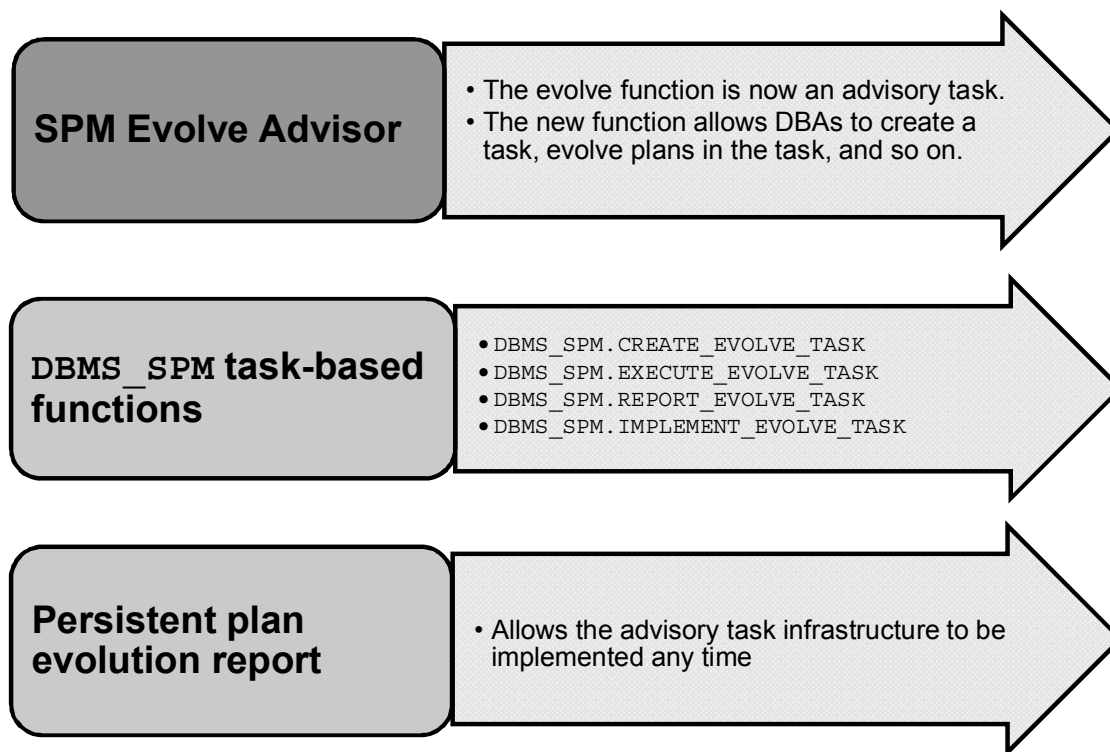
Starting with Oracle Database 12c, the database can use adaptive SPM. With adaptive SQL plan management, DBAs no longer have to manually run the verification or evolve process for non-accepted plans. They can go back days or weeks later and review what plans were evolved during each of the nightly maintenance window. When the SPM Evolve Advisor is enabled, it runs a verification or evolve process for all SQL statements that have non-accepted plans during the nightly maintenance window. All the non-accepted plans are ranked and the evolve process is run for them. The newly found plans are ranked the highest.

If the non-accepted plan performs better than the existing accepted plans in the SQL plan baseline, then the plan is automatically accepted and becomes usable by the optimizer.

After the verification is complete, a persistent report is generated detailing how the non-accepted plan performs compared to the accepted plan performance.

As the evolve process is now an AUTOTASK DBAs can also schedule their own evolve job at end time and use `DBMS_SPM.REPORT_AUTO_EVOLVE_TASK` to view the results of auto job.

Automatically Evolving SQL Plan Baseline



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

SPM Evolve Advisor is a SQL advisor that evolves plans that have recently been added to the SQL plan baseline. The advisor simplifies plan evolution by eliminating the manual chore of evolution.

New functions introduced in the DBMS_SPM package work on the advisory task infrastructure. The new functions such as DBMS_SPM.CREATE_EVOLVE_TASK, DBMS_SPM.EXECUTE_EVOLVE_TASK, and so on allow database administrators (DBAs) to create a task, evolve one or more plans in the task, fetch the report of the task, and implement the results of a task. The task can be executed multiple times, and the report of a given task can be fetched multiple times. SPM can schedule an evolve task as well as rerun an evolve task. There is also a function to implement the results of a task, which in this case would be to accept the currently non-accepted plans.

Using the task infrastructure allows the report of the evolution to be persistently stored in the advisory framework repository. It also allows Enterprise Manager to schedule SQL plan baselines evolution and fetch reports on demand. The new function supports HTML and XML reports in addition to TEXT. In Oracle Database 12c, the new task-based functions in DBMS_SPM retain the time_limit specification as a task parameter. It has the same default values as Oracle Database 11g. When ACCEPT_PLANS is true (default), SQL plan management automatically accepts all plans recommended by the task. When set to false, the task verifies the plans and generates a report if its findings, but does not evolve the plans. Users can view the report by using the appropriate function, and then execute a new implement function to accept the successful plans.

SQL Management Base Enhancements

- Prior to Oracle Database 12c, Pre-12c
`DBMS_XPLAN.DISPLAY_SQL_PLAN_BASELINE` displayed the execution plan by compiling the statement with the baseline.
- In Oracle Database 12c, when a new plan is added to the plan history of a SQL statement, plan rows are also stored in the SQL management base (SMB). When 12c
`DBMS_XPLAN.DISPLAY_SQL_PLAN_BASELINE` is executed, you fetch the plan data from the SMB.
- Easier diagnosability when plan cannot be reproduced

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle Database 11g, `DBMS_XPLAN.DISPLAY_SQL_PLAN_BASELINE` displayed the execution plan by compiling the statement with the baseline, mainly by using outlines. The SQL Plan Management (SPM) did not store new plans added to the log history until they were accepted. When customers upgrade to Oracle Database 12c, all plan rows are populated during execution of the plans.

Starting in Oracle Database 12c Release 1, the SMB stores the plans rows for new plans added to the plan history of a SQL statement. The `DBMS_XPLAN.DISPLAY_SQL_PLAN_BASELINE` function fetches and displays the plan from the SMB. For plans created before Oracle Database 12c Release 1 (12.1), the function must compile the SQL statement and generate the plan because the SMB does not store the rows.

Quiz

The `report_evolve_task` function is called to display the results of an evolve task.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: a

Lesson Road Map

- Adaptive SQL Plan Management
- Adaptive Execution Plans
- Optimizer Statistics Management

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Adaptive Execution Plans

- A query plan changes during execution because runtime conditions indicate that optimizer estimates are inaccurate.
- All adaptive execution plans rely on statistics that are collected during query execution.
- The two adaptive plan techniques are:
 - Dynamic plans
 - Reoptimization
- The database uses adaptive execution plans when `OPTIMIZER_FEATURES_ENABLE` is set to 12.1.0.1 or later, and `OPTIMIZER_ADAPTIVE_REPORTING_ONLY` initialization parameter is set to the default of `FALSE`.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Adaptive Execution Plans feature enables the optimizer to automatically adapt a poorly performing execution plan at run time and prevent a poor plan from being chosen on subsequent executions. The optimizer instruments its chosen plan so that at run time, it can be detected if the optimizer's estimates are not optimal. Then the plan can be automatically adapted to the actual conditions. An adaptive plan is a plan that changes after optimization when optimizer estimates prove inaccurate.

The optimizer can adapt plans based on statistics that are collected during statement execution. All adaptive mechanisms can execute a plan that differs from the plan which was originally determined during hard parse. This improves the ability of the query-processing engine (compilation and execution) to generate better execution plans.

The two Adaptive Plan techniques are:

- **Dynamic plans:** A dynamic plan chooses among subplans during statement execution. For dynamic plans, the optimizer must decide which subplans to include in a dynamic plan, which statistics to collect to choose a subplan, and thresholds for this choice.
- **Re-optimization:** In contrast, reoptimization changes a plan for executions after the current execution. For re-optimization, the optimizer must decide which statistics to collect at which points in a plan and when re-optimization is feasible.

Note: `OPTIMIZER_ADAPTIVE_REPORTING_ONLY` controls reporting-only mode for adaptive optimizations. When set to `TRUE`, adaptive optimizations run in reporting-only mode where the information required for an adaptive optimization is gathered, but no action is taken to change the plan.

Dynamic Plans

- The final decision is based on statistics that are collected during execution.
- Alternate subplans are precomputed and stored in the cursor.
- Statistic collectors are inserted at key points in the plan.
- If statistics prove to be out of range, subplans can be swapped.
- It requires buffering near the swap point to avoid returning rows to users.
- Only join methods and the distribution method can change.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A dynamic plan is an execution plan that has different built-in plan options. During the first execution, before a specific subplan becomes active, the optimizer makes a final decision about which option to use. The optimizer bases its choice on observations made during the execution up to this point. A dynamic plan enables the final plan for a statement to differ from the default plan, thereby potentially improving query performance.

A subplan is a portion of a plan that the optimizer can switch to as an alternative at run time.

During statement execution, the statistics collector buffers a portion of rows. Portions of the plan preceding the statistics collector can have alternate subplans, each of which is valid for a subset of possible values returned by the collector.

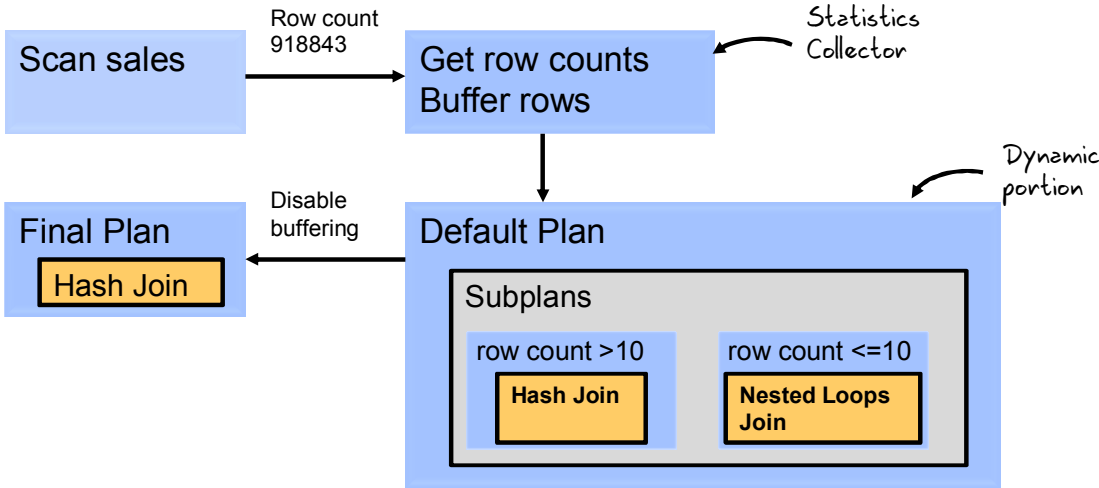
Often the set of values for a subplan is a range. If a statistic falls into the valid values of a subplan that was not the default plan, then the optimizer chooses the alternative subplan. After the optimizer chooses a subplan, buffering is disabled. The statistics collector stops collecting rows, and passes them through instead. On subsequent executions of the child cursor, the optimizer disables buffering, and chooses the same final plan.

With dynamic plans, the execution plan adapts to the optimizer's poor plan choices, and correct decisions can be made during the first execution.

Note: The new column in `V$SQL IS_RESOLVED_ADAPTIVE_PLAN` indicates if the final plan was not the default plan. Information found via dynamic plans is persisted as SQL plan directives.

Dynamic Plan: Adaptive Process

```
SQL> SELECT s.cust_id, c.cust_last_name
FROM   sh.sales s, sh.customers c
WHERE  s.cust_id = c.cust_id;
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The code example shows a join of the sales and customers tables. A dynamic plan for this statement could show two possible plans, one with a nested loop and the other with a hash join.

The following graphic shows the adaptive process. For the query given in the slide, the dynamic portion of the default plan contains two subplans, each of which uses a different join method. The collector retrieves row count statistics, and also buffers up to 10 rows.

If the row count is below 10, then the optimizer chooses the nested loop; otherwise, the optimizer chooses the hash join. Because the row count in sales table is over 10, the optimizer chooses a hash join, stops buffering, and makes the final plan.

Dynamic Plans: Example

A.1. The default plan show NLJ

```
SQL> explain plan for
select /*Q10*/ product_name
from order_items o, product_information p
where o.unit_price = 15
      and quantity > 1
      and p.product_id = o.product_id;

Explained.

SQL> select * from table(dbms_xplan.display(format=>'basic +note'));

PLAN_TABLE_OUTPUT
-----
Plan hash value: 384646879

-----
| Id | Operation                | Name                |
-----
| 0  | SELECT STATEMENT         |                     |
| 1  | NESTED LOOPS              |                     |
| 2  | NESTED LOOPS              |                     |
| 3  | TABLE ACCESS FULL       | ORDER_ITEMS         |
| 4  | INDEX UNIQUE SCAN        | PRODUCT_INFORMATION_PK |
| 5  | TABLE ACCESS BY INDEX ROWID | PRODUCT_INFORMATION |
-----
```

A.2. The final plan shows HJ

```
SQL> select /*Q10*/ product_name
from order_items o, product_information p
where o.unit_price = 15
      and quantity > 1
      and p.product_id = o.product_id;

PRODUCT_NAME
-----
Screws <B.28.S>

13 rows selected.

SQL> select * from table(dbms_xplan.display_cursor(format=>'basic +note'));

PLAN_TABLE_OUTPUT
-----
EXPLAINED SQL STATEMENT:
-----
select /*Q10*/ product_name from order_items o, product_information p
where o.unit_price = 15      and quantity > 1      and p.product_id =
o.product_id

Plan hash value: 2615131494

-----
| Id | Operation                | Name                |
-----
| 0  | SELECT STATEMENT         |                     |
| 1  | HASH JOIN                |                     |
| 2  | TABLE ACCESS FULL       | ORDER_ITEMS         |
| 3  | TABLE ACCESS FULL       | PRODUCT_INFORMATION |
-----
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A statement that has a dynamic plan could show two possible plans: one plan with a nested loop before execution and another with a hash join after execution. This information is displayed in the plan output table.

The Note section of the execution plan indicates whether the plan is dynamic: this is a dynamic plan.

Reoptimization: Statistics Feedback

- Statistics feedback (formerly known as Cardinality feedback) was introduced in Oracle Database 11g, Release 2.
- Statistics feedback is useful for queries where the data volume being processed is stable over time.
- During query execution optimizer estimates are compared to execution statistic: if they vary significantly then a new plan will be chosen for subsequent executions

Note: `CURSOR_SHARING` parameter is either `EXACT` or `FORCE`.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Statistics feedback (formerly known as cardinality feedback) was introduced in Oracle Database 11g, Release 2. This feature does a very limited form of re-optimization, after the first execution is completed. It automatically improves plans for queries for which the optimizer estimate improper cardinalities in the plan. The optimizer misestimates cardinalities for a variety of reasons, such as missing or inaccurate statistics, or complex predicates. Statistics feedback is not used for volatile tables. It is useful for queries where the data volume being processed is stable over time.

The optimizer may enable cardinality monitoring feedback for the cursor in the following cases: tables with no statistics, multiple conjunctive or disjunctive filter predicates on a table, and predicates containing complex operators for which the optimizer cannot accurately compute selectivity estimates.

The feature works as follows: During query execution, optimizer estimates are compared to execution statistics. If they vary significantly, a new plan is chosen for subsequent executions. Statistics are gathered for the actual data volume and data type seen during execution. If the original optimizer estimates vary significantly, the statement is hard parsed during the next execution by using the execution statistics. Statements are monitored only once. If they do not show significant differences initially, they do not change in the future. Only individual table cardinalities are examined. Information is stored only in the cursor and is lost if the cursor ages out.

Statistics Feedback: Monitoring Query Executions

```
SQL> SELECT /*+ gather_plan_statistics */ product_name
      FROM order_items o, product_information p
      WHERE o.unit_price = 15
      AND    quantity > 1
      AND    p.product_id = o.product_id;
```

Id	Operation	Name	Starts	E-Rows	A-Rows
0	SELECT STATEMENT		1	1	13
1	NESTED LOOPS		1	1	13
2	NESTED LOOPS		1	4	13
3	TABLE ACCESS FULL	ORDER_ITEMS	1	4	13
4	INDEX UNIQUE SCAN	PRODUCT_INFORMATION_PK	13	1	13
5	TABLE ACCESS BY INDEX ROWID	PRODUCT_INFORMATION	13	1	13

Predicate Information (identified by operation id):

3	- filter(("O"."UNIT_PRICE"=15 AND "QUANTITY">1))
4	- access("P"."PRODUCT_ID"="O"."PRODUCT_ID")

Initial cardinality estimate is more than 10X off.

Predicate Information

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Execution of the query given in the code box is monitored. The query has a combination of an equality filter predicate and an inequality filter predicate which causes the misestimation in the cardinality of the `order_items` table after these filters are applied. The optimizer chooses the given plan where you can see from the execution statistics (estimation: E-Rows and actual: A-Rows columns in the plan) that the cardinality of `order_items` is underestimated.

Statistics Feedback: Reparsing Statements

```
SQL> SELECT /*+ gather_plan_statistics */ product_name
FROM order_items o, product_information p
WHERE o.unit_price = 15
AND quantity > 1
AND p.product_id = o.product_id;
```

Id	Operation	Name	Starts	E-Rows	A-Rows
0	SELECT STATEMENT		1	13	13
1	HASH JOIN		1	13	13
2	TABLE ACCESS FULL	ORDER_ITEMS	1	13	13
3	TABLE ACCESS FULL	PRODUCT_INFORMATION	1	288	288

Predicate Information (identified by operation id):

1 - access("P"."PRODUCT_ID"="O"."PRODUCT_ID")
 2 - filter(("O"."UNIT_PRICE"=15 AND "QUANTITY">1))

Note

- cardinality feedback used for this statement

Cardinality estimates are now correct and trigger the join method to change.

Statistics feedback appears in the notes section of the plan.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Execution statistics are used to reparse the statement during the second execution of the query in the code box. During the second execution, the actual cardinality is fed back to the optimizer.

And this time the optimizer gets the cardinality estimate right, and the optimizer displays a different plan.

Automatic Reoptimization

- The optimizer automatically changes a plan during subsequent executions of a SQL statement.
- Join statistics are also included.
- Statements are continually monitored to see if statistics fluctuate over different executions.
- It works with adaptive cursor sharing for statements with bind variables.
- `IS_REOPTIMIZABLE` column is added in `V$SQL`.
- Information found at execution time is persisted as SQL plan directives.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In reoptimization, the optimizer adapts a plan only on subsequent statement executions. Deferred optimization can fix any suboptimal plan chosen because of incorrect optimizer estimates, such as a suboptimal distribution method or an incorrect choice of degree of parallelism. At the end of the first execution, the optimizer has a more complete set of statistics, and this information can improve plan selection in the future. A query may be reoptimized several times, each time creating a larger and more precise set of optimizer adjustments.

Reoptimization fixes plan problems that dynamic plans cannot solve. Whereas dynamic plans help change relatively local parts of a plan, dynamic plans are not feasible for some global plan changes. For example, a query with an inefficient join order might cause suboptimal performance, but it is not feasible to adapt the join order during execution. In these cases, the optimizer automatically considers re-optimization. For reoptimization, the optimizer must decide which statistics to collect and when.

As of Oracle Database 12c, join statistics are also included. Statements are continually monitored to verify if statistics fluctuate over different executions. It works with adaptive cursor sharing for statements with bind variables. This feature improves the ability of the query processing engine (compilation and execution) to generate better execution plans.

The optimizer uses the statistics in the following ways:

- The statistics collectors and their associated optimizer estimates determine if automatic reoptimization is an option. If statistics differ from estimates by more than a threshold, the optimizer looks for a replacement plan.
- After the optimizer identifies a query as a reoptimization candidate, the database submits all collected statistics to the optimizer.

Note: `IS_REOPTIMIZABLE` column shows whether the next execution matching this child cursor will trigger a reoptimization or if the child cursor contains reoptimization information, but will not trigger reoptimization because the cursor was compiled in reporting mode.

Quiz

A query plan changes during execution because runtime conditions indicate that optimizer estimates are inaccurate.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: a

Lesson Road Map

- Adaptive SQL Plan Management
- Adaptive Execution Plans
- **Optimizer Statistics Management**

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

SQL Plan Directives

- A SQL plan directive is additional information and instructions that the optimizer can use to generate a better plan:
 - Collect missing statistics.
 - Create column group statistics.
 - Perform dynamic sampling.
- Directives can be used for multiple statements:
 - Directives are collected on query expressions.
- They are persisted to disk in the `SYSAUX` tablespace.
- Directives are automatically maintained:
 - Created as needed during compilation or execution:
 - Missing statistics, cardinality misestimates
 - Purged if not used after a year

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

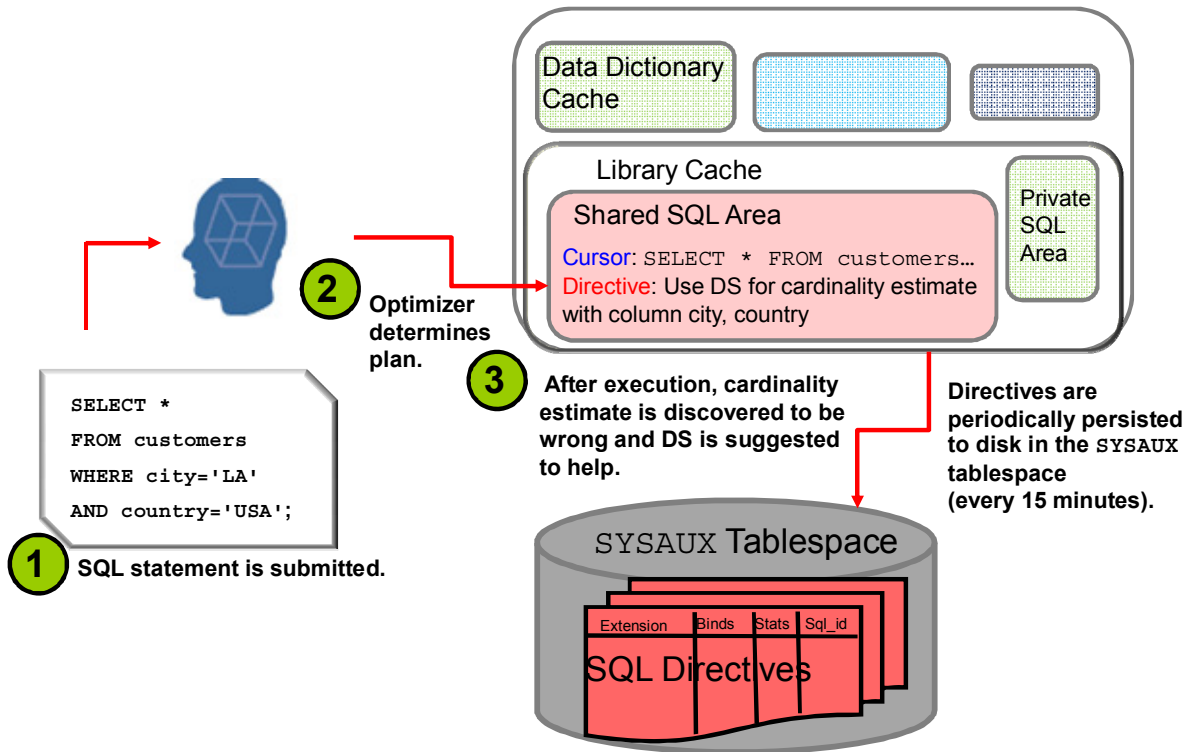
In previous releases, the database stored compilation and execution statistics in the shared SQL area, which is non-persistent. Starting in Oracle Database 12c, the database can use a SQL plan directive, which is additional information and instructions that the optimizer can use to generate a more optimal plan. For example, a SQL plan directive might instruct the optimizer to collect missing statistics, create column group statistics, or perform dynamic sampling. During SQL compilation or execution, the database analyzes the query expressions that are missing statistics and cause the optimizer to misestimate cardinality to create SQL plan directives. When the optimizer generates an execution plan, the directives give the optimizer additional information about objects that are referenced in the plan.

SQL plan directives are not tied to a specific SQL statement or `SQL ID`. The optimizer can use SQL plan directives for SQL statements that are nearly identical because SQL plan directives are defined on a query expression. For example, directives can help the optimizer with queries that use similar patterns, such as web-based queries that are the same except for a select list item. The database stores SQL plan directives persistently in the `SYSAUX` tablespace. When generating an execution plan, the optimizer can use SQL plan directives to obtain more information about the objects that are accessed in the plan.

Directives are automatically maintained, created as needed, purged if not used after a year.

Directives can be monitored in `DBA_SQL_PLAN_DIR_OBJECTS`. SQL plan directives improve plan accuracy by persisting both compilation and execution statistics in the `SYSAUX` tablespace, allowing them to be used by multiple SQL statements.

Creating SQL Plan Directives

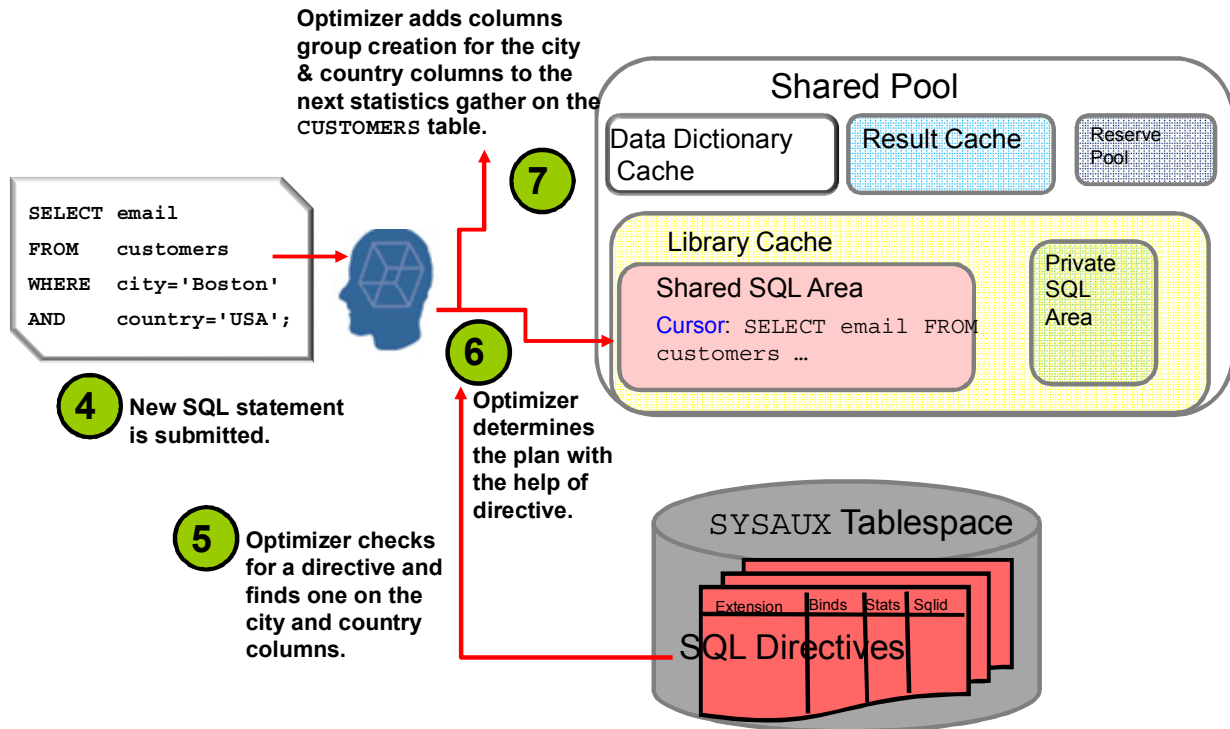


ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The slide shows how SQL Directives are created. The SQL statement is submitted, and then the optimizer determines the plan. After execution, when cardinality estimates are discovered to be wrong, dynamic sampling (DS) is suggested to help. Directives are periodically persisted to disk in the SYSAUX tablespace.

Using SQL Plan Directives



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When a new SQL statement is submitted, the optimizer checks for the directives and finds one on the city and country columns. After the optimizer gets the directive, it determines the plan with the help of the directive. The optimizer adds columns group creation for the city and country columns to the next statistics gather on the CUSTOMERS table.

SQL Plan Directives: Example

B.1. Manually Flush Directives (auto flush done every 15 minutes)

```
SQL> exec dbms_spd.flush_sql_plan_directive
```

PL/SQL procedure successfully completed.

B.2. Display Directives

```
SQL> col state format a5
SQL> col subobject_name format a11
SQL> col object_name format a13
SQL> select object_name, SUBOBJECT_NAME, type, state, reason from dba_sql_plan_directives d,
and object_name in ('ORDER_ITEMS', 'PRODUCT_INFORMATION');
```

OBJECT_NAME	SUBOBJECT_N	TYPE	STATE	REASON
ORDER_ITEMS	UNIT PRICE	DYNAMIC_SAMPLING NEW	SINGLE TABLE CARDINALITY MISESTIMATE	
ORDER_ITEMS	QUANTITY	DYNAMIC_SAMPLING NEW	SINGLE TABLE CARDINALITY MISESTIMATE	
ORDER_ITEMS		DYNAMIC_SAMPLING NEW	SINGLE TABLE CARDINALITY MISESTIMATE	

B.3. Use Directives

```
SQL> explain plan for
select /*Q10*/ product_name
from order_items o, product_information p
where o.unit_price = 15
and quantity > 1
and p.product_id = o.product_id;
```

Explained.

```
SQL> select * from table(dbms_xplan.display(format=>'basic +note'));
```

PLAN_TABLE OUTPUT

Plan hash value: 2615131494

Id	Operation	Name
0	SELECT STATEMENT	
1	HASH JOIN	
2	TABLE ACCESS FULL	ORDER_ITEMS
3	TABLE ACCESS FULL	PRODUCT_INFORMATION

Note

```
- dynamic sampling used for this statement (level=2)
- 1 Sql Plan Directive used for this statement
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

This slide shows SQL directives examples.

You can monitor SQL plan directives by using the DBA_SQL_PLAN_DIR_OBJECTS view.

The Notes section of the plan display functionality shows you information about SQL Plan Directives also, such as one SQL plan directive used for this statement.

Online Statistics Gathering for Bulk-Load

- In Oracle Database 12c, the database automatically gathers table statistics during the following types of bulk loads:
 - `CREATE TABLE AS SELECT`
 - `INSERT INTO ... SELECT` into an empty table by using a direct path insert.
- Statistics are available immediately after load.
- No additional table scan is required to gather statistics.
- All internal maintenance operations that use CTAS (MV refresh) will benefit.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

With online statistics gathering, statistics are automatically created as part of a bulk load operation such as a `CREATE TABLE AS SELECT` operation or an `INSERT INTO ... SELECT` operation on an empty table.

Online statistic gathering eliminates the necessity to manually gather statistics after a bulk data load has occurred. It behaves in a similar manner to the statistics gathering done during a `CREATE INDEX` or `REBUILD INDEX` command. All internal maintenance operations that use `CREATE TABLE AS SELECT` (CTAS) ((MV) Materialized View refresh) benefit. In a data warehouse, you frequently need to load a large amount of data into the database. For example, a sales record data warehouse might load sales data nightly where online statistics gathering is useful.

Oracle Database 12c now gathers the statistics automatically, and the principal benefits are improved performance and manageability of bulk load operations by eliminating user intervention to gather statistics after the load. And also by removing an additional full table scan required for separate statistics gathering operations.

While gathering online statistics, the database does not gather index statistics or histograms. If index statistics or histograms are required, then Oracle recommends running `DBMS_STATS.GATHER_TABLE_STATS` with the options parameter set to `GATHER AUTO` after the bulk load. In this case, `GATHER_TABLE_STATS` only gathers missing statistics, thus index statistics or histograms, and not table and column statistics collected during the bulk load.

Concurrent Statistics Enhancements in Oracle Database 12c

- Auto statistics gather job uses concurrency.
- Prevent concurrent statistics gathering to make the system overwhelmed through careful resource usage capping.
- Allow gathering index statistics concurrently.
- Allow gathering of statistics for multiple partitioned tables concurrently.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Prior to Oracle Database 12c, the automatic statistics gather job gathered statistics one segment at a time. It now uses concurrency.

In Oracle Database 12c, you can run multiple statistics gathering tasks concurrently, which may provide considerable improvements in statistics gathering time by increasing the resource utilization rate, in particular, on multi-CPU systems.

Employ smart scheduling mechanisms to have maximum degree of concurrency (to the extent that resources are available), for example, allow gathering of statistics for multiple partitioned tables concurrently.

Statistics for Global Temporary Tables

- In Oracle Database 12c, global temporary tables can have a different set of optimizer statistics for each session.
- `GLOBAL_TEMP_TABLE_STATS` preference of the `DBMS_STATS` package controls whether to gather session or shared statistics for global temporary tables.
- `DBMS_STATS` commits changes to session-specific global temporary tables, but not to transaction-specific global temporary tables.
- The cursor using the session-specific statistics is not shared by other sessions.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A global temporary table is a special table that stores intermediate session-private data for a specific duration. Prior to Oracle Database 12c, like for permanent tables, only one set of statistics was available for all occurrences of global temporary tables.

Starting with Oracle Database 12c, you can maintain session-private statistics for global temporary tables. You can choose to gather statistics for a global temporary table in one session and use the statistics only for this session. Meanwhile, you can continue to maintain a shared version of the statistics that is usable by all other sessions which have not yet gathered session-private statistics. Session level DML monitoring is also provided to the global temporary tables. Session-specific statistics for global temporary tables allow queries to be optimized more accurately based on your own data in a global temporary table.

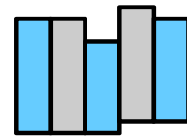
Session-specific statistics have the following characteristics:

- Dictionary views that track statistics, such as `DBA_TAB_STATISTICS`, `DBA_IND_STATISTICS`, `DBA_TAB_HISTOGRAMS`, and so on have the `SCOPE` column to indicate whether the statistics are `SHARED` or `SESSION`-specific.
- Session-specific statistics reside only in memory and are never persisted to disk. The database automatically deletes the statistics when the session is exited.

- You can set the `GLOBAL_TEMP_TABLE_STATS` preference of the `DBMS_STATS` package to controls whether to gather session or shared statistics for global temporary tables. `SESSION` is the default.
- `DBMS_STATS` commits changes to session-specific global temporary tables, but not to transaction-specific global temporary tables so that data in transaction-specific global temporary tables is not lost.

Histogram Enhancements

- A histogram is a structure that represents the distribution of data for each column in a table.
- The optimizer uses histograms to compute the selectivity of filter and join predicates.
- In Oracle Database 12c, two new types of histograms are introduced:
 - Top Frequency histograms
 - Hybrid histograms
- With sampling set to `AUTO`, only frequency and hybrid histograms are created.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Histograms represent the data distribution in a table column. Oracle Database creates histograms on columns that have a data skew, to improve cardinality estimates.

In Oracle Database 12c, two new types of histograms have been introduced for columns, which have more than 254 distinct values to improve the cardinality estimates generated via histograms:

- Top Frequency histograms
- Hybrid histograms

These new histograms have a more accurate frequency of endpoint values, and enable the optimizer to choose better plans.

If Oracle Database 12c creates new histograms, and if the sampling percentage is `AUTO`, they are either frequency or hybrid, but not height-balanced.

Note: If you upgrade the database from Oracle Database 11g to 12c, any height-based histograms created before the upgrade remains in use.

Top Frequency Histograms

- Traditionally a frequency histogram is created only if NDV is less than the number of histogram buckets specified.
- Top frequency histogram created if:
 - Sampling percentage set to AUTO
 - Column contains more than *nb* distinct values
 - Top *nb* frequent values percentage is above *p*% of rows
- A Top Frequency histogram provides more accurate cardinality estimates.
- A Top Frequency histogram is indicated by the TOP-FREQUENCY value in the DBA_TAB_COL_STATISTICS.HISTOGRAM column.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Prior to Oracle Database 12c, a frequency histogram was created only if the number of distinct values (NDV) for the column was less than the number of histogram buckets specified (up to 254). With Oracle Database 12c, if a column contains more than 254 distinct values, and if a small number of distinct values occupy more than 99% of the rows, the database creates a Top Frequency histogram by using the small number of extremely popular distinct values. A Top Frequency histogram can produce a better histogram for highly popular values by ignoring statistically insignificant unpopular values.

A Top Frequency histogram is indicated by the TOP-FREQUENCY value in the DBA_TAB_COL_STATISTICS.HISTOGRAM column.

Starting in Oracle Database 12c, the database creates Top Frequency histograms when the following conditions are true:

- The sampling percentage is AUTO. In this case, the database constructs frequency histograms from a full table scan. For all other sampling percentage specifications, the database derives frequency histograms from a sample.
- The number of distinct values in the column is greater than the number of buckets (*nb*) created or defaulted.
- The percentage of rows occupied by the top *nb* frequent values is equal to or greater than threshold *p*, where *p* is $(1-(1/nb))*100$.

Note: Starting in Oracle Database 12c, the database constructs frequency histograms from a full table scan when the sampling size is AUTO (which is the default).

Hybrid Histograms

- A hybrid histogram combines characteristics of both height-based histograms and frequency histograms.
- It stores the endpoint repeat count value, which is the number of times the endpoint value is repeated, for each endpoint in the histogram.
- In Oracle Database 12c, the database creates hybrid histograms when the following conditions are true:
 - The sampling percentage is `AUTO`.
 - The criteria for top frequency histograms do not apply.
 - NDV is greater than *nb*, where *nb* is the user-specified number of buckets. If no number is specified, *nb* defaults to 254.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A hybrid histogram combines characteristics of both height-based histograms and frequency histograms.

The height-based histogram sometimes produces inaccurate estimates. For example, a value that occurs as an endpoint value of only one bucket but almost occupies two buckets is not considered as popular.

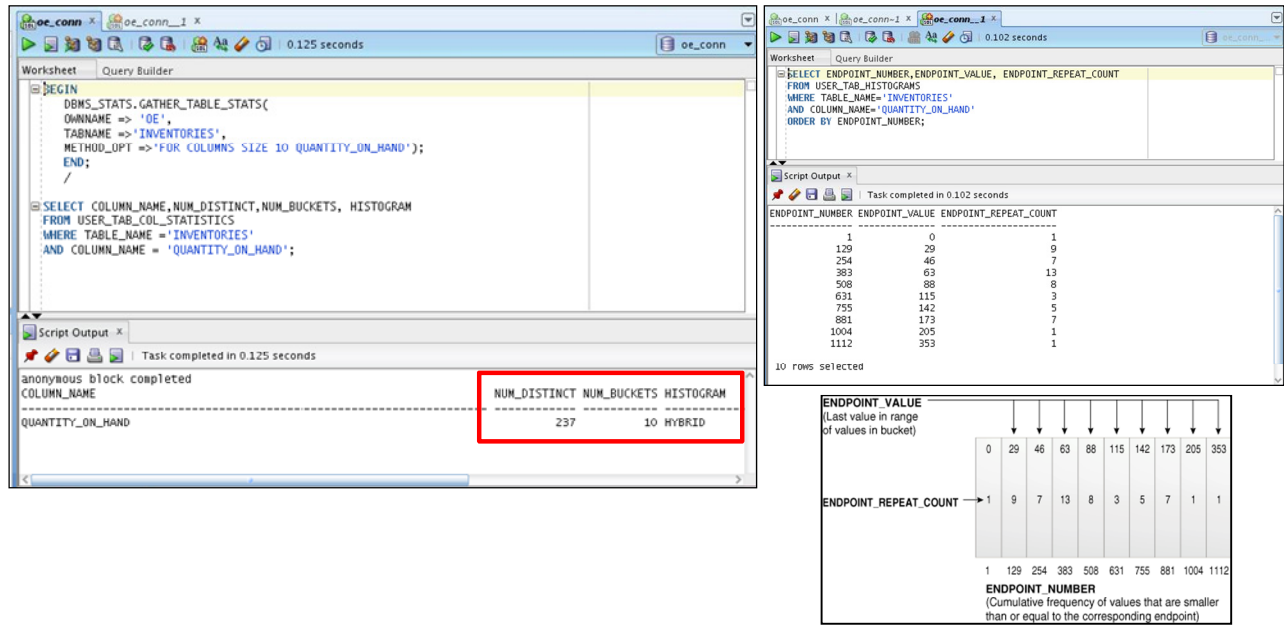
To solve this problem, a hybrid histogram stores the endpoint repeat count value and the cumulative frequency of values that are smaller than the corresponding endpoint value for each bucket in the histogram. By using these two pieces of information, the optimizer can obtain accurate estimates for almost popular values.

In Oracle Database 12c, the database creates hybrid histograms when the following conditions are true:

- The sampling percentage is `AUTO`.
- The criteria for top frequency histograms do not apply.
- NDV is greater than *nb* where *nb* is the user-specified number of buckets or the default of 254.

Hybrid Histograms: Example

You can view hybrid histograms by using the USER_TAB_COL_HISTOGRAMS table.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In the example in the slide, the `oe.inventories.quantity_on_hand` column has 237 distinct values and 1112 rows. The histogram distributes the data into 10 buckets, each with roughly the same number of rows. In query output, one row corresponds to each bucket in the histogram. Oracle Database added a special 0 bucket to this histogram because the value in the first bucket (29) is not the minimum value for the `quantity_on_hand` column. The 0 bucket holds the minimum value of 0 for `quantity_on_hand`.

The graphic represents the hybrid histogram for `inventories.quantity_on_hand`. The x-axis shows the endpoint (bucket) numbers, which represent the cumulative frequency of the associated values. The diagram shows that 63 is the value at the right border of the fourth bucket, and that this value is repeated 13 times in the bucket.

Extended Statistics Enhancements

- `DBMS_STATS` enables you to identify and create extended statistics.
- There are two types of extended statistics:
 - Column group statistics
 - Expression statistics
- Extended statistics are automatically maintained when statistics are gathered on the table.
- Oracle Database 12c can detect and create the necessary column groups for you based on a workload.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Extended statistics allow statistics to be gathered on a group of columns within a table. `DBMS_STATS` enables you to identify and create extended statistics, which are statistics that can improve cardinality estimates when multiple predicates exist on different columns of a table.

There are two types of extended statistics:

- Column group statistics are useful when multiple columns from the same table are used in `WHERE` clause predicates.
- Expression statistics are useful when a column is used as part of a complex expression in `WHERE` clause predicates.

Starting in this release, Oracle Database can determine which column groups are required in a given workload or SQL tuning set (STS), and then can create the associated column groups. Thus, for any given workload, you no longer need to know which columns from each table will be used together. By monitoring a workload, Oracle Database 12c records the necessary column groups information.

Subsequent slides show you how to detect useful column groups for a specific workload.

Capturing Column Group Usage

Enable workload monitoring



```
SQL> CONNECT / AS SYSDBA
Connected.
SQL> -- Switch on seed column usage for 300 seconds
SQL> EXEC dbms_stats.seed_col_usage(null, null, 300)
PL/SQL procedure successfully completed.
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In an Oracle SQL*Plus session, connect as SYS and run the PL/SQL program shown in the slide to enable monitoring for 300 seconds (the first argument refers to SQLSET_NAME and the second to OWNER_NAME).

The new SEED_COL_USAGE procedure iterates over the SQL statements in the specified SQL Tuning Set (here, all the statements that will be executed in the system for the next 300 seconds), compiles them, and then seeds column usage information in the data dictionary for the columns that appear in these statements.

Capturing Column Group Usage: Running the Workload

```
SQL> EXPLAIN PLAN FOR
2 SELECT *
3 FROM customers
4 WHERE cust_city='Los Angeles'
5 AND cust_state_province='CA'
6 AND country_id=52790;
Explained.
```

PLAN_TABLE_OUTPUT

Plan hash value: 2008213504

Id	Operation	Name	Rows
0	SELECT STATEMENT		1
1	TABLE ACCESS FULL	CUSTOMERS	1

8 rows selected.

Actual number of rows returned by the 1st query is 932. Optimizer underestimates because it assumes each predicate will reduce the number of rows.

```
SQL> EXPLAIN PLAN FOR
2 SELECT country_id, cust_state_province, count(cust_city)
3 FROM customers
4 GROUP BY country_id, cust_state_province;
```

Id	Operation	Name	Rows	Byte	Cost (XCPU)	Time
0	SELECT STATEMENT		1949	31184	408 (1)	00:00:01
1	HASH GROUP BY		1949	31184	408 (1)	00:00:01
2	TABLE ACCESS STORAGE FULL	CUSTOMERS	55500	867K	406 (1)	00:00:01

Actual number of rows returned by the 2nd query is 145. Optimizer overestimates because it assumes no relationship between country and state.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Following from the previous example, the database must observe a representative workload to determine the appropriate column groups. You do not need to run the queries themselves during the monitoring period. Instead, you can run `EXPLAIN PLAN` for some longer-running queries in your workload to ensure that the database is recording column group information for these queries.

The examples in the slide show the explain plans for two queries on the customer's table.

Reviewing Column Group Usage

```
SQL> SELECT dbms_stats.report_col_usage ('SH', 'CUSTOMERS')
        FROM dual;
```

COLUMN USAGE REPORT FOR SH.CUSTOMERS

1. COUNTRY_ID	: EQ
2. CUST_CITY	: EQ
3. CUST_STATE_PROVINCE	: EQ
4. (CUST_CITY, CUST_STATE_PROVINCE, COUNTRY_ID)	: FILTER
5. (CUST_STATE_PROVINCE, COUNTRY_ID)	: GROUP_BY

EQ means that the column was used in the equality predicate in query 1.

In query 2, the GROUP_BY columns are used in the GROUP BY expression.

FILTER means that columns are used together as filter predicates rather than as joins and so on. It comes from query 1.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The new `REPORT_COL_USAGE` procedure reports the recorded column group usage information.

The example in the slide reviews the column group usage information recorded for the `CUSTOMERS` table.

All three columns appeared in the same `WHERE` clause, so the report shows them as a group filter.

In the second query, two columns appeared in the `GROUP BY` clause, so the report labels them as `GROUP_BY`.

The sets of columns in the `FILTER` and `GROUP_BY` report are candidates for column groups.

Creating Column Groups Detected During Workload Monitoring

```
SQL> SELECT dbms_stats.create_extended_stats
              ('SH', 'CUSTOMERS')
FROM dual;
```

```
#####
EXTENSIONS FOR SH.CUSTOMERS
.....<<<<<<<<
1. (CUST_CITY, CUST_STATE_PROVINCE, COUNTRY_ID) :
   SYS_STUMZ$C3AIHLPBROI#SKA58H_N      created
2. (CUST_STATE_PROVINCE, COUNTRY_ID) :
   SYS_STU#S#WF25Z#QAHIE#MOFFMM_      created
#####
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The example in the slide creates column groups for the `CUSTOMERS` table based on the usage information captured during the monitoring window.

The database created two column groups for `CUSTOMERS`: one column group for the filter predicate and one group for the group by operation.

Note: After being created, you should regather statistics on the `CUSTOMERS` table.

Automatic Dynamic Sampling

- Starting in Oracle Database 12c, the optimizer automatically decides if dynamic sampling is useful and what dynamic sampling level will be used for all SQL statements.
- Automatic dynamic sampling is enabled when the `OPTIMIZER_DYNAMIC_SAMPLING` initialization parameter is set to the value of 11.
- Increase the scope to non-parallel statements.
- Extend capabilities to include joins and group by.
- Make results of dynamic sampling queries persistent in cache to eliminate overhead of repeated sampling.
- Additional statistics sources (for example, extent maps) compensate for stale statistics on volatile tables.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Prior to Oracle Database 12c, dynamic sampling always occurred during optimization, and only when a table in the query had no statistics. Starting in Oracle Database 12c, the optimizer can use automatic dynamic sampling. In this case, the optimizer automatically decides whether dynamic sampling is useful and which dynamic sampling level to use for all SQL statements.

The primary factor in the decision is whether available statistics are sufficient to generate a good plan. If the statistics are insufficient, the optimizer uses dynamic sampling. The scope of the statistics gathered by dynamic sampling has been increased to include joins, group by and nonparallel statements.

The statistics gathered by dynamic sampling are now persistent and may be used by other queries to eliminate overhead of repeated sampling.

It is also helpful to compensate for stale statistics on volatile tables.

Automatic dynamic sampling is enabled when the `OPTIMIZER_DYNAMIC_SAMPLING` initialization parameter is set to the value of 11 (the default value being 2).

Quiz

In Oracle Database 12c, the database creates hybrid histograms if the sampling percentage is AUTO.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: a

Summary

In this lesson, you should have learned about:

- Adaptive SQL Plan Management
- Enhancements to the SQL management base
- How to use a dynamic plan to improve query performance
- Reoptimization for adaptive execution plans
- How to use SQL plan directives to generate a better plan
- Statistics gathering performance improvements
- How to use new histograms
- Enhancements to extended statistics and how to detect useful column groups for a specific workload
- Enhancements to dynamic sampling

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice

- 19-1: Using dynamic plans
- 19-2: Using re-optimization
- 19-2: Using concurrent statistic gathering

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

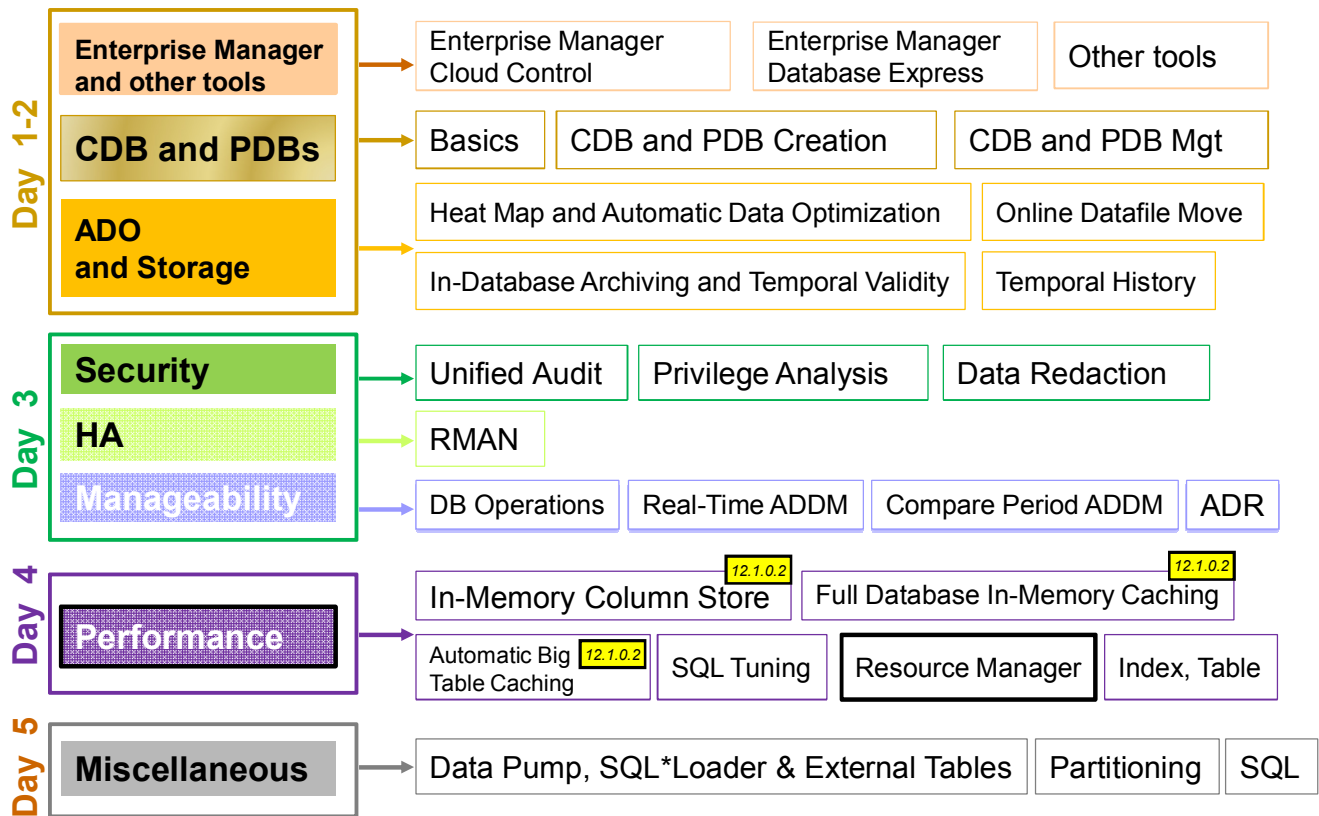
20

Resource Manager and Other Performance Enhancements

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are several Resource Manager enhancements to handle resources within CDBs and PDBs. There is also a new multiprocess multithreaded Oracle architecture and enhancements on Database Smart Flash Cache and undo segments.

Objectives

After completing this lesson, you should be able to:

- Use Resource Manager in a CDB environment
- Control resources consumed by IM Column Store population operations
- Describe the multiprocess multithreaded Oracle architecture
- Describe Database Smart Flash Cache enhancements
- Use temporary undo for your temporary tables
- Limit the size of the PGA

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For a complete understanding of Oracle Pluggable Database new feature and usage, refer to the following guides in the Oracle documentation:

Oracle Database Administrator's Guide 12c Release 1 (12.1)

- *Chapter "Using Oracle Resource Manager for Pluggable Databases with SQL*Plus"*
- *Chapter "Using Oracle Resource Manager for Pluggable Databases with Cloud Control"*
- *Chapter "Managing Resources with Oracle Database Resource Manager Runaway Queries"*

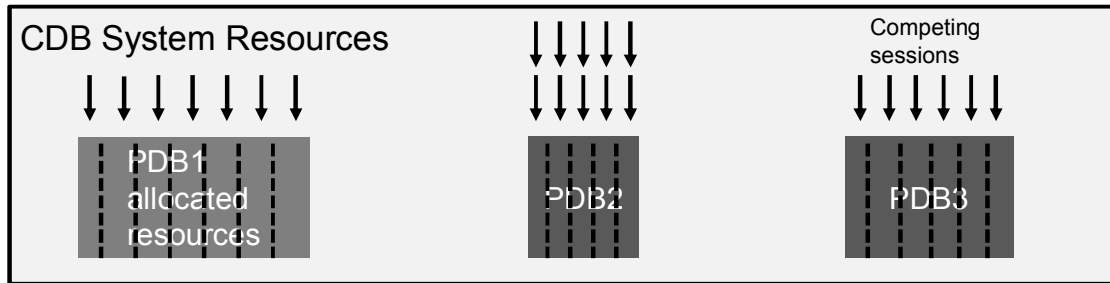
Refer to other sources of information:

- *Oracle Database 12c New Features Demo Series* demonstrations under Oracle Learning Library:
 - *Multitenant Architecture*
 - *Basics of CDB and PDB Architecture*

Resource Manager and Pluggable Databases

In a CDB, Resource Manager manages resources:

- Between PDBs
- Within each PDB



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In a non-CDB, you can use Resource Manager to manage multiple workloads that are contending for system and database resources. However, in a CDB, you can have multiple workloads within multiple PDBs competing for system and CDB resources.

In a CDB, Resource Manager can manage resources on two basic levels:

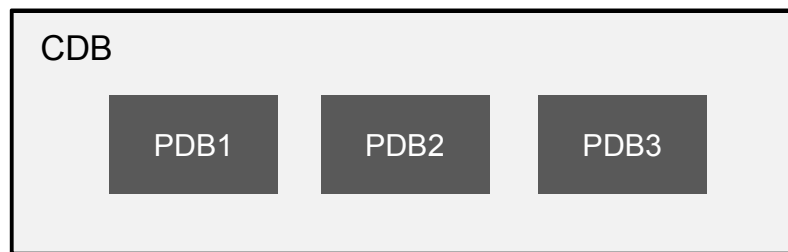
- **CDB level:** Resource Manager can manage the workloads for multiple PDBs that are contending for system and CDB resources. You can specify how resources are allocated to PDBs, and you can limit the resource utilization of specific PDBs.
- **PDB level:** Resource Manager can manage the workloads within each PDB.

Resource Manager allocates the resources in two steps:

1. It allocates a portion of the system's resources to each PDB.
2. In a specific PDB, it allocates a portion of system resources obtained in Step 1 to each session connected to the PDB.

Managing Resources Between PDBs

- PDBs compete for resources: CPU, Exadata I/O, parallel servers
 - System shares are used to allocate resources for each PDB.
 - Limits are used to cap resource utilization of each PDB.
- When a new PDB is plugged in, the CDB DBA can specify a default or explicit allocation.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

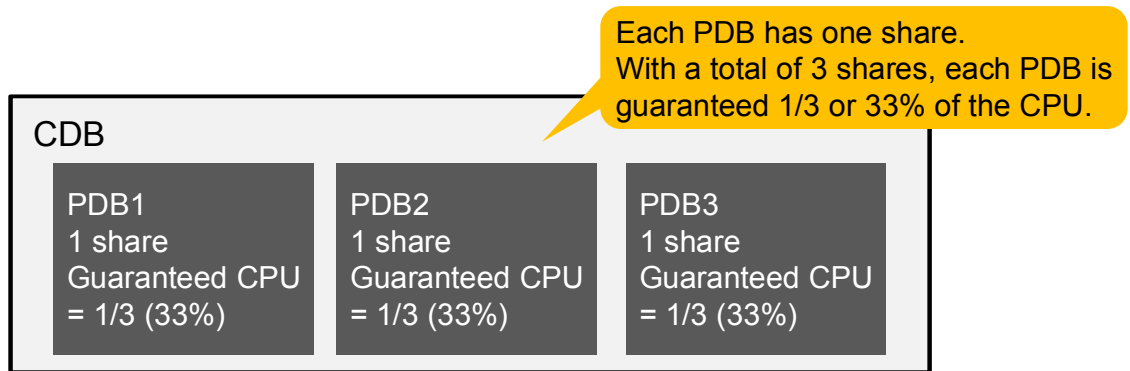
In a CDB with multiple PDBs, some PDBs are typically more important than others. The Resource Manager enables you to prioritize the resource (CPU and I/O, as well as allocation of parallel execution slaves in the context of parallel statement queuing) usage of specific PDBs. This is done by granting different PDBs different shares of the system resources so that more resources are allocated to the more important PDBs.

In addition, limits can be used to restrain the system resource usage of specific PDBs.

When a PDB is plugged in to a CDB and no directive is defined for it, the PDB uses the default directive for PDBs. As the CDB DBA, you can control this default and you can also create a specific directive for each new PDB.

Note: No consumer groups nor shares can be defined for the root container.

CDB Resource Plan Basics: Share



- Specify resources using “shares”.
 - Recomputation is not required when PDBs are added or removed.
 - PDB1 is guaranteed 33% of the CPU.
 - PDB1 is limited to 100% of the CPU.
 - PDB1 is actually using 20% of the CPU.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To allocate resources among PDBs, you assign a share value to each PDB. A higher share value results in more resources for a PDB.

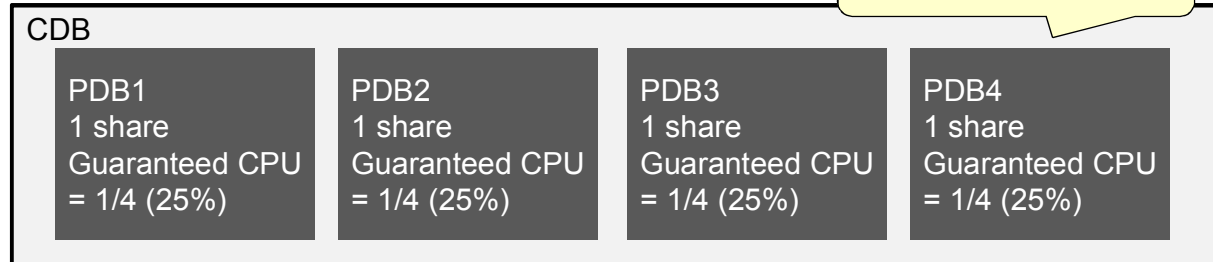
In this example, each existing PDB is assigned one share. With a total of three shares, each PDB is guaranteed to get at least 33% of the system resources.

Depending on the workload, each PDB can get up to 100% of the system resources but may actually use only 20% of these resources.

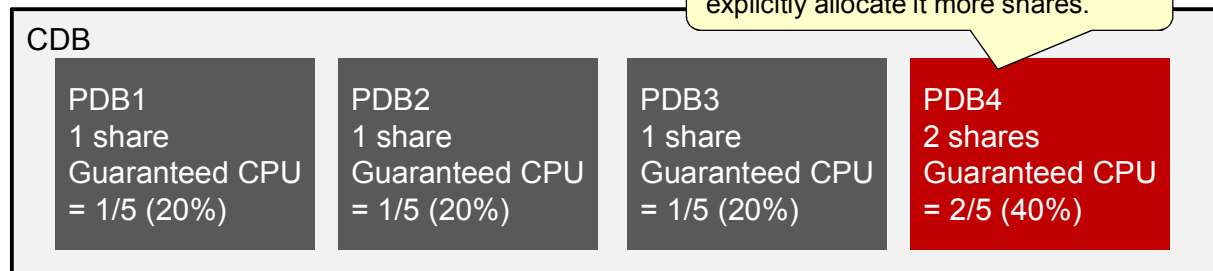
CDB Resource Plan Basics: Share

Default allocation: One share

New PDB gets the default allocation: 1 share.



If this PDB is more important, you can explicitly allocate it more shares.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If a new PDB is added, it automatically gets the default share value of one.

In the first example in the slide, with a total of four shares, each PDB is guaranteed to get 25% of the system resources.

Note: Each added PDB in the CDB resource plan uses the default directive for PDBs. You can change this default directive if its default value is not suitable for your plan.

However, if you consider PDB4 to be of higher priority than the others, you can explicitly assign it more shares. In the second example in the slide, PDB4 gets two shares while the other PDBs get one share. With a total of five shares, PDB1, PDB2, and PDB3 is guaranteed to get one-fifth (20%) of the system resources because they were assigned one share each. Because PDB4 was assigned two shares, it is guaranteed to get two-fifths (40%) of the system resources.

CDB Resource Plan Basics: Limits

- Two limits can be defined for each PDB:
 - Utilization limit for CPU, Exadata I/Os and parallel servers
 - Parallel server limit to override the utilization limit
- Default values: 100%
- You can change default values.

PDB1

```
utilization_limit = 30
```

```
Parallel_server_limit = 50
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A limit restrains the system resource usage of a specific PDB. You can specify limits for CPU, Exadata I/Os, and parallel execution servers.

To limit the CPU, Exadata I/Os, and parallel servers (`PARALLEL_SERVER_TARGET` initialization parameter) usage of each PDB, you can create a plan directive by using the `UTILIZATION_LIMIT` parameter expressed as a percentage of the system resources the PDB can use. Resource Manager throttles the PDB sessions so that the CPU, Exadata I/Os, and parallel servers utilization for the PDB does not exceed the utilization limit. In the example in the slide, PDB1 gets a maximum of 30% of the system CPU, Exadata I/Os bandwidth, and available parallel servers for the CDB instance.

For parallel execution servers, you can override the value defined by the `UTILIZATION_LIMIT` by creating a plan directive that uses the `PARALLEL_SERVER_LIMIT` parameter. The `PARALLEL_SERVER_LIMIT` value corresponds to a percentage of `PARALLEL_SERVERS_TARGET`. For example, if the `PARALLEL_SERVERS_TARGET` initialization parameter is set to 200 and the `PARALLEL_SERVER_LIMIT` is set to 50% for a PDB, then the maximum number of parallel servers the PDB can use is 100 ($200 \times .50$).

When you do not explicitly define limits for a PDB, the PDB uses the default limits for PDBs that are set to 100%. These are the values corresponding to the default directive that is automatically added to your plan.

Note: You can also change the default directive attribute values for PDBs by using the `UPDATE_CDB_DEFAULT_DIRECTIVE` procedure in the `DBMS_RESOURCE_MANAGER` package.

CDB Resource Plan: Full Example

PDB/Directive Name	Shares	Utilization Limit	Parallel Server Limit
(Default Allocation)	(1)	(100%)	(100%)
(Autotask Allocation)	(-1)	(90%)	(100%)
PDB1	1	30%	50%
PDB2	1	30%	80%
PDB3	1	30%	30%
PDB4	2	100%	100%

PDB1 is

- Guaranteed 1/5 (20%) of CPU and Exadata disk bandwidth
- Limited to 30% of CPU and Exadata disk bandwidth
- Limited to 50% of the parallel servers

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

This example show you a possible CDB resource plan created in the root container.

The default allocation directive applies to PDBs for which specific directives have not been defined.

Default allocation is one share and both corresponding limits are set to their default of 100%. These are the default values for the default allocation directive.

The autotask allocation directive applies to automatic maintenance tasks that are run in the root or in PDBs.

The autotask allocation is -1 share, which means that the automated maintenance tasks get an allocation of 20% of the system resources. You should always change this default value. By default, the autotask allocation gets a utilization limit of 90% and 100% for its parallel server limit.

PDB1 is allocated one share which represents one-fifth or 20% of the CPU, Exadata disk bandwidth, and queued parallel queries will be selected one-fifth of the time compared to the other PDBs. In addition, a utilization limit of 30% of the resources is set as well as a 50% limit of the available parallel servers.

Root

Creating a CDB Resource Plan

1. Create a pending area.
2. Create a CDB resource plan.

```
SQL> EXEC DBMS_RESOURCE_MANAGER.CREATE_CDB_PLAN( -
        plan      => 'daytime_plan', -
        comment => 'CDB resource plan for mycdb
```

3. Create directives for the PDBs.

```
SQL> EXEC DBMS_RESOURCE_MANAGER.CREATE_CDB_PLAN_DIRECTIVE( -
        plan      => 'daytime_plan', -
        pluggable_database => 'salespdb', -
        shares     => 3, -
        utilization_limit  => NULL, -
        parallel_server_limit => NULL)
```

4. (Optional) Update the default PDB directives.
5. (Optional) Update the default autotask directives.
6. Validate and submit the pending area.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

CDB resource plan management is done directly from the root container.

The slide displays the steps you need to perform to create a CDB resource plan.

You use the `DBMS_RESOURCE_MANAGER` package to create a CDB resource plan and define the directives for the plan. If you try to define a CDB resource plan in a PDB, you get an error.

1. Connect to the root to create a pending area by using the `CREATE_PENDING_AREA` procedure.
2. Create a CDB resource plan named `daytime_plan` by using the `CREATE_CDB_PLAN` procedure.
3. Create the CDB resource plan directives for the PDBs by using the `CREATE_CDB_PLAN_DIRECTIVE` procedure. Each directive specifies how resources are allocated to a specific PDB. In the example, the `salespdb` PDB gets three shares and default limits. You could define other directives for the `hrpdb` PDB that would get 1 share and 70% for both limits.
4. When you are done, simply validate and submit your pending area.

As in a non-CDB, you enable the Resource Manager plan for a CDB by setting the `RESOURCE_MANAGER_PLAN` initialization parameter in the root. If you disable a CDB resource plan, then some directives in PDB resource plans are disabled. You can also associate a CDB plan to scheduler window.

Setting Default Directives

- Setting default PDB directive:

```
SQL> EXEC DBMS_RESOURCE_MANAGER.UPDATE_CDB_DEFAULT_DIRECTIVE( -  
    plan                        => 'daytime_plan', -  
    new_shares                  => 1, -  
    new_utilization_limit       => 50, -  
    new_parallel_server_limit   => 50)
```

- Setting autotask CDB resource plan directive:

```
SQL> EXEC DBMS_RESOURCE_MANAGER.UPDATE_CDB_AUTOTASK_DIRECTIVE( -  
    plan                        => 'daytime_plan', -  
    new_shares                  => 1, -  
    new_utilization_limit       => 75, -  
    new_parallel_server_limit   => 75)
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

All other PDBs in this CDB use the default PDB directive that you can optionally update by using the `UPDATE_CDB_DEFAULT_DIRECTIVE`. In the example in the slide, you change only the limits' defaults to 50%.

Similarly, if the current autotask CDB resource plan directive does not meet your requirements, update the directive by using the `UPDATE_CDB_AUTOTASK_DIRECTIVE` procedure. Here, all three parameters are changed to 1% and 75% respectively.

When you are done, simply validate and submit your pending area.

Viewing CDB Resource Plan Directives

```
SQL> SELECT    plan, pluggable_database, shares,
              utilization_limit, parallel_server_limit
FROM          dba_cdb_rsrc_plan_directives
ORDER BY plan;
```

Plan	Pluggable Database	Shares	Utilization Limit	Parallel Server Limit
DAYTIME_PLAN	ORA\$DEFAULT_PDB_DIRECTIVE	1	50	50
DAYTIME_PLAN	ORA\$AUTOTASK	1	75	75
DAYTIME_PLAN	SALESPDB	3		
DAYTIME_PLAN	HRPDB	1	70	70
DEFAULT_CDB_PLAN	ORA\$DEFAULT_PDB_DIRECTIVE	1	100	100
DEFAULT_CDB_PLAN	ORA\$AUTOTASK		90	100
DEFAULT_MAINTENANCE_PLAN	ORA\$DEFAULT_PDB_DIRECTIVE	1	100	100
DEFAULT_MAINTENANCE_PLAN	ORA\$AUTOTASK		90	100
ORA\$INTERNAL_CDB_PLAN	ORA\$DEFAULT_PDB_DIRECTIVE			
ORA\$INTERNAL_CDB_PLAN	ORA\$AUTOTASK			



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

This example uses the `DBA_CDB_RSRC_PLAN_DIRECTIVES` view to display all of the directives defined in all of the CDB resource plans in the root container.

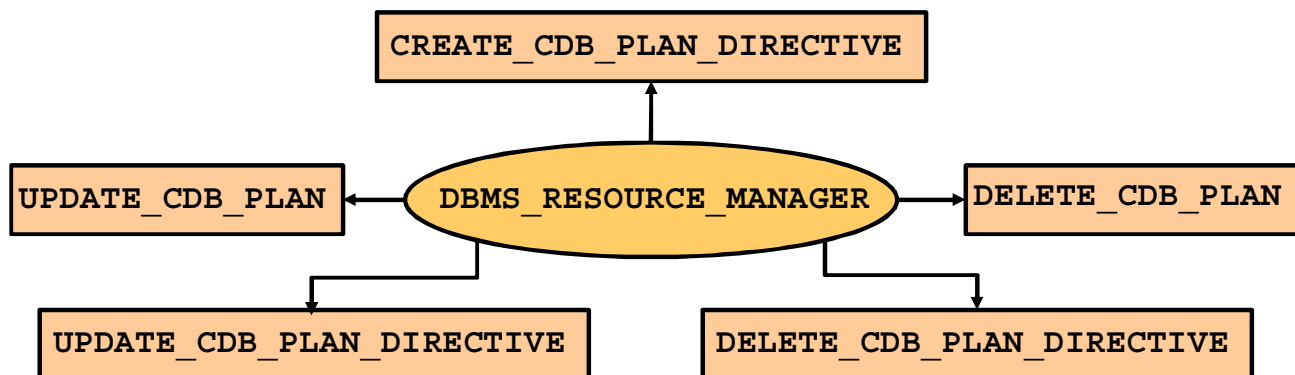
The `ORA$INTERNAL_CDB_PLAN` is the default CDB resource plan. It is a very simple plan where no resources are managed.

Note: From the root container, you can use `DBA_CDB_RSRC_PLANS` view to display all of the CDB resource plans defined in the root container.

Maintaining a CDB Resource Plan

```
SQL> CONNECT sys AS SYSDBA
```

```
SQL> EXEC DBMS_RESOURCE_MANAGER.CREATE_PENDING_AREA();
```



```
SQL> EXEC DBMS_RESOURCE_MANAGER.VALIDATE_PENDING_AREA();
```

```
SQL> EXEC DBMS_RESOURCE_MANAGER.SUBMIT_PENDING_AREA();
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

1. You can update a CDB resource plan to change its comment by using the `UPDATE_CDB_PLAN` procedure.
2. When you create a PDB in a CDB, you can create a CDB resource plan directive for the PDB by using the `CREATE_CDB_PLAN_DIRECTIVE` procedure. The directive specifies how resources are allocated to the new PDB.
3. You can delete the CDB resource plan directive for a PDB using the `DELETE_CDB_PLAN_DIRECTIVE` procedure. You might delete the directive for a PDB if you unplug or drop the PDB. However, you can retain the directive, and if the PDB is plugged into the CDB in the future, the existing directive applies to the PDB.
4. You can update the CDB resource plan directive for a PDB using the `UPDATE_CDB_PLAN_DIRECTIVE` procedure. The directive specifies how resources are allocated to the PDB.
5. You can delete a CDB resource plan using the `DELETE_CDB_PLAN` procedure. You might delete a CDB resource plan if the plan is no longer needed. You can enable a different CDB resource plan, or you can disable Resource Manager for the CDB. If you delete an active CDB resource plan, then some directives in PDB resource plans become disabled.

Managing Resources Within a PDB

- In a non-CDB database, workloads within a database are managed with resource plans.
- In a PDB, workloads are also managed with resource plans, also called PDB resource plans.
- The functionality is similar except for the following differences:

Non-CDB Database	PDB Database
Multi-level resource plans	Single-level resource plans only
Up to 32 consumer groups	Up to 8 consumer groups
Subplans	No subplans

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A CDB resource plan determines the amount of resources allocated to each PDB. A PDB resource plan determines how the resources allocated to a specific PDB are allocated to consumer groups within that PDB. A PDB resource plan is similar to a resource plan for a non-CDB. Specifically, a PDB resource plan allocates resource among the consumer groups within a PDB.

In a CDB, the following restrictions apply to PDB resource plans:

- PDB resource plan cannot have a multilevel scheduling policy.
- PDB resource plan can have a maximum of eight consumer groups.
- PDB resource plan cannot have subplans.

Note: If you try to create a PDB plan in the root, you get an error.

Managing PDB Resource Plans

- Connect to the PDB to manage the PDB plan.
- Use exactly the same procedures as for managing a resource plan in a non-CDB environment.
- For `CREATE_PLAN_DIRECTIVE` procedure:
 - New `SHARE` argument
 - Replace `MAX_UTILIZATION_LIMIT` and `PARALLEL_TARGET_PERCENTAGE` arguments with `UTILIZATION_LIMIT` and `PARALLEL_SERVER_LIMIT`
- You can view CDB and PDB resource plans using `V$RSRC_PLAN`.

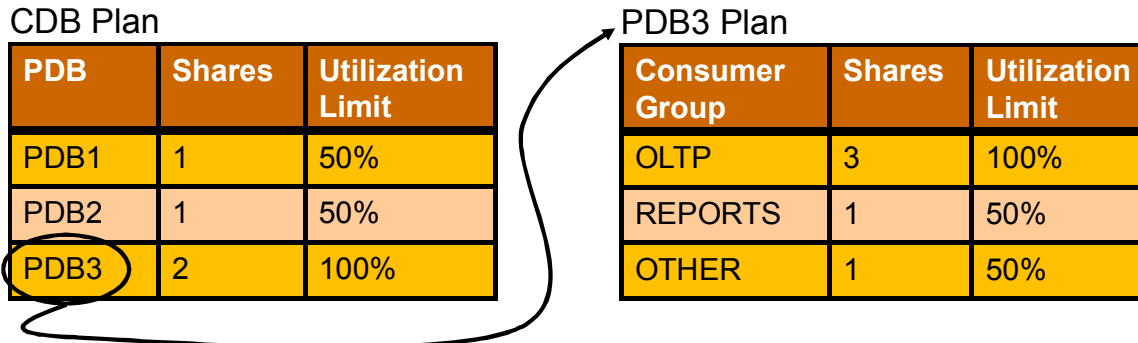
The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

`V$RSRC_PLAN` exists in previous releases of the Oracle Database. The resource plan with `con_id=1` is the active CDB resource plan.

Putting It Together

- How do CDB and PDB resource plans work together?
 - Resources allocated to a PDB, based on CDB resource plan
 - Resource allocated to a consumer group based on the PDB resource plan



- What does this mean for PDB3 Reports?
 - Guaranteed 50% (2/4) x 20% (1/5) = 10% of the resources
 - Limited to 100% x 50% = 50% of the resources

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Resources are allocated to each PDB based on the CDB resource plan in action. Each PDB receives a certain percentage of the system resources and Resource Manager distribute that lot to each consumer group based on that PDB's resource plan in action.

An example is shown in the slide. The example uses SHARES and UTILIZATION LIMIT for simplicity.

As you can see, the REPORTS consumer group is guaranteed to receive 10% of the total system resources available. This is because PDB3 is allocated two shares out of four in the CDB resource plan, which represents 50% of the resources, and out of this 50%, the REPORTS consumer group receives one share out of five, which represents 20% of the given 50%. Similarly, the REPORTS consumer group in PDB3 is limited to 50% of total system resources.

Considerations

- Setting a PDB resource plan is optional. If not specified, all sessions within the PDB are treated equally.
- A non-CDB is plugged into a CDB with a plan:
 - The plan runs exactly the same on the PDB if:
 - Consumer groups ≤ 8
 - No subplans
 - All allocations on level 1
 - The plan is converted.
 - The original plan is stored in the dictionary with the `LEGACY` status.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

`V$RSRC_PLAN` exists in previous releases of the Oracle Database. The resource plan with `con_id=1` is the active CDB resource plan.

If a non-CDB with a plan is plugged into a CDB as a new PDB, the plan acts exactly the same way in the PDB if it does not violate any of the PDBs restrictions:

- The number of consumer groups does not exceed 8.
- There are no subplans.
- All allocations are on level 1.

However, if it violates any of these restrictions, the plan is converted to something equivalent. If the plan is enabled, the converted version is used. If the converted plan does not meet your expectations, you can still work the original plan, stored in the dictionary with the `LEGACY`.

Runaway Queries and Resource Manager

- New parameters to trigger consumer group switching:
 - SWITCH_IO_LOGICAL
 - SWITCH_ELAPSED_TIME
- New meta consumer group called LOG_ONLY
- New column added to V\$SQL_MONITOR:
 - RM_LAST_ACTION
 - RM_LAST_ACTION_REASON
 - RM_LAST_ACTION_TIME
 - RM_CONSUMER_GROUP
- V\$RSRCMGRMETRIC and V\$RSRCMGRMETRIC_HISTORY always populated

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To better track runaway queries, Resource Manager is enhanced to include the following new features:

- In addition to existing switch conditions (switch_io_reqs, switch_io_megabytes, switch_time), new parameters are added to dbms_resource_manager.create_plan_directive:
 - switch_io_logical: Number of logical I/Os that will trigger the action specified by switch_group
 - switch_elapsed_time: Elapsed time that will trigger the action specified by switch_group
- There is a new meta-consumer group called LOG_ONLY. This can be used as the argument for the switch_group parameter. It means that the DBA wants to log the runaway query without changing its consumer group or taking other action.
- Resource Manager integrates the runaway query information with SQL Monitor. To retain important results for Resource Manager, it pins up to five SQLs per consumer group. SQL Monitor does not purge these SQL executions until they are unpinned. Here are the new columns added to V\$SQL_MONITOR:
 - RM_LAST_ACTION: The most recent action that was taken on this SQL operation by Resource Manager. Its value is one of the following: CANCEL_SQL, KILL_SESSION, LOG_ONLY, SWITCH TO <CG NAME>

- RM_LAST_ACTION_REASON: The reason for the most recent action that was taken on this SQL operation by Resource Manager. Its value is one of the following: SWITCH_CPU_TIME, SWITCH_IO_REQS, SWITCH_IO_MBS, SWITCH_ELAPSED_TIME, SWITCH_IO_LOGICAL
- RM_LAST_ACTION_TIME: The time of the most recent action that was taken on this SQL operation by Resource Manager
- RM_CONSUMER_GROUP: The current consumer group for this SQL operation

Note: `v$rsrsrcmgrmetric` and `v$rsrsrcmgrmetric_history` will always produce a row every minute regardless of whether there is a Resource Manager plan set.

Controlling IM Column Store Repopulation Resource Consumption

- IM column store population is CPU intensive.
- There are different types of IM column store populations:

Description	Default Consumer Group
Population of in-memory segments (e.g.: Segments with priority other than NONE)	ora\$autotask
On-demand population (e.g.: When a user accesses an in-memory segment with NONE priority)	User's consumer group

- ➔ Manage the CPU usage of population operations by changing their priority:
- Enabling one of the out-of-box resource plans (DEFAULT_PLAN)
 - Creating your own resource plan
 - Mapping ORACLE_FUNCTION attribute with 'INMEMORY' value to another consumer group

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

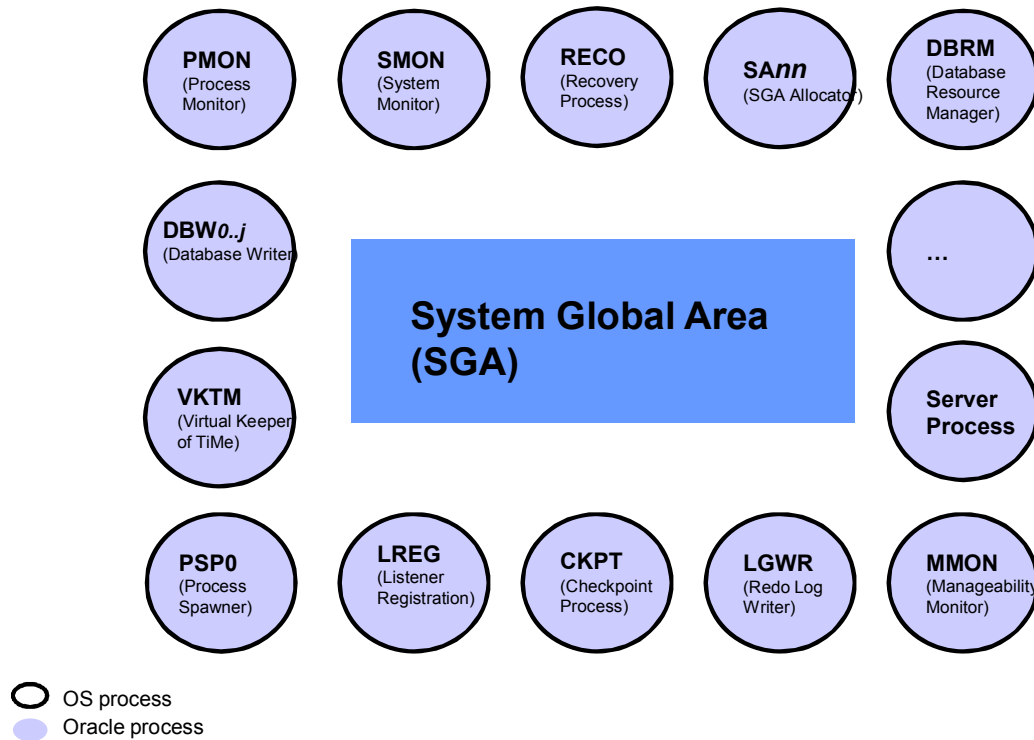
By using Resource Manager, you can manage the CPU usage of IM column store population operations and change their priority as needed by enabling one of the out-of-box resource plans, like DEFAULT_PLAN, or by creating your own resource plan.

By default, all IM column store population operations are run in the ora\$autotask consumer group, except for “on-demand population” which runs in the consumer group the user accessing the data is mapped to. If the ora\$autotask consumer group does not exist in the resource plan, the population runs in OTHER_GROUPS. The other operations in ora\$autotask include automated maintenance operations like gathering statistics and segment analysis.

To change the consumer group for IM column store population operations, the DBA uses the DBMS_RESOURCE_MANAGER.SET_CONSUMER_GROUP_MAPPING procedure to map it to another group. Example:

```
SQL> EXEC DBMS_RESOURCE_MANAGER.SET_CONSUMER_GROUP_MAPPING
      ( attribute      => 'ORACLE_FUNCTION',
        value          => 'INMEMORY',
        consumer_group => 'BATCH_GROUP' )
```

Default UNIX/Linux Architecture

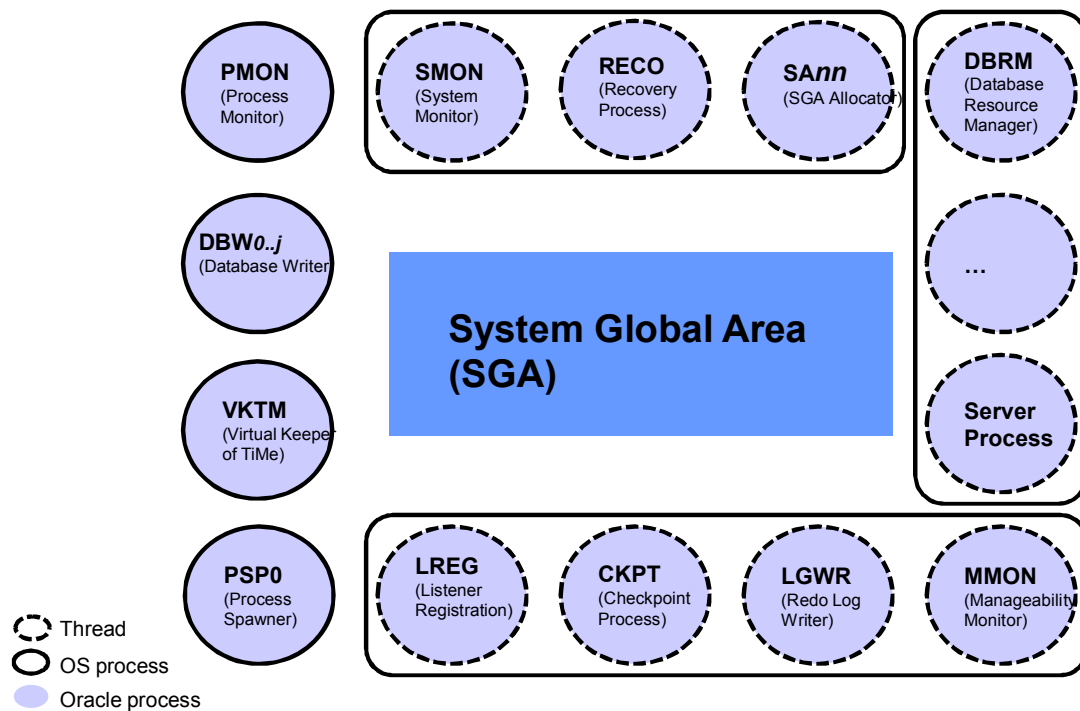


ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Oracle process execution architecture for an Oracle Database Instance depends on the operating system. For example, on Windows an Oracle background process is a thread of execution within an Operating System (OS) process. On Linux and UNIX, an Oracle background process runs as an OS process. This situation is represented in the slide where every Oracle background process and foreground process, generically called Oracle process, runs as a specific OS process.

Multi-Process Multi-Threaded UNIX/Linux Architecture



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Multi-Process (MP) model makes every Oracle process run as a separate OS process. This model suffers from inefficient resource utilization and sharing.

Starting with Oracle Database 12c, the multithreaded Oracle (MT) model enables certain Oracle processes to execute as operating system threads in the same address space. In threaded mode, some background processes on UNIX and Linux run as processes (processes containing one thread), whereas the remaining Oracle processes run as threads within OS processes. The OS process to Oracle process allocation is random based. This is what is called the Multi-Process / Multi-Threaded hybrid architecture.

This is illustrated in the slide where Oracle processes PMON, DBW, VKTM, and PSP run in their own OS process, while all the others run as threads in different OS processes.

Notes

- As of Oracle Database 12c release 12.1, and because of their importance, it was decided to always have PMON, DBW, VKTM, and PSP running as OS processes. This can change in future releases.
- Each OS process also runs a special thread called SCM, which is basically an internal listener thread. All thread creation is routed through this thread. This special thread is not shown in the slide.

Multi-Process Multi-Threaded Architecture: Benefits and Setup

- CPU and memory usage reduction
- Better system reliability
- Better performance for parallel operations

```
SQL> CONNECT sys AS SYSDBA
Enter password:
Connected.
SQL> ALTER SYSTEM SET threaded_execution=true SCOPE=SPFILE;
System altered.
SQL> SHUTDOWN IMMEDIATE
Database closed.
Database dismounted.
ORACLE instance shut down.
SQL> STARTUP
...
Database opened.
SQL>
```



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Note

In some tests, memory usage was almost divided by half when using multi-process multi-threaded feature.

Setting up an Oracle database instance for using multi-process multi-threaded architecture is done by starting up the Oracle database instance with the initialization parameter `THREADED_EXECUTION` set to `TRUE`.

To be able to start up an Oracle database instance with `THREADED_EXECUTION` set to `TRUE`, you need to use a `SYSDBA` connection that was authenticated via a password file.

Multi-Process Multi-Threaded Architecture: Considerations

- Threads still use PGA for private memory.
- Threads still use SGA memory for inter-process communication.
- The listener does not spawn threads.
- Password file for SYSDBA authentication is required.
- As a best practice, set the following parameters:

```
DEDICATED_THROUGH_BROKER_LISTENER = on
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- An Oracle process running as a thread still uses PGA for private memory.
- An Oracle process running as a thread still uses SGA memory for inter-process communication.
- The database listener does not spawn threads. Instead, the connection requests, handled by a dispatcher, are handed out to the spawning thread for the given instance. Each OS process running multiple Oracle processes also run an internal listener thread called SCMN. All thread creation is routed through this thread.
- Multi-process Multi-threaded mode requires password file for SYSDBA authentication while starting the database instance. Without using SYS password, an ORA-1031: insufficient privileges error is triggered on startup.
- Threaded execution connections cannot run over a static service definition. Thus, the DBA must ensure that client connections are routed over a dynamically registered service, either by a TNS-entry or by using EZConnect.
- Also set the DEDICATED_THROUGH_BROKER_LISTENER parameter to ON in your listener.ora file. When the parameter is set, the listener knows that it should not spawn an OS process when a connect request is received, instead it passes the request to the database so that a database thread is spawned and answers the connection.

Multi-Process Multi-Threaded Architecture: Monitoring

```
SQL> select spid, stid, pname from v$process order by spid;
```

SPID	STID	PNAME
31562	31562	PMON
31566	31566	PSP0
31570	31570	VKTM
31576	31576	SCMN
31576	31580	GEN0
31576	31627	SMON
31576	31633	LREG
31576	31624	LG01
...	...	...
31590	31608	DIA0
31590	31590	SCMN
31590	31642	D000
...	...	...
31598	31602	OFSD
31598	31598	SCMN
31612	31612	DBW0

47 rows selected.
SQL>

```
$ ps -ef | grep orcl
```

oracle	598	19126	0	16:30	pts/0	00:00:00	grep orcl
oracle	31562	1	0	14:04	?	00:00:01	ora_pmon_orcl
oracle	31566	1	0	14:04	?	00:00:01	ora_psp0_orcl
oracle	31570	1	1	14:04	?	00:01:52	ora_vkrm_orcl
oracle	31576	1	0	14:04	?	00:00:05	ora_u004_orcl
oracle	31590	1	0	14:04	?	00:00:35	ora_u005_orcl
oracle	31598	1	0	14:04	?	00:00:00	ora_u006_orcl
oracle	31612	1	0	14:04	?	00:00:00	ora_dbw0_orcl

```
$ ps -eLo "pid tid comm args" | grep ora_
```

pid	tid	comm	args
31562	31562	ora_pmon_orcl	ora_pmon_orcl
31566	31566	ora_psp0_orcl	ora_psp0_orcl
31570	31570	ora_vkrm_orcl	ora_vkrm_orcl
31576	31576	ora_scmn_orcl	ora_u004_orcl
31576	31579	oracle	ora_u004_orcl
31576	31580	ora_gen0_orcl	ora_u004_orcl
31576	31583	ora_mman_orcl	ora_u004_orcl

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The slide shows you how to look at the various threads and OS processes to relate them to Oracle processes.

The SPID and STID columns in V\$PROCESS shows the relation between the OS process ID and the thread ID.

At the operating system level you can retrieve the relationship between the OS process ID and the corresponding thread ID, Oracle process and OS process names.

Database Smart Flash Cache Enhancements

- Multiple flash drives in the flash cache
 - Dynamic enable/disable for flash cache devices
- Change to in-memory parallel query (PQ) algorithm
 - When Auto DOP is enabled
 - Buffer cache divided into OLTP and DW sections
 - Separate cache replacement algorithm used for DW section

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle Database 11g R2, the Smart Flash Cache was limited to one device. For systems where there was a need for a flash cache larger than could be provided by a single flash device, ASM diskgroup or a volume manager was required to present the multiple devices as a single device or file.

In Oracle Database 12c, up to 16 devices can be specified to be part of the flash cache. These devices can be enabled and disabled dynamically, but resizing the flash cache dynamically is not supported.

In Oracle Database 12c, an object based algorithm has been added to the standard LRU algorithm for managing the database buffer cache with a flash cache to allow in-memory parallel query to take advantage of the flash cache. The specialized object-based policy segregates the buffer cache, including the flash cache, with a portion of the buffers being replaced by the specialized object-based algorithm, while the other portion being replaced by the traditional LRU algorithm, which also include the original smart flash cache algorithm. In the RAC environment, if a large object is smaller than the combined size of the combined buffer caches and flash caches size for the cluster, the object is partitioned and fit into memory and flash completely. With in-memory parallel query, it can eliminate disk reads for this query, or intelligently read from disks in parallel to increase the read bandwidth.

Note: This new algorithm is enabled when using automatic degree of parallelism (Auto DOP) by setting the initialization parameter `PARALLEL_DEGREE_POLICY` to `AUTO`.

Enabling and Disabling Flash Devices

- Specify flash devices at instance startup.

```
db_flash_cache_file = /dev/raw/sda, /dev/raw/sdb  
db_flash_cache_size = 32G, 64G
```

- Disable a flash device.

```
db_flash_cache_size = 0, 64G
```

- Enable a flash device.

```
db_flash_cache_size = 32G, 64G
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle Database 12c, you specify the flash devices and their sizes at instance startup in the SPFILE or test parameter file. Even though you cannot change the size of the flash cache dynamically, you can enable and disable individual devices as shown.

The `DB_FLASH_CACHE_FILE` parameter is not dynamic and can only be set at instance startup. Specify a comma separated list of up to 16 OS devices. The example shown is appropriate for a Linux system.

In this example, the flash device `/dev/raw/sda` is initialized at startup with 32 GB. With the `ALTER SYSTEM SET` command you can disable `/dev/raw/sda` by setting the size to 0. You can also enable `/dev/raw/sda` back to the same size it was previously that is 32 GB. You cannot change the size of the device using this method, or change the devices specified.

If there is only one device in the list of devices and its size is set to 0, the flash cache is disabled.

In-Memory PQ Algorithm: Benefits

- Allows parallel query to utilize smart flash cache
- Allows smart flash cache and disk to be read in parallel
- Does not thrash the buffer cache or flash cache
- Seeks an optimal balance between I/O and CPU resources

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The in-memory PQ object replacement algorithm is used to support Data Warehouse scalability by making use of the flash cache on a compute node in a large cluster. The advantages of this algorithm are also apparent in smaller instances that run a mixed workload, or a workload that changes from OLTP to DSS at times. This algorithm does not replace the LRU algorithm, but exists in the same instance.

This algorithm:

- Allows parallel query to utilize smart flash cache
- Reads the smart flash cache and disk in parallel
- Does not force already cached parts of a object out of the cache for more of the same object
- Does not force buffer blocks being used by the main LRU algorithm out of the buffer cache or flash cache
- Seeks an optimal balance between I/O and CPU resources

Smart Flash Cache: New Statistics

Several new statistics to measure the use of the object replacement algorithm:

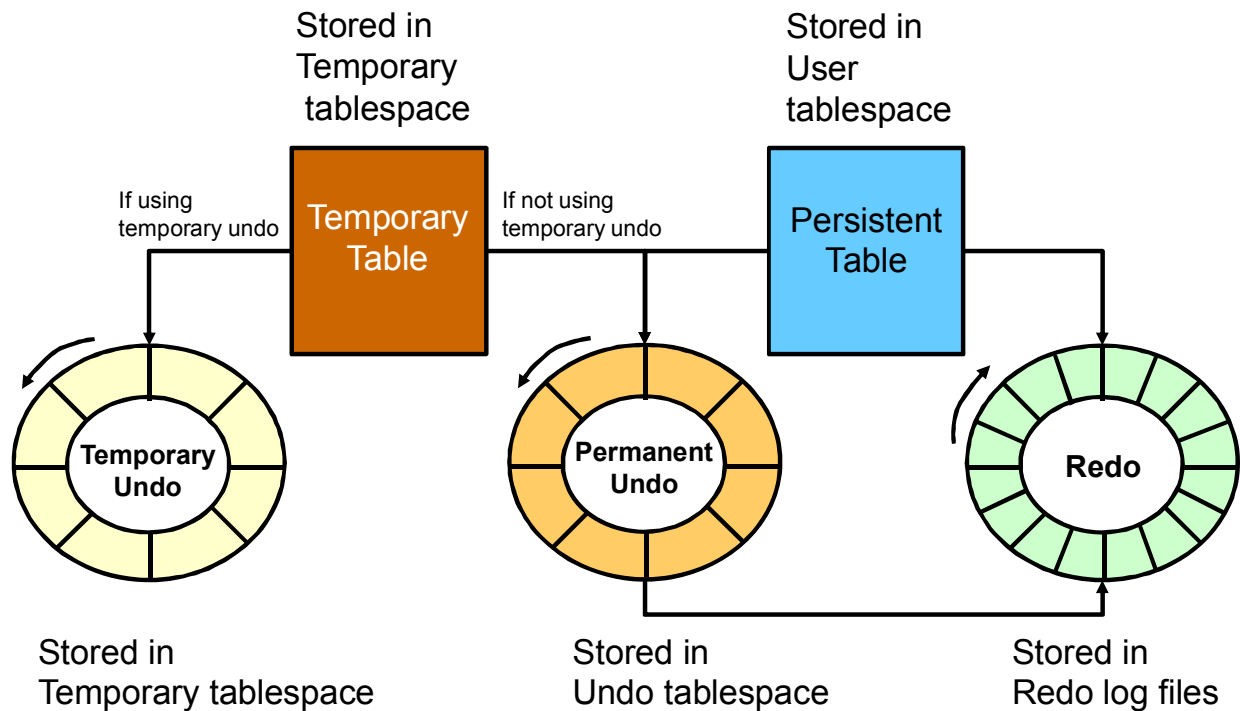
- Data warehousing scanned chunks
- Data warehousing scanned chunks - disk
- Data warehousing scanned chunks - flash
- Data warehousing scanned chunks - memory
- Data warehousing scanned objects

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- **Data warehousing scanned chunks:** Number of 1 MB chunks that have been scanned using the 12c in-memory PQ data warehousing scan
- **Data warehousing scanned chunks - disk:** Number of 1 MB chunks that have been scanned using the 12c in-memory PQ data warehousing scan, and the request was satisfied by direct read on disk
- **Data warehousing scanned chunks - flash:** Number of 1 MB chunks that have been scanned using the 12c in-memory PQ data warehousing scan, and the request was satisfied by buffers in Database Smart Flash Cache
- **Data warehousing scanned chunks - memory:** Number of 1 MB chunks that have been scanned using the 12c in-memory PQ data warehousing scan, and the request was satisfied by buffers in memory
- **Data warehousing scanned objects:** Number of objects that have been scanned using the 12c in-memory PQ data warehousing scan

Temporary Undo: Overview



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Temporary tables are widely used as scratch areas for staging intermediate results. This is because changing those tables is much faster than changing nontemporary tables. The performance gain is mainly due to the fact that no redo entries are directly generated for changes on temporary tables. However, the undo for operations on temporary tables (and indexes) is still logged to the redo log.

Undo for temporary tables is useful for consistent reads and transaction rollbacks during the life of that temporary object. Beyond this scope the undo is superfluous. Hence it need not be persisted in the redo stream. For instance, transaction recovery just discards undo for temporary objects.

Starting with Oracle Database 12c it is possible for undo generated by temporary tables' transactions to be stored in a separate undo stream directly in the temporary tablespace to avoid for that undo to be logged in the redo stream. This new mode is called temporary undo.

Note: Temporary undo segment is session private. It stores undo for the changes to temporary tables (temporary objects in general) belonging to the corresponding session.

Temporary Undo: Benefits and Setup

- Temporary undo reduces the amount of undo stored in the undo tablespaces.
- Temporary undo reduces the size of the redo log.
- Temporary undo enables DML operations on temporary tables in a physical standby database with the Oracle Active Data Guard option.

```
SQL> ALTER SESSION SET TEMP_UNDO_ENABLED = TRUE;
```

```
SQL> ALTER SYSTEM SET TEMP_UNDO_ENABLED = TRUE;
```

- Temporary undo mode is selected when a session first uses a temporary object.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Enabling temporary undo provides the following benefits:

- Temporary undo reduces the amount of undo stored in the undo tablespaces. Less undo in the undo tablespaces can result in more realistic undo retention period requirements for undo records.
- Temporary undo reduces the size of the redo log. Performance is improved because less data is written to the redo log, and components that parse redo log records, such as LogMiner, perform better because there is less redo data to parse.
- Temporary undo enables data manipulation language (DML) operations on temporary tables in a physical standby database with the Oracle Active Data Guard option. However, data definition language (DDL) operations that create temporary tables must be issued on the primary database.

You can enable temporary undo for a specific session or for the whole system. When you enable temporary undo for a session, the session creates temporary undo without affecting other sessions. When a session uses temporary objects for the first time, the current value of the `TEMP_UNDO_ENABLED` initialization parameter is set for the rest of the session. Therefore, if temporary undo is enabled for a session and the session uses temporary objects, temporary undo cannot be disabled for the session. Similarly, if temporary undo is disabled for a session and the session uses temporary objects, then temporary undo cannot be enabled for the session. When you enable temporary undo for the system, all existing sessions and new sessions create temporary undo.

Temporary Undo Monitoring

```
SQL> SELECT to_char(BEGIN_TIME, 'dd/mm/yy hh24:mi'),
            TXNCOUNT, MAXCONCURRENCY, UNDOBLKCNT, USCOUNT,
            NOSPACERRCNT
      FROM   V$TEMPUNDOSTAT;
```

TO_CHAR(BEGIN_	TXNCOUNT	MAXCONCURRENCY	UNDOBLKCNT	USCOUNT	NOSPACERRCNT
...					
19/02/14 22:19	0	0	0	0	0
19/02/14 22:09	0	0	0	0	0
...					
19/02/14 13:09	0	0	0	0	0
19/02/14 12:59	3	1	24	1	0

576 rows selected.

```
SQL>
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

V\$TEMPUNDOSTAT shows various statistics related to the temporary undo log for this database instance. It displays a histogram of statistical data to show how the system is working. Each row in the view keeps statistics collected in the instance for a 10-minute interval. The rows are in the descending order of the BEGIN_TIME column value. This view contains a total of 576 rows, spanning a four-day cycle. This view is similar to the V\$UNDOSTAT view.

The example shows you some of the important columns of the V\$TEMPUNDOSTAT view:

- BEGIN_TIME: Identifies the beginning of the time interval.
- TXNCOUNT: Total Number of transactions that have bound to temp undo segment within the corresponding time interval.
- MAXCONCURRENCY: Highest number of transactions executed concurrently which modified temporary objects within the corresponding time interval.
- UNDOBLKCNT: Total number of temporary undo blocks consumed during the corresponding time interval.
- USCOUNT: Temp undo segments created during the corresponding time interval.
- NOSPACERRCNT: Total number of times the error *no space left for temporary undo* was raised during the corresponding time interval.

Note: For more information about V\$TEMPUNDOSTAT, refer to the *Oracle Database Reference Guide*.

Limiting the Size of the Program Global Area

- PGA memory usage may exceed the value set by `PGA_AGGREGATE_TARGET` due to the following reasons:
 - `PGA_AGGREGATE_TARGET` acts as a target, not a limit
 - `PGA_AGGREGATE_TARGET` only controls allocations of tunable memory
- Use the `PGA_AGGREGATE_LIMIT` initialization parameter to specify a hard limit on PGA memory usage
- When `PGA_AGGREGATE_LIMIT` is reached
 - The sessions that are using the most untunable memory will have their calls aborted.
 - If the total PGA memory usage is still over the limit, the sessions that are using the most untunable memory will be terminated.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In automatic PGA memory management mode, Oracle Database attempts to adhere to the value specified in `PGA_AGGREGATE_TARGET` by dynamically controlling the amount of PGA memory allotted to work areas. However, at times PGA memory usage may exceed the value specified by `PGA_AGGREGATE_TARGET` for the following reasons:

- `PGA_AGGREGATE_TARGET` acts as a target, not a limit.
- `PGA_AGGREGATE_TARGET` only controls allocations of tunable memory.

Excessive PGA usage can lead to high rates of swapping. If this occurs, consider using the `PGA_AGGREGATE_LIMIT` initialization parameter to limit overall PGA usage.

`PGA_AGGREGATE_LIMIT` enables you to specify a hard limit on PGA memory usage. If the `PGA_AGGREGATE_LIMIT` value is exceeded, Oracle Database aborts or terminates the sessions or processes that are consuming the most untunable PGA memory. Parallel queries are treated as a single unit.

By default, `PGA_AGGREGATE_LIMIT` is set to the greater of 2 GB, 200% of the `PGA_AGGREGATE_TARGET` value, or 3 MB times the value of the `PROCESSES` parameter. However, it will not exceed 120% of the physical memory size minus the total SGA size. The default value is written to the alert log. A warning message is written in the alert log if the amount of physical memory on the system cannot be determined.

SYS and background sessions are exempt from the limit, but not job queue processes.

Summary

In this lesson, you should have learned how to:

- Use Resource Manager in a CDB environment
- Control resources consumed by IM Column Store population operations
- Describe the multi-process multi-threaded Oracle architecture
- Describe Database Smart Flash Cache enhancements
- Use temporary undo for your temporary tables
- Limit the size of the PGA

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 20: Overview

- 20-1: Using Resource Manager to handle resources shared in a CDB and its PDBs
- 20-2: Setting up and monitoring multi-process multi-threaded architecture

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

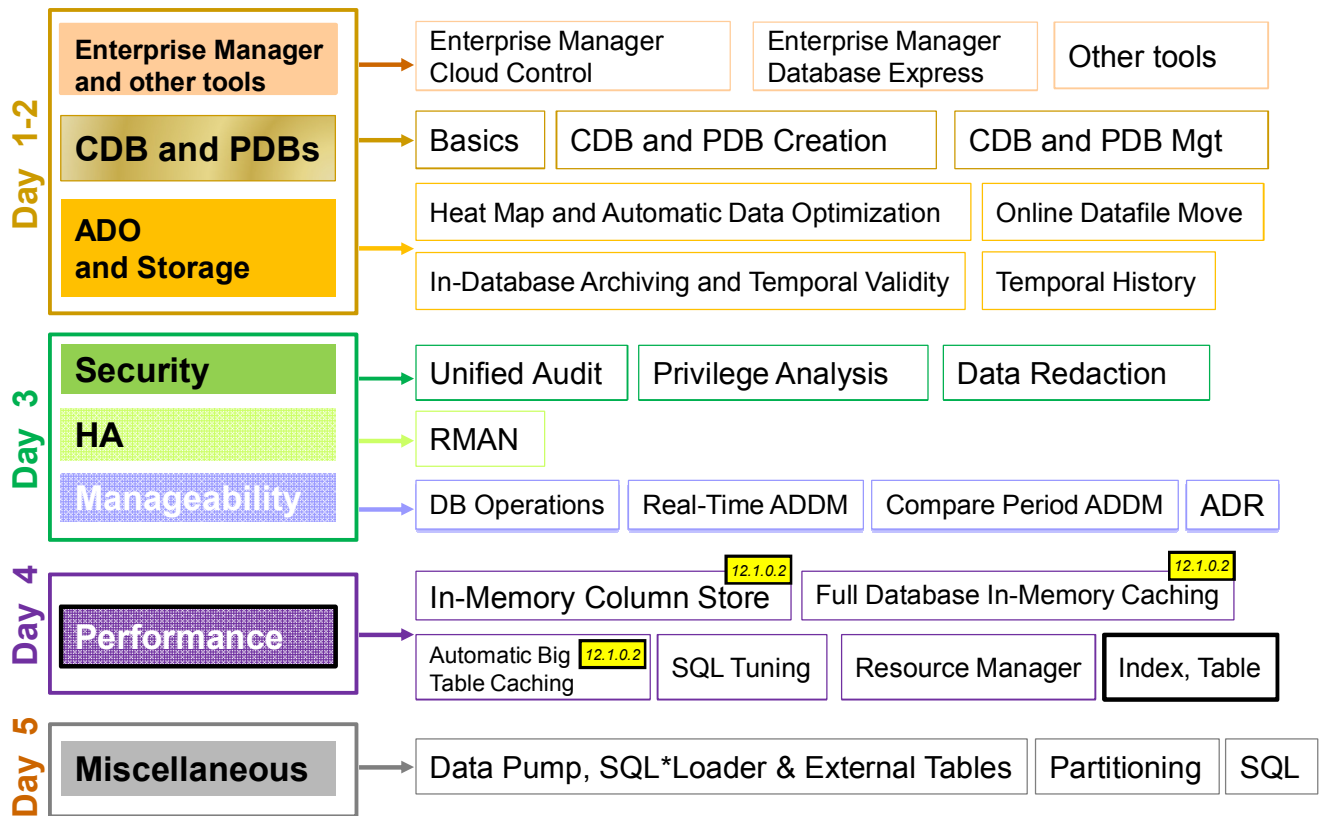
21

Tables, Indexes, and Online Operations Enhancements

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

New features for tables and indexes in Oracle Database 12c include invisible columns and multiple indexes on the same set of columns, compression, and Online Operations enhancements.

Objectives

After completing this lesson, you should be able to:

- Explain multiple indexes on the same set of columns
- Describe the support for invisible and hidden columns
- Describe online redefinition enhancements
- Describe Advanced Row Compression
- Make more DDL statements nonblocking

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For a complete understanding of multiple indexes on the same sets of columns and invisible columns on tables, refer to the following guide in the Oracle documentation:

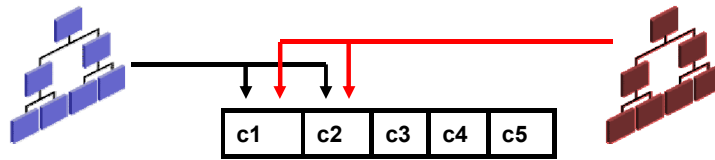
- *Oracle Database Administration Guide 12c Release 1 (12.1)*
 - “Managing Tables” Chapter
 - “Managing Indexes” Chapter

For a complete understanding of compression on tables, refer to the following guide in the Oracle documentation:

- *Oracle Database Concepts 12c Release 1 (12.1)*
 - “Tables and Table Clusters” Chapter – Table Compression
- *Oracle Database PL/SQL Packages and Types Reference.*

Why Multiple Indexes on the Same Set of Columns?

- Perform application migrations without dropping an existing index and recreating it with different attributes.
- Several different indexes:
 - Unique and nonunique index



- Different partitioning: Local and global

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

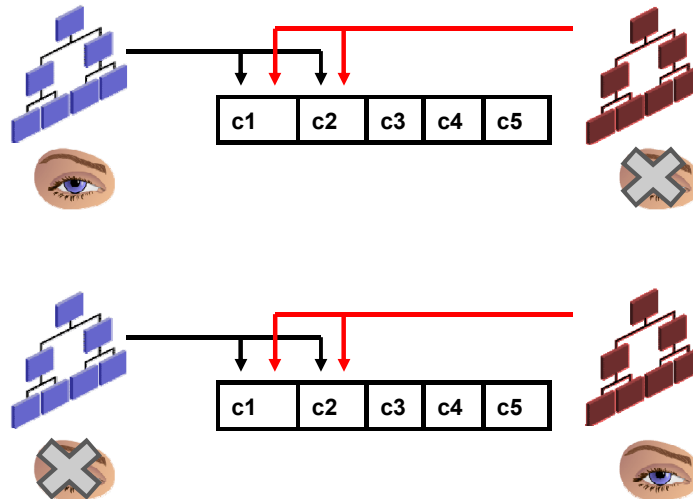
You can create multiple indexes on the same set of columns when at least one of the following index characteristics is different:

- The indexes are of different types. However, the following exceptions apply:
 - You cannot create a B-tree index and a B-tree cluster index on the same set of columns.
 - You cannot create a B-tree index and an index-organized table on the same set of columns.
- The indexes use different partitioning:
 - Indexes that are not partitioned and indexes that are partitioned
 - Indexes that are locally partitioned and indexes that are globally partitioned
 - Indexes that differ in partitioning type (range or hash)
- The indexes have different uniqueness properties: You can create both a unique and a nonunique index on the same set of columns.

When you have multiple indexes on the same set of columns, only one of these indexes is visible at a time.

Creating Multiple Indexes on the Same Set of Columns

Only **one** of the indexes **visible at a time**.



If `OPTIMIZER_USE_INVISIBLE_INDEXES=TRUE`, the optimizer uses the invisible index.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When you have multiple indexes on the same set of columns, only one of these indexes can be visible at a time.

If you are creating a visible index, any existing indexes on the set of columns must be invisible. Alternatively, you can make the other indexes on the same set of columns invisible or create the index on the set of columns as invisible.

If `OPTIMIZER_USE_INVISIBLE_INDEXES` is set to true, the optimizer can use the invisible index to create a plan.

```
SQL> create table T1 as select * from hr.employees;
SQL> create index t1_i1 on T1 (employee_id) invisible;
SQL> create index t1_i2 on T1 (employee_id) reverse;
SQL> alter session set optimizer_use_invisible_indexes = true;
SQL> explain plan for select employee_id, last_name from T1 where
employee_id=100;
```

Execution Plan

Plan hash value: 2242215931

	Id	Operation	Name	
	0	SELECT STATEMENT		
	1	TABLE ACCESS BY INDEX ROWID BATCHED	T1	
*	2	INDEX RANGE SCAN	T1_I1	

Quiz

When multiple indexes exist on the same set of columns, the optimizer chooses the best one to improve the query performance.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

Only one of the multiple indexes on the same set of columns is visible. Therefore, the optimizer does not have several indexes to choose from. The optimizer can only decide whether to use the visible one or not.

Invisible and Hidden Columns in SQL*Plus

- Invisible columns are user-specified hidden columns.

```
SQL> CREATE TABLE mytab (col1 VARCHAR2(10),
                           col2 NUMBER INVISIBLE);
```

```
SQL> DESC mytab
```

Name	Null?	Type
-----	-----	-----
COL1		VARCHAR2(10)

The invisible column is not displayed

- Can be made visible later
- Hidden columns do not affect existing applications that access the table.
- A hidden column can be accessed only by referring to its name explicitly in queries and other operations.

```
SQL> INSERT INTO mytab (col1, col2) values ('A',1);
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Invisible columns are introduced so that developers can enhance applications to meet constant changes in business rules and requirements. Invisible columns are user-specified hidden columns. With invisible columns, users can access the application while the developer is enhancing it to meet new business requirements. You expose the invisible column after the new business requirement is incorporated into the application. You can add a hidden column to a table, and then make it visible later with an `ALTER TABLE MODIFY` statement.

Users can seamlessly add a hidden column to the table without affecting existing applications that access the table. You can use an `INVISIBLE` column as a partitioning key when it is specified as a part of `CREATE TABLE`. You can specify the invisible column as a virtual column. You cannot implicitly specify a value for an `INVISIBLE` column in the `VALUES` clause of an `INSERT` statement. You must specify the `INVISIBLE` column in the column list. You can access this hidden column only by referring to its name explicitly in queries and other operations.

Note: Invisible columns are not the same as system-generated hidden columns.

You can make invisible columns visible, but you cannot make hidden columns visible.

```
SQL> ALTER TABLE mytab MODIFY (col2 VISIBLE);
```


SET COLINVISIBLE and DESCRIBE Commands

The SET COLINVISIBLE command displays the invisible columns in a DESCRIBE command.

```
SQL> SET COLINVISIBLE ON
```

```
SQL> DESC mytab
```

Name	Null?	Type
COL1		VARCHAR2 (10)
COL2 (INVISIBLE)		NUMBER

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The SET COLINVISIBLE command enables users to display the invisible column existence in a DESCRIBE command. The syntax has an ON/OFF option. ON means that metadata for the invisible column is displayed in the DESCRIBE command, and OFF means that no metadata is displayed.

When a table does not contain invisible columns, the COLINVISIBLE flag does not affect the DESCRIBE command.

The code example shows the use of the SET COLINVISIBLE command and the effect on the DESCRIBE result.

Quiz

The `SET COLINVISIBLE` command does not allow you to display the invisible columns definition in a `DESCRIBE` command.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

When set to `ON`, the command `SET COLINVISIBLE` displays the invisible columns definition.

Online Redefinition: **Tables with VPD**

Redefine a table with virtual private database (VPD) policies without changing the properties of any of the table's columns.

1. Apply a VPD policy on HR.EMP table using DBMS_RLS.ADD_POLICY procedure

```
SQL> EXEC DBMS_RLS.ADD_POLICY ( object_schema => 'hr', -
    object_name => 't1', policy_name => 't1_pol', -
    function_schema => 'sys', policy_function=> 'auth_sal', -
    statement_types => 'select, insert, update, delete')
```

2. Start the redefinition process:
 - a. Verify that the table is a possible candidate for online redefinition.
 - b. Create the interim table.
 - c. Start the redefinition.

```
SQL> EXEC DBMS_REDEFINITION.START_REDEF_TABLE ( uname => 'hr', -
    orig_table => 't1', int_table => 'int_t1', -
    options_flag => DBMS_REDEFINITION.CONST_USE_PK, -
    copy_vpd_opt => DBMS_REDEFINITION.CONST_VPD_AUTO)
```

3. Complete the redefinition process.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You apply a Virtual Private Database (VPD) policy on a table to restrict the data selected or modified by users during SELECT and DML operations.

If the original table being redefined has VPD policies, you can use the `copy_vpd_opt` parameter in the `DBMS_REDEFINITION.START_REDEF_TABLE` procedure to handle these policies during online redefinition.

The possible values for the `copy_vpd_opt` parameter are the following:

- `DBMS_REDEFINITION.CONST_VPD_NONE`: Specify this value if there are no VPD policies on the original table. This value is the default. If this value is specified and VPD policies exist for the original table, then an error is raised.
- `DBMS_REDEFINITION.CONST_VPD_AUTO`: Specify this value to copy the VPD policies automatically from the original table to the new table during online redefinition. Only the table owner and the user invoking online redefinition can access the interim table during online redefinition.
- `DBMS_REDEFINITION.CONST_VPD_MANUAL`: Specify this value to copy the VPD policies manually from the original table to the new table during online redefinition.
 - VPD policies are specified for the original table, and column mappings are done between the original table and the interim table.
 - You want to add or modify VPD policies during online redefinition of the table.

Online Redefinition: **dml_lock_timeout**

The **dml_lock_timeout** parameter in the `FINISH_REDEF_TABLE` procedure specifies how long the procedure waits for the pending DML to commit.

1. Verify that the table can be redefined online.

```
SQL> EXEC DBMS_REDEFINITION.CAN_REDEF_TABLE (...);
```

2. Create the interim table.

3. Start the redefinition.

```
SQL> EXEC DBMS_REDEFINITION.START_REDEF_TABLE (...);
```

4. Copy the dependent objects.

```
SQL> EXEC DBMS_REDEFINITION.COPY_TABLE_DEPENDENTS (...);
```

5. Complete the redefinition process.

```
SQL> EXEC DBMS_REDEFINITION.FINISH_REDEF_TABLE (uname => 'hr', -  
          orig_table          => 't1', int_table => 'int_t1' -  
          dml_lock_timeout    => 100)
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When you complete the online redefinition process, you execute the `FINISH_REDEF_TABLE` procedure. During this procedure, the original table is locked in exclusive mode for a very short time, independent of the amount of data in the original table. However, `FINISH_REDEF_TABLE` will wait for all pending DML to commit before completing the redefinition.

You can use the `dml_lock_timeout` parameter in the `FINISH_REDEF_TABLE` procedure to specify how long the procedure waits for pending DML to commit. The parameter specifies the number of seconds to wait before the procedure ends gracefully.

- When you specify a non-NULL value for this parameter, you can restart the procedure, and it continues from the point at which it timed out.
- When the parameter is set to NULL, the procedure does not time out. In this case, if you stop the procedure manually, then you must abort the online table redefinition by using the `ABORT_REDEF_TABLE` procedure and start over from the beginning of step 5 (as shown in the slide).

Advanced Row Compression: New Feature Name and Syntax

11g	<i>COMPRESS</i>	
Method	CREATE and ALTER TABLE	Typical Apps
Basic	COMPRESS [BASIC]	DSS
OLTP	COMPRESS FOR OLTP	OLTP, DSS

SQL> CREATE TABLE tab1 COMPRESS [BASIC];		
--	--	--

SQL> CREATE TABLE tab1 COMPRESS FOR OLTP;		
---	--	--

12c	<i>ROW STORE COMPRESS</i>	
Method	CREATE and ALTER TABLE	Typical Apps
Basic	ROW STORE COMPRESS	DSS
Advanced	ROW STORE COMPRESS ADVANCED	OLTP, DSS

SQL> CREATE TABLE tab1 ROW STORE COMPRESS [BASIC];		
--	--	--

SQL> CREATE TABLE tab1 ROW STORE COMPRESS ADVANCED;		
---	--	--

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 11g extended the compression technology to support regular data manipulation operations such as `INSERT`, `UPDATE`, and `DELETE`. Consequently, compression could be used for all kinds of workload, be it online transaction processing (OLTP) or data warehousing.

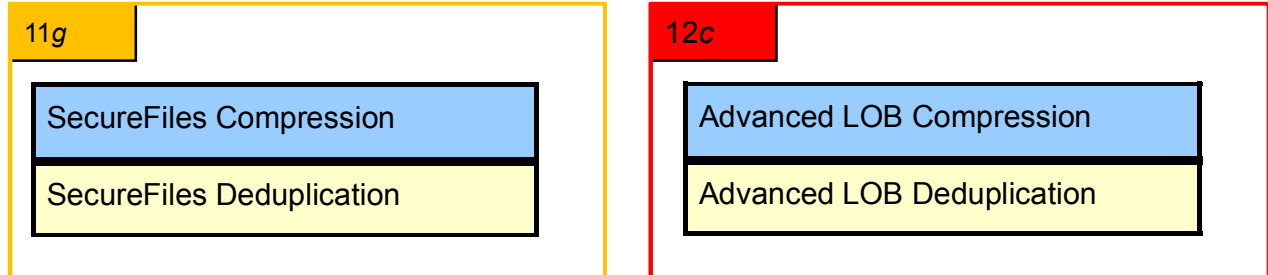
Oracle Database 12c brings incremental enhancements for OLTP Compression. The OLTP Compression feature was released in 11g Release 1 as a part of Advanced Compression Option (ACO). The OLTP Compression, now named Advanced Row Compression, increases savings and provides higher compression ratios. One problem with OLTP and BASIC Compression is that when a row is updated, all the columns of the row are uncompressed up to the highest referenced column for the update. Now only those columns that are updated are uncompressed. When OLTP compression did not support more than 255 columns, row store compression supports more than 255 columns. The shrink space operation is now supported for row store compressed table.

Row Store or Column Store are attributes of a table, like the organization clause. Compression and locking options are options of the store.

ROW STORE and COMPRESS are the new attributes to compress a table (ROW STORE versus COLUMN STORE or HCC).

ADVANCED and BASIC are types of compression, used uniformly for tables, indexes, and LOBs to convey that it is part of ACO.

LOB Compression: New Name



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

SecureFile LOB

With Oracle Database 12c, the names of two features in the Advanced Compression Option related to SecureFiles has changed:

- SecureFiles Compression is now called Advanced LOB Compression.
- SecureFiles Deduplication is now called Advanced LOB Deduplication.

Using the Compression Advisor

- A compression advisor helps to determine optimal compression ratios.

```
BEGIN
  DBMS_COMPRESSION.GET_COMPRESSION_RATIO (
    'USERS', 'HR', 'EMP', NULL, DBMS_COMPRESSION.COMP_ADVANCED,
    blkcnt_cmp, blkcnt_uncmp, row_cmp, row_uncmp, cmp_ratio,
    comptype_str,1000,1);
  DBMS_OUTPUT.PUT_LINE('Block count compressed = ' || blkcnt_cmp);
  DBMS_OUTPUT.PUT_LINE('Block count uncompressed = ' || blkcnt_uncmp);
  DBMS_OUTPUT.PUT_LINE('Row count per block compressed = ' || row_cmp);
  DBMS_OUTPUT.PUT_LINE('Row count per block uncompressed=' || row_uncmp);
  DBMS_OUTPUT.PUT_LINE('Compression type = ' || comptype_str);
  DBMS_OUTPUT.PUT_LINE('Comp ratio= ' || blkcnt_uncmp/blkcnt_cmp || 'to 1');
END;
```

- New compression types in COMPRESS_FOR column in DBA_TABLES view

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The compression advisor has been enhanced to display the new advanced compression type.

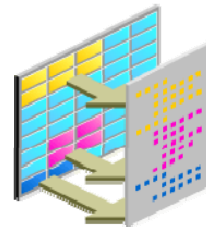
SQL> set serveroutput on

```
DECLARE
  blkcnt_cmp pls_integer;
  blkcnt_uncmp pls_integer;
  row_cmp pls_integer;
  row_uncmp pls_integer;
  cmp_ratio pls_integer;
  comptype_str varchar2(100);
BEGIN
  DBMS_COMPRESSION.GET_COMPRESSION_RATIO (
    'USERS', 'HR', 'EMP', NULL, DBMS_COMPRESSION.COMP_ADVANCED,
    blkcnt_cmp, blkcnt_uncmp, row_cmp, row_uncmp, cmp_ratio,
    comptype_str,1000,1);
  DBMS_OUTPUT.PUT_LINE('Block count compressed = ' || blkcnt_cmp);
  DBMS_OUTPUT.PUT_LINE('Block count uncmp = ' || blkcnt_uncmp);
  DBMS_OUTPUT.PUT_LINE('Compression type = ' || comptype_str);
  DBMS_OUTPUT.PUT_LINE('Ratio = ' || blkcnt_uncmp/blkcnt_cmp || ' to 1');
END;
/
Block count compressed = 5
Block count uncompressed = 7
Compression type = "Compress Advanced"
Compression ratio = 1.4 to 1
PL/SQL procedure successfully completed.
```

Enhanced Online DDL Capabilities

You can use the new **ONLINE** keyword to allow the execution of DML statements during the following DDL operations:

- DROP INDEX
- DROP CONSTRAINT
- ALTER INDEX UNUSABLE
- SET COLUMN UNUSED



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Beginning with Oracle Database 12c Release 1 (12.1), you can use the new **ONLINE** keyword to allow the execution of DML statements during the following DDL operations:

- DROP INDEX
- DROP CONSTRAINT
- ALTER INDEX UNUSABLE
- SET COLUMN UNUSED

This enhancement enables simpler application development, especially for application migrations. There are no application disruptions for schema maintenance operations.

DROP INDEX / CONSTRAINT



- **ONLINE** indicates that DML operations on the table or partition are allowed while dropping the index.
- DROP INDEX online is supported for partitioned and nonpartitioned indexes.

```
SQL> DROP INDEX schema.index ONLINE FORCE;
```

- DROP CONSTRAINT online enables you to drop an integrity constraint from a database with a few restrictions:
 - Cannot drop a constraint with CASCADE
 - Cannot drop a referencing constraint

```
SQL> ALTER TABLE hr.employees  
      DROP CONSTRAINT emp_email_uk ONLINE;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The DROP INDEX statement is used to remove an index or domain index from a database. Drop index online is supported for partitioned and nonpartitioned indexes.

ONLINE: Specify **ONLINE** to indicate that DML operations on the table or partition are allowed while dropping the index.

FORCE: FORCE applies only to domain indexes. This clause drops the domain index even if the index type routine invocation returns an error or if the index is marked **IN PROGRESS**. Without **FORCE**, you cannot drop a domain index if its index type routine invocation returns an error or if the index is marked **IN PROGRESS**.

Restrictions on dropping indexes:

- You cannot drop a domain index if the index or any of its index partitions is marked **IN_PROGRESS**.
- You cannot specify the **ONLINE** clause when dropping a domain index, a cluster index, or an index on a queue table.

Restrictions on dropping constraints:

- You cannot drop a constraint with **CASCADE**.
- You cannot drop a constraint that holds references for other dependencies.

Index UNUSABLE

- Specify the `UNUSABLE` clause to mark the index or index partitions or index subpartitions `UNUSABLE`.
- Specify the **ONLINE** clause to indicate that DML operations on the table or partition are allowed while marking the index `UNUSABLE`.

```
SQL> ALTER INDEX hr.i_emp_ix UNUSABLE ONLINE;
```

```
SQL> SELECT status FROM user_indexes
      WHERE table_name='EMP';
```

INDEX_NAME	STATUS
I_EMP_IX	UNUSABLE

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Specify the `UNUSABLE` keyword to mark the index or index partitions or index subpartitions `UNUSABLE`. The space allocated for an index or index partition or subpartition is freed immediately when you mark the object `UNUSABLE`. An unusable index must be rebuilt, or it must be dropped and re-created, before it can be used. While one partition is marked `UNUSABLE`, the other partitions of the index are still valid. As of Oracle 12c, the `ALTER INDEX UNUSABLE` clause is made online by specifying an `ONLINE` keyword to indicate that DML operations on the table or partition are allowed while marking the index `UNUSABLE`. If you specify this clause, the database does not drop the index segments.

Note: You cannot specify `UNUSABLE` for an index on a temporary table.

SET UNUSED Column

- The `SET UNUSED` clause can be the first step to free space in the database by dropping columns no longer needed.
- The **ONLINE** keyword indicates that DML operations on the table are allowed while marking the columns `UNUSED`.

```
SQL> CREATE TABLE emp (ename VARCHAR2(20), id NUMBER);
SQL> INSERT INTO emp VALUES('Tim',4);
SQL> SELECT * FROM emp;
SQL> ALTER TABLE emp SET UNUSED (ename) ONLINE;

SQL> DESC emp;
Name                Null?    Type
-----
ID                  NUMBER
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The `drop_column_clause` can be the first step to free space in the database by dropping columns that you no longer need or by marking them to be dropped at a future time when the demand on system resources is less.

You specify the `SET UNUSED` keyword to mark one or more columns as unused. When you specify this clause for a column in an external table, the clause is transparently converted to an `ALTER TABLE ... DROP COLUMN` statement. Because any operation on an external table is a metadata-only operation, there is no difference in the performance of the two commands. Unused columns are treated as if they were dropped, even though their column data remains in the table rows. After marking a column `UNUSED`, you have no access to it. A `SELECT *` query does not retrieve data from unused columns. In addition, the names and types of columns marked `UNUSED` are not displayed during a `DESCRIBE`, and you can add a new column to the table with the same name as an unused column.

You specify the `ONLINE` keyword to indicate that DML operations on the table are allowed while marking the column or columns `UNUSED`.

You cannot specify the `ONLINE` clause when marking a column with a `DEFERRABLE` constraint as unused. A subsequent `DROP UNUSED COLUMNS` physically removes all unused columns from a table, similar to a `DROP COLUMN`.

Summary

In this lesson, you should have learned how to:

- Explain multiple indexes on the same set of columns
- Describe the support for invisible and hidden columns
- Describe online redefinition enhancements
- Describe Advanced Row Compression
- Make more DDL statements nonblocking

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 21: Overview

- 21-1: Using invisible/visible columns
- 21-2: Using Advanced Row Compression

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Module

Miscellaneous

ORACLE

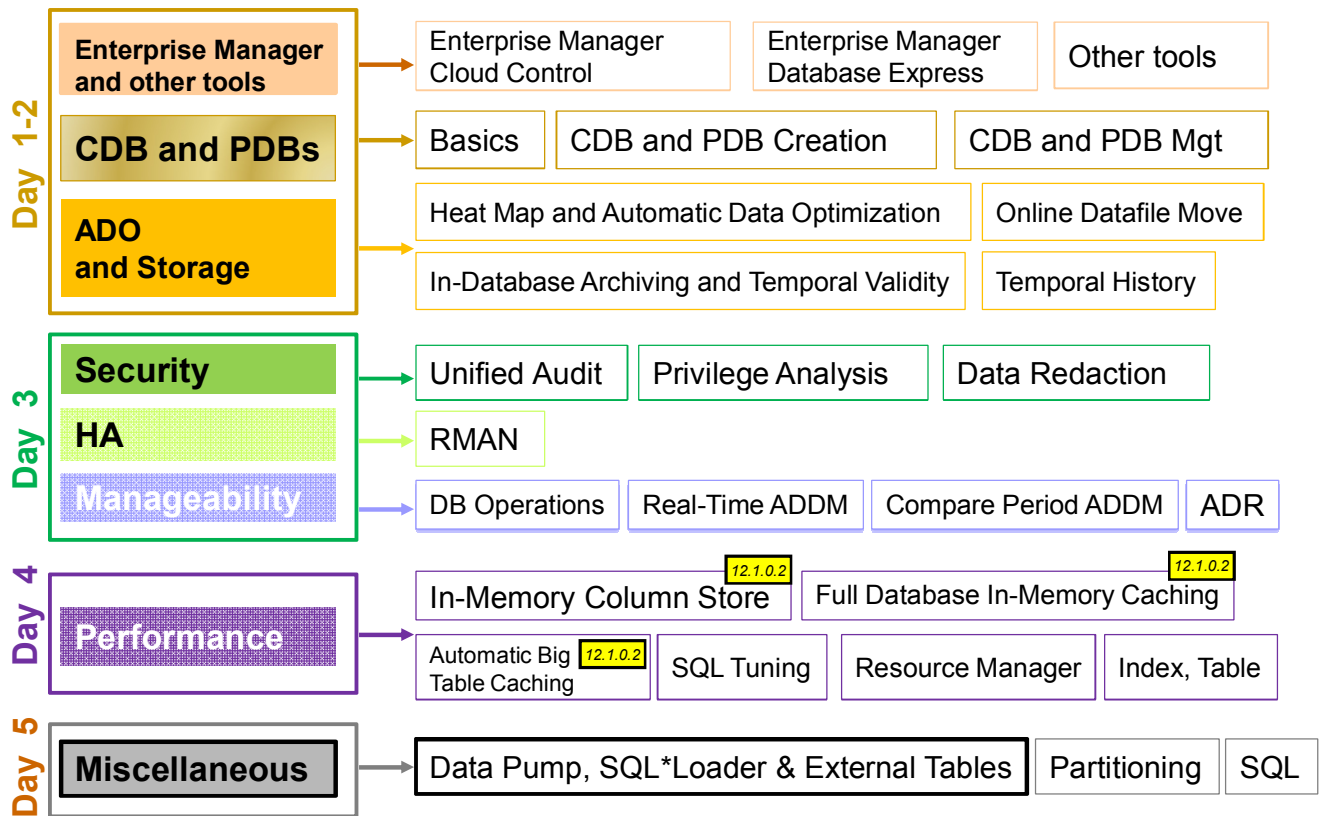
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Data Pump, SQL*Loader, and External Tables

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are several **Oracle Data Pump**, **SQL*Loader**, and **External Tables** enhancements. You will also examine **SQL*Loader Express Mode** feature.

Objectives

After completing this lesson, you should be able to:

- Describe Oracle Data Pump Full Transportable
- Describe each Oracle Database 12c new feature for Data Pump, SQL*Loader, and External Tables
- Describe applicable use cases and configuration details for the Oracle Data Pump and SQL*Loader new features
- Describe SQL*Loader Express Mode

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For a complete understanding of Oracle Data Pump new features and usage, refer to the following guides in the Oracle documentation:

- *Oracle Database Utilities 12c Release 1 (12.1)*

Refer to other sources of information available:

- My Oracle Support note: *"Reduce Transportable Tablespace Downtime Using Cross Platform Incremental Backups"* (Doc ID 1389592.1)
- Oracle White Paper: *"Upgrading to Oracle Database 12c"*
(<http://www.oracle.com/technetwork/database/upgrade/upgrading-oracle-database-wp-12c-1896123.pdf>)
- Oracle White Paper : *Oracle Database 12c: Full Transportable Export/Import*
(<http://www.oracle.com/technetwork/database/enterprise-edition/full-transportable-wp-12c-1973971.pdf>)

Full Transportable Export/Import: Overview

⇒ Upgrading to a new release of Oracle Database

⇒ Transporting a database to a new computer system

- Transport a non-CDB (Oracle Database 11.2.0.3 and up) into another non-CDB.
- Transport a non-CDB (11.2.0.3 and +) into a CDB as a PDB.
- Transport a PDB into another PDB.
- Transport a PDB into a non-CDB.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The full transportable export/import feature is useful in several scenarios:

- **Upgrading to a new release of Oracle Database:** You can use full transportable export/import to upgrade a database from 11.2.0.3 or higher to Oracle Database 12c. To do so, install Oracle Database 12c and create an empty database. Next, use full transportable export/import to transport the 11.2.0.3 database into the Oracle Database 12c database.
- **Transporting a database to a new computer system:** You can use full transportable export/import to move a database from one computer system to another. You might want to move a database to a new computer system to upgrade the hardware or to move the database to a different platform.
- **Transporting a non-CDB into a non-CDB or CDB:** Transported into a CDB, the transported database becomes a PDB associated with the CDB. Full transportable export/import can move an 11.2.0.3 or higher database into an Oracle Database 12c database efficiently.

Full Transportable Export/Import: Usage

A full transportable export exports all objects and data necessary to create a complete copy of the database.

- TRANSPORTABLE=ALWAYS parameter
- FULL parameter

```
$ expdp user_name FULL=y ... TRANSPORTABLE=always
```

A full transportable import imports a dump file only if it has been created by using the transportable option during export.

- TRANSPORT_DATAFILES
- If NETWORK_LINK used, requires:
 - TRANSPORTABLE=ALWAYS parameter

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A full transportable export exports all objects and data necessary to create a complete copy of the database. A mix of data movement methods is used:

- Objects residing in transportable tablespaces have only their metadata unloaded into the dump file set. The data itself is moved when you copy the data files to the target database. The data files that must be copied are listed at the end of the log file for the export operation.
- Objects residing in non-transportable tablespaces (for example, SYSTEM and SYSAUX) have both their metadata and data unloaded into the dump file set, using direct path unload and external tables.

The example shows a full transportable export using the VERSION parameter. The VERSION parameter is required if the source database is an 11.2.0.3 or a higher Oracle Database 11g release.

Performing a full transportable export has the following restrictions:

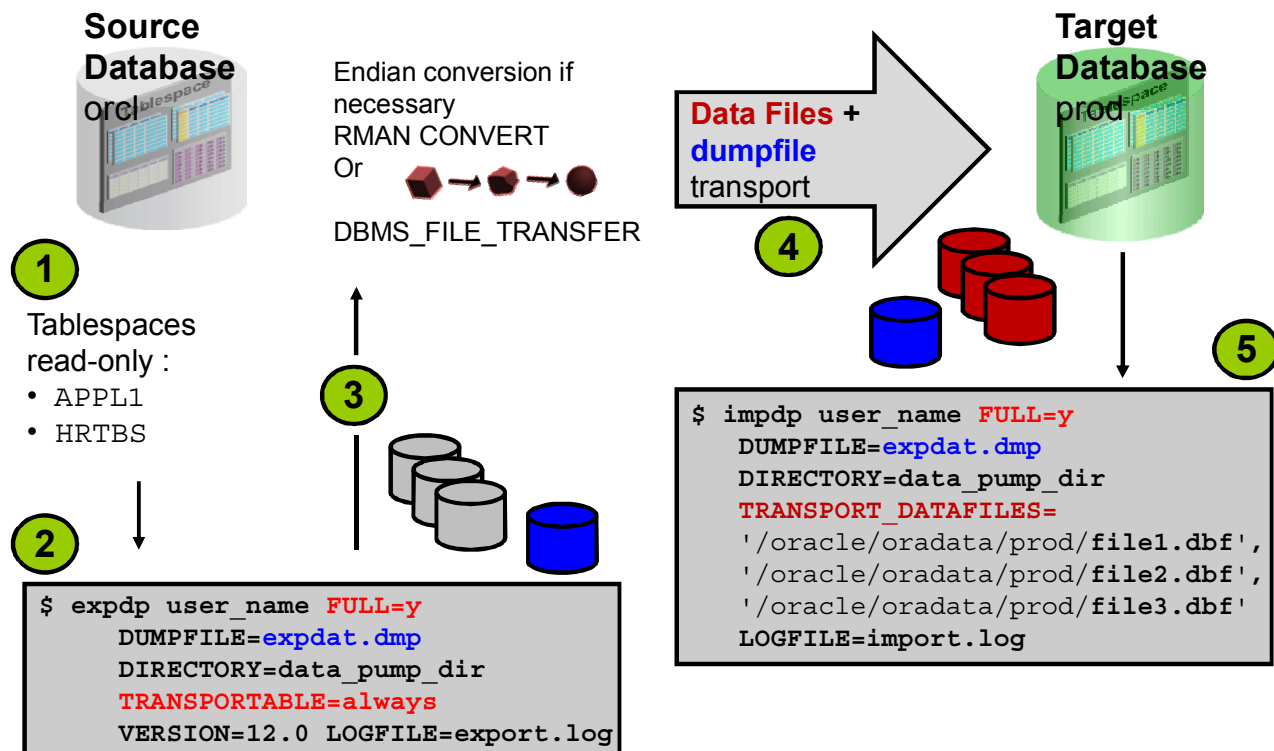
- If the database being exported contains either encrypted tablespaces or tables with encrypted columns (either Transparent Data Encryption (TDE) columns or SecureFile LOB columns), then the `ENCRYPTION_PASSWORD` parameter must also be supplied.
- The source and target databases must be on platforms with the same endianness if there are encrypted tablespaces in the source database.
- If the source platform and the target platform are of different endianness, you must convert the data being transported so that it is in the format of the target platform. Use either the `DBMS_FILE_TRANSFER` package or the `RMAN CONVERT` command.
- A full transportable export is not restartable.
- All objects with storage that are selected for export must have all of their storage segments either entirely within administrative, non-transportable tablespaces (`SYSTEM / SYSAUX`) or entirely within user-defined, transportable tablespaces. Storage for a single object cannot straddle the two kinds of tablespaces.
- When transporting a database over the network using full transportable export, tables with `LONG` or `LONG RAW` columns that reside in administrative tablespaces (such as `SYSTEM` or `SYSAUX`) are not supported.
- When transporting a database over the network using full transportable export, auditing cannot be enabled for tables stored in an administrative tablespace (such as `SYSTEM` and `SYSAUX`) if the audit trail information itself is stored in a user-defined tablespace.
- If both the source and target databases are running Oracle Database 12c Release 1 (12.1), then to perform a full transportable export, either the Oracle Data Pump `VERSION` parameter must be set to at least 12.0. or the `COMPATIBLE` database initialization parameter must be set to at least 12.0 or later.
- Full transportable exports are supported from an 11.2.0.3 source database.

Full Transportable Import

Performing a full transportable import has the following requirements:

- If you are using a network link, then the database specified on the `NETWORK_LINK` parameter must be Oracle Database 11g release 2 (11.2.0.3) or later, and the Oracle Data Pump `VERSION` parameter must be set to at least 12. (In a non-network import, `VERSION=12` is implicitly determined from the dump file.)
- If the source platform and the target platform are of different endianness, then you must convert the data being transported so that it is in the format of the target platform. You can use the `DBMS_FILE_TRANSFER` package or the `RMAN CONVERT` command to convert the data.
- A full transportable import of encrypted tablespaces is not supported in network mode or dump file mode if the source and target platforms do not have the same endianness.
- When transporting a database over the network using full transportable import, tables with `LONG` or `LONG RAW` columns that reside in administrative tablespaces (such as `SYSTEM` or `SYSAUX`) are not supported.
- When transporting a database over the network using full transportable import, auditing cannot be enabled for tables stored in an administrative tablespace (such as `SYSTEM` and `SYSAUX`) if the audit trail information itself is stored in a user-defined tablespace.

Full Transportable Export/Import: Example



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To perform a Full Transportable operation, perform the following steps:

1. Before the export, make all of the user-defined tablespaces in the database read-only.
2. Invoke the Oracle Data Pump export utility as a user with DATAPUMP_EXP_FULL_DATABASE role and specify the full transportable export options: FULL=Y, TRANSPORTABLE=ALWAYS.
The LOGFILE parameter is important because it will contain the list of data files that need to be transported for the import operation.
To perform the operation on an 11.2.0.3 database, use the VERSION parameter. Full transportable import is supported only for Oracle Database 12c databases.
3. Before the import, transport the dump file.
4. Transport the data files that you may have converted. If you are transporting the database to a platform different from the source platform, then determine if cross-platform database transport is supported for both the source and target platforms. If both platforms have the same endianness, no conversion is necessary. Otherwise you must do a conversion of each tablespace in the database either at the source or target database using either DBMS_FILE_TRANSFER or RMAN CONVERT command.
5. Invoke the Oracle Data Pump import utility as a user with DATAPUMP_IMP_FULL_DATABASE role and specify the full transportable import options: FULL=Y, TRANSPORT_DATAFILES.
6. Make the source tablespaces read-write. You can perform this operation before step 5.

Transporting a Database Over the Network: Example

Transport a database over the network: perform an import using the `NETWORK_LINK` parameter.

1. Create a database link in the target to the source database.
2. Make the user-defined tablespaces in the source database read-only.
3. Transport the datafiles for all of the user-defined tablespaces from the source to the target location.
4. Perform conversion of the datafiles if necessary.
5. Import in the target database.

```
$ impdp username full=Y network_link=sourcedb  
transportable=always  
transport_datafiles='/D1/sales01.dbf','/D1/cust01.dbf'  
encryption_password=pass1 version=12 logfile=import.log
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To transport a database over the network, use import with the `NETWORK_LINK` parameter. The import is performed using a database link, and there is no dump file involved.

- If the source database is an 11.2.0.3 database or a higher Oracle Database 11g release database, then the `VERSION` parameter is required and must be set to 12. If the source database is an Oracle Database 12c database, then the `VERSION` parameter is not required.
- If the source database contains any encrypted tablespaces or tablespaces containing tables with encrypted columns, then you must specify the `ENCRYPTION_PASSWORD` parameter.
- The Oracle Data Pump network import copies the metadata for objects contained within the user-defined tablespaces and both the metadata and data for user-defined objects contained within the administrative tablespaces, such as `SYSTEM` and `SYSAUX`.
- When the import is complete, the user-defined tablespaces are in read-write mode.
- Make the user-defined tablespaces read-write again at the source database.

Disabling Logging for Oracle Data Pump Import

- Delivers faster data loads
- Useful for large data loads and populating new databases
- Recommended full database backup after data load

Notes

- Logging is not completely eliminated. A small amount of log activity is still generated by the import.
- Logging is not disabled if the database is in `FORCE LOGGING` mode.

```
$ impdp ... TRANSFORM=DISABLE_ARCHIVE_LOGGING:Y
```

1

```
$ impdp ... TRANSFORM=DISABLE_ARCHIVE_LOGGING:Y:INDEX
```

2

```
$ impdp ... TRANSFORM=DISABLE_ARCHIVE_LOGGING:Y
TRANSFORM=DISABLE_ARCHIVE_LOGGING:N:TABLE
```

3

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Data Pump provides an option to disable logging during an import operation. The result is quicker imports, because redo log information is not written to disk or archived, allowing greater I/O bandwidth to be dedicated to data loading. Disabling logging is particularly useful for large data loads or jobs that initially populate a new database.

If you disable logging, Oracle recommends that you perform a full Recovery Manager (RMAN) backup after the Oracle Data Pump import operation is completed. Also, in the unlikely event of media failure in the middle of the import, you must restart the import.

If `DISABLE_ARCHIVE_LOGGING` is set to `Y`, such as in example 1, then the logging attributes for the specified object types, `TABLE` and/or `INDEX` are disabled before the data is imported. If it is set to `N` (the default), then archive logging is not disabled during import. After the data has been loaded, the logging attributes for the objects are restored to their original settings. If no object type is specified, then the `DISABLE_ARCHIVE_LOGGING` behavior is applied to both `TABLE` and `INDEX` object types.

Note that the operations against the master table that is used by Oracle Data Pump to coordinate its activities are logged, even if logging is disabled for the Oracle Data Pump import session. If the database is in `FORCE LOGGING` mode, logging is not disabled during the Oracle Data Pump load even if the option to do so is specified.

- Example 2 disables logging for `INDEX` only.
- Example 3 shows a different way to achieve the same outcome as example 2.

Exporting Views as Tables

- Using `VIEWS_AS_TABLES` to export:
 - A table definition and the **view data**
 - Dependent objects (such as constraints and grants)
- Useful for:
 - Exporting a data subset
 - Exporting a denormalized data set spanning multiple tables

```
$ expdp ... VIEWS_AS_TABLES=employees_v TABLES=departments
```

1

```
$ expdp ... VIEWS_AS_TABLES=managers_v,oe.orders_v
```

2

```
$ impdp ... VIEWS_AS_TABLES=managers_v REMAP_TABLE=managers_v:managers
```

3

```
$ impdp hr1/hr1 VIEWS_AS_TABLES=employees_v NETWORK_LINK=hr_link  
REMAP_TABLE=employees_v:employees TABLE_EXISTS_ACTION=append
```

4

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Data Pump provides the `VIEWS_AS_TABLES` option to export a view as a table. Instead of just exporting the view definition SQL, Oracle Data Pump writes into the dump file a corresponding table definition and all the data visible in the view.

At import time, Oracle Data Pump can create a table with the same geometry (columns and data types) as the original view and populate it with the exported data.

Objects depending on the view are also exported as if they were defined on the table. For example, grants that are associated with the view are recorded as grants on the corresponding table in the export dump file. Dependent objects that do not apply to tables are not exported.

The ability to export views as tables can be used to export data subsets for development and testing, data replication, and offline archival purposes.

You can also use this feature to export a denormalized data set spanning multiple related tables. This kind of operation is commonly encountered when data is extracted from transactional systems and loaded into decision support systems and data warehouses.

Exporting views as tables is limited only to relational views with scalar columns. More complex views, including views that reference object types or functions, are not supported.

The `VIEWS_AS_TABLES` parameter unloads view data in unencrypted format. If you unload sensitive data, Oracle strongly recommends that you enable encryption on the export operation and that you ensure that the imported table is encrypted.

The slide shows four examples using the `VIEWS_AS_TABLES` command-line parameter.

- Example 1 is a simple case, where the `VIEWS_AS_TABLES` parameter specifies a single view (`hr.employees_v`). You can use this parameter by itself or with the `TABLES` parameter. If you use either parameter, Oracle Data Pump performs a table-mode export.
- Example 2 exports two views as tables (`hr.managers_v` and `oe.orders_v`).
- Example 3 uses the export dump file that was created in example 2. It demonstrates using the `VIEWS_AS_TABLES` parameter in conjunction with `impdp` to selectively import `managers_v` as the `managers` table.
- Example 4 shows the `VIEWS_AS_TABLES` parameter being used in conjunction with a network import. In this mode, the syntax for the `VIEWS_AS_TABLES` parameter is the same as that supported by Oracle Data Pump export (`expdp`). In this example, the view `employees_v` is sourced from the database specified by the `hr_link` database link. The data from `employees_v@hr_link` is imported into the `employees` table in the `hr1` schema on the target database. Because `TABLE_EXISTS_ACTION=append` is specified, the data from the source view rows are appended to the `hr1.employees` table if it already exists.

Specifying the Encryption Password

- This new option prompts users to specify encryption.

password:

```
$ expdp ... ENCRYPTION_PWD_PROMPT=Y ...
...
Password: <Specify password here. Input not echoed>
...
```

- With this feature, encryption password security is enhanced because it is supplied silently at run time.
 - The password is not visible by using commands like `ps`.
 - The password is not stored in scripts.
- Concurrent use of the `ENCRYPTION_PASSWORD` parameter and `ENCRYPTION_PWD_PROMPT=Y` is prohibited and generates an error.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Earlier versions of the Oracle Data Pump utilities (`expdp` and `impdp`) provided support for encrypting data written to the export dump file. Although you can use an Oracle Wallet for transparent encryption, password-based encryption is also available.

To support password-based encryption and enable a password to be specified, a command-line parameter, `ENCRYPTION_PASSWORD`, was added to the Oracle Data Pump utilities. Before Oracle Database 12c, using the `ENCRYPTION_PASSWORD` parameter was the only way to set a password for password-based encryption in conjunction with Oracle Data Pump export. The issue with this mechanism is that the encryption password can be easily compromised. Nonprivileged system users can typically use utilities such as `ps` on Linux and UNIX to view command-line parameters, including the encryption password. Also, storing the encryption password inside scripts is generally considered to be poor practice from a security perspective.

With Oracle Database 12c, the Oracle Data Pump utilities have the ability to prompt the user for an encryption password without the value being echoed as it is entered. The new command-line parameter, `ENCRYPTION_PWD_PROMPT`, can be set to `Y` to instruct the utilities to prompt the user for the encryption password. The password is read from the standard input stream, and terminal echo is suppressed while the password is being entered.

Compressing Tables During Import

- Compression method can now be specified at import time.
 - Independent of compression settings present at export time.
- Command-line syntax and example:

```
$ impdp ... TRANSFORM = TABLE_COMPRESSION_CLAUSE:NONE | "<comp-clause>" ...
```

```
$ impdp ... TRANSFORM = TABLE_COMPRESSION_CLAUSE:"ROW STORE COMPRESS"
```

- Syntax notes:
 - NONE => compression clause omitted from CREATE TABLE
 - <comp-clause> => valid table compression clause

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Using earlier versions of Oracle Data Pump, tables are imported with the same compression settings that were present during export. Oracle Database 12c enables Oracle Data Pump to specify the compression method at import time regardless of the settings in the source database.

The option to compress (or uncompress) tables during import is provided by a new metadata transform parameter, `TABLE_COMPRESSION_CLAUSE`, which can be set on the `impdp` command line or programmatically by using the `DBMS_DATAPUMP.METADATA_TRANSFORM` PL/SQL procedure.

If the parameter is set to `NONE`, the table compression clause is omitted and imported tables get the default compression setting associated with the tablespace. This option is particularly useful when you want to apply a uniform compression method to a series of tables that may have had different compression settings in the source database.

Otherwise, the parameter value must be a valid table compression clause, such as `NOCOMPRESS`, `ROW STORE COMPRESS`, and `ROW STORE COMPRESS ADVANCED`.

Table compression clauses containing more than one keyword must be enclosed in double quotation marks when they are specified on the `impdp` command line. Because quotation marks in the command line can have special meaning in some operating systems, consider using a parameter file to set the transform parameter.

Creating SecureFile LOBs During Import

- Option to specify LOB storage method at import time
- Command-line syntax and example:

```
$ impdp ... TRANSFORM = LOB_STORAGE:SECUREFILE|BASICFILE|  
                        DEFAULT|NO_CHANGE ...
```

```
$ impdp ... LOB_STORAGE:SECUREFILE
```

- Syntax notes:
 - DEFAULT => no lob storage clause for CREATE TABLE
 - NO_CHANGE => settings specified in dump file

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 12c enables Oracle Data Pump to specify the LOB storage method at import time regardless of the settings in the source database. This capability provides a straightforward mechanism for users to migrate from BasicFile LOBs to SecureFile LOBs.

The option to specify the LOB storage method during import is provided by a new metadata transform parameter, `LOB_STORAGE`, which can be set on the `impdp` command line or programmatically by using the `DBMS_DATAPUMP.METADATA_TRANSFORM` PL/SQL procedure.

By setting the parameter to `SECUREFILE` or `BASICFILE`, you specify the corresponding LOB storage method for all LOBs that are associated with the import session. If you set the parameter to `DEFAULT`, the LOB storage clause is omitted and imported tables get the default setting that is associated with the session or instance. If you set the parameter to `NO_CHANGE`, the LOB storage clauses that are specified in the export dump file are used.

Quiz

Setting `TRANSFORM=TABLE_COMPRESSION_CLAUSE:NONE` on the `impdp` command causes all the tables created by the `impdp` session to be uncompressed.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: b

Imported tables get the default compression setting associated with the tablespace.

SQL*Loader Support for Direct-Path Loading of Identity Columns

- An identity column is a numeric column that is assigned an integer value from a sequence generator for each `INSERT` performed on the table.
- SQL*Loader supports direct-path loading of identity columns:

Column specification	Action performed by SQL*Loader
<code>col1 NUMBER GENERATED ALWAYS AS IDENTITY</code>	<code>col1</code> cannot be loaded explicitly. The column value is always provided by the sequence generator.
<code>col2 NUMBER GENERATED BY DEFAULT AS IDENTITY</code>	<code>col2</code> can be loaded explicitly. If it is not, the column value is provided by the sequence generator. Explicitly loaded <code>NULL</code> values cause the corresponding row to be rejected because identity columns implicitly carry the <code>NOT NULL</code> constraint.
<code>col3 NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY</code>	<code>col3</code> can be loaded explicitly. If it is not, the column value is provided by the sequence generator. Explicitly loaded <code>NULL</code> values are replaced by sequence values.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

SQL*Loader new features start with support for direct-path loading of identity columns.

An identity column is a numeric column that is assigned an increasing or decreasing integer value from a sequence generator for each `INSERT` statement that is performed on the associated table. You specify identity columns by using the `GENERATED ... AS IDENTITY` clause in association with a `CREATE TABLE` or `ALTER TABLE` statement. Identity columns are new in Oracle Database 12c.

In conjunction with this new feature, SQL*Loader supports direct-path loading of identity columns. The table on the slide describes the actions that are performed by SQL*Loader for different types of identity columns. These actions occur automatically and are consistent with the normal behavior of identity columns.

SQL*Loader and External Table Enhancements

- Wildcards allowed in names of data files
 - `INFILE ('emp*.dat') or ('emp?.dat')`
- Support for CSV files with embedded newlines
 - `FIELDS CSV WITH EMBEDDED ...`
- Table-level `NULLIF` and Date formats can be specified once for all character and date fields
 - `FIELDS DATE FORMAT "DD-Month-YYYY"`
 - `FIELDS TERMINATED BY "," NULLIF = "NA"`
- With `FIELD NAMES` clause, the first record in the data file contains the names and order of the fields
 - `FIELD NAMES FIRST FILE`
- Specific to external tables: `ALL FIELDS OVERRIDE`
 - Default attributes for columns

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The following statements can be applied to SQL*Loader and External Tables. The syntax may differ when using SQL*Loader or External Tables.

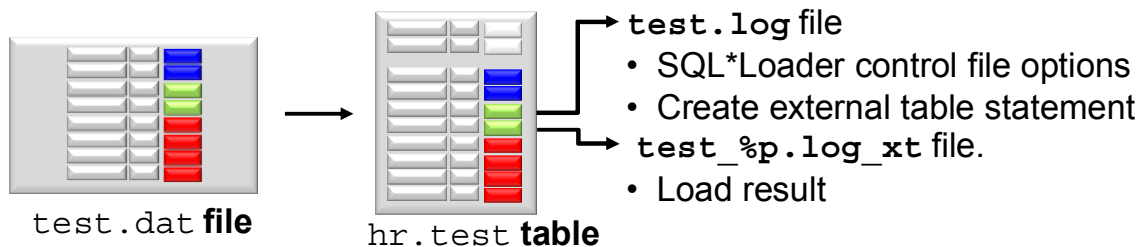
- Wildcards are allowed in data file names: "*" is for multiple characters and "?" is for single characters.
- CSV files are supported in Oracle Database 12c under conditions. Read *Oracle Database Utilities 12c Release 1 (12.1)* to get the conditions under which CSV files data can be loaded.
- Table-level `NULLIF` and date format can be specified once for all fields and can be overridden for an individual field. It applies to all character and date fields for `NULLIF` and to all date fields for date format.
- The first record containing the names and order of the fields can be in the first file or every file.
- `FIRST FILE` indicates that the first data file contains a list of field names for the data in the first record. `ALL FILES` indicates that all data files contain a list of field names for the data in the first record. The first record is skipped in each data file when the data is processed. The fields can be in a different order in each data file.
- The last statement applies only to external tables. The `ALL FIELDS OVERRIDE` clause tells the access driver that all fields are present and that they are in the same order as the columns in the external table. You only need to specify fields that have a special definition.

SQL*Loader Express Mode

It is simple to use:

- Specify a table name to initiate an Express Mode load:
 - The table columns must be scalar data types.
 - The data file contains only delimited character data.
 - SQL*Loader uses table column definitions to determine input data types.
- There is no need to create a control file.

```
$ sqlldr hr TABLE=test
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If you activate SQL*Loader Express Mode, specifying only the username and the `TABLE` parameter, it uses default settings for a number of other parameters. You can override most of the defaults by specifying additional parameters on the command line.

SQL*Loader express mode generates two files. The names of the log files come from the name of the table (by default).

- A log file that includes:
 - The control file output
 - A SQL script for creating the external table and performing the load using a SQL `INSERT AS SELECT` statement.

Neither the control file nor the SQL script are used by SQL*Loader express mode. They are made available to you in case you want to use them as a starting point to perform operations using regular SQL*Loader or stand-alone external tables.

- A log file similar to a SQL*Loader log file that describes the result of the operation. The "%p" represents the process id of the SQL*Loader process.

An example of the input data file named `test.dat` containing two records to load and excerpts of the two log files generated after the load completed. The table `HR.TEST` was created with three columns, `C1 NUMBER`, `C2 VARCHAR2(10)`, and `C3 VARCHAR2(10)`.

```
$ more test.dat
3, C, WWW
4, D, UUU

$ sqlldr hr TABLE=test

$ more test.log

...
Express Mode Load, Table: TEST
Data File:      test.dat
  Bad File:      test_%p.bad    ...
Table TEST, loaded from every logical record.
Insert option in effect for this table: APPEND
Column Name      Position   Len    Term Encl Datatype
-----
C1                FIRST           *      , CHARACTER
C2                NEXT           *      , CHARACTER
C3                NEXT           *      , CHARACTER

Generated control file for possible reuse:
OPTIONS(EXTERNAL_TABLE=EXECUTE, TRIM=LRTRIM)
LOAD DATA INFILE 'test' APPEND INTO TABLE TEST
FIELDS TERMINATED BY "," ( C1, C2, C3)

End of generated control file for possible reuse.

...
creating external table "SYS_SQLLDR_X_EXT_TEST"
CREATE TABLE "SYS_SQLLDR_X_EXT_TEST"
("C1" NUMBER,"C2" CHAR(1),"C3" VARCHAR2(20)) ORGANIZATION external(
  TYPE oracle_loader DEFAULT DIRECTORY SYS_SQLLDR_XT_TMPDIR_00000
  ACCESS PARAMETERS
    ( RECORDS DELIMITED BY NEWLINE CHARACTERSET US7ASCII
      BADFILE 'SYS_SQLLDR_XT_TMPDIR_00000': 'test_%p.bad'
      LOGFILE 'test_%p.log_xt' READSIZE 1048576
      FIELDS TERMINATED BY "," LRTRIM REJECT ROWS WITH ALL NULL FIELDS ("C1"
CHAR(255),"C2" CHAR(255),"C3" CHAR(255)))
  location ('test.dat')) REJECT LIMIT UNLIMITED

executing INSERT statement to load database table TEST
INSERT /*+ append parallel(auto) */ INTO TEST (C1, C2, C3)
SELECT  "C1",  "C2",  "C3" FROM "SYS_SQLLDR_X_EXT_TEST"  ...
Table TEST:    2 Rows successfully loaded.
```

Summary

In this lesson, you should have learned how to:

- Describe Oracle Data Pump Full Transportable
- Describe each Oracle Database 12c new feature for Oracle Data Pump, SQL*Loader, and External Tables
- Describe applicable use cases and configuration details for the Oracle Data Pump and SQL*Loader new features
- Describe SQL*Loader Express Mode

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 22: Overview

- 22-1: Transporting a database by using the Full Transportable mode
- 22-2: Loading data by using SQL*Loader Express Mode *(optional)*

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

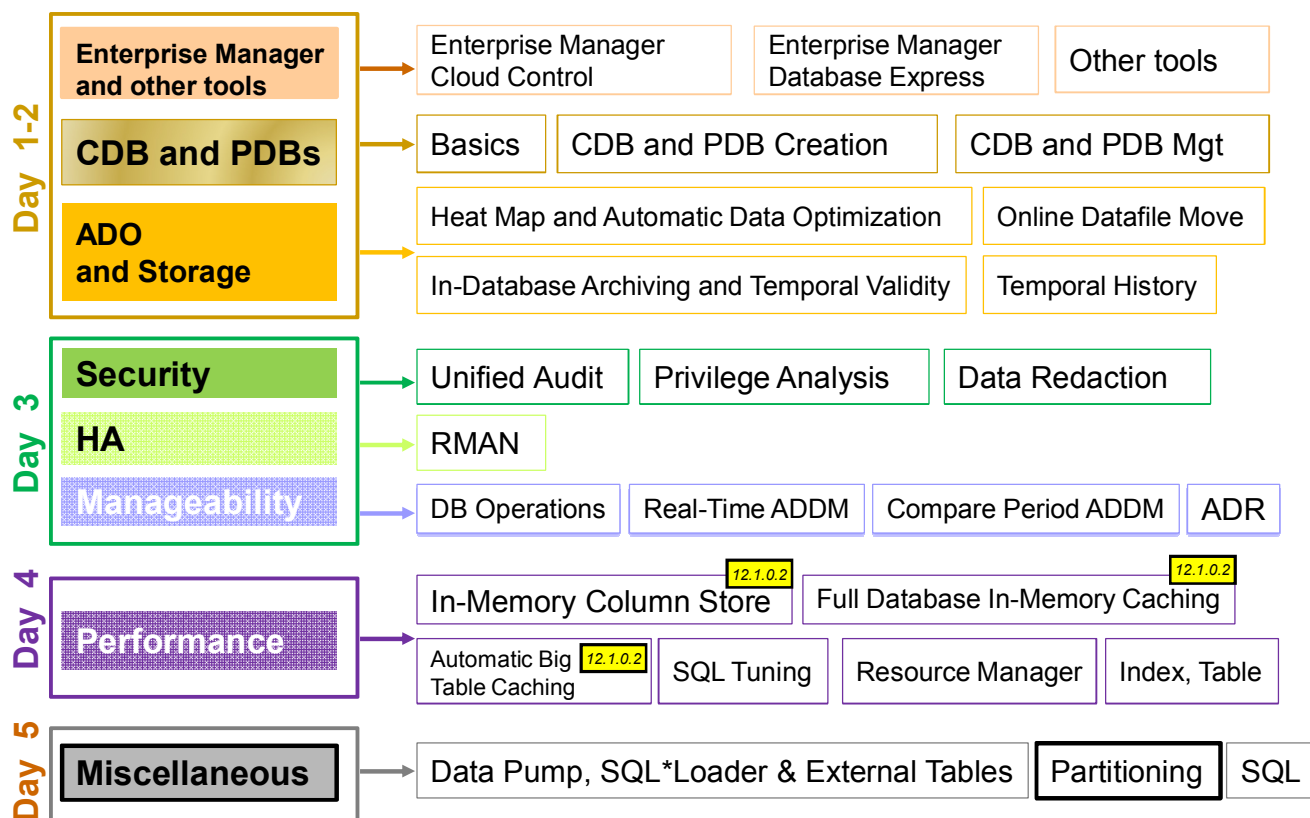
Partitioning Enhancements

23

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are several **Partitioning** enhancements.

Objectives

After completing this lesson, you should be able to:

- Explain Interval Reference Partitioning
- Maintain multiple partitions in a single operation
- Describe partial local and global indexes
- Create and maintain partial local and global indexes
- Describe Asynchronous Global Index maintenance

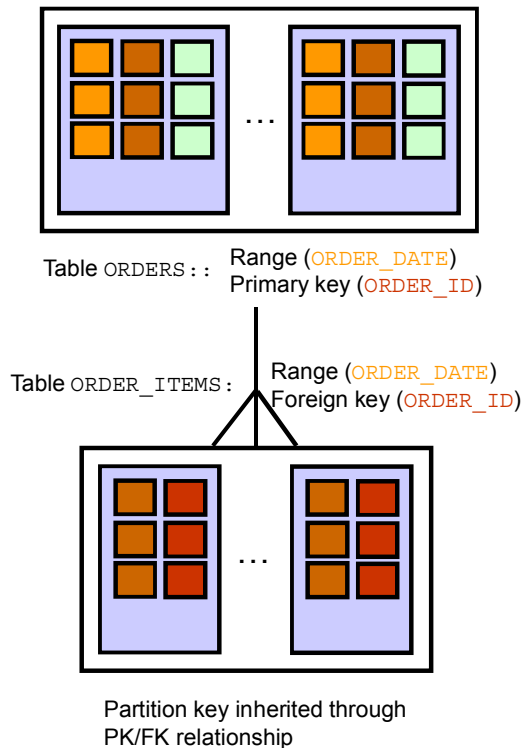
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For a complete understanding of partitioning enhancements and usage, refer to the following guides in the Oracle documentation:

- *Oracle Database VLDB and Partitioning Guide 12c Release 1 (12.1)*
- *Oracle Database Data Warehousing Guide 12c Release 1 (12.1)*

Enhancements to Reference Partitioning



- Supports Interval Reference Partitioning
- Provides the CASCADE option for the TRUNCATE PARTITION operation
- Provides the CASCADE option for the EXCHANGE [SUB] PARTITION operation

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

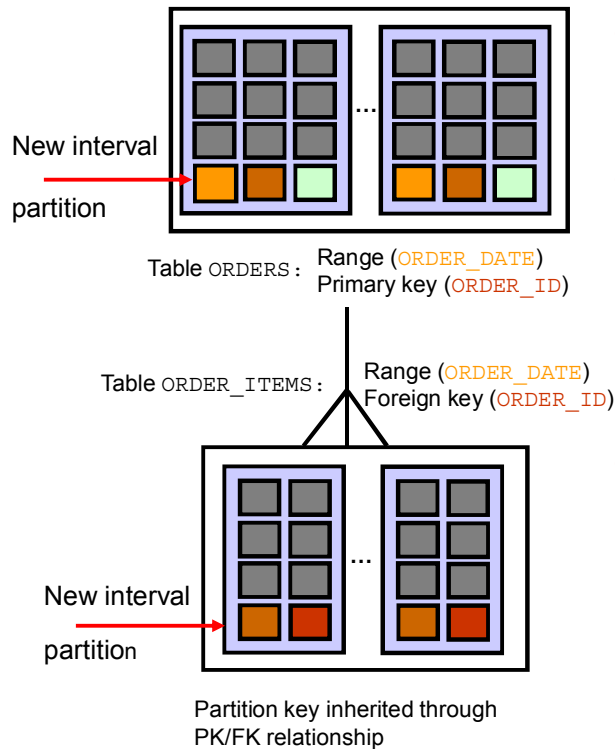
Reference partitioning was introduced in Oracle 11g. It equipartitions two tables related by referential constraints. When you partition a table by reference, partition maintenance operations subsequently performed on the parent table automatically cascade to the child table. Therefore, you cannot perform partition maintenance operations directly on a reference-partitioned table.

In Oracle Database 12c Release 1, reference partitioning is enhanced in three new ways:

- Interval Reference Partitioning allows reference partitioning with interval partitioned parent tables.
- Provide a CASCADE option for TRUNCATE PARTITION operations, which cascades the operation to reference-partitioned child tables.
- Provide a CASCADE option for EXCHANGE [SUB] PARTITION operations, which cascades the operation to reference-partitioned child tables.

Additionally, TRUNCATE TABLE is enhanced with a CASCADE option, which cascades the operation to child tables, (which do not have to be reference partitioned) based on referential constraints.

Interval Reference Partitioning



- Interval-partitioned tables can be used as parent tables for reference partitioning.
- Partitions in the reference-partitioned table corresponding to interval partitions in the parent table are created upon insertion into the reference partitioned table.
- Whenever an interval partition is created in a parent table, the partition name in the child table is inherited from the associated parent table.

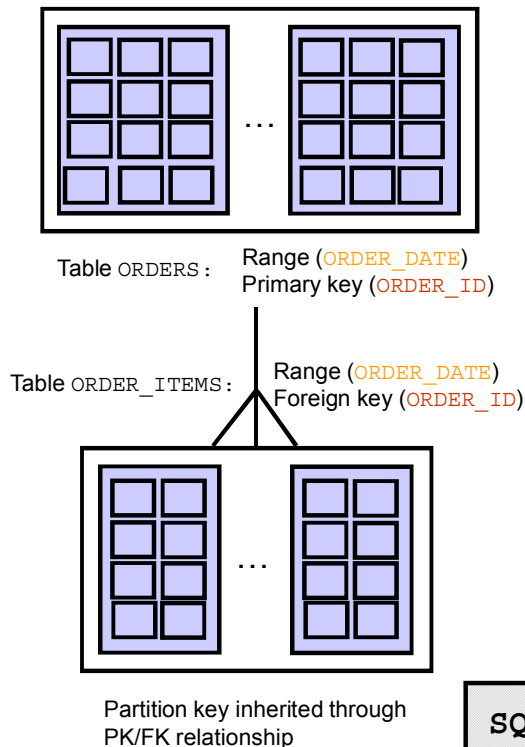
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Interval partitioned tables can be used as parent tables for reference partitioning. Partitions in the interval section of an interval reference-partitioned table (child table) are created when a row is inserted into the child table. Therefore, it is possible that an interval partition exists in the parent table, but it does not exist in the child table because no one inserted any row into that partition of the child table. When an interval partition is created in a parent table, the partition name in the child table is inherited from the associated parent table.

If the child table has a table-level default tablespace, it is used as the tablespace for the new interval partition. Otherwise, the tablespace is inherited from the parent table partition.

TRUNCATE TABLE CASCADE



If you specify the **CASCADE** option for **TRUNCATE TABLE**, it:

- Truncates all child tables that reference the table with an enabled **ON DELETE CASCADE** referential constraint.
- Succeeds if a parent and child are connected by multiple referential constraints and at least one of the constraints has **ON DELETE CASCADE** enabled.

```
SQL> TRUNCATE TABLE hr.orders CASCADE;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

As of Oracle Database 12c, you can add a **CASCADE** option for truncate operations to cascade the operation based on referential constraints. This affects **TRUNCATE TABLE**, **ALTER TABLE TRUNCATE PARTITION**, and **ALTER TABLE TRUNCATE SUBPARTITION** operations.

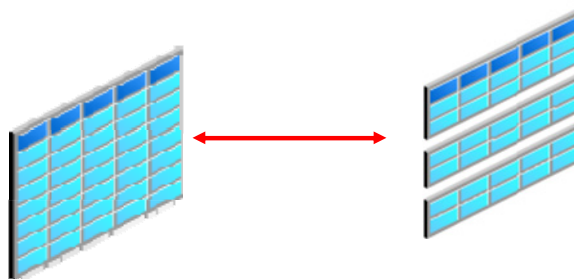
If you specify the **CASCADE** option for **TRUNCATE TABLE**, Oracle Database truncates all the child tables that reference the table with an enabled **ON DELETE CASCADE** referential constraint. This cascading action applies recursively to grandchildren, great grandchildren, and so on. After determining the set of tables to be truncated based on the enabled **ON DELETE CASCADE** referential constraints, an error is raised if any table in this set is referenced through an enabled constraint from a child outside of the set. If a parent and child are connected by multiple referential constraints, a **TRUNCATE TABLE CASCADE** operation targeting the parent succeeds if at least one constraint has **ON DELETE CASCADE**.

The **TRUNCATE** can be targeted at any level in a reference-partitioned hierarchy and cascades to child tables starting from the targeted table. Privileges are not required on the child table. The **CASCADE** option is ignored if you specify it for a table that does not have reference-partitioned children. Any other options specified for the operation, such as **DROP STORAGE** or **UPDATE INDEXES**, apply to all the tables affected by the operation.

The **CASCADE** options are off by default.

Multipartition Maintenance Operations

- Add / truncate / drop partition allows you to add / truncate / drop multiple partitions in one single operation.
- Split partition and merge partition operations allow you to roll out data into smaller partitions or to roll up data into larger partition
- Prior to Oracle Database 12c, Oracle allowed splitting into and merging of only two partitions using the `ALTER TABLE split` and `merge partition` DDLs.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Split partition and merge partitions operations allow you to roll out data into smaller partitions or to roll up data into a larger partition.

Prior to Oracle Database 12c, Oracle allowed splitting into and merging of only two partitions by using the `alter table split` and `merge partition` DDLs. As a result, to split a partition into N partitions or to merge N partitions into one, $(N-1)$ DDLs must be issued. For instance, you may want to roll up data for the last year by merging all the monthly partitions. Because only two partitions can be merged at a time, it requires issuing 11 `alter table merge partition` DDLs. This is not only cumbersome, but also expensive due to multiple data reads and writes.

An alternative approach to merge multiple range partitions is to load the data from the N source partitions into a new nonpartitioned table by using the `CREATE TABLE ... AS SELECT` (CTAS) statement. Next, you drop the first $(N-1)$ source partitions and exchange the N -th source partition with the new table. The N -th source partition now contains data from the N partitions that were to be merged and may be renamed to the target partition name. This approach reduces data movement compared to issuing $(N-1)$ merge partition DDLs. For example, when rolling up monthly partitions for the last year, the customer will issue a CTAS, 11 drop partition and an exchange partition DDL to achieve the same result with lesser data movement.

Adding Multiple Partitions

- Add multiple new partitions with the `ADD PARTITION` clause of the `ALTER TABLE` statement.
- Local and global index operations are the same as when adding a single partition.
- Add multiple range partitions that are listed in ascending order of their upper bound values to the high end of a Range-partitioned or Composite Range-partitioned table.
- Add multiple list partitions to a table by using new sets of partition values if the `DEFAULT` partition does not exist.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can add multiple new partitions with the `ADD PARTITION` clause of the `ALTER TABLE` statement. When adding multiple partitions, local and global index operations are the same as when adding a single partition. Note that both `ADD PARTITION` and `ADD PARTITIONS` are synonymous.

You can add multiple range partitions that are listed in ascending order of their upper bound values to the high end (after the last existing partition) of a Range-partitioned or Composite Range-partitioned table, provided the `MAXVALUE` partition is not defined. Similarly, you can add multiple list partitions to a table by using new sets of partition values if the `DEFAULT` partition does not exist.

Creating a Range-Partitioned Table

```
SQL> CREATE TABLE sales
      ( prod_id NUMBER(6),
        cust_id NUMBER,
        time_id DATE,
        channel_id CHAR(1),
        promo_id NUMBER(6),
        quantity_sold NUMBER(3),
        amount_sold NUMBER(10,2)
      )
PARTITION BY RANGE (time_id)
(PARTITION sales_q1_2012 VALUES LESS THAN
(TO_DATE('01-APR-2012','dd-MON-yyyy'))
TABLESPACE tsa
,PARTITION sales_q2_2012 VALUES LESS THAN
(TO_DATE('01-JUL-2012','dd-MON-yyyy'))
TABLESPACE tsb
,PARTITION sales_q3_2012 VALUES LESS THAN
(TO_DATE('01-OCT-2012','dd-MON-yyyy'))
TABLESPACE tsc
,PARTITION sales_q4_2012 VALUES LESS THAN
(TO_DATE('01-JAN-2013','dd-MON-yyyy'))
TABLESPACE tsd );
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The slide example creates a SALES table with four partitions, one for each quarter of 2012. This example shows how to create a range-partitioned table in Oracle Database 11g.

- Each partition is given a name:
 - sales_q1_2012
 - sales_q2_2012
 - sales_q3_2012
 - sales_q4_2012
- The time_id column is the partitioning column, whereas its value represents the partitioning key of a specific row.
- The VALUES LESS THAN clause determines the partition bound: Rows with partitioning key values that compare less than the ordered list of values specified by the clause are stored in the partition.
- Each partition is contained in a separate tablespace, tsa, tsb, tsc, and tsd.

For example, a row with time_id=17-MAR-2012 would be stored in partition sales_q1_2012.

Adding Multiple Partitions

You can add multiple partitions by using a single SQL statement by specifying the individual partitions as follows:

```
SQL> ALTER TABLE sales ADD  
      PARTITION sales_q1_2013 VALUES LESS THAN  
        (TO_DATE('01-APR-2013','dd-MON-yyyy')),  
      PARTITION sales_q2_2013 VALUES LESS THAN  
        (TO_DATE('01-JUL-2013','dd-MON-yyyy')),  
      PARTITION sales_q3_2013 VALUES LESS THAN  
        (TO_DATE('01-OCT-2013','dd-MON-yyyy')),  
      PARTITION sales_q4_2013 VALUES LESS THAN  
        (TO_DATE('01-JAN-2014','dd-MON-yyyy'));
```



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle Database 11g, you can add one partition to a partitioned table.

In Oracle Database 12c, you can add multiple partitions by using a single statement and specifying the individual partitions. For example, as in the slide example, you add multiple partitions to the range-partitioned `sales` table created earlier, named `sales_q1_2012`, `sales_q2_2012`, `sales_q3_2012`, and `sales_q4_2012`.

Truncating Multiple Partitions

- Truncate multiple partitions from a range- or list-partitioned table with the `ALTER TABLE ... TRUNCATE PARTITION` statement.
- The corresponding partitions of local indexes are truncated in the operation.
- Global indexes must be rebuilt unless the `UPDATE INDEXES` clause is specified.

```
SQL> ALTER TABLE sales TRUNCATE PARTITIONS  
      sales_q1_2012, sales_q2_2012,  
      sales_q3_2012, sales_q4_2012;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Use the `ALTER TABLE ... TRUNCATE PARTITION` statement to remove all rows from a table partition. Truncating a partition is similar to dropping a partition, except that the partition is emptied of its data, but not physically dropped.

The corresponding partitions of local indexes are truncated in the operation.

Global indexes must be rebuilt unless the `UPDATE INDEXES` clause is specified.

The slide example truncates four partitions in the range-partitioned `sales` table.

Dropping Multiple Partitions

- Remove multiple partitions or subpartitions from a range- or list-partitioned table with the following clauses of the `ALTER TABLE` statement:
 - `DROP PARTITION`
 - `DROP SUBPARTITION`

```
SQL> ALTER TABLE sales DROP PARTITIONS sales_q1_2012,  
                                           sales_q2_2012,  
                                           sales_q3_2012,  
                                           sales_q4_2012;
```

- Local and global index operations are the same as when dropping a single partition.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can remove multiple partitions or subpartitions from a range or list-partitioned table with the `DROP PARTITION` and `DROP SUBPARTITION` clauses of the `ALTER TABLE` statement. For example, the statement in the slide drops multiple partitions from the range-partitioned table, `sales`. Note that you cannot drop all the partitions of a table.

When you drop multiple partitions, local and global index operations are the same as when you drop a single partition.

Splitting into Multiple Partitions

- Redistribute one partition content into multiple partitions.
- Each new partition obtains a new segment and inherits all unspecified physical attributes from the current source partition.
- Specify a list of new partition descriptions similar to the create partitioned table SQL statements, instead of using the AT or VALUES clauses.

```
SQL> ALTER TABLE part_tab SPLIT PARTITION p0 INTO  
(PARTITION p01 VALUES LESS THAN (25),  
PARTITION p02 VALUES LESS THAN (50),  
PARTITION p03 VALUES LESS THAN (75),  
PARTITION p04);
```

```
SQL> ALTER TABLE part_tab SPLIT PARTITION p0 INTO  
(PARTITION p01 VALUES LESS THAN (25),PARTITION p02);
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can redistribute the contents of one partition into multiple partitions with the `SPLIT PARTITION` clause of the `ALTER TABLE` statement when a partition becomes too large and causes backup, recovery, or maintenance operations to take a long time to complete or it is felt that there is simply too much data in the partition and therefore impacts performance.

When splitting multiple partitions, the segment associated with the current partition is preserved and new (empty) segments are created. Each new partition obtains a new segment and inherits all of the unspecified physical attributes from the current source partition.

You can use the extended split syntax to specify a list of new partition descriptions similar to the create partitioned table SQL statements, instead of using the `AT` or `VALUES` clauses. Additionally, the range or list values clause for the last new partition description is derived based on the high bound of the source partition and the bound values specified for the first (N-1) new partitions resulting from the split.

The first slide example demonstrates splitting a partition `p0` into multiple partitions namely `p01`, `p02`, `p03`, and `p04`.

In the second example in the slide, partition `p02` has the high bound of the original partition `p0`.

Splitting into Multiple Partitions Rules

- Split a range partition into N partitions:
 - $(N-1)$ values of the partitioning key column must be specified within the range of the partition at which to split the partition.
- Split a list partition into N partitions:
 - $(N-1)$ lists of literal values must be specified.
 - Each list defines the first $(N-1)$ partitions into which rows with corresponding partitioning key values are inserted.
- When splitting a `DEFAULT` list partition or a `MAXVALUE` range partition into multiple partitions:
 - The first $(N-1)$ new partitions are created by using the literal value lists or high bound values specified.
 - The N^{th} new partition resulting from the split have the `DEFAULT` value or `MAXVALUE`.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To split a range partition into N partitions, $(N-1)$ values of the partitioning key column must be specified within the range of the partition at which to split the partition. The new noninclusive upper bound values specified must be in ascending order. The high bound of N^{th} new partition is assigned the value of the high bound of the partition being split. The names and physical attributes of the N new partitions resulting from the split can be optionally specified.

To split a list partition into N partitions, $(N-1)$ lists of literal values must be specified, each of which defines the first $(N-1)$ partitions into rows with corresponding partitioning key values are inserted. The remaining rows of the original partition are inserted into the N^{th} new partition whose value list contains the remaining literal values from the original partition. No two value lists can contain the same partition value. The $(N-1)$ value lists that are specified cannot contain all of the partition values of the current partition because the N^{th} new partition would be empty. Also, the new $(N-1)$ value lists cannot contain any partition values that do not exist for the current partition.

When splitting a `DEFAULT` list partition or a `MAXVALUE` range partition into multiple partitions, the first $(N-1)$ new partitions are created by using the literal value lists or high bound values specified, whereas the N^{th} new partition resulting from the split have the `DEFAULT` value or `MAXVALUE`. The `SPLIT_TABLE_SUBPARTITION` clause is extended similarly to allow split of a range or list subpartition into N new subpartitions.

Splitting into Multiple Partitions: Examples

```
SQL> ALTER TABLE sales SPLIT PARTITION sales_2012 INTO
(PARTITION sales_Q1_2012 VALUES LESS THAN
  (TO_DATE('01-APR-2012','dd-MON-yyyy')),
PARTITION sales_Q2_2012 VALUES LESS THAN
  (TO_DATE('01-JUL-2012','dd-MON-yyyy')),
PARTITION sales_Q3_2012 VALUES LESS THAN
  (TO_DATE('01-OCT-2012','dd-MON-yyyy')),
PARTITION sales_Q4_2012);
```

```
SQL> ALTER TABLE customers SPLIT PARTITION europe INTO
(PARTITION northern_europe VALUES ('France', 'Sweden'),
PARTITION southern_europe VALUES ('Spain', 'Portugal'),
PARTITION rest_europe);
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The first slide example splits the `sales_2012` partition of the partitioned table `sales` into four partitions corresponding to the quarters of the 2012 year. In this example, the partition `sales_2012` implicitly becomes the high bound of the split partition.

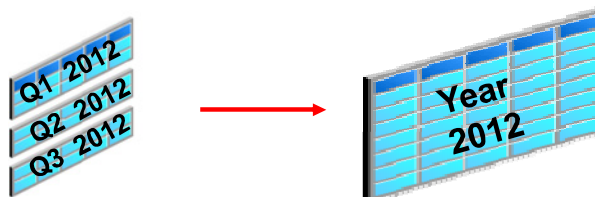
In the second slide example, the sample table `customers` partitioned by list, splits the partition `Europe` into three partitions: `western_europe`, `southern_europe`, and `rest_europe`.

Merging Multiple Range Partitions

- Merge the contents of two or more partitions or subpartitions into one new partition or subpartition.

```
SQL> ALTER TABLE sales
      MERGE PARTITIONS sales_q1_2012, sales_q2_2012,
                        sales_q3_2012, sales_q4_2012
      INTO PARTITION sales_2012;
```

```
SQL> ALTER TABLE sales
      MERGE PARTITIONS sales_q1_2012 TO sales_q4_2012
      INTO PARTITION sales_2012;
```



- The new partition inherits the partition upper bound of the highest of the original partitions.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle Database 11g, you can merge the contents of two partitions into one partition.

In Oracle Database 12c, you can merge the contents of two or more partitions or subpartitions into one new partition or subpartition. You can then drop the original partitions or subpartitions with the `MERGE PARTITIONS` and `MERGE SUBPARTITIONS` clauses of the `ALTER TABLE` SQL statement as shown in the slide. One reason for merging range partitions is to keep historical data online in larger partitions. For example, you can have daily partitions, with the oldest partition rolled up into weekly partitions, which can then be rolled up into monthly partitions, and so on.

When merging multiple range partitions, the partitions must be adjacent and specified in the ascending order of their partition bound values. The new partition inherits the partition upper bound of the highest of the original partitions.

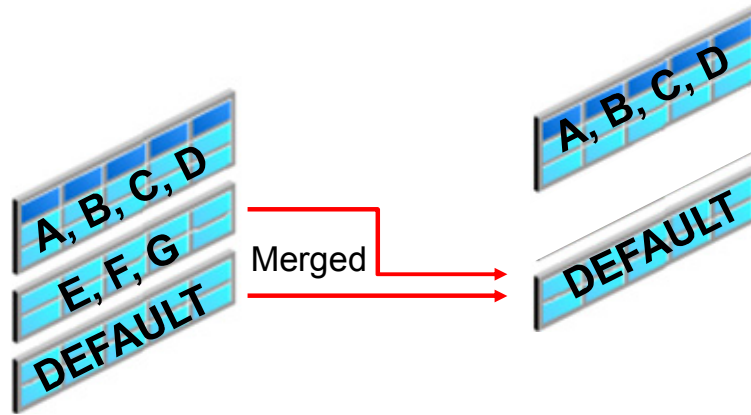
In the first slide example, you merge the four partitions of the `sales` range-partitioned table. These four partitions that correspond to the four quarters of the oldest year are merged into a single partition containing the entire sales data for the year.

If you are using range partitioning, you can rewrite the first statement by using the `TO` keyword, as displayed in the second example in the slide.

Be aware of performance impact of this type of operation.

Merging List and System Partitions

- When merging multiple list partitions, the resulting partition-value list is the union of the set of partition value list of all of the partitions to be merged.
- A `DEFAULT` list partition merged with other list partitions results in a `DEFAULT` partition.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When merging multiple list partitions, the resulting partition value list is the union of the set of partition-value list of all of the partitions to be merged. A `DEFAULT` list partition merged with other list partitions results in a `DEFAULT` partition.

Quiz

You can redistribute the contents of one partition into multiple partitions with the `SPLIT PARTITION` clause of the `ALTER TABLE` statement.

- a. True
- b. False

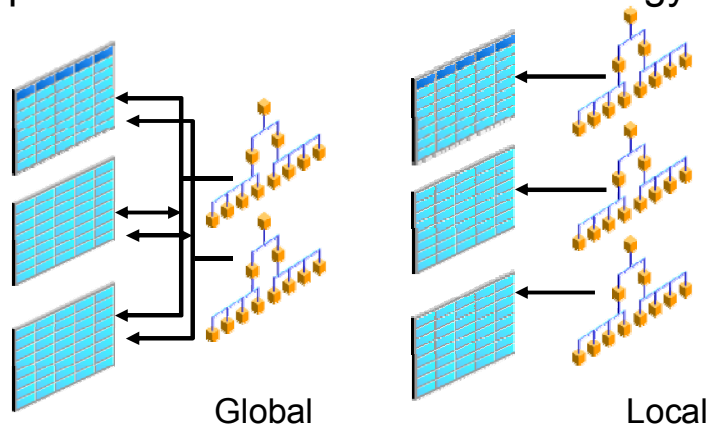
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: a

Partitioned Indexes: Review

- Indexes can be partitioned like tables.
- A partitioned index can exist on a nonpartitioned table.
- A global partitioned index uses a different partition strategy than that of the table.
- A local partitioned index follows the strategy of the table.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The same management benefits and performance improvement that can be achieved by partitioning tables is achieved by partitioning indexes. When an index is partitioned, each partition is a complete separate index. There is no master index to decide which index to use because this piece of information is inherent in the partition information. The syntax used to specify the partitioning of an index is very similar to that used to partition a table.

Global indexes can be defined on nonpartitioned and partitioned tables and do not follow the partitioned table partitioning key. Local indexes can be defined only on partitioned tables. A local index has the same partitioning key as the table partitioning key. You can use only range and hash partitioning for global indexes.

Local indexes are automatically maintained; that is, changes made on the partitions on the table are automatically repeated on the local indexes; however, depending on the DDL, you may have to use the update indexes clause to maintain associated local index during partition maintenance operations.

A local index segment is always built for the equivalent data (table) segment. Therefore, for a composite partitioned tables, you have only local subpartitioned index segments and nothing on the logical partition top level.

Partial Indexes for Partitioned Tables

- Local and global indexes can be created on a **subset of the partitions** of a table, enabling more flexibility in index creation.
- This feature supports global indexes that index a certain subset of table partitions or subpartitions, and not others.
- This feature is supported by using a *default* table indexing property
- When a table is created or altered, a default indexing property can be specified for the table or its partitions.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Indexes typically occupy considerable space because each table in the database could have many indexes defined on it.

Global indexes index all table partitions and the index structure has no correlation with the table partitioning. Prior to Oracle Database 12c, Oracle did not provide the ability to index a subset of partitions and to exclude the others. There is no operation that a DBA may use prior to Oracle Database 12c to exclude non-active partitions from a global index.

In Oracle Database 12c, Partial Global Indexes save space and improve performance during loads and queries.

Local and global indexes can be created on a subset of the partitions of a table, enabling more flexibility in index creation.

This feature supports global indexes that include or index a certain subset of table partitions or subpartitions, and exclude the others. This operation is supported by using a default table indexing property. When a table is created or altered, a default indexing property can be specified for the table or its partitions.

Partial Index Creation on a Table

- When an index is created as **PARTIAL** on a table:
 - Local indexes: An index partition is created usable if indexing is turned on for the table partition, and unusable otherwise
 - Global indexes: Include only those partitions for which indexing is turned on, and exclude the others
- **FULL** is the default if neither **FULL** or **PARTIAL** is specified at index creation.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can override this behavior by specifying the **FULL** property at the index creation. **FULL** is the default if neither **FULL** or **PARTIAL** is specified. In this case, the index creation is similar to the behavior of Oracle Database releases prior to Oracle Database 12c. All local index partitions are created **USABLE** and a global index refers to all partitions of the table.

You cannot create a partial unique global index. A unique global index always has to be full.

Note

- In a local index, all keys in a particular index partition refer only to rows stored in a single underlying table partition. A partial local index is created by specifying the **LOCAL INDEXING PARTIAL** attribute.
- In a global index, the keys may refer to rows stored in multiple underlying table partitions or subpartitions. A partial global index is created by specifying the **GLOBAL INDEXING PARTIAL** attribute.

Specifying the INDEXING Clause at the Partition and Subpartition Levels

```
SQL> CREATE TABLE orders ( order_id NUMBER(12),
                           order_date DATE
                           CONSTRAINT order_date_nn NOT NULL,
...
CONSTRAINT order_total_min CHECK (order_total >= 0)
)

INDEXING OFF
PARTITION BY RANGE (ORDER_DATE)
(PARTITION ord_p1 VALUES LESS THAN (TO_DATE('01-MAR-2010','DD-
MON-YYYY')) INDEXING ON,
PARTITION ord_p2 VALUES LESS THAN (TO_DATE('01-JUL-2010','DD-
MON-YYYY')) INDEXING OFF,
PARTITION ord_p3 VALUES LESS THAN (TO_DATE('01-OCT-2010','DD-
MON-YYYY')) INDEXING ON,
PARTITION ord_p4 VALUES LESS THAN (TO_DATE('01-MAR-2011','DD-
MON-YYYY')),
PARTITION ord_p5 VALUES LESS THAN (TO_DATE('01-MAR-2012','DD-
MON-YYYY')));
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle Database 12c, the CREATE TABLE syntax is extended to support specification of the default indexing attributes clause INDEXING. You can use ALTER TABLE .. MODIFY DEFAULT ATTRIBUTES clause to change the default property of INDEXING ON/OFF. This changes the default for future partitions (and subpartitions). The INDEXING clause may also be specified at the partition and subpartition levels.

The slide example creates a table ORDERS with the following items:

- Partitions ORD_P1 and ORD_P3, which are included in all partial global indexes
- Partial local index partitions corresponding to the preceding two table partitions, which are created **USABLE** by default. The other local index partitions corresponding to table partitions with **INDEXING OFF** and table partitions with no **INDEXING** clause (ORD_P4 and ORD_P5), are created **UNUSABLE**.

Creating a Partial Local or Global Index

- An index, local or global, can be created to follow the table indexing properties by specifying the `INDEXING PARTIAL` clause.

```
SQL> CREATE INDEX orders_gidx_ordertotal  
      ON orders (order_total)  
      GLOBAL INDEXING PARTIAL;
```

- An index, local or global, can still be created to override the table indexing properties by specifying the `INDEXING FULL` clause.

```
SQL> CREATE INDEX orders_gidx_ordertotal  
      ON orders (order_total)  
      GLOBAL INDEXING FULL;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The `ORDERS_GIDX_ORDERTOTAL` global index in the slide example will be created to index only those table partitions that have `INDEXING ON`, and exclude the remaining partitions. Local indexes may also be created `PARTIAL` as discussed earlier, using the `LOCAL` clause.

Explain Plan: LOCAL INDEX ROWID

A partial local index is created on two partitions of the table on the column ORDER_DATE.

```
SQL> EXPLAIN PLAN FOR SELECT order_mode, order_status FROM hr.tab_part1
WHERE order_mode='direct';
SQL> select * from table(dbms_xplan.display) ;
```

Id	Operation	Name	Pstart	Pstop
0	SELECT STATEMENT			
1	VIEW	VW_TE_2		
2	UNION-ALL			
3	CONCATENATION			
4	PARTITION RANGE SINGLE		1	1
5	TABLE ACCESS FULL	TAB_PART1	1	1
6	PARTITION RANGE SINGLE		3	3
7	TABLE ACCESS BY LOCAL INDEX ROWID BATCHED	TAB_PART1 LIDX_ORDERDATE	3	3
8	INDEX RANGE SCAN			
9	PARTITION RANGE OR		KEY (OR)	KEY (OR)
10	TABLE ACCESS FULL	TAB_PART1	KEY (OR)	KEY (OR)

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The LIDX_ORDERDATE partial local index is composed of two partitions indexing partitions 1 and 3 of the table that have INDEXING ON.

In this example, the partial local index is used to access rows in the partition 3 but not to access rows of partition 1. Most of the rows in partition 1 contain the value selected.

Explain Plan: GLOBAL INDEX ROWID

A partial global index is created on two partitions of the table on the column ORDER_MODE.

```
SQL> EXPLAIN PLAN FOR SELECT order_mode, order_status FROM hr.tab_part1
      WHERE order_mode='direct';
SQL> select * from table(dbms_xplan.display) ;
```

Id	Operation	Name	Pstart	Pstop
0	SELECT STATEMENT			
1	VIEW	VW_TE_2		
2	UNION-ALL			
3	TABLE ACCESS BY GLOBAL INDEX ROWID			
	BATCHED	TAB_PART1	ROWID	ROWID
4	INDEX RANGE SCAN	GIDX_ORDERMODE		
5	PARTITION RANGE OR		KEY (OR)	KEY (OR)
6	TABLE ACCESS FULL	TAB_PART1	KEY (OR)	KEY (OR)

ORACLE

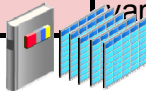
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The GIDX_ORDERMODE partial global index in the example in the slide indexes only those table partitions that have INDEXING ON, partitions 1 and 3 upon the five partitions of the table.

In this example, the partial global index is used to access rows in the table partitions 1 and 3 but not to access other rows of nonindexed partitions.

Affected Data Dictionary Views: Overview

Data Dictionary View / New columns	Description
DBA ALL USER_PART_TABLES New column DEF_INDEXING=ON/OFF	Display the object-level partitioning information for the partitioned tables.
DBA ALL USER_TAB_PARTITIONS New column INDEXING=ON/OFF	Display partition-level partitioning information, partition storage parameters, and partition statistics.
DBA ALL USER_TAB_SUBPARTITIONS New column INDEXING=ON/OFF	Display subpartition-level partitioning information, subpartition storage parameters, and subpartition statistics.
DBA ALL USER_INDEXES New column INDEXING=PARTIAL/FULL	Display indexes.
DBA ALL USER_IND_PARTITIONS DBA ALL USER_IND_SUBPARTITIONS Column STATUS=USABLE/UNUSABLE	Display for each index partition, the partition-level partitioning information, the storage parameters for the partition, and various partition statistics.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The following is a quick review of some of the affected views:

- The **DBA_PART_TABLES** view displays the object-level partitioning information for all partitioned tables in the database. The new column **DEF_INDEXING** has one of two values **ON** or **OFF**, specifying the default indexing **ON** or indexing **OFF** at the table level.
- The **DBA_TAB_PARTITIONS** view describes partition-level partitioning information, partition storage parameters, and partition statistics generated by the **DBMS_STATS** package for all partitions. The new column **INDEXING** has one of two values **ON** or **OFF**, specifying indexing **ON** or **OFF** at the partition level.
- The **DBA_TAB_SUBPARTITIONS** view describes, for each table subpartition, the subpartition name, the name of the table and partition to which it belongs, and its storage attributes. The new column **INDEXING** has one of two values **ON** or **OFF**, specifying indexing **ON** or **OFF** at the subpartition level.
- The **DBA_INDEXES** view describes indexes. To gather statistics for this view, you use the **DBMS_STATS** package. The new column **INDEXING** is **FULL** if the index is full or **PARTIAL** if the index is partial.
- The **DBA_IND_PARTITIONS** view displays, for each index partition, the partition-level partitioning information, the storage parameters for the partition, and various partition statistics generated by the **DBMS_STATS** package. The column **STATUS** is **USABLE** if the partial local index partition is used and **UNUSABLE** if not. Similar to **DBA_IND_PARTITIONS**, **DBA_IND_SUBPARTITIONS** also contains the **STATUS** column, which accepts the values **USABLE** or **UNUSABLE**.

Asynchronous Global Index Maintenance

- DROP PARTITION and TRUNCATE PARTITION operations by using the UPDATE INDEXES clause are optimized when maintaining global indexes:
 - Making the index maintenance a metadata-only operation
- Maintenance operations on indexes can be performed with the automatic scheduler job to clean up all global indexes:
 - SYS.PMO_DEFERRED_GIDX_MAINT_JOB
 - Executing DBMS_PART.CLEANUP_GIDX procedure



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The partition maintenance operations DROP PARTITION and TRUNCATE PARTITION requesting global indexes maintenance are optimized making the index maintenance a metadata-only operation. This functionality enables maintenance of a list of data object numbers in metadata, where index entries corresponding to the drop and truncated objects that are invalid are ignored.

Maintenance operations on indexes can be performed with the automatic scheduler job SYS.PMO_DEFERRED_GIDX_MAINT_JOB to clean up all global indexes. This job is scheduled to run at 2:00 A.M. on a daily basis by default. You can run this job at any time by using DBMS_SCHEDULER.RUN_JOB if you want to proactively clean up the indexes.

You can also modify the job to run with a different schedule based on your specific requirements. However, Oracle recommends that you do not drop the job.

DBMS_PART Package

- The DBMS_PART package enables you to maintain and manage operations on partitioned objects.
- The new package contains the CLEANUP_GIDX procedure:
 - This procedure identifies and cleans up the global indexes to ensure efficiency in terms of storage and performance.

```
SQL> DESC dbms_part
PROCEDURE CLEANUP_GIDX
Argument Name      Type              In/Out Default?
-----
SCHEMA_NAME_IN     VARCHAR2          IN          DEFAULT
TABLE_NAME_IN      VARCHAR2          IN          DEFAULT
```

- You can also force cleanup of an index that needs maintenance by using one of the following options:

```
SQL> ALTER INDEX ... REBUILD
                        [PARTITION];
```

```
SQL> ALTER INDEX ... COALESCE [PARTITION]
                        CLEANUP;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The DBMS_PART package provides an interface for maintenance and management operations on partitioned objects. As a consequence of prior partition maintenance operations with asynchronous global index maintenance, global indexes can contain entries pointing to data segments that no longer exist. These stale index rows do not cause any correctness issues or corruptions during any operation on the table or index, whether these are queries, DMLs, DDLs, or analyze.

The DBMS_PART.CLEANUP_GIDX package procedure identifies and cleans up these global indexes to ensure efficiency in terms of storage and performance.

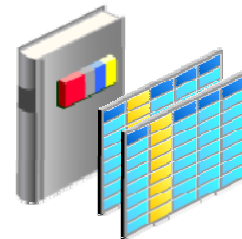
You can also force a cleanup of an index needing maintenance by using one of the following options:

- **DBMS_PART.CLEANUP_GIDX:** This PL/SQL package procedure gathers the list of global indexes in the system that may require cleanup and runs the operations necessary to restore the indexes to a clean state.
- **ALTER INDEX REBUILD [PARTITION]:** This SQL statement rebuilds the entire index or index partition as was done prior to Oracle Database 12.1 releases; the resulting index (partition) does not contain any stale entries.
- **ALTER INDEX COALESCE [PARTITION] CLEANUP:** This SQL statement cleans up any orphaned entries in index blocks.

Global Index Maintenance Optimization During Partition Maintenance Operations

Partial Global Index Optimization:

- A new column `ORPHANED_ENTRIES` is added to the following data dictionary views:
 - `DBA | ALL | USER_INDEXES`
 - `DBA | ALL | USER_IND_PARTITIONS`
- This column specifies whether or not a global index (partition) contains stale entries due to deferred index maintenance during one of the following operations:
 - `DROP/TRUNCATE PARTITION`
 - `MODIFY PARTITION INDEXING OFF`
- The column will have one of three values:
 - YES, NO, or N/A



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- The column `ORPHANED_ENTRIES` is added to the dictionary views `DBA_INDEXES` and `DBA_IND_PARTITIONS`. This column specifies whether or not a global index (partition) contains stale entries due to deferred index maintenance during `DROP/TRUNCATE PARTITION`, or `MODIFY PARTITION` operations.
- The column can have one of the following three values:
 - YES: The index (partition) contains orphaned entries.
 - NO: The index (partition) does not contain any orphaned entries.
 - N/A: The property is not applicable; this is the case for local indexes, or indexes on nonpartitioned tables.

Quiz

Which values can be set for the new INDEXING column in the DBA_INDEXES view? (Select all that apply):

- a. PARTIAL
- b. NO
- c. ON
- d. NA
- e. FULL

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: a, e

Summary

In this lesson, you should have learned how to:

- Explain Interval Reference Partitioning
- Maintain multiple partitions in a single operation
- Describe partial local and global Indexes
- Create and maintain partial local and global indexes
- Describe Asynchronous Global Index maintenance

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 23: Overview

- 23-1: Using partial local and global indexing on partitioned tables

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

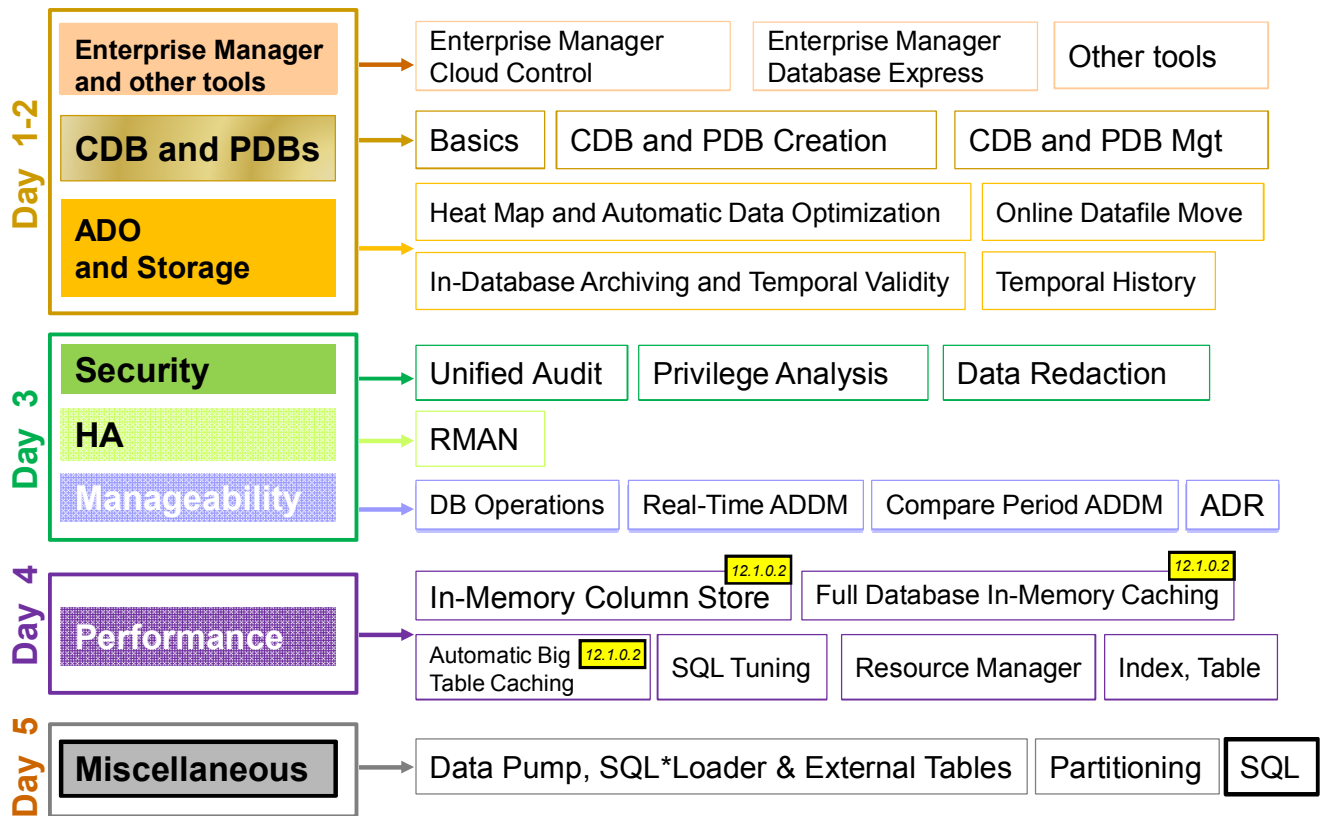
24

SQL Enhancements and Migration Assistant for Unicode

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are several **SQL** enhancements.

The **Migration Assistant for Unicode** replaces the Character Set Scanner.

Objectives

After completing this lesson, you should be able to:

- Enumerate the increase in the length limits for VARCHAR2, NVARCHAR2, and RAW data types in Oracle SQL
- Understand Oracle Database Migration Assistant for Unicode 6.1
- Make SecureFiles the default storage mechanism for LOBs
- Use the SQL row-limiting clause in a query

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For a complete understanding of the Oracle Data Redaction new feature usage, refer to the following guide in the Oracle documentation:

- *Oracle Database Reference 12c Release 1 (12.1) – “MAX_STRING_SIZE” Chapter*
- *Oracle Database SQL Language Reference 12c Release 1 (12.1) – “Data Types” Chapter*

Refer to other sources of information:

- *Oracle Database 12c New Features Demo Series* demonstrations under Oracle Learning Library:
 - *SQL Row_Limiting_Clause*
- *Oracle Database 12c: Installation and Upgrade* course

Increased Length Limits of Data Types

- The maximum size for the VARCHAR2, NVARCHAR2, and RAW data types is increased to 32767 bytes.

VARCHAR2 and NVARCHAR2 columns with a declared column length of **4000 bytes or less** and RAW columns with a declared column length of 2000 bytes or less are stored inline.

VARCHAR2 and NVARCHAR2 columns with a declared column length of **greater than 4000 bytes** and RAW columns with a declared column length of greater than 2000 bytes are called extended character data type columns and are stored out-of-line.

- MAX_STRING_SIZE controls the maximum size of the extended data types in SQL.

```
MAX_STRING_SIZE = { STANDARD | EXTENDED }
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle Database 12c, you can specify a maximum size of 32767 bytes for the VARCHAR2, NVARCHAR2, and RAW data types. This enables users to store longer strings in the database. Prior to this release, the maximum size was 4000 bytes for the VARCHAR2 and NVARCHAR2 data types and 2000 bytes for the RAW data type. The declared length of the VARCHAR2, NVARCHAR2, or RAW column affects how the column is internally stored.

- VARCHAR2 and NVARCHAR2 columns with a declared column length of 4000 bytes or less and RAW columns with a declared column length of 2000 bytes or less are stored inline.
- VARCHAR2 and NVARCHAR2 columns with a declared column length of greater than 4000 bytes and RAW columns with a declared column length of greater than 2000 bytes are called “extended character data type columns” and are stored out-of-line.

MAX_STRING_SIZE controls the maximum size of the extended data types in SQL:

- STANDARD means the length limit of the data types used prior to Oracle 12c.
- EXTENDED means the 32767 bytes limit in Oracle Database 12c.

Extended character data types have the following restrictions:

- Not supported in clustered tables and index-organized tables
- No intrapartition parallel DDL, UPDATE, and DELETE DML
- No intrapartition parallel direct-path inserts on tables stored in a tablespace managed with Automatic Segment Space Management (ASSM).

Configuring Database for Extended Data Type

1. Shut down the database instance.
2. Restart the database in UPGRADE mode.
3. Change the setting of MAX_STRING_SIZE to EXTENDED;

```
SQL> ALTER SYSTEM SET MAX_STRING_SIZE = EXTENDED;
```

4. Run the \$ORACLE_HOME/rdbms/admin/utl32k.sql script as SYSDBA.
5. Restart the database instance.

Note

- You cannot change the value from EXTENDED to STANDARD.
- In a RAC environment, shut down all instances.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You cannot create a table with extended character data type columns until you configure the database with the instance parameter MAX_STRING_SIZE set to EXTENDED.

The value of the MAX_STRING_SIZE initialization parameter is also enforced to be the same on all RAC nodes in a RAC environment.

Using VARCHAR2, NVARCHAR2, and RAW Data Types

- You can create a table with extended character data type columns:

```
SQL> CREATE TABLE long_varchar(id NUMBER,vc VARCHAR2(32767));
```

- You can modify the size of existing VARCHAR2, NVARCHAR2, and RAW columns:

```
SQL> ALTER TABLE t MODIFY (varchar_column VARCHAR2(32767));
```

- You can add a 32k column to an existing table:

```
SQL> ALTER TABLE t ADD (long_varchar_column VARCHAR2(12345));
```

- Data Pump export, import, and SQL*Loader can use extended Data Types.
- Indexes prevent data type extension on existing columns.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can create a table with extended character data type columns as described in the slide. You can modify the size of existing VARCHAR2, NVARCHAR2, and RAW columns with the ALTER TABLE MODIFY (column...) statement. In this case, Oracle Database performs an in-place length extension and does not migrate the inline storage to external LOB storage.

Note: Oracle recommends that you do not excessively increase the size of an existing VARCHAR2 column beyond 4000 bytes, as it can give rise to the following problems:

- Row chaining may occur.
- Any data stored inline has to be read in its entirety, whether or not a column is selected. Extended character data type columns stored inline can therefore negatively impact the performance.
- To migrate to the new out-of-line storage of extended character data type columns, you have to **recreate** the tables. Otherwise, the inline storage of the column is preserved during any type of table reorganization such as ALTER TABLE MOVE.

You can add a 32k column to an existing table via ALTER TABLE ADD DDL if the table satisfies the conditions required for 32k types, as described in the earlier CREATE TABLE section.

Data Pump export, import, and SQL*Loader can use extended Data Types.

Existing indexes prevent data type extension on existing columns. You must drop indexes first, then modify the column with the new extended value, and finally recreate the indexes.

Database Migration Assistant for Unicode

Unicode Update: Latest version is Unicode Standard 6.1.

- Is an industry standard that allows text and symbols from all languages to be consistently represented and manipulated by computers

The Database Migration Assistant for Unicode 6.1 (DMU):

- Is a Unique next-generation migration tool that provides a streamlined end-to-end solution for migrating databases from standard character sets to Unicode
- Has `CSSCAN` and `CSALTER` utilities removed from the database installation and is unsupported
- Supports migration of all Oracle data types that contains textual data and deals with database objects
- Significantly reduces migration down time
- Is located in `$ORACLE_HOME/dmu`



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

DMU is a unique next-generation migration tool that provides a streamlined end-to-end solution for migrating databases from standard character sets to Unicode. The standard `CSSCAN` and `CSALTER` utilities are removed from the database installation and desupported. The DMU supports the migration of almost all Oracle data types that may directly or indirectly contain textual data and deals with database objects, such as materialized views, indexes, constraints, and triggers, which are affected by conversion of tables so that they are properly synced up after the migration. It significantly reduces migration down time. It is not compatible with CDBs.

Features:

- Guided end-to-end migration workflow
- Intuitive graphical user interface
- Advanced data analysis and cleansing tools
- Validation mode to check data integrity for compliance with the Unicode Standard

Benefits:

- Alleviates costly manual workload through automated migration tasks
- Simplifies data preparation process and prevents data loss
- Robust error-handling and failure recovery
- Health check on data integrity in databases encoded with the Unicode Standard

SecureFiles

- SecureFiles are made default storage mechanisms for large objects (LOBs).
- The default value for `DB_SECUREFILE` is `PREFERRED` when the `init.ora` parameter `COMPATIBLE` is set to 12.0.0.0.0 or higher.
- To change this behavior, set the value to `PERMITTED`.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Beginning with Oracle Database 12c, SecureFiles are now the default for large object (LOB) storage. If no storage type is explicitly specified, new LOB columns use SecureFiles LOB storage.

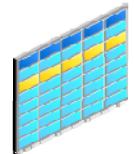
When the `COMPATIBLE` instance parameter is set to 12.0.0.0.0 or higher, the default value for `DB_SECUREFILE` is `PREFERRED`. To change this behavior, set the value to `PERMITTED`. SecureFiles provide optimal performance for storing unstructured data in the database. Making SecureFiles the default for unstructured data helps to ensure that the database delivers the best possible performance when you are managing unstructured data.

SQL Row-Limiting Clause

Use a row-limiting clause to limit the rows returned by a query.

- Specify the number of rows to return with the `FETCH FIRST/NEXT` keywords.
- Specify the percentage of rows to return with the `PERCENT` keyword.
- Use the `OFFSET` keyword to specify that the returned rows begin with a row after the first row of the full result set.

Queries that order data and limit row output are widely used and are often referred to as `Top-N` queries.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle Database 12c Release 1, SQL `SELECT` syntax has been enhanced to allow a row limiting clause, which limits the number of rows that are returned in the result set.

Limiting the number of rows returned can be valuable for reporting, analysis, data browsing, and other tasks. Queries that order data and then limit row output are widely used and are often referred to as `Top-N` queries.

You can specify the number of rows or percentage of rows to return with the `FETCH FIRST/NEXT` keywords. You can use the `OFFSET` keyword to specify that the returned rows begin with a row after the first row of the full result set.

SQL Row-Limiting Clause: Examples

```
SQL> SELECT employee_id, first_name
      FROM employees
      ORDER BY employee_id
      FETCH FIRST 5 ROWS ONLY ;
```

Script Output x | Task completed in 0

EMPLOYEE_ID	FIRST_NAME
100	Steven
101	Neena
102	Lex
103	Alexander
104	Bruce

```
SQL> SELECT employee_id, first_name
      FROM employees
      ORDER BY employee_id
      OFFSET 5 ROWS FETCH NEXT 5 ROWS ONLY ;
```

Returns the
5 employees
with the lowest
employee_id

Script Output x | Task completed in 0

EMPLOYEE_ID	FIRST_NAME
105	David
106	Valli
107	Diana
108	Nancy
109	Daniel

Returns the 5 employees
with the next set of lowest
employee_id

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You specify the row limiting clause in the SQL `SELECT` statement by placing it after the `ORDER BY` clause. Note that an `ORDER BY` clause is not required.

- **OFFSET:** Use this clause to specify the number of rows to skip before row limiting begins. The value for offset must be a number. If you specify a negative number, offset is treated as 0. If you specify `NULL` or a number greater than or equal to the number of rows that are returned by the query, 0 rows are returned.
- **ROW | ROWS:** Use these keywords interchangeably. They are provided for semantic clarity.
- **FETCH:** Use this clause to specify the number of rows or percentage of rows to return.
 - **FIRST | NEXT:** Use these keywords interchangeably. They are provided for semantic clarity.
 - **row_count | percent PERCENT:** Use `row_count` to specify the number of rows to return. Use `percent PERCENT` to specify the percentage of the total number of selected rows to return. The value for percent must be a number.

The first code example returns the five employees with the lowest `employee_id`.

The second code example returns the five employees with the next set of lowest `employee_id`.

Quiz

To use Extended Character Data types, you must set `MAX_STRING_SIZE` to `EXTENDED`.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: a

Quiz

You can limit the number of rows returned by a query by using:

- a. A SQL row-limiting clause in the query
- b. The session parameter `SQL_ROW_LIMITING`

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: a

Summary

In this lesson, you should have learned how to:

- Enumerate the increase in the length limits for VARCHAR2, NVARCHAR2, and RAW data types in Oracle SQL
- Understand Oracle Database Migration Assistant for Unicode 6.1
- Making the SecureFiles the default storages mechanism for LOBs
- Use the SQL row-limiting clause in a query

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 25: Overview

- 25-1: Using 32K VARCHAR2 data type
(*optional*)
- 25-2: Querying a table using a SQL row-limiting clause
(*optional*)

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

New Processes, Views, Parameters, Packages, and Privileges



ORACLE

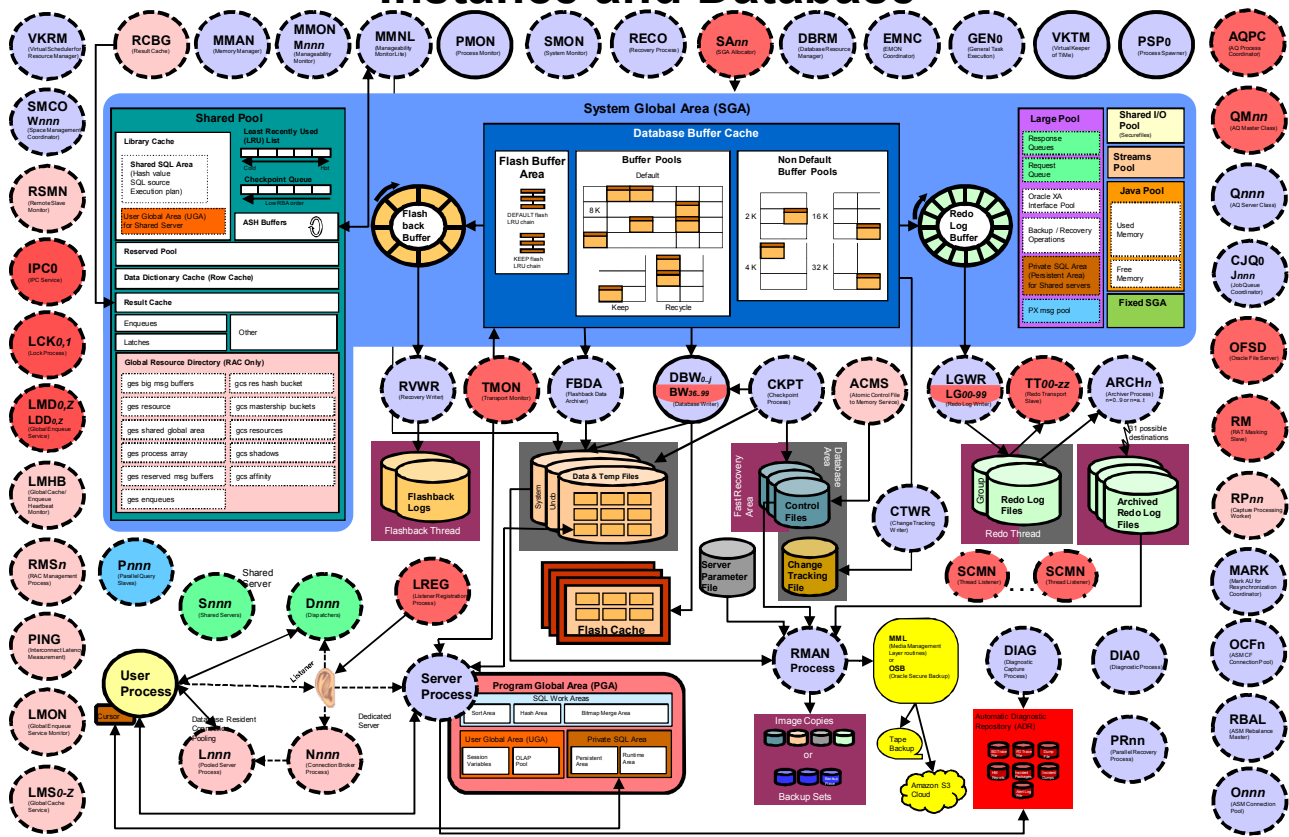
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Note

For a complete list of views, parameters, packages, and privileges on Oracle Database 12c new features, refer to the following guide in the Oracle documentation:

- *Oracle Database Reference 12c Release 1 (12.1)*
- *Oracle Database PL/SQL Packages and Types Reference 12c Release 1 (12.1)*
- *Oracle Database Security Guide 12c Release 1 (12.1)*

Instance and Database

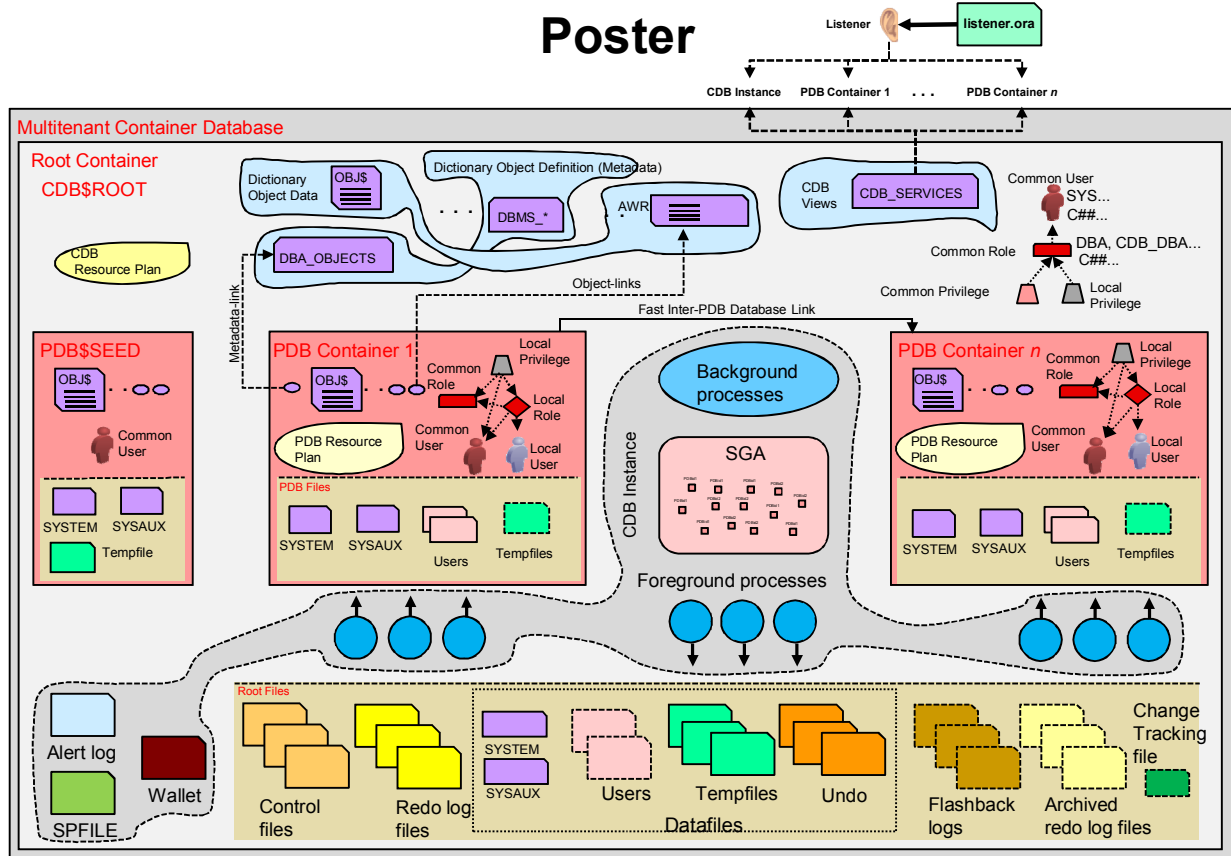


Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Caption:

1. Circle elements represent Oracle processes. If they are surrounded by dotted lines (equal dotted elements), it means they can run either as threads or OS processes. If they are surrounded by a solid line, it means they can run as only OS processes. The SCMN case is an exception. When using the Multi-Process Multi-Threaded architecture, each OS process, running more than one Oracle Process, also runs a special thread called SCMN that is basically an internal listener thread. All thread creation is routed through this thread.
2. Darker circle elements are new elements for DB 12c.
3. The two main different color nuances for circle elements are to make the distinction between RAC and non-RAC processes.
4. Files are represented by cylinders.
5. Storage location for those files are divided into three main areas: Fast Recover Area, Database Area, and Automatic Diagnostic Repository. Areas are designated by background rectangles by using three different colors. Exception is the server parameter file. If you find a file type part of two areas, it means that some corresponding files can be in both areas.
6. Inside one circle element you may see two names. This is to indicate that the second (smaller letters) is a slave of the first.

Multitenant Architecture: General Architecture Poster



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

CDB and PDB

View

- CDB_xxx: All of the objects in the CDB across all PDBs
- DBA_xxx: All of the objects in a container or PDB
- CDB_pdb\$: All PDBs within CDB
- CDB_tablespaces: All tablespaces within CDB
- CDB_users: All users within CDB (common and local)
- V\$PDBS: Displays information about PDBs associated with the current instance
- V\$CONTAINERS: Displays information about PDBs and the root associated with the current instance
- PDB_PLUG_IN_VIOLATIONS: Displays information about PDB violations after compatibility check with CDB
- RC_PDBS: Recovery catalog view about PDB backups

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

CDB and PDB

Parameter

- `ENABLE_PLUGGABLE_DATABASE`: Required to create a CDB at CDB instance startup
- `PDB_FILE_NAME_CONVERT`: Maps names of existing files to new file names when processing a `CREATE PLUGGABLE DATABASE` statement
- `CDB_COMPATIBLE`: Enables you to get behavior similar to a non-CDB
- `PDB_OS_CREDENTIAL` **12.1.0.2**: Enables another OS user than `oracle` to connect to PDBs

Package

- `DBMS_PDB.DESCRIBE`
- `DBMS_PDB.CHECK_PLUG_COMPATIBILITY`

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Heat Map and ADO

View

- DBA_HEAT_MAP_SEG_HISTOGRAM
- DBA_HEAT_MAP_SEGMENT
- V\$HEAT_MAP_SEGMENT

- DBA_ILMOBJECTS
- DBA_ILMPOLICIES, DBA_ILMDATAMOVEMENTPOLICIES
- DBA_ILMTASKS, DBA_ILMEVALUATIONDETAILS
- DBA_ILMRESULTS

Parameter

- HEAT_MAP: Activates activity tracking and statistics collection

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Heat Map and ADO

Package

- DBMS_HEAT_MAP
 - BLOCK_HEAT_MAP
 - EXTENT_HEAT_MAP
- DBMS_ILM
 - EXECUTE_ILM, STOP_ILM
 - PREVIEW_ILM, ADD_TO_ILM, REMOVE_FROM_ILM
 - EXECUTE_ILM_TASK
- DBMS_ILM_ADMIN
 - CUSTOMIZE
 - DISABLE_ILM, ENABLE_ILM
 - CLEAR_HEAT_MAP_ALL, CLEAR_HEAT_MAP_TABLE
 - SET_HEAT_MAP_START
 - SET_HEAT_MAP_ALL, SET_HEAT_MAP_TABLE

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In-Database Archiving and Temporal Validity

New column

- `ORA_ARCHIVE_STATE` in application tables

Package

- `DBMS_FLASHBACK_ARCHIVE`
 - `ENABLE_AT_VALID_TIME`
 - `SET_CONTEXT_LEVEL`

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Security: Auditing

View

- UNIFIED_AUDIT_TRAIL
- AUDIT_UNIFIED_POLICIES
- AUDIT_UNIFIED_ENABLED_POLICIES

New columns in UNIFIED_AUDIT_TRAIL

- FGA_POLICY_NAME
- DP_XXX (Data Pump operations)
- RMAN_XXX
- OLS_XXX
- DV_XXX (Database Vault operations)
- XS_XXX (Real Application Security operations)

Package

- DBMS_AUDIT_MGMT.FLUSH_UNIFIED_AUDIT_TRAIL

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Security: Privilege Analysis

View

- DBA_USED_PRIVS
- DBA_USED_SYSPRIVS, DBA_USED_OBJPRIVS
- DBA_USED_PUBPRIVS
- DBA_USED_OBJPRIVS_PATH, DBA_USED_SYSPRIVS_PATH
- DBA_UNUSED_PRIVS
- DBA_UNUSED_OBJPRIVS, DBA_UNUSED_SYSPRIVS
- DBA_UNUSED_OBJPRIVS_PATH
- DBA_UNUSED_SYSPRIVS_PATH
- DBA_PRIV_CAPTURES

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Security: Privilege Analysis and New Privileges

Package

- DBMS_PRIVILEGE_CAPTURE.CREATE_CAPTURE
- DBMS_PRIVILEGE_CAPTURE.ENABLE_CAPTURE
- DBMS_PRIVILEGE_CAPTURE.DISABLE_CAPTURE
- DBMS_PRIVILEGE_CAPTURE.GENERATE_RESULT
- DBMS_PRIVILEGE_CAPTURE.DROP_CAPTURE

Privilege

- SYSBACKUP
- SYSDG
- SYSKM
- PURGE DBA_RECYCLEBIN

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Security: Oracle Data Redaction

View

- REDACTION_POLICIES
- REDACTION_COLUMNS
- REDACTION_VALUES_FOR_TYPE_FULL

Package

- DBMS_REDACT.ADD_POLICY
- DBMS_REDACT.ALTER_POLICY
- DBMS_REDACT.DROP_POLICY
- DBMS_REDACT.ENABLE_POLICY
- DBMS_REDACT.DISABLE_POLICY
- DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES

Privilege

- EXEMPT REDACTION POLICY
- EXEMPT DDL REDACTION POLICY
- EXEMPT DML REDACTION POLICY

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

HA: Flashback Data Archive

Package

- `DBMS_FLASHBACK_ARCHIVE.SET_CONTEXT_LEVEL`
- `DBMS_FLASHBACK_ARCHIVE.GET_SYS_CONTEXT`

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Manageability: Database Operations

New Column

- DBOP_NAME
- DBOP_EXEC_ID in
 - V\$SQL_MONITOR
 - V\$ACTIVE_SESSION_HISTORY
 - CDB_HIST_ACTIVE_SESS_HISTORY
 - DBA_HIST_ACTIVE_SESS_HISTORY

Parameter

- STATISTICS_LEVEL=TYPICAL
- CONTROL_MANAGEMENT_PACK_ACCESS=DIAGNOSTIC+TUNING

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Manageability: Database Operations

Package

- `DBMS_SQL_MONITOR.BEGIN_OPERATION`
- `DBMS_SQL_MONITOR.END_OPERATION`
- `DBMS_SQL_MONITOR.REPORT_SQL_MONITOR`
- `DBMS_SQL_MONITOR.REPORT_SQL_MONITOR_LIST`
- `DBMS_SQL_MONITOR.REPORT_SQL_MONITOR_XML`
- `DBMS_SQL_MONITOR.REPORT_SQL_MONITOR_LIST_XML`

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Manageability: ADDM

Package DBMS_ADDM:

New functions

- REAL_TIME_ADDM_REPORT
- COMPARE_INSTANCES
- COMPARE_DATABASES
- COMPARE_CAPTURE_REPLAY_REPORT
- COMPARE_REPLAY_REPLAY_REPORT

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Performance: In-Memory Column Store

Package

- `DBMS_INMEMORY.REPOPULATE`

Parameter

- `INMEMORY_SIZE`
- `INMEMORY_QUERY`
- `INMEMORY_CLAUSE_DEFAULT`
- `INMEMORY_FORCE`

New Columns

- `INMEMORY_PRIORITY`, `INMEMORY_DISTRIBUTE`,
`INMEMORY_COMPRESSION` in `DBA_TABLES`,
`DBA_TAB_PARTITIONS`
- `DEF_INMEMORY_PRIORITY`, `DEF_INMEMORY_DISTRIBUTE`,
`DEF_INMEMORY_COMPRESSION` in `DBA_TABLESPACES`

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Performance: In-Memory Column Store

View

- V\$IM_COLUMN_LEVEL
- V\$IM_COL_CU
- V\$IM_HEADER
- V\$IM_SEGMENTS
- V\$IM_USER_SEGMENTS
- V\$IM_SEGMENTS_DETAIL
- V\$IM_SEG_EXT_MAP
- V\$IM_TBS_EXT_MAP
- V\$IM_SMU_HEAD
- V\$IM_SMU_CHUNK

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Performance: Full Database In-Memory Caching

New Columns

- `FORCE_FULL_DB_CACHING` in `V$DATABASE`

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Performance: Automatic Big Table Caching

Parameter

- DB_BIG_TABLE_CACHE_PERCENT_TARGET

View

- V\$BT_SCAN_CACHE
- V\$BT_SCAN_OBJ_TEMPS

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Performance: SQL Tuning

Package

- DBMS_SPM.REPORT_AUTO_EVOLVE_TASK
- DBMS_SPM.CREATE_EVOLVE_TASK
- DBMS_SPM.EXECUTE_EVOLVE_TASK
- DBMS_SPM.REPORT_EVOLVE_TASK
- DBMS_SPM.IMPLEMENT_EVOLVE_TASK
- DBMS_STATS.SEED_COL_USAGE
- DBMS_STATS.REPORT_COL_USAGE

Parameter

- OPTIMIZER_ADAPTIVE_REPORTING_ONLY

New Column

- IS_RESOLVED_ADAPTIVE_PLAN, IS_REOPTIMIZABLE in V\$SQL

View

- DBA_SQL_PLAN_DIR_OBJECTS

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Performance: Resource Manager

View

- DBA_CDB_RSRC_PLAN_DIRECTIVES

Package DBMS_RESOURCE_MANAGER: New procedures

- CREATE_CDB_PLAN / UPDATE_CDB_PLAN / DELETE_CDB_PLAN
- CREATE_CDB_PLAN_DIRECTIVE / UPDATE_CDB_PLAN_DIRECTIVE / DELETE_CDB_PLAN_DIRECTIVE
- UPDATE_CDB_DEFAULT_DIRECTIVE
- UPDATE_CDB_AUTOTASK_DIRECTIVE

Package DBMS_RESOURCE_MANAGER: Updated procedure

- SET_CONSUMER_GROUP_MAPPING: New values for the ORACLE_FUNCTION attribute
 - INMEMORY_PREPOPULATE
 - INMEMORY_POPULATE
 - INMEMORY_REPOPULATE
 - INMEMORY_TRICKLE

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Performance: Multi-Process Multi-Threaded

Parameter

- `THREADED_EXECUTION=TRUE`

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Performance: Database Smart Flash Cache

Parameter change

- `DB_FLASH_CACHE_FILE=file1, file1`
- `DB_FLASH_CACHE_SIZE=size1, size2`

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Performance: Temporary UNDO

View

- V\$TEMPUNDOSTAT

Parameter

- TEMP_UNDO_ENABLED=TRUE

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Performance: Online Operations

Package

- `DBMS_REDEFINITION.START_REDEF_TABLE (...,
copy_vpd_opt =>
DBMS_REDEFINITION.CONST_VPD_AUTO)`
- `EXEC DBMS_REDEFINITION.FINISH_REDEF_TABLE (...,
dml_lock_timeout => 100);`

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Miscellaneous: Partitioning

New Column

- DEF_INDEXING in
 - DBA_PART_TABLES
- INDEXING in
 - DBA_TAB_PARTITIONS
 - DBA_TAB_SUBPARTITIONS
 - DBA_INDEXES
- ORPHANED_ENTRIES in
 - DBA_INDEXES
 - DBA_IND_PARTITIONS

Package

- DBMS_PART.CLEANUP_GIDX

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Miscellaneous: SQL

Parameter

- MAX_STRING_SIZE=standard|extended
- DB_SECUREFILE=preferred

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Appendix C: Data Comparison

Package

- DBMS_COMPARISON

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

B

Pluggable Databases: Other Creation Methods

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

This appendix explains how to create Pluggable Databases by using methods that were not detailed in the lesson titled “Creating Container Database and Pluggable Databases.”

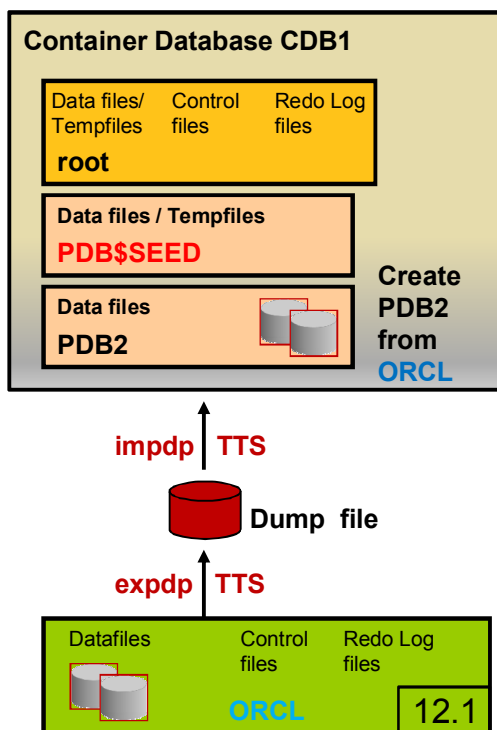
Note: For a complete understanding of Oracle Pluggable Database new feature and usage, refer to the following guides in the Oracle documentation:

- *Oracle Database Administrator’s Guide 12c Release 1 (12.1)*
- *Oracle Database PL/SQL Packages and Types Reference 12c Release 1 (12.1) – “DBMS_PDB” chapter*

You can also refer to the OTN white papers:

- *“Oracle Database 12c: Full Transportable Export/Import”*
- *“Overview of Cross-Platform Data Transport using Backup Sets”*

Plugging a Non-CDB into CDB Using Data Pump



1. Perform TTS or TDB export from **ORCL**.
2. Connect to the root as a common user with the `CREATE PLUGGABLE DATABASE` privilege.
3. Create new PDB2 (Method 1).
4. Perform TTS or TDB import into CDB1 / with:
 - **ORCL dumpfile**
 - **ORCL datafiles**
6. Check application data:

```
SQL> CONNECT sys@PDB2
SQL> SELECT * FROM HR.EMP;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If you choose the transportable tablespace (TTS) or database (TDB) export/import method, the method is the same as using Data Pump export/import to move data between two non-CDBs.

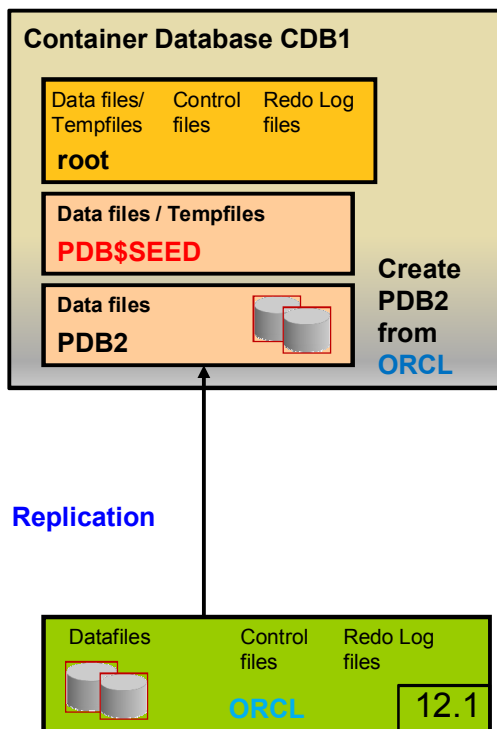
1. First, perform a TTS or TDB export from **ORCL** database.
2. Then, connect to the root of the CDB as a common user.
3. To create the new PDB2 that will be the container for **ORCL** data, use the method 1 as explained previously.
4. Finally, perform a TTS or TDB import into PDB2 of CDB using both **ORCL** dump file and data files. The import command uses the connection to the PDB2 service.
5. Then connect to PDB2 to check with `dba_tables` view that your tables are in PDB2. Check that the application data has been imported.

```
SQL> SELECT * FROM HR.EMP;
```

If you cannot use transportable databases or tablespaces to move the data, then use a full conventional Data Pump export/import to move the data and metadata to the PDB.

Note: To get detailed information on how to perform a TDB, refer to the lesson covering Data Pump Enhancements or to the *Oracle Database Administrator's Guide 12c Release 1 (12.1) Chapter Transporting Data*.

Plugging a Non-CDB into CDB Using Replication



Method using replication:

1. Connect to the root as a common user with the CREATE PLUGGABLE DATABASE privilege.
2. Create new PDB2 (Method 1).
3. Open PDB2 in read/write mode.
4. Configure unidirectional replication environment from ORCL to PDB2.
5. Check application data:

```
SQL> CONNECT sys@PDB2
SQL> SELECT *
      2  FROM dba_tables;
SQL> SELECT * FROM HR.EMP;
```

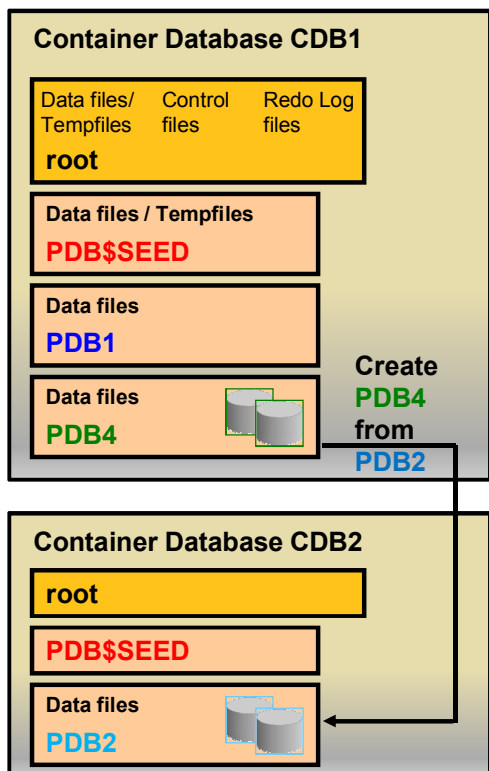
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If you choose the replication method, the steps would be the following:

1. Connect to the root as a common user with the CREATE PLUGGABLE DATABASE privilege.
2. Use method 1 as explained previously to create the new PDB2 that will be the container for ORCL data.
3. Open PDB2 in read/write mode.
4. Configure an Oracle GoldenGate unidirectional replication environment with the non-CDB ORCL as the source database and the PDB2 as the destination database.
5. When the data at PDB2 catches up with the data at the non-CDB ORCL, fail over to PDB2.

Cloning PDBs Across CDBs



1. In CDB1, create a **link_cdb2** to a common user with the SYSDBA privilege in CDB2.
2. Use any method to define the location of the new **pdb4** files.
3. Connect to CDB2 as SYS.
4. Quiesce **PDB2**:


```
SQL> ALTER PLUGGABLE DATABASE pdb2 CLOSE;
SQL> ALTER PLUGGABLE DATABASE pdb2 OPEN READ ONLY;<
```
5. Connect to CDB1 as SYS.
6. Clone **PDB4** from **PDB2**:


```
SQL> CREATE PLUGGABLE DATABASE pdb4 FROM PDB2@link_cdb2;
```
7. Reopen **PDB2**.
8. Open **PDB4** in read/write mode.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

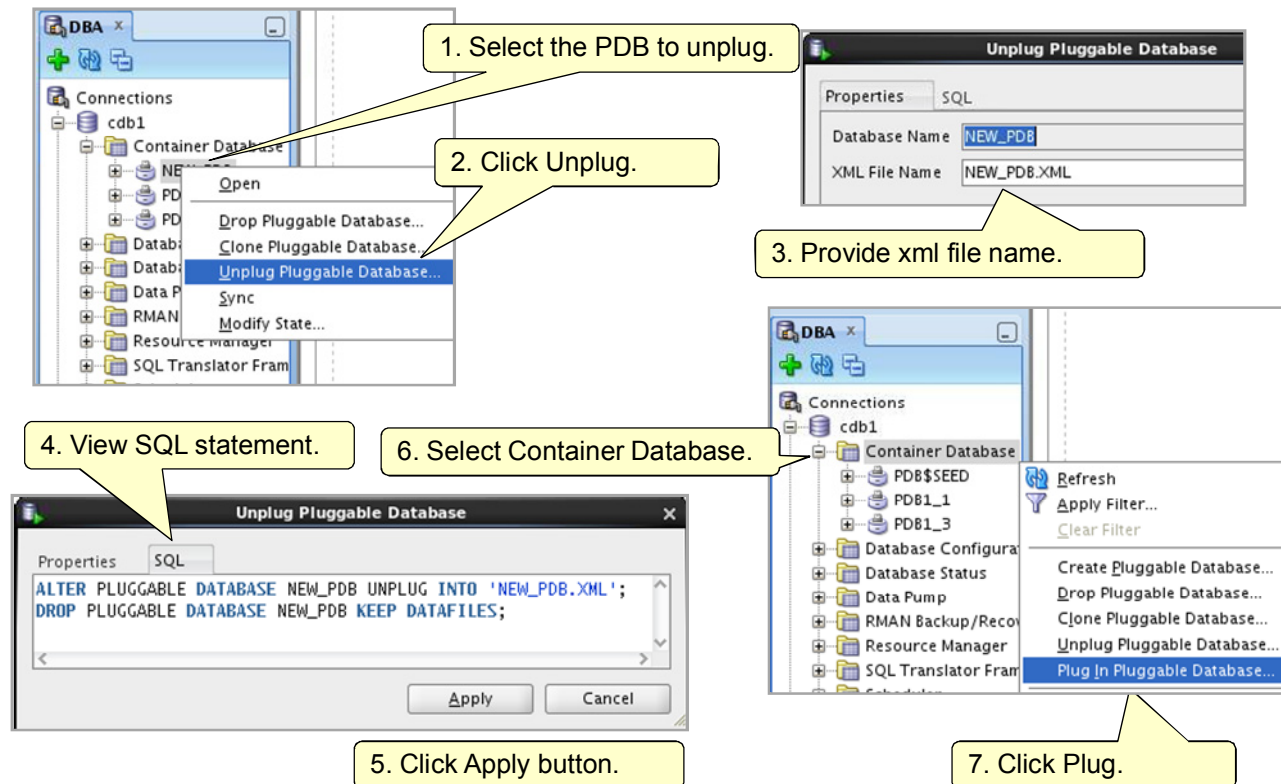
This technique copies a source PDB from a CDB and plugs the copy in to another CDB. In this case, you must specify a database link to the remote CDB in the **FROM** clause.

The database link connects to the remote source PDB from the CDB that will contain the new PDB.

There are different ways to specify the target locations of the files of the cloned PDB, either **FILE_NAME_CONVERT** clause in the **CREATE PLUGGABLE DATABASE** statement, or omit this clause and use either **DB_CREATE_FILE_DEST** or **PDB_FILE_NAME_CONVERT** initialization parameter.

There are some restrictions to this operation. The source and target CDB platforms must meet these requirements: they must have the same endianness and compatible database options installed, use the same character set and national character set.

Plugging Unplugged PDB: Using SQL Developer



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can use SQL Developer to perform the unplug and plug operations.

First select the PDB to unplug and close it. Then click Unplug option. Provide a name for the xml file that will be generated. You can view the SQL statement before applying it. Then to plug the unplugged PDB, select the Container Database and choose Plug option.

Note: In Oracle Database 12.1.0.2, if describing a remote PDB, the PDB name can specify the name of a database link through which that PDB may be accessed.

```
SQL> create database link pdblink connect to system identified by
*****
```

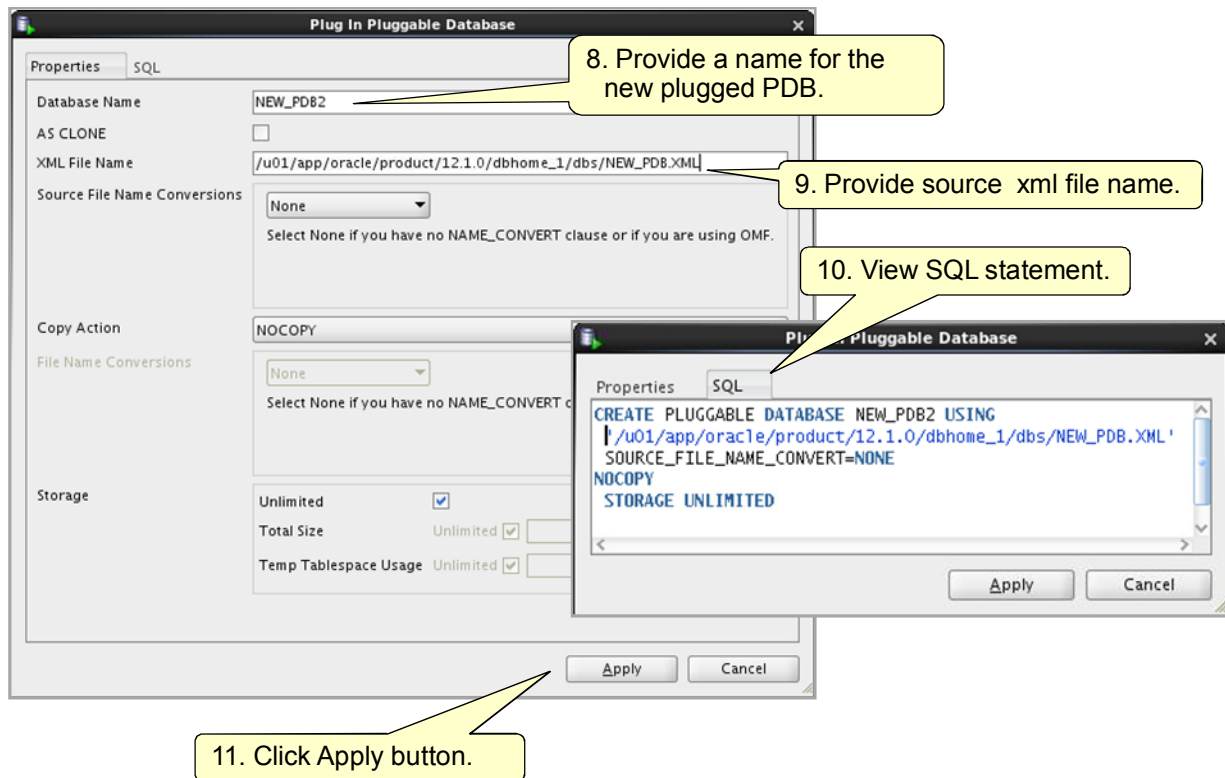
```
using 'PDB_SAMPLES';
```

Database link created.

```
SQL> exec dbms_pdb.describe('/tmp/pdb_samples.xml',
'PDB_SAMPLES@pdblink')
```

PL/SQL procedure successfully completed.

Plugging Unplugged PDB: Using SQL Developer

**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To plug the PDB, provide a name for the new plugged PDB and the XML file source file. Before applying the statement, you can view it.

The last action would be the PDB opening.

Schema and Data Change Management

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Objectives

After completing this lesson, you should be able to explain:

- Why and when to use the new Schema Change Plans feature
- How to use schema change plans
- Why and when to use the new Data Comparisons feature
- How to use data comparisons

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Note

For information about licensing the *Database Lifecycle Management Pack*, refer to the following guide in the Oracle documentation:

- *Oracle Enterprise Manager Licensing Information 12c Release 1 (12.1)*

Refer to other sources of information:

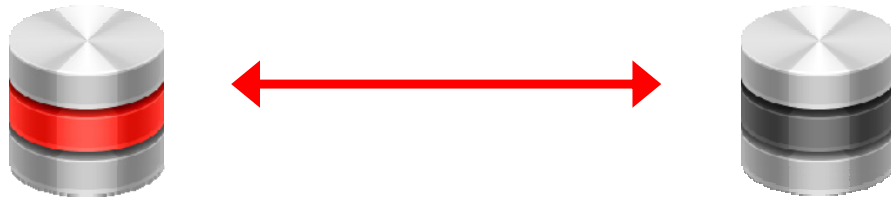
- *Oracle Enterprise Manager Cloud Control 12c Demo Series* demonstrations under Oracle Learning Library:
 - *Managing Schema Change Plans*
 - *Performing Data Comparison*

Schema Change Plans and *Data Comparison* operations are available only with Enterprise Manager Cloud Control.

Database Lifecycle Management Pack: New Features

Schema change plans:

- Save schema changes into a change plan.
- Apply change to multiple targets.



Data comparisons:

- Compare reference data across environments.
- Identify differences between test and production.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

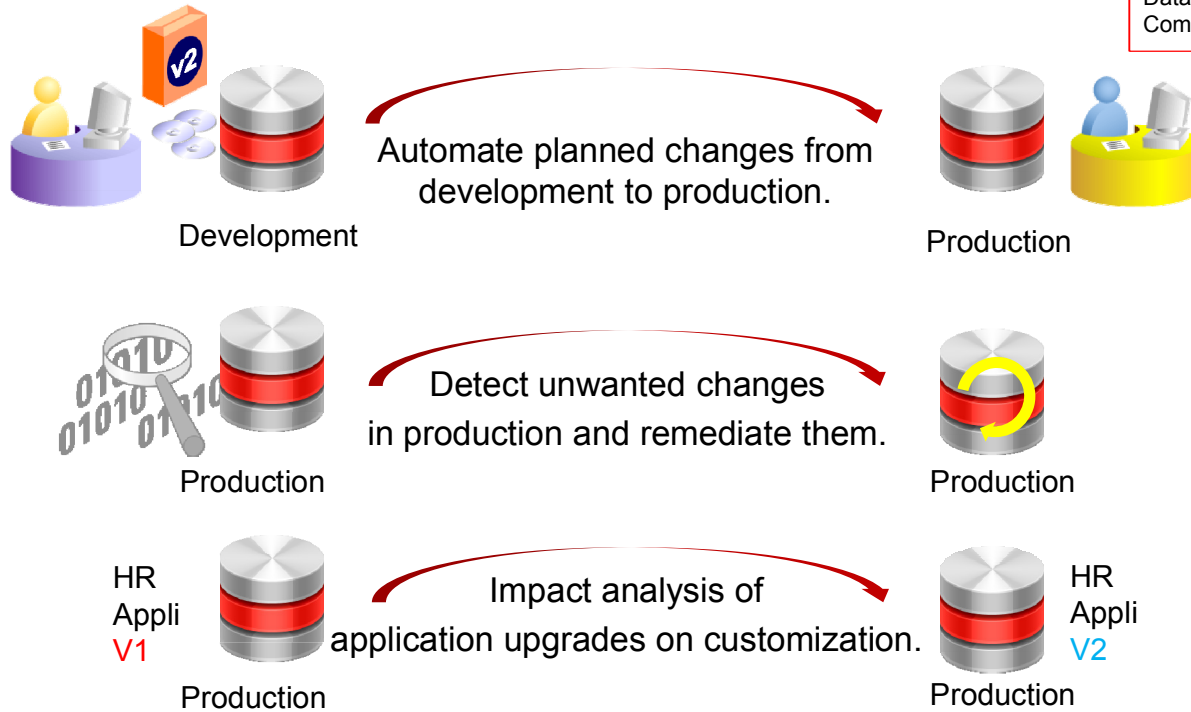
The Database Lifecycle Management Pack is all about making sure that enterprises can manage all changes, either proactively as they promote a database from development test to production, or reactively when any unwanted changes are made in a production environment that are causing problems. Enterprise Manager provides this capability to automatically identify and detect these changes so that organizations can take immediate actions and automate the process of applying the corrective actions without scripts and manual intervention.

The Database Lifecycle Management Pack allows you to:

- Group schema changes together into what is called a schema change plan based on comparison between two databases and propagate those change plans from one environment to another
- Perform data comparisons where you can compare data across environments, and identify the divergences in data between your test and production environments

Change Management Pack Features

**Schema
Change
Plans**
Data
Comparisons



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Change Management Pack Capabilities

The Oracle Enterprise Manager 11g Release 1 Change Management pack gives database administrators the ability to evaluate, plan for, and implement database schema changes to support new application requirements without error and data loss while minimizing down time.

The Oracle Enterprise Manager 11g Release 1 Change Management pack allows you to:

- Automate plan changes from development into production
- Detect unwanted changes in production and remediate them
- Analyze the impact of an application upgrade on customization

For example, when you have a custom application or a custom object or module and perform an application upgrade, there are some triggers added behind the scene, and columns may be added to the tables. Some modules may also be affected by database objects changes. Changes on your application objects are analyzed and reported.

How do you automate planned changes from development to production?

Change Management Pack Components

1. Create Dictionary Baselines:
 - Contain the definitions of a set of objects captured at a specific time
 - Can be used for comparisons
2. Perform Dictionary Comparisons:
 - Detect discrepancies between two versions of objects definitions
3. Perform Dictionary Synchronizations:
 - Make the definitions of a set of objects in the destination database the same as those in the source

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Enterprise Manager 11g Release 1, the key features of the Change Management pack that enable administrators to evaluate, plan, and implement database schema changes are the following:

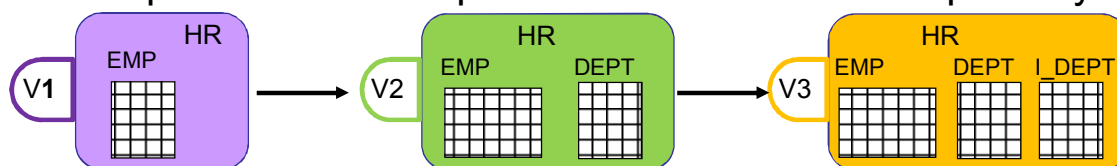
- Dictionary baselines: A point in time of the definition of the database and its associated database objects
- Dictionary comparisons: A complete list of differences between a baseline or a database and another baseline or a database
- Dictionary synchronizations: A process of promoting changes from a database definition capture in a baseline or from a database to a target database

Dictionary Baselines

Contain definitions of a database or subset of a database captured at a specific time

- Non-schema object types:
 - Tablespaces
 - Users, profiles, roles, grants
- All schema objects for an application:
 - Tables, views, indexes
 - Procedures, packages, triggers
 - init.ora

Provides point-in-time snapshots stored in the EM repository



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A dictionary baseline is an object that contains a set of database definitions captured at a certain point in time and stored in the Enterprise Manager repository.

When creating a baseline, you create a corresponding baseline scope specification, which describes the names and the types of database objects and schemas from which they should be captured.

At a later time, or at regular intervals, you capture additional versions of the same baseline. A scope specification identifies the database objects to be captured in a baseline. (Scope specifications also identify objects to process in dictionary comparisons and synchronizations.) When the scope of a baseline is specified, you cannot change the scope specification. This restriction ensures that all versions of the baseline are captured by using the same set of rules, which means that differences between versions result from changes in the database, not from scope specification changes. The baselines can be used as references for future comparisons with other databases or other baselines to analyze if changes took place.

- Baseline scope specification can include:
 - Schemas and object types to capture
For example, you can capture all tables, indexes, and views of schemas `HR` and `SH`.
 - Non-schema objects such as users, roles, and tablespaces, and privilege grants to users and roles
You can specify schemas to exclude, and object types.
 - Individual schema objects by specifying the type, schema and name of each object
 - `init.ora` parameters

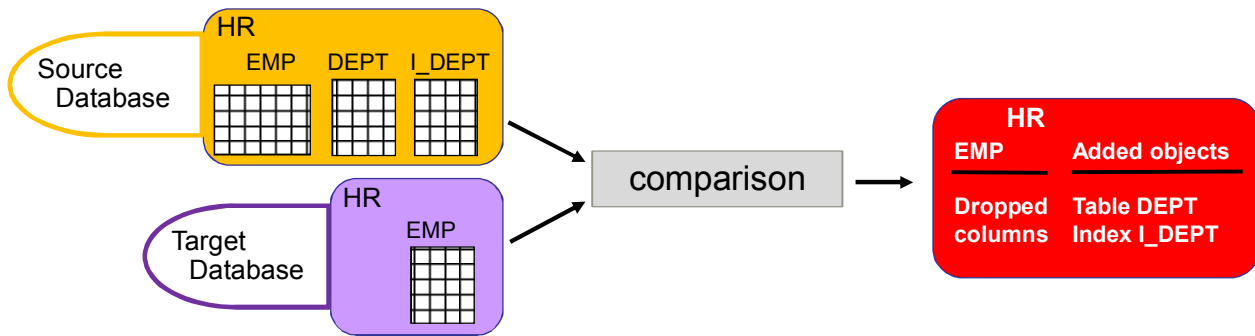
Baselines are good references to compare to other baselines or databases.

How Would Developers Use Baselines?

Several developers work on the same test database and make some changes in the same application. At different points in time, an automated job regularly creates a new version of the baseline, including changes made by the different developers over time. The reported changes are stored outside the database, in the Enterprise Manager repository. It is auditable.

Dictionary Comparisons

- Contain the results of comparing the definitions of database objects at a specific time
- Can compare:
 - Database with database
 - Baseline with database
 - Schema with schema
- Can define what you want to compare



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The next step in the process of propagating the identified changes to another database is to compare between the reference database or baseline and the target database.

You will perform a dictionary comparison that identifies differences in database object definitions between a baseline from the development database and the production database, or the test database and the production database, or two schemas within a single database/baseline.

A comparison specification is defined by a source and a target, scope, and owner. The scope specification describes the names and types of database object definitions to be included in the comparison and the schemas that contain these object definitions.

Comparisons identify differences in any attribute value between objects of any type. For example, comparisons show the differences between the definitions in the original baseline for the application and those in the current database. After creating a new comparison version, it identifies the differences between the original definitions at the start of the development cycle, and those same definitions at the current time.

Schema objects in the source are compared to objects in the same schema in the target. For example, the HR.EMP table in the development source database is compared to HR.EMP in the production target database.

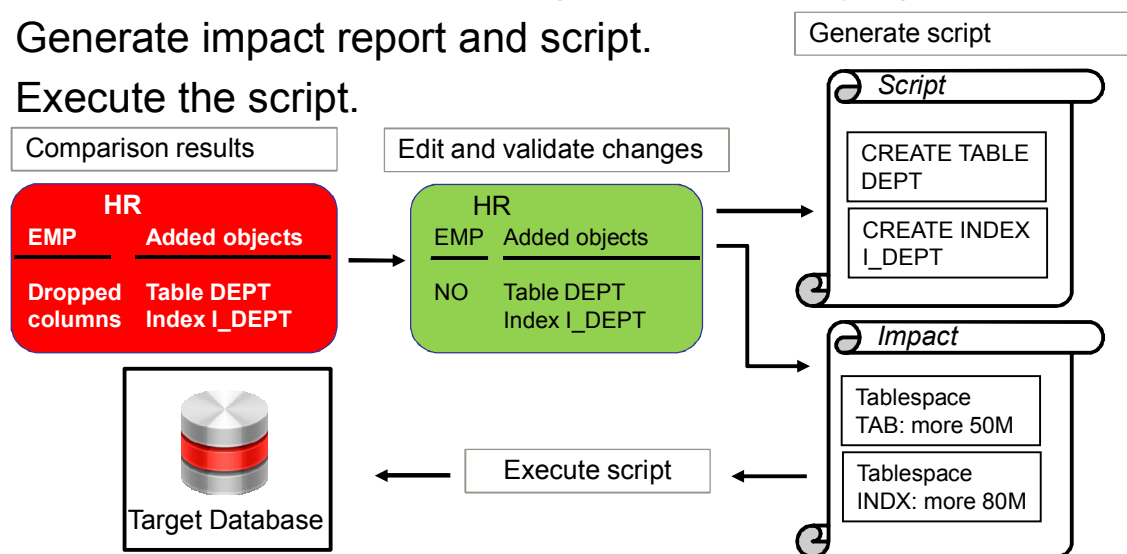
Any discrepancy between the source and target databases is reported:

- A new column is added to or dropped from a table in either one of the source databases or target database.
- A new index is created in the source database and not in the target database or vice versa.

Dictionary Synchronization

Dictionary synchronization includes three stages after compare source database or baseline with target:

1. View and edit validated changes before applying.
2. Generate impact report and script.
3. Execute the script.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The last step is the dictionary synchronization that will synchronize differences in database object definitions between two databases, or a baseline and a database. The basic action of a database synchronization is to create or modify selected object definitions in a database to match those in another database or in a baseline.

Synchronization is performed in three steps:

1. View the list of suggested changes to selectively exclude objects from synchronization, before the generation of the script. Interactive synchronization mode gives you this chance at the second step to examine the results of comparison.
2. Validate the suggested changes. The SQL script is generated. You can still view the SQL script to eliminate some SQL statements and ask for the regeneration of the script.
3. Proceed with the execution of the script.

Comparing Change Propagation and 11g SQL Scripts

Category	Change Propagation	SQL Scripts
Intelligent validation of changes	Yes	No
Preview and edit changes before apply	Yes	No
Automatically save and version changes	Yes	No
Display log of changes applied	Yes	Yes
Maintain history of change logs	Yes	No (else manually)
Centrally managed	Yes	No
Allows execution without revealing database passwords	Yes	No
Designed to handle database changes across multiple database environments	Yes	No

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Change Propagation Benefits Using Change Management Pack Rather than SQL Scripts

- Change propagation with Change Management Pack provides an intelligent validation of changes. The method using 11g SQL scripts is time consuming and potentially error prone.
- The comparison allows you to preview and edit the changes. The changes are automatically identified. It is easier to handle all the changes suggested after comparison. You just have to evict or accept them.
- You can keep track of the different versions of the changes through the GUI Enterprise Manager tool. This is more difficult to handle this with scripts: you have to manually create a directory to host the spool file and to maintain a history of change logs.
- The changes are centrally stored in the Enterprise Manager repository.
- The passwords are not exposed as this is the case with SQL scripts. Enterprise Manager provides this capability to store credentials that are very safe regarding secure identification as the developers and DBAs do not have to remember passwords to create baselines in a development database or apply changes in a target database.
- It is designed to handle database changes application across multiple database environments such as development, test, reporting, production databases again and again. The changes are captured in one place and can then be applied in several target databases.

Database Lifecycle Management Pack Schema Change Plans

Change plans allow users to specify, group, and package object metadata changes.

Create change plans from:

- Ad hoc changes
- Comparison-based differences
- Developer tools

Apply changes to multiple targets.

Role-based workflow

- Developer: Create/store change plan via SQL Developer
- DBA: Review/apply change plan



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The application developers need an automated mechanism to capture the development changes to hand off to the DBA instead of sending scripts back and forth which is time-consuming and potentially error prone process. Schema Change Plan provides the automated way for developers and DBAs to work on application schema changes via a common and single object, the change plan.

This object is created by developers to collect and store application schema changes, which is reviewed and applied into any target system by DBAs. This is therefore a container object for change requests.

According to your role in the enterprise, you use different tools to manage the change plans.

1. The developer creates and stores a change plan with SQL Developer tool in the Enterprise Manager repository. These changes can be ad hoc changes, such as newly created tables and indexes, added or dropped columns in tables, modified procedures, added triggers, modified views. The DBA can also create change plans relying on comparison-based changes. For example, if you compare two databases that have a common table, but in one database, the table contains an additional column that the same table in the other database does not have, then this is a change that can be included in a change plan. These changes are traditionally manually collected in a script. Enterprise Manager 12c Database Lifecycle Management introduces this new object, the change plan in which developers or DBAs store all application changes. Change plans are not associated with a source database.

2. Using Enterprise Manager GUI tool, the DBA reviews the changes within the change plan, edit the changes and validate them or not, and finally requests the SQL script generation. When the script generation is completed, the DBA executes the SQL script to apply all the accepted changes to several databases.

If developers are not allowed to access the test or production environment, the key thing introduced is the ability to work within a role-based workflow. If you look at the process at the bottom of the slide, the developer and DBA have different roles. The developer uses SQL Developer to manage the application development providing some schema changes. He creates a change plan to store changes and stores it in Enterprise Manager repository.

Then, the DBA uses Enterprise Manager to review and edit the schema change plan. After he validated the changes, he requests the SQL script generation and executes it at a scheduled time to apply the changes to multiple targets.

Change Requests

- A change plan contains change requests for one or more metadata objects.
- A change request can be a request to:
 - Create an object
 - Drop an object
 - Modify one or more attributes of an object
- When a change plan is deployed to synchronize the target:
 - The change is analyzed in the context of the database being deployed
 - A relevant PL/SQL script is generated, based on the metadata at the target database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A schema change plan contains change requests for one or more metadata objects. A change request can take three forms:

- Creates a database object: It contains the complete definition of the database object. For example, there is a new table that has been created in a development database and that needs to be added in the target database application.
- Drops a database object: It consists of the type, schema, and name of the object, and an instruction to drop that object. For example, a table or an index does not need to exist anymore in the target database because the developer dropped it from the application.
- Modifies a database object: it contains one or more instructions pertaining to specific object attributes. These can take various forms:
 - Modify the value of an attribute: for example, change a table's tablespace to USERS or the data type of column C1 from VARCHAR2 (50) to VARCHAR2 (100)
 - Add or remove a sub-object, such as a table column or partition. For example, modify one or more attributes of an object if some columns need to be added or dropped in the table in the target database.
 - Change the value of an initialization parameter

An object can be monitored so that in case any change occurs on this object, the change plan will be informed of the type of change applied on this monitored object.

A schema change plan is not a SQL script. The change requests consist of abstract specifications of the changes to be made. Deploying a change plan to a database results in a script that is appropriate for that database.

When a schema change plan is validated, what should also be considered during the deployment phase, is that the changes are analyzed in the context of the target database to determine whether the changes can be carried out at all. For example, if a change suggests to add a column in the target database table in comparison to the same table from the test database, and if the column, meanwhile, had already been manually added to the target table, the analysis reports that it is useless to apply the change. Another example, if a change request seeks to modify a table that does not exist in the destination database, it cannot be done. Similarly, if a change request adds a foreign key constraint to a table and the constraint references a table that does not exist, the constraint cannot be added to the table.

It determines whether the change may have harmful consequences. For example, if a table column has the data type `VARCHAR2 (200)`, a request to change the data type to `VARCHAR2 (100)` may result in script execution failure or truncation of existing data.

It creates the DDL statements needed to carry out the change request.

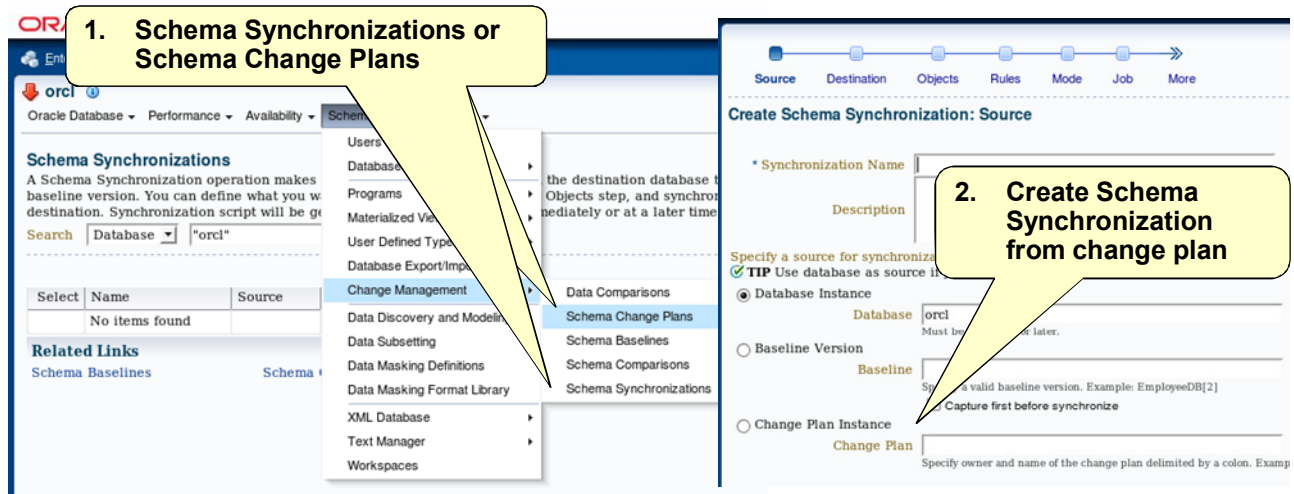
It determines the dependencies between objects affected by the schema change plan deployment and orders the DDL statements appropriately.

This is an intelligent analysis, which validates the changes made in the test database against the actual definitions that are in the target database to check if the changes can be applied successfully.

It generates the Impact Report indicating when a change request cannot be carried out or may result in a problem. The DBA can examine the Impact Report, along with the generated script, before executing the script.

The PL/SQL script is generated for the specific target database, based on the metadata at the target database and this step still allows the DBA to exclude certain actions to be performed on the target database before execution takes place.

Schema Synchronization



- If Schema Synchronizations, then Create Schema Synchronization from the
 - “Database” or “Baseline” or “Change plan” screen
- If Schema Change Plans, then “Create Synchronization from Change Plan” is selected.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Schema change plans complement the existing Change Management Dictionary Synchronization component as it is a new source for synchronization.

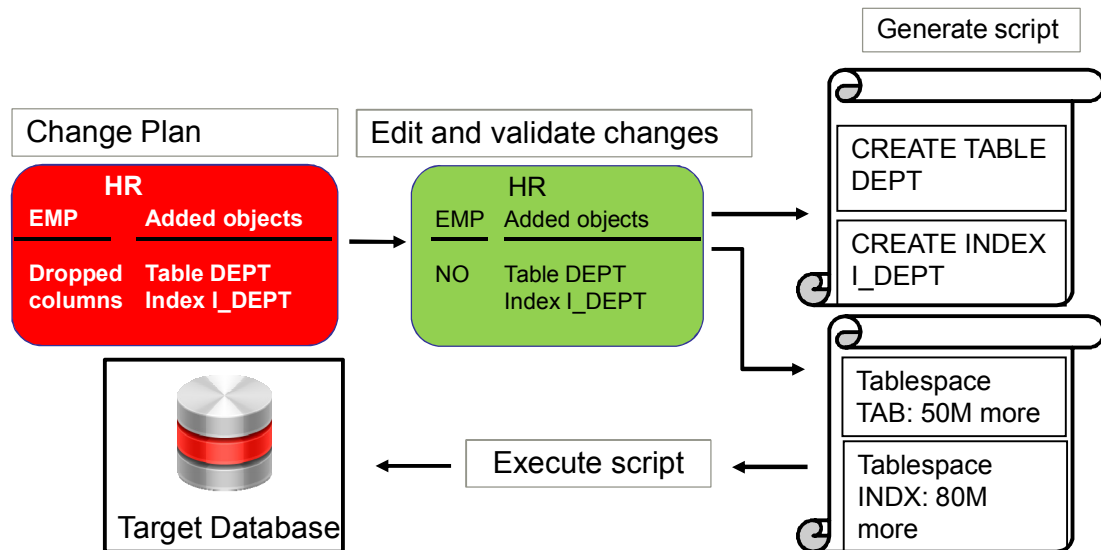
Schema Synchronization operation makes the definitions of set of objects in the destination database the same as those in the source and is still performed in three steps.

The step before synchronization is still the comparison of the source with the target database, but this source can be either a database or a baseline version baseline or a change plan. In the previous Oracle Change Management pack version, the source for comparison was restricted to either a baseline or a source database. The DBA just needs to review all changes contained in the schema plan changes and validate them before requesting the script generation.

You can perform the first step of the synchronization in two ways:

- Click the “Change Management” option in the “Schema” menu, then the “Schema Synchronizations” option and then “Create Schema Synchronization” where you will be prompted to choose the source of the synchronization. In Enterprise Manager Cloud Control, you now have the change plan as a possible source.
- Click the “Change Management” option in the “Schema” menu, then the “Schema Change Plans” option, and select the change plan amongst the available ones to synchronize a target with. Then, click the “Create Synchronization from a Change plan” button. The analysis is performed to display the inconsistencies in the change requests. The generation of the script still allows you to exclude certain actions to be performed on the target database before execution takes place.

Schema Synchronization



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The step before synchronization is still the comparison of the source with the target database where the source can be now a change plan. The DBA reviews all changes issued after the comparison, validates them before requesting the script generation.

After the “Create Synchronization from a Change Plan” analysis is performed, it displays the inconsistencies in the change requests. The generation of the script still allows you to exclude certain actions to be performed on the target database before execution takes place.

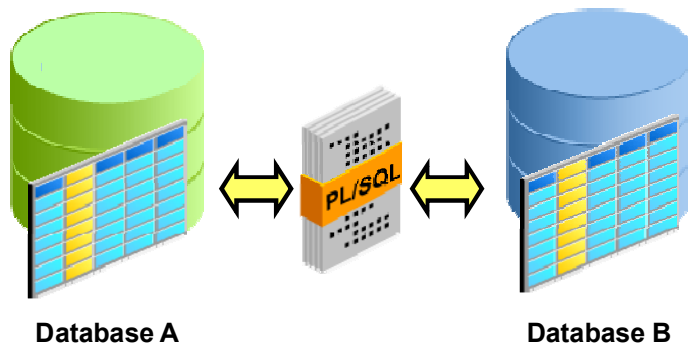
Database Lifecycle Management Pack

Data Comparisons

Schema Change
Plans
**Data
Comparisons**

Data Comparisons fills a critical gap to allow:

- Application vendors to compare seed data
- Application customers to compare configuration data between different sites
- DBAs to determine how seed data customizations will be affected by application upgrades



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Data Comparisons of Database Lifecycle Management pack fills a critical gap in the Change Management Pack lacking this capability of data comparison.

Application vendors need to compare seed data between application versions.

Customers of applications may want to compare their configuration data between different sites.

DBAs may want to know how their seed data customizations will be affected during the upgrade of the application database. This can be done by cloning the production database to a test database, upgrading the test database and then comparing the seed data between test and production. There are a number of such applications of data comparisons.

Data Comparisons is a new feature added to the Database Lifecycle Management Pack. This is accessible by Enterprise Manager Cloud Control GUI pages that provide an interface to access the `DBMS_COMPARISON` package, available in the Oracle Database.

DBMS_COMPARISON

Data Comparisons provides a GUI interface to access the DBMS_COMPARISON package.

- Compares data between a referenced and a candidate database
 - Requires a database link between the two databases
 - Referenced and candidate can be the same database
- Can be used for different types of data:
 - Seed data – Provided with an application on installation
 - Configuration data – Parameters for the application that are set up by the user
 - Master data – Data families that are of interest to the business (for example, customers, suppliers, products, employees, and so on)
 - Transaction data – Records the operation of business processes

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Data Comparisons is dependent on the DBMS_COMPARISON package of Oracle Database. DBMS_COMPARISON is a package appearing in Oracle Database 11g that enables to compare database objects in different databases and identify differences between them.

DBMS_COMPARISON compares data in database objects between a referenced database and a candidate database.

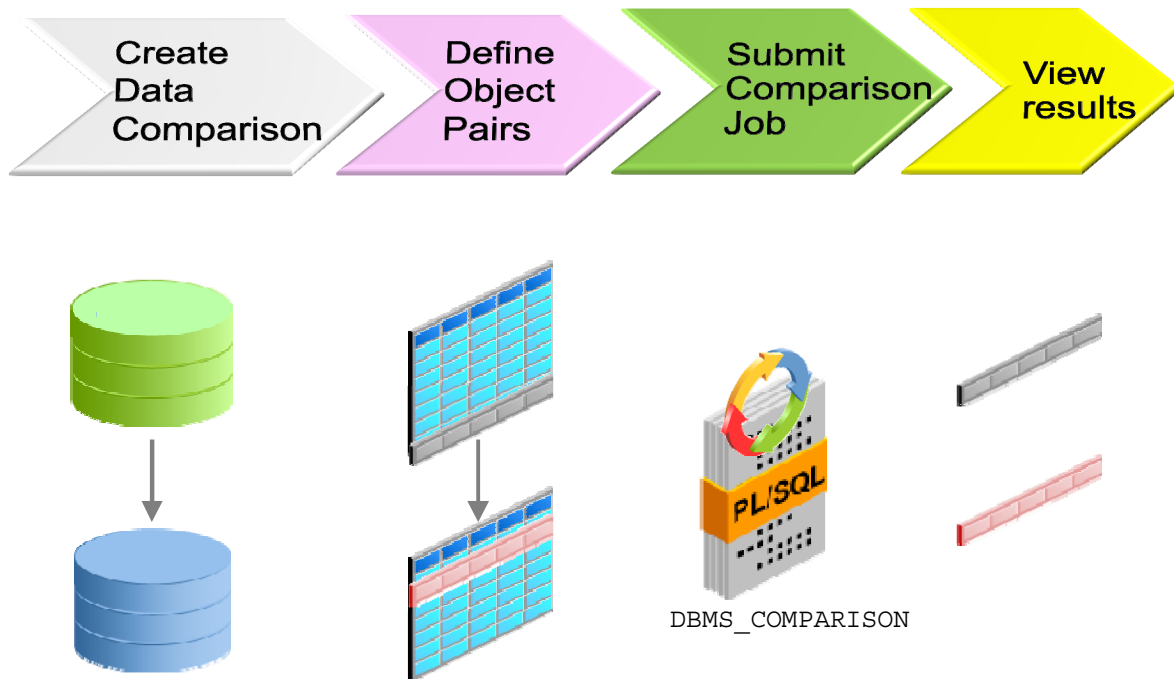
Referenced database is the one that runs the subprograms in the DBMS_COMPARISON package. Be advised that the referenced database carries an additional processing load and requires some space to store the row IDs of differing rows (not the entire rows themselves). If you compare data between a production system and a test system, it might be appropriate to process and store the results on the test system.

An object in the referenced database is compared with one in the candidate database.

The referenced and candidate databases can be one and the same. Otherwise, a database link created in the referenced database needs to be passed to DBMS_COMPARISON to allow it to access data in the candidate database.

- Seed data that is required by the application to function is data that comes with the application on installation. It can be provided with the system for training, testing, or as template for the data that user enters. For example, Siebel ships with seed data in a List of Values (LOV) definition, default mappings of views to responsibilities, and predefined queries.
- Configuration data consists of parameters for the application that are set up by the user. Depending on the kind of application, configuration data may include a chart of accounts, a supplier's payment terms, or a list of diagnostic codes.
- Master data refers to families of data that are of interest to the business (customers, suppliers, products, employees). These are typically the resources of the business and they fit into subject categories. All businesses, regardless of the type of business, store data about their people, products, financial, information, and physical resources.
- Transaction data records the operation of the business processes. Examples of transactions include orders, invoices, or payment records. This type of data is expected to grow quickly and form the bulk of data in transaction processing databases.

Flow



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The flow is simple.

1. You create a data comparison between two databases or within the same database.
2. You define the objects for which data comparison is needed.
3. You submit a job to perform the comparison.
4. View the results to verify if there are any differences in terms of data between the objects compared.

Guidelines

- Referenced database must be of Oracle Database 11g Release 1 or later and candidate database must be Oracle Database 10g Release 1 or later.
- Database character sets must be the same.
- Data can be compared for tables, single-table views, materialized views, and synonyms.
- An index must uniquely identify rows in both objects.
- Data cannot be compared for some data types (for example, LONG, LONG RAW, ROWID, CLOB, and BLOB).
 - These columns can, however, be excluded from the comparison.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The referenced database always executes the comparison; therefore, the Oracle Database version must be Oracle Database 11g or later. The candidate database for comparisons must be of Oracle Database 10g or later. It is either a remote database accessed by a database link or the same database to compare different schema objects.

The database character sets must be the same for the referenced and candidate databases.

Data Comparisons allows you to compare tables, single-table views, materialized views, and synonyms for tables. To compare rows from a referenced object with rows of a candidate object, the rows must be uniquely identified. A primary key constraint or a unique constraint on one or more non-NULL columns can satisfy this requirement.

So to compare rows from a referenced object with rows of a candidate object, the database objects must have one of the following types of indexes:

- A single-column index on a number, time stamp, interval, or `DATE` data type column
- A composite index that includes only number, time stamp, interval, or `DATE` data type columns. Each column in the composite index must either have a `NOT NULL` constraint or be part of the primary key.

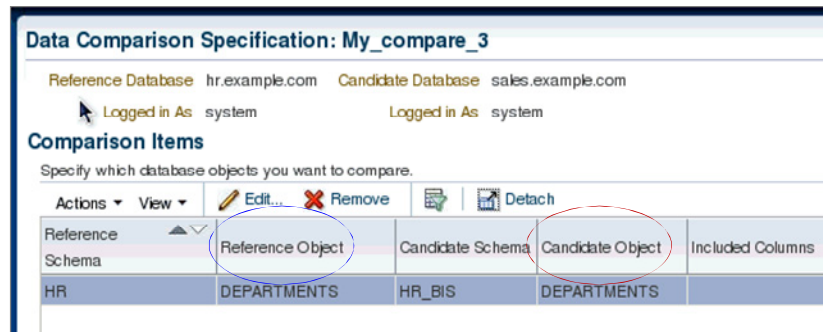
In case these constraints are not present on a table, then the user needs to create an index whose columns satisfy this requirement. In case this requirement is not satisfied, you get an error message while viewing the results such as:

```
ORA-23626: No eligible index on table SCOTT.BONUS ORA-06512:
at "SYS.DBMS_COMPARISON", line 5002 ORA-06512: at
"SYS.DBMS_COMPARISON", line 448 ORA-06512: at line 2
```

Most data types are considered during comparisons between two objects. But certain data types are not supported during comparison (examples: LONG, LONG RAW, ROWID, CLOB, BLOB). These columns can, however, be excluded from the comparisons.

Creating a Data Comparison

1. Create a data comparison.
 - Reference database
 - Candidate database
2. Select object pairs or add multiple objects to the list.



3. Submit comparison job.
4. View comparison results.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

From the Enterprise menu, choose a target database and then in a database context, from the Schema menu, click Change Management and then Data Comparisons.

1. Create the data comparison: Provide the names of the referenced and candidate databases that can be either different databases or the same database. The referenced database is the one that contains the objects of reference.
2. Through the “Actions” menu, define object pairs by specifying the referenced and candidate objects. If the referenced database is the same as the candidate database, the objects are from the same database. In the example in the slide, the comparison is defined between the `HR.DEPARTMENTS` table from `HR` database with the `HR_BIS.DEPARTMENTS` table from `SALES` database. Select one or more columns of the referenced or candidate objects for comparison being common to both objects. Optionally, select an index used for comparison if several exist, specify a where clause per pair of objects being compared, and configure the comparison to a point in time. You declare object pairs for comparison or multiple objects. Adding multiple objects enables a bulk inclusion of multiple objects from the referenced database into the specification. You can search and select multiple objects, such as tables and views from the referenced database, and then edit each item as needed.
3. Submit the comparison job.
4. When the comparison job has completed, view the results of the comparison.

Comparison Job and Results

Data Comparisons

A data comparison operation compares the objects in a candidate database with the objects in a reference database. Objects are to be compared and submitted for comparison immediately or at a later date. After the comparison job completion, select the data comparison job to view the results. The results will be displayed when you delete the comparison job.

Actions View Create... Edit Spec Submit Comparison Job... View Results Delete

Name	Reference Database	Candidate Database	Comparison Start	Job Status	Owner	Description
My_compare	hr.example.com	sales.example.com	2011-06-23 at 09:32	Completed with Error	DB_ADMIN	
My_compare_3	hr.example.com	sales.example.com	2011-06-23 at 09:43	Succeeded	SYSMAN	
My_compare_2	hr.example.com	sales.example.com	2011-06-23 at 09:34	Succeeded	SYSMAN	

Data Comparison Results: My_compare_3

View row differences between Candidate Database sales.example.com

View View Row Differences Detach

Reference Schema	Reference Object	Candidate Schema	Candidate Object	Result	Reference Only Rows	Candidate Only Rows	Non-identical Rows
HR	DEPARTMENTS	HR_BIS	DEPARTMENTS	≠	14	1	1

There are differences.

Reference-only rows
Candidate-only rows
Non-identical rows

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The first screenshot displays the comparison jobs that were submitted. Three comparisons were performed with different configurations.

The second screenshot shows the result of one of the completed comparisons My_Compare_3. Data Comparisons attempts to compare all tables. Look for rows in the "Result" column with the ≠ symbol, indicating that there are differences between referenced row and candidate row data. If the symbol is =, then all rows are identical according to the selected columns for Comparisons. If for any reason (for example, a missing unique index on any of the involved columns) the comparison failed, you can view the error messages by selecting the Messages tab that is not displayed on this window but would be at the bottom of the window. An error message is indicated with a red X instead of the ≠ symbol.

You can also view the SQL statements executed to perform the comparisons by clicking the Executed Statements tab. You would see the following detailed statements such as:

- create database link that creates the link between the referenced database and the candidate database
- dbms_comparison.create_comparison that creates a named comparison with the list of objects to be compared
- dbms_comparison.compare that executes the comparison

If you want to view the details of the differences between two objects, click View Row Differences.

Results: Reference Only Rows

- Reference-only rows
- Candidate-only rows
- Non-identical rows

Row Data Differences: my_compare_3

Reference Database: hr.example.com Candidate Database: sales.example.com

Logged in As: system Logged in As: system

Reference Object: HR.DEPARTMENTS Candidate Object: HR_BIS.DEPARTMENTS

Index Columns: DEPARTMENT_ID

Show ☒ Reference Only Rows ☐ Candidate Only Rows ☐ Non-identical Rows

Rows are categorized based on their index column values. Reference only and candidate only rows are those with index column values present on both sides, but with one or more other column values differing. Non-identical rows are those with index column values present on both sides, but with one or more other column values differing.

View

Row Source	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
Reference	10	Administration	200	1700
Reference	140	Control And Credit		1700
Reference	150	Shareholder Services		1700
Reference	160	Benefits		1700
Reference	170	Manufacturing		1700
Reference	180	Construction		1700

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Row Data Differences page allows you to view:

- Reference-only rows
- Candidate-only rows
- Non-identical changed rows
- All of them

The Row Source column indicates the origin of each row of data as a whole. Furthermore, data in a row differing between referenced and candidate are displayed in contrasting colors, indicating whether the source of the data comes from the referenced object (in blue) or the candidate object (in red).

The comparison is shown based on a key column (depending on a chosen unique index). If the key column value is different, the row appears as a candidate-only or reference-only row. If other columns are different, the row appears as a non-identical row.

In the example in the slide, you chose to view the rows that exist with index column values in the referenced object but not in the candidate object. For example, the rows with DEPARTMENT_ID 10, 140 do not exist in the HR_BIS.DEPARTMENTS table.

Results: Candidate-Only Rows

- Reference-only rows
- **Candidate-only rows**
- Non-identical rows

The screenshot shows the 'Row Data Differences' tool for a comparison named 'My_compare_3'. The 'Reference Database' is 'hr.example.com' and the 'Candidate Database' is 'sales.example.com'. The 'Reference Object' is 'HR.DEPARTMENTS' and the 'Candidate Object' is 'HR_BIS.DEPARTMENTS'. The 'Index Columns' are 'DEPARTMENT_ID'. The 'Show' section has 'Candidate Only Rows' selected. Below, a table lists rows categorized as 'Candidate'. A green callout box points to the 'Candidate' label with the text 'Candidate - only rows'. Another green callout box points to the row with 'DEPARTMENT_ID' 280 with the text 'All rows existing in candidate object but not in referenced object'.

Row Source	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
Candidate	280	NEW_Payroll		1700

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In the example in the slide, you chose to view the candidate-only rows, those with index column values that exist only in the candidate object but not in the referenced object. For example, the rows with `DEPARTMENT_ID` 280 does not exist in the referenced object `HR.DEPARTMENTS` whereas it does in the candidate object `HR_BIS.DEPARTMENTS`.

Results: Non-Identical Rows

- Reference-only rows
- Candidate-only rows
- Non-identical rows between **referenced object** and **candidate object**

Row Data Differences: My_compare_3

Reference Database: **hr.example.com** Candidate Database: **sales.example.com**
 Logged in As: **system** Logged in As: **system**
 Reference Object: **HR.DEPARTMENTS** Candidate Object: **HR_BIS.DEPARTMENTS**
 Index Columns: DEPARTMENT_ID

Show ☐ Reference Only Rows ☐ Candidate Only Rows ☒ Non-identical Rows

Rows are categorized based on their index column values. Reference only and candidate only rows are those with index column values present on both sides, but with one or more differences in the other (non-index) column values.

Non-identical rows

Row Source	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
Reference	240	Government Sales		1700
Candidate	240	Government Sales		1800

All rows existing in both referenced and candidate objects but with different values

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In the example in the slide, you chose to view the non-identical rows. This means those with index column values present on both sides, but with one or more differences in the other (non-index) column values. In this example, the row with DEPARTMENT_ID 240 exists in both objects HR.DEPARTMENTS and HR_BIS.DEPARTMENTS but with LOCATION_ID 1700 in the reference object and 1800 in the candidate object.

Quiz

The schema change plan in the target database is used to:

- a. Modify one or more attributes of an object
- b. Create a missing object
- c. Drop an object
- d. All of the above

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: d

Quiz

The Data Comparisons job shows the following differences between two compared objects:

- a. Reference-only rows
- b. Candidate-only rows
- c. Non-identical rows between referenced object and candidate object
- d. All of the above

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Answer: d

Summary

In this lesson, you should have learned how to explain:

- Why and when to use the new Schema Change Plans feature
- How to use schema change plans
- Why and when to use the new Data Comparisons feature
- How to use data comparisons

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice

C-1: Using Schema Change Plans (*Optional demonstration*)

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

New and Enhanced Features in Other Courses



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Further Information

For more information about topics that are not covered in this course, refer to the following:

- Other Oracle University Oracle Database 12c ILT courses
- Oracle Database 12c: New Features Self Studies
 - A comprehensive series of self-paced online courses covering all new features in detail
 - Demonstrations for all topics covered in self-paced online courses:
<http://www.oracle.com/goto/oll>
- Oracle Learning Library OBEs and Demos
 - <http://www.oracle.com/technology/obe/demos/admin/demos.html>
 - <http://www.oracle.com/technology/obe/start/index.html>

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For more information about topics that are not covered in this course, refer to the following:

- Oracle University Oracle Database 12c instructor-led courses
- Oracle Database 12c: New Features self-paced online courses
- Oracle By Example series and demos: Oracle Database 12c
 - Oracle by Example (OBE) tutorials provide hands-on, step-by-step instructions on how to implement various technology solutions to business problems. In addition to the following OBE tutorials, you can also access more product training at the Oracle University Knowledge Center.
 - Demos provide an automated demonstration of a particular task with explanations on how the task is performed.
 - Tutorials provide concept explanations, demos, and step-by-step instructions for a particular product or topic.

Suggested Oracle University ILT Courses

- Enterprise Manager Cloud Control
 - *Using Oracle Enterprise Manager Cloud Control 12c*
 - *Oracle Enterprise Manager Cloud Control 12c: Install and Upgrade Workshop*
 - *Oracle Enterprise Manager Cloud Control 12c: Advanced Administration Workshop*
 - *Oracle Enterprise Manager Cloud Control 12c: Advanced Configuration Ed 1.1*
- Installation and Upgrade
 - *Oracle Database 12c: Install and Upgrade workshop*
- Security
 - *Oracle Database 12c: Security*

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered within a solid red rectangular bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To obtain more information about the key cloud computing technologies used by the Oracle products and other new Oracle Database features, follow additional training from Oracle University.

Suggested Oracle University ILT Courses

- Real Application Cluster
 - *Oracle Database 12c: High Availability New Features Ed 1*
 - *Oracle Database 12c: Clusterware Administration*
 - *Oracle Database 12c: ASM Administration Ed 1*
 - *Oracle Database 12c: RAC Administration*
- Exadata
 - *Exadata Database Machine Administration workshop Ed1*
- Performance
 - *Oracle Database 12c: Performance Management and Tuning*
 - *Oracle Database 12c: SQL Tuning for Developers*
- Data Guard
 - *Oracle Database 12c: Data Guard Administration Ed 1*

The Oracle University logo, featuring the word "ORACLE" in white capital letters on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To obtain more information about the key cloud computing technologies used by the Oracle products and other new Oracle Database features, follow additional training from Oracle University.